Appunti di Algoritmi e Strutture Dati - 2 parte

Alessio Novel

Strutture dati speciali....3
Insiemi disgiunti....3
Realizzazione basata su liste....5
Realizzazione basata su alberi....5
Euristiche....6
Programmazione dinamica....10
Domino....12
Hateville....16
Zaino....19
Zaino con memoization....22
Variante dello zaino, senza limiti....25
Sottosequenza comune massimale....28
String matching approssimato....35
Prodotto di catena di matrici....38
Insieme indipendente di intervalli pesati....49
Strutture di dati e progettazione di algoritmi....54
Cammini minimi....54
Algoritmo prototipo....58
Algoritmo di Dijkstra....60
Algoritmo di Johnson....63
Algoritmo di Fredman-Tarjan....64
Algoritmo di Bellman-Ford-Moore....64
Cammini minimi su DAG....67
Cammini minimi da sorgente multipla....69
Greedy....73
Insieme indipendente di intervalli....74
Resto....78
Scheduling....81
Zaino frazionario....83
Compressione di Huffman....85
Alberi di copertura di peso minimo....90
Algoritmo generico....92
Algoritmo di Kruskal....95
Algoritmo di Prim....97
Ricerca locale....101
Flusso massimo....102
Dimostrazione correttezza....111

Complessità....115
Abbinamento (matching) massimo nei grafi bipartiti....117
Backtrack....118
....120
Enumerazione....120
Sottoinsiemi e permutazioni....122
Subset sum....128
Giochi....130
Backtracking iterativo....136
Algoritmi probabilistici....136
Algoritmi Montecarlo....137
Algoritmi Las Vegas....144
Teoria dell’ NP-completezza....150
Riducibilità polinomiale....152
Classi ,....157
Certificati polinomiali....158
Classe....159
Problemi -Completi....160
Tecniche risolutive per problemi intrattabili....166
Algoritmi pseudo polinomiali....167
NP-completezza “forte”....171
Algoritmi di approssimazione....174
Non approssimabilità....179
Algoritmi euristici....179
Greedy....179
Ricerca locale....181
Algoritmi branch-&-bound....182
Commesso viaggiatore....184
Algoritmi di ordinamento....187

Strutture dati speciali

Insiemi disgiunti

In alcune applicazioni siamo interessati a gestire una collezione di insiemi dinamici disgiunti.
Una collezione di **insiemi dinamici disgiunti** è una collezione $S=\{S_1,\ldots,S_k\}$ di insiemi dove:

- $\forall i,j\ :\ i\neq j\Rightarrow S_i\cap S_j=\varnothing$
- $¿i=1¿k\ S_i=S,\ \ dove\ \ |S|=n$

Un esempio pratico di utilizzo di questa struttura dati è quello delle componenti connesse di un grafo.
Le **operazioni fondamentali** sono:

- Creare n insiemi disgiunti, ognuno composto da un unico elemento
- $merge()$, ovvero unire più insiemi (posso farlo al massimo $n-1$ volte)
- $find()$, ovvero identificare l'insieme a cui appartiene un elemento

Ogni insieme è rappresentato da un **rappresentante** univoco. Il rappresentante dell'insieme S_i può essere un qualunque membro di S_i.
Le operazioni di ricerca del rappresentante su uno stesso insieme devono restituire sempre lo stesso oggetto (solo successivamente all'unione con un altro insieme il rappresentante può cambiare).

Per la memorizzazione, invece di memorizzare oggetti, utilizziamo gli interi $1\ldots n$ e assumiamo che l'associazione intero-oggetto sia memorizzata esternamente.

Specifica

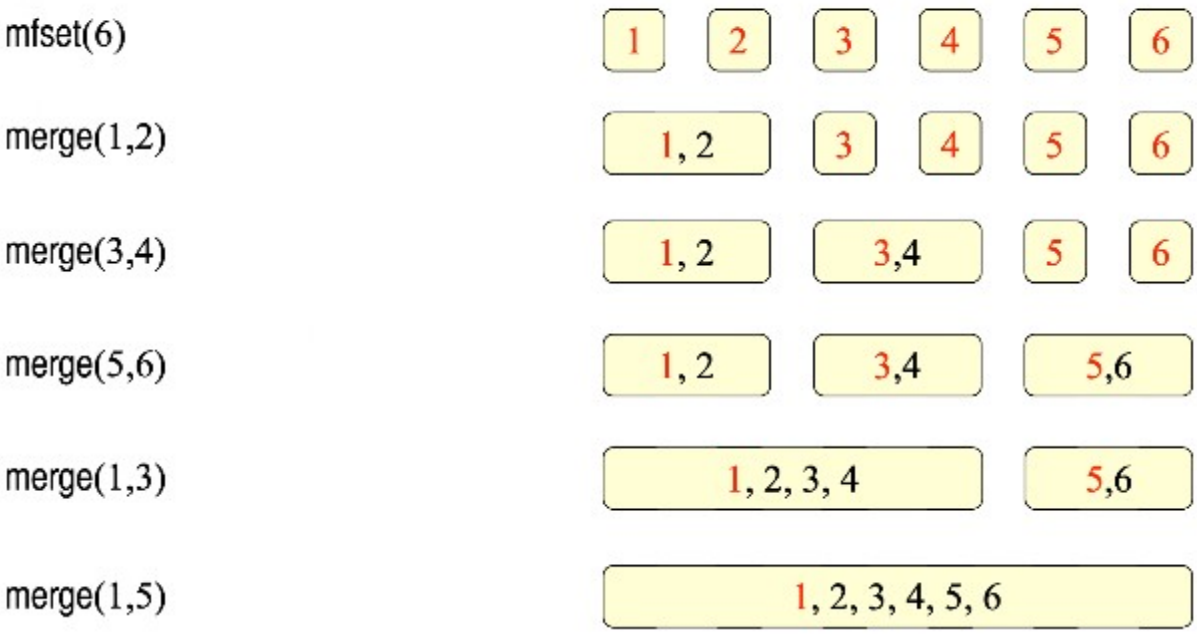
MFSET

% Crea n componenti $\{1\},\ldots,\{n\}$
MFSET Mfset(**int** n)

% Restituisce il rappresentante della componente contenente x
int find(**int** x)

% Unisce le componenti che contengono x e y
merge(**int** x, **int** y)

Esempio



Vediamo ora un'applicazione pratica degli insiemi disgiunti, risolvendo il problema di trovare le componenti connesse di un grafo non orientato **dinamico**.

- Si inizia con ogni vertice che rappresenta una componente connessa
- Per ogni $(u, v) \in E$ si esegue $merge(u, v)$
- Alla fine ogni insieme disgiunto rappresenta una componente connessa

```
MFSET cc(GRAPH G)
MFSET M = Mfset(G.n)
foreach u ∈ G.V() do
    foreach v ∈ G.adj(u) do
        M.merge(u, v)
return M
```

Complessità

$O(n) + m$ operazioni merge()

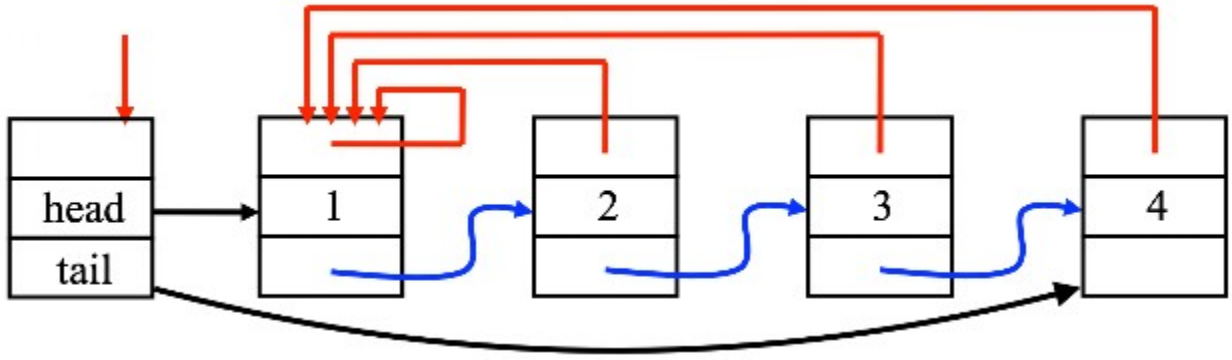
Se l'operazione $merge()$ ha costo $O(1)$ (come vedremo dopo), l'inizializzazione costa $O(n+m)$ e tutte le successive merge costano $O(1)$.

Questo algoritmo è interessante per la capacità di gestire grafi dinamici (in cui gli archi vengono aggiunti dopo la creazione del grafo).

Realizzazione basata su liste

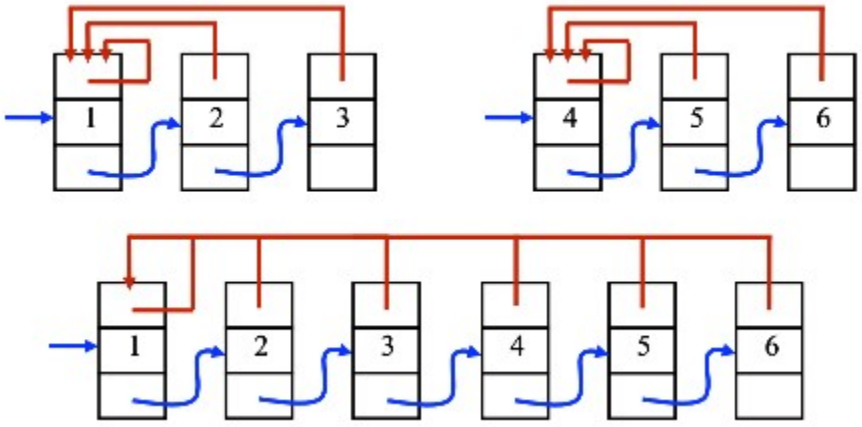
Nella rappresentazione basata su insiemi di liste ogni insieme viene rappresentato da una lista concatenata.
Il **primo oggetto** di ogni lista è il **rappresentante** dell'insieme.
Ogni elemento della lista contiene:

- Un oggetto
- Un puntatore all'elemento successivo
- Un puntatore al rappresentante



L'operazione $find(x)$ consiste nel restituire il rappresentante di x, il quale è puntato direttamente dall'elemento che contiene l'oggetto x.
Quindi l'operazione $find(x)$ richiede tempo $O(1)$.

L'operazione $merge(x, y)$ consiste nell' "appendere" la lista che contiene y alla lista che contiene x, modificando i puntatori ai rappresentanti nella lista "appesa".
Il costo nel caso pessimo per n operazioni è $O(n^2)$ (continuo ad appendere la lista più lunga a quella più corta) e dunque il costo ammortizzato è $O(n)$.



Questo costo non è molto buono.

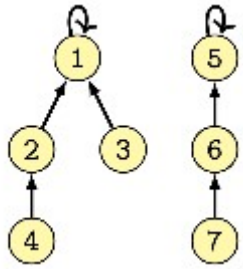
Realizzazione basata su alberi

Nella rappresentazione basata su insiemi di alberi ogni insieme viene rappresentato da un albero, creando una foresta.

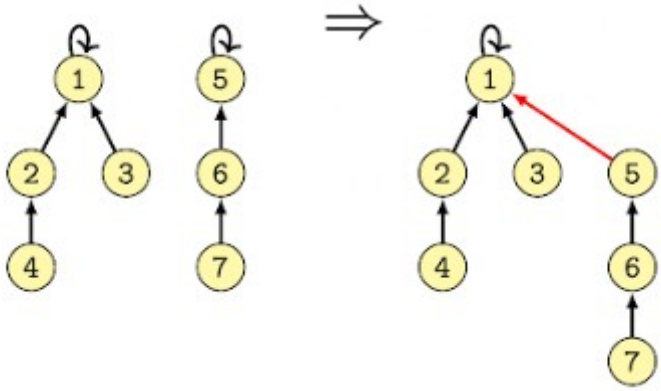
La **radice** è il **rappresentante** dell'insieme.
Ogni nodo dell'albero contiene:

- Un oggetto
- Un puntatore al padre

La radice ha un puntatore a sé stessa.



L'operazione $merge(x, y)$ consiste nell' "agganciare" l'albero radicato in y ad x, modificando il puntatore al padre di y (x e y sono i rappresentanti)
Il costo è $O(1)$ (bisogna fare una singola operazione) una volta trovati i rappresentanti, e il costo per trovarlo dipende dalla funzione $find()$.



L'operazione $find(x)$ consiste nel risalire la lista dei padri di x fino a trovare la radice, e restituire la radice come rappresentante.
Abbiamo costo $O(n)$ nel caso pessimo (nel caso in cui l'albero è costruito in modo da risultare una lista)

Euristiche

Vediamo ora le cosiddette **tecniche euristiche**, ovvero delle tecniche per migliorare le due realizzazioni che abbiamo visto.
Cosa vuol dire euristico? Un **algoritmo euristico** è un particolare tipo di algoritmo progettato per:

- Risolvere un problema più velocemente (qualora i metodi classici siano troppo lenti)
- Trovare una soluzione approssimata (qualora i metodi classici falliscano nel trovare una soluzione esatta)

Le euristiche applicate agli insiemi disgiunti sono:

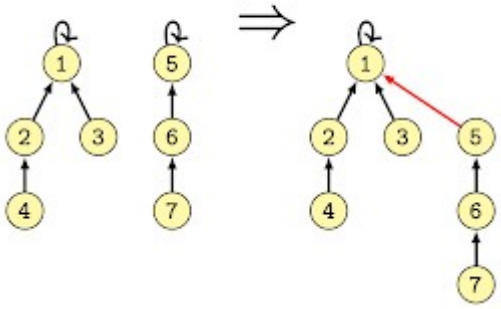
- Euristica del **peso** (per le **liste**)
- Euristica del **rango** (per gli **alberi**)
- Euristica della **compressione dei cammini** (per gli **alberi**)

L'**euristica del peso** (per le liste) è una strategia usata per diminuire il costo dell'operazione $merge()$.
Consiste nel memorizzare nelle liste l'informazione della loro lunghezza, così da far in modo di agganciare la lista più corta a quella più lunga durante una $merge()$. Notiamo che la lunghezza della lista può essere mantenuta in tempo $O(1)$.
Parlando di complessità, è possibile dimostrare che il costo delle operazioni $merge()$ è limitato superiormente da $O(\log\ n)$.
Infatti ci possono essere al massimo $n-1$ operazioni di $merge()$ che fanno concretamente qualcosa. Utilizzando questa euristica, ogni volta che un oggetto cambia rappresentante, si ritroverà in un insieme che contiene almeno il doppio degli elementi dell'insieme originale a cui apparteneva.
Utilizziamo il metodo dell'analisi ammortizzata basata sui crediti, e assegniamo ad ogni oggetto un numero di crediti pari a $\lfloor \log n \rfloor$ al momento della creazione dell'insieme corrispondente.
Ad ogni operazione $merge()$, consumiamo un credito per ognuno degli oggetti che cambia rappresentante; ogni singolo oggetto può essere coinvolto in al più $\lfloor \log n \rfloor$ (per via del raddoppiamento), quindi un totale di $O(n\ \log\ n)$ crediti è sufficiente a pagare qualunque sequenza di unioni.

L'**euristica sul rango** (per gli alberi) è una strategia usata per diminuire il costo dell'operazione $find()$.
Ogni nodo contiene informazioni sul proprio **rango**.
Il rango $rank[x]$ di un nodo x è il numero di archi del cammino più lungo fra x e una foglia sua discendente. Il rango quindi è l'altezza del sottoalbero associato al nodo.
Questo viene fatto con l'obiettivo di **mantenere bassa l'altezza degli alberi**.

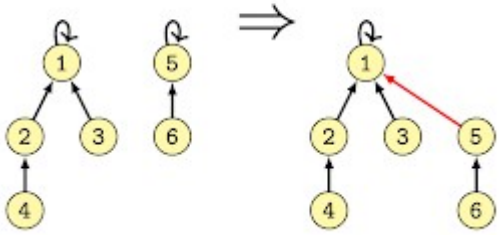
Alberi di rango uguale

- Si aggancia un albero alla radice dell'altro (indifferente)
- L'altezza cresce di 1



Alberi di rango diverso

- Si aggancia l'albero con rango più basso all'albero con rango più alto
- L'altezza resta inalterata



MFSET

```
int[] parent
int[] rank

MFSET Mfset(int n)
    MFSET t = new MFSET
    t.parent = int[1...n]
    t.rank = int[1...n]
    for i = 1 to n do
        t.parent[i] = i
        t.rank[i] = 0
    return t
```

merge(int x, int y)

```
r_x = find(x)
r_y = find(y)
if r_x ≠ r_y then
    if rank[r_x] > rank[r_y] then
        parent[r_y] = r_x
    else if rank[r_y] > rank[r_x] then
        parent[r_x] = r_y
    else
        parent[r_x] = r_y
        rank[r_y] = rank[r_y] + 1
```

Per giustificare la bontà di questa scelta, cerchiamo di calcolare una limitazione superiore all'altezza dell'albero.

Teorema: Un albero MFSET con radice r ottenuto tramite euristica sul rango ha almeno $2^{rank[r]}$ nodi.

Dimostrazione (informale)

- **Induzione caso base**: All'inizio tutti gli alberi sono costituiti da un nodo singolo, con rank 0;
- quindi ogni albero ha almeno $2^0 = 1$ nodi, corretto.

- **Induzione caso 1**: Facendo merge di due alberi con radici x, y con rango $rank[x] > rank[y]$
  - il rango finale è pari a $rank[x]$
  - Per induzione, il numero di nodi è $\geq 2^{rank[x]} + 2^{rank[y]}$, che è maggiore di $2^{rank[x]}$.
- **Induzione caso 2**: Facendo merge di due alberi con radici x, y con rango $rank[x] = rank[y]$,
  - il rango finale è pari a $rank[x] + 1$
  - Per induzione, il numero di nodi è $\geq$

$$2^{rank[x]} + 2^{rank[y]} = 2^{rank[x]} + 2^{rank[x]} = 2^{rank[x]+1}$$

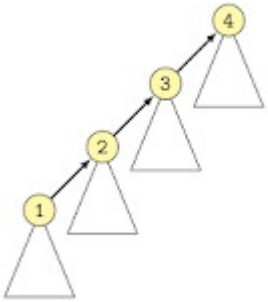
come volevasi dimostrare

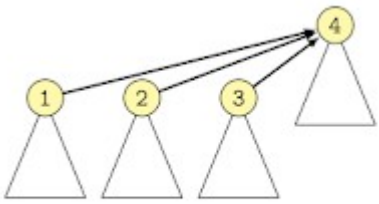
Avendo che, dato un albero contenente k nodi con radice r, vale $k \geq 2^{rank[r]}$, possiamo applicare il logaritmo da entrambi i lati e ottenere che $rank[r] \leq \log\ k$.

Di conseguenza possiamo dire che un albero $MFSET$ con radice r ed n nodi ha altezza inferiore a $\log\ n$, e quindi la complessità con euristica sul rango di $find()$, e quindi di $merge()$, è $O(\log\ n)$.

L'**euristica sulla compressione dei cammini** invece consiste nell'appiattire l'albero durante le operazioni di $find(x)$, in questo modo le ricerche successive di x saranno svolte in $O(1)$. Dunque modifichiamo la funzione $find()$ in modo che alteri l'albero durante la sua esecuzione, in pratica ogni nodo che viene incontrato dalla $find()$ nel percorso di risalita dal generico nodo x alla radice, viene reso figlio della radice:

```
int find(int x)
if parent[x] ≠ x then
    parent[x] = find(parent[x])
return parent[x]
```





È importante notare che il numero di operazioni della nuova versione di $find()$ aumenta solo per un fattore costante rispetto alla vecchia versione e che pertanto la sua complessità nel caso pessimo non aumenta.

Il vantaggio introdotto dalla compressione dei cammini si ha quando viene effettuata una sequenza di operazioni formata da m $find()$ e $merge()$ con $m \geq n$ in un insieme di n elementi, infatti la complessità è $O(n + m \cdot \alpha(n))$ invece di $O(n + m \ \log \ n)$ senza la compressione.
$\alpha(n)$, ovvero la funzione inversa della funzione di Ackermann, cresce lentissimamente, ne segue che ciascune delle m operazioni $find()$ e $merge()$ della sequenza ha complessità ammortizzata $O\left(\frac{n + m \cdot \alpha(n)}{m}\right)$, che è pressoché $O(1)$ per ogni valore "realistico" di n ed m.

Applicando entrambe le euristiche degli alberi, ovvero quella del rango e quella dei cammini, abbiamo che il rango non rappresenta più l'altezza del nodo, ma il limite superiore all'altezza del nodo. Infatti **non** viene calcolato il **rango corretto** (è troppo difficile mantenere le informazioni di rango corretto dopo l'appiattimento e non porta nessun vantaggio concreto farlo).

Ecco un riassunto delle complessità:

Algoritmo	find()	merge()
Liste	$O(1)$	$O(n)$
Alberi	$O(n)$	$O(1)^+$
Liste + Euristica sul peso	$O(1)$	$O(\log n)^*$
Alberi + Euristica sul rango	$O(\log n)$	$O(1)^+$
Alberi + Euristica sul rango + Compressione cammini	$O(1)^*$	$O(1)$

* Complessità ammortizzata
\+ Si considera solo l'operazione di unione, non si considera l'identificazione dei rappresentanti tramite find()

Programmazione dinamica

La programmazione dinamica è una filosofia simile al Divide-et-impera, ma portata all'estremo. Si basa sulla massima "Il pesce grosso mangia tutti i pesci piccoli".
In altri termini, se non si sa con esattezza quali sottoproblemi risolvere, si può provare a risolverli tutti e conservare i risultati ottenuti per poterli usare successivamente nel risolvere il problema di dimensione più grande.
Questa tecnica è detta **programmazione dinamica**, perché indica un processo di programmazione (o pianificazione) di un metodo risolutivo che sia applicabile dinamicamente per dimensioni via via crescenti del problema.
Rispetto alla tecnica Divide-et-impera, la programmazione dinamica presenta tre grandi differenze:

- È iterativa e non ricorsiva
- Affronta i sottoproblemi "dal basso verso l'alto" anziché "dall'alto verso il basso"

- Memorizza i risultati dei sottoproblemi in una tabella

Mentre il Divide-et-impera risulta vantaggioso quando i sottoproblemi più piccoli da risolvere sono indipendenti, la programmazione dinamica è conveniente quando i sottoproblemi non sono indipendenti, ma ne condividono altri.
Procedendo dal basso verso l'alto la programmazione dinamica risolve un sottoproblema comune alla volta, mentre il Divide-et-impera lo risolverebbe più volte.

Perché la programmazione dinamica sia applicabile occorre che:

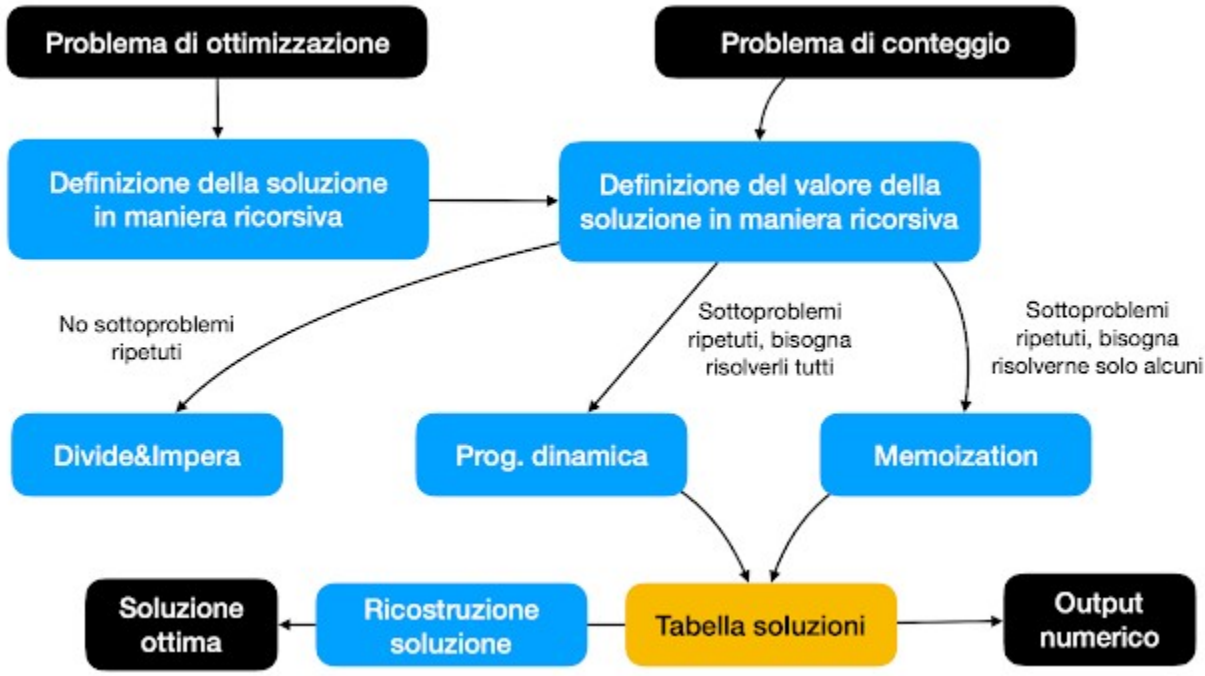
1. Sia possibile combinare le soluzioni dei sottoproblemi per trovare la soluzione di un problema più grande
2. Le decisioni prese per risolvere in modo ottimo un sottoproblema rimangano valide quando il sottoproblema diviene un "pezzo" di un problema più grande
3. Una tecnica Divide-et-impera richieda di risolvere più volte gli stessi sottoproblemi e sia pertanto inutilizzabile da un punto di vista computazionale

Le proprietà 1 e 2 prendono il nome di **sottostruttura ottima**. Ma non sono sufficienti; in pratica, affinché un algoritmo basato sulla programmazione dinamica abbia complessità polinomiale, occorre inoltre che:

4. Ci sia un numero polinomiale di sottoproblemi da risolvere
5. Per evitare di risolvere più di una volta lo stesso sottoproblema, si usi una tabella in cui si memorizzino le soluzioni di tutti i sottoproblemi, senza preoccuparsi se la soluzione di un particolare sottoproblema verrà poi utilizzata oppure no
6. Il tempo per combinare le soluzioni dei sottoproblemi e trovare la soluzione del problema più grande sia polinomiale

In sintesi la programmazione dinamica è un metodo per spezzare un problema ricorsivamente in sottoproblemi, dove ogni sottoproblema viene risolto una volta sola memorizzando la soluzione dei sottoproblemi in una tabella. Nel caso in cui un sottoproblema debba essere risolto nuovamente, si usa la sua soluzione in tabella. La tabella è facilmente indirizzabile (lookup in $O(1)$).

L'approccio generale è il seguente:



Vediamo un po' di problemi per capire come utilizzare la tecnica della programmazione dinamica.

Domino

Definizione:
Il gioco del domino è basato su tessere di dimensione 2×1. Scrivere un algoritmo efficiente che prenda in input un intero n e restituisca il numero di possibili disposizioni di n tessere in un rettangolo $2 \times n$.

Esempio

I casi (a)-(e) della figura rappresentano le cinque disposizioni possibili con cui è possibile riempire un rettangolo 2×4.



Quante disposizioni ci sono con $n=7$?

Proviamo un approccio "standard" ricorsivo.
Definiamo una formula ricorsiva $DP[n]$ che permette di calcolare il numero di disposizioni possibili quando si hanno n tessere.
Vediamo prima i casi base:

- Quante disposizioni sono possibili se non ho tessere (ovvero con $n=0$)?
  Con $n=0$, esiste una sola disposizione possibile (nessuna tessera)
- Quante disposizioni sono possibili se ho solo una tessera (ovvero con $n=1$)?
  Con $n=1$, esiste una sola disposizione possibile (tessera verticale)

Ora vediamo come comportarci quando abbiamo n tessere con $n \geq 2$:

- Cosa succede se decido di mettere l'ultima tessera in verticale?
  Se metto una tessera in verticale, risolverò il problema di dimensione $n-1$
- Cosa succede se decido di mettere l'ultima tessera in orizzontale?
  Se metto una tessera in orizzontale, ne devo mettere due; risolverò il problema di dimensione $n-2$

Queste due possibilità si sommano insieme (perchè è un problema di conteggio) e otteniamo:

$$DP[n] = \begin{cases} 1 & n \leq 1 \\ DP[n-2] + DP[n-1] & n > 1 \end{cases}$$

Notiamo che $DP[n]$ è pari al $(n+1)$-*esimo* numero della serie di Fibonacci.

Un algoritmo ricorsivo che risolve questo problema è il seguente:

```
int domino1(int n)
if n ≤ 1 then
    return 1
else
    return domino1(n − 1) + domino1(n − 2)
```

E la sua equazione di ricorrenza è la seguente:

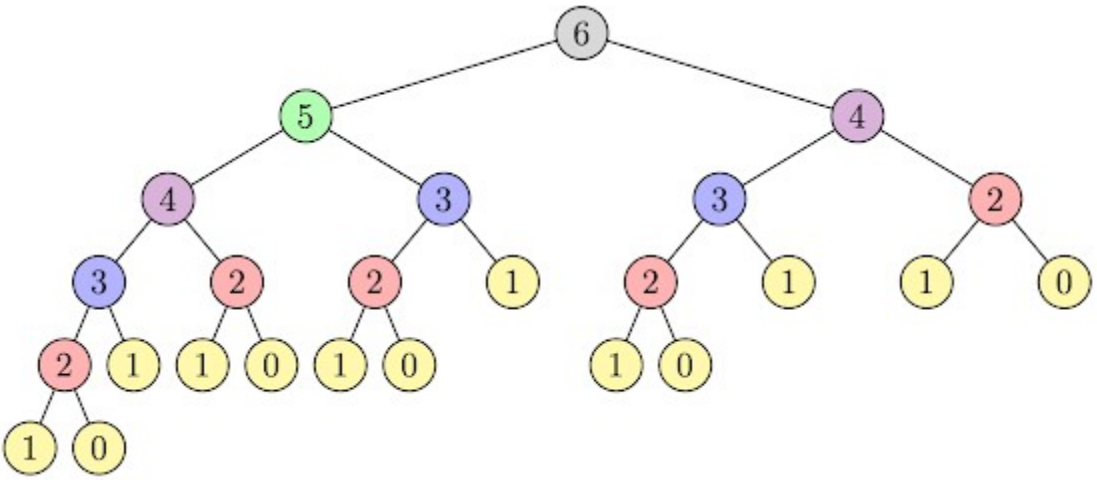
$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n-1) + T(n-2) + 1 & n > 1 \end{cases}$$

La sua complessità può essere calcolata con il Teorema di Ricorrenza lineare di ordine costante:

- $a_1 = 1, a_2 = 1, a = a_1 + a_2 = 2, \beta = 0$
- Complessità: $\Theta(a^n \cdot n^\beta)$

$$T(n) = \Theta(2^n)$$

Notiamo però che molti sottoproblemi vengono ripetuti e ricalcolati più volte:



Per evitare di risolvere un sottoproblema più di una volta, memorizziamo il risultato ottenuto risolvendo un particolare problema in una **tabella DP** (vettore, matrice o dizionario).
La tabella deve contenere un elemento per ogni sottoproblema che dobbiamo risolvere.

Per costruire questa tabella, memorizziamo i casi base direttamente nelle posizioni relative applicando un'**iterazione bottom-up**:

- Si parte dai problemi risolubili come casi base
- Si sale verso problemi via via più grandi …
- … fino a raggiungere il problema originale

Vediamo quindi l'algoritmo iterativo che risolve il problema del domino:

```
int domino2(int n)
DP = new int[0...n]
DP[0] = DP[1] = 1
for i = 2 to n do
    DP[i] = DP[i − 1] + DP[i − 2]
return DP[n]
```

n	0	1	2	3	4	5	6	7
DP []	1	1	2	3	5	8	13	21

Notiamo facilmente che la complessità in tempo dell'algoritmo $domino2(n)$ è $\Theta(n)$ e che la complessità in spazio è anch'essa $\Theta(n)$.
La complessità in tempo non è migliorabile, ma volendo possiamo ridurre la complessità in spazio evitando di usare un vettore ma salvandosi semplicemente le ultime due caselle, visto che servono solo ed esclusivamente quelle per calcolare la casella successiva:

```
int domino3(int n)
int DP_0 = 1
int DP_1 = 1
int DP_2 = 1
for i = 2 to n do
    DP_0 = DP_1
    DP_1 = DP_2
    DP_2 = DP_0 + DP_1
return DP_2
```

n	0	1	2	3	4	5	6	7
DP_0	-	-	1	1	2	3	5	8
DP_1	1	1	1	2	3	5	8	13
DP_2	1	1	2	3	5	8	13	21

Ora abbiamo che la complessità in spazio è costante e quindi $\Theta(1)$.

In realtà questo algoritmo non cresce linearmente sebbene abbiamo detto che la complessità in tempo dell'algoritmo $domino2(n)$ è $\Theta(n)$.
Visto che la serie generata (che è quella di fibonacci) cresce rapidamente, il numero di bit necessari a rappresentare i risultati crescerà anch'esso rapidamente.
Per capire quanti bit sono necessari per rappresentare $F(n)$ ci viene in aiuto la formula di Binet.

Formula di Binet per i numeri di Fibonacci

$$DP[n-1] = F(n) = \frac{\phi^n}{\sqrt{5}} - \frac{(1-\phi)^n}{\sqrt{5}}$$

dove ϕ è la sezione aurea:

$$\phi = \frac{1+\sqrt{5}}{2} = 1{,}6180339887\ldots$$

$$\frac{1}{\phi} = \phi - 1 = \frac{2}{1+\sqrt{5}} = 0{,}6180339887\ldots$$



Questo ci dice che il numero di bit per rappresentare la serie di Fibonacci cresce linearmente.
Infatti il logaritmo in base 2 di $F(n)$ ha complessità $\Theta(n)$.
Di conseguenza tutto quello che abbiamo visto deve essere moltiplicato per n:

Nel modello di costo logaritmico, le tre versioni hanno complessità:

Funzione	Complessità (Tempo)	Complessità (Spazio)
domino1()	$O(n2^n)$	$O(n^2)$
domino2()	$O(n^2)$	$O(n^2)$
domino3()	$O(n^2)$	$O(n)$

Questa cosa non c'entra niente con la programmazione dinamica ma era per far vedere che le cose che avevamo detto prima non erano totalmente vere.

Hateville

Hateville è un villaggio particolare, composto da n case, numerate da 1 a n lungo una singola strada. Ad Hateville ognuno odia i propri vicini della porta accanto, da entrambi i lati.
Quindi, il vicino i odia i vicini $i-1$ e $i+1$ (se esistenti).
Hateville vuole organizzare una sagra e vi ha affidato il compito di raccogliere i fondi. Ogni abitante i ha intenzione di donare una quantità $D[i]$, ma non intende partecipare ad una raccolta fondi a cui partecipano uno o entrambi i propri vicini.

L'obiettivo è scrivere un algoritmo che dato il vettore D, restituisca la quantità massima di fondi che può essere raccolta.
Bonus: restituire un sottoinsieme di indici $S \subseteq \{1,\dots,n\}$ tale che il totale dei fondi raccolti $T = \sum_{i \in S} D[i]$ sia massimale.

Dobbiamo trovare una formula ricorsiva che permetta di calcolare il sottoinsieme di case che, se selezionate, dia origine alla maggior quantità di donazioni.
Per definire la formula ricorsiva ridefiniamo il problema.
Sia $HV(i)$ uno dei possibili insiemi di indici da selezionare per ottenere una donazione ottimale delle prime i case di Hateville numerate da 1 a n. Di conseguenza $HV(n)$ è la soluzione del problema originale.

Consideriamo l'abitante i:

- Se non accettiamo la sua donazione abbiamo che $HV(i) = HV(i-1)$
- Se accettiamo la sua donazione abbiamo che $HV(i) = \{i\} \cup HV(i-2)$

Per decidere se accettare o meno dobbiamo prendere l'insieme di indici che ci porta alla somma delle donazioni più alta, quindi:

$$HV(i) = highest(HV(i-1),\ \{i\} \cup HV(i-2))$$

(Usiamo highest e non max visto che abbiamo un insieme di indici e non un valore).

Per convincerci della correttezza di questa affermazione vediamo il **Teorema della Sottostruttura ottima**.
Sia P_i il problema dato dalle prime i case. Sia S_i una soluzione ottima per il problema P_i. Ne consegue che:

1. Se $i \notin S_i$, allora $S_i = S_{i-1}$
2. Se $i \in S_i$, allora $S_i = S_{i-2} \cup \{i\}$

Sia $\|S\| = \sum_{k \in S} D[k]$ il totale di donazioni di un insieme S.
Dimostriamo il caso 1, ovvero $i \notin S_i$.

S_i è una soluzione ottima anche per P_{i-1}.
Se così non fosse, esisterebbe una soluzione S'_{i-1} per il problema P_{i-1} tale che $\|S'_{i-1}\| > \|S_i\|$.
Ma allora S'_{i-1} sarebbe una soluzione anche per P_i tale che $\|S'_{i-1}\| > \|S_i\|$, ed è assurdo.
Dimostriamo il caso 2, ovvero $i \in S_i$.
$i-1 \notin S_i$, altrimenti non sarebbe una soluzione ammissibile.
Quindi $S_i - \{i\}$ è una soluzione ottima per P_{i-2}.
Se così non fosse, esisterebbe una soluzione S'_{i-2} per il problema P_{i-2} tale che $\|S'_{i-2}\| > \|S_i - \{i\}\|$
.
Ma allora $S'_{i-2} \cup \{i\}$ sarebbe una soluzione per P_i tale che $\|S'_{i-2} \cup \{i\}\| > \|S_i\|$, ed è assurdo.

Ci mancano i casi base:

- Se Hateville non ha abitanti allora l'insieme di indici sarà vuoto → $HV(0) = \varnothing$
- Se Hateville ha un solo abitante allora l'insieme di indici sarà composto da un elemento, ovvero l'unico abitante esistente -> $HV(1) = \{1\}$

Mettendo tutto insieme otteniamo:

$$HV(i) = \begin{cases} \emptyset & i = 0 \\ \{1\} & i = 1 \\ highest(HV(i-1), HV(i-2) \cup \{i\}) & i \geq 2 \end{cases}$$

La domanda è: vale la pena scrivere un algoritmo ricorsivo, basato su divide-et-impera, per risolvere il problema di Hateville? La risposta è no perchè, come prima, ci sono dei sottoproblemi che vengono risolti e ricalcolati più volte.

Di conseguenza riutilizziamo l'idea che abbiamo utilizzato nell'esercizio del Domino e creiamo una tabella con un algoritmo iterativo bottom-up:

Esempi

i	0	1	2	3	4	5	6	7
D		10	5	5	8	4	7	12
HV	$\emptyset$	$\{1\}$	$\{1\}$	$\{1,3\}$	$\{1,4\}$	$\{1,3,5\}$	$\{1,4,6\}$	$\{1,3,5,7\}$

i	0	1	2	3	4	5	6	7
D		10	1	1	10	1	1	10
HV	$\emptyset$	$\{1\}$	$\{1\}$	$\{1,3\}$	$\{1,4\}$	$\{1,4\}$	$\{1,4,6\}$	$\{1,4,7\}$

Abbiamo due problemi:

- Dobbiamo definire bene la funzione $highest()$ (è solo una cosa concettuale al momento)
- Memorizzare insiemi di indici nella tabella è **costoso**

Proviamo quindi a concentrarci solo sul valore lasciando stare gli indici.

Sia $DP[i]$ il **valore** della massima quantità di donazioni che possiamo ottenere dalle prime i case di Hateville.
$DP[n]$ è il valore della soluzione ottima.
Di conseguenza la formula ricorsiva è la seguente:

$$DP[i] = \begin{cases} 0 & i = 0 \\ D[1] & i = 1 \\ \max(DP[i-1], DP[i-2] + D[i]) & i \geq 2 \end{cases}$$

L'algoritmo iterativo che risolve il problema Hateville è il seguente:

```
int hateville(int[] D, int n)
int[] DP = new int[0...n]
DP[0] = 0
DP[1] = D[1]
for i = 2 to n do
    DP[i] = max(DP[i − 1], DP[i − 2] + D[i])
return DP[n]
```

Esempi

i	0	1	2	3	4	5	6	7
D		10	5	5	8	4	7	12
DP	0	10	10	15	18	19	25	31

i	0	1	2	3	4	5	6	7
D		10	1	1	10	1	1	10
DP	0	10	10	11	20	20	21	30

Il problema ora è che abbiamo solo il valore della soluzione massimale, ma non abbiamo la soluzione vera e propria (ovvero gli indici degli abitanti).

Però notiamo che dalla tabella è possibile ricostruire la soluzione originale guardando i valori.
Analizziamo l'elemento $DP[i]$:

- Se $DP[i] = DP[i-1]$, allora la casa i non è stata selezionata
- Se $DP[i] = DP[i-2] + D[i]$, allora la casa i è stata selezionata
- Se entrambe le equazioni sono vere, allora una vale l'altra

Per ricostruire la soluzione fino ad i procediamo in maniera ricorsiva:

- Se $DP[i] = DP[i-1]$, si prende la soluzione fino a $i-1$ (senza aggiungere nulla)
- Altrimenti, si prende la soluzione fino a $i-2$ e si aggiunge i

```
SET solution(int[] DP, int[] D, int i)
if i == 0 then
    return ∅
else if i == 1 then
    return {1}
else if DP[i] == DP[i − 1] then
    return solution(DP, D, i − 1)
else
    SET sol = solution(DP, D, i − 2)
    sol.insert(i)
    return sol
```

```
SET hateville(int[] D, int n)
[...]
return solution(DP, D, n)
```

Note: per come è stato costruito il codice, D non è necessario

La complessità computazionale di $solution(n)$ è $\Theta(n)$.
La complessità computazionale e spaziale di $hateville(n)$ è $\Theta(n)$.
Notiamo che possiamo migliorare la complessità di $hateville(n)$ (come in Domino) solo se non ci serve ricostruire la soluzione, perchè se ci serve ricostruire la soluzione abbiamo bisogno per forza della tabella completa.

Zaino

Dato un insieme di oggetti, ognuno caratterizzato da un peso e un profitto, e uno "zaino" con un limite di capacità, individuare un sottoinsieme di oggetti:

- il cui peso sia inferiore alla capacità dello zaino
- il valore totale degli oggetti sia massimale, i.e. più alto o uguale al valore di qualunque altro sottoinsieme valido di oggetti

Come input quindi abbiamo un vettore w (weight) dove $w[i]$ è il peso dell'oggetto $i\text{-}esimo$, un vettore p (profit) dove $p[i]$ è il profitto dell'oggetto $i\text{-}esimo$ e la capacità C dello zaino.

L'output è un insieme $S \subseteq \{1,\dots,n\}$ tale che:

- Il peso totale deve essere minore o uguale alla capacità

  Ovvero $w(S) = \sum_{i \in S} w[i] \leq C$
- Il profitto totale deve essere massimizzato

  Ovvero $argmax_S \ \ p(S) = \sum_{i \in S} p[i]$

Vediamo come definire matematicamente il valore della soluzione.
Dato uno zaino di capacità C e n oggetti caratterizzati da un peso w e un profitto p, definiamo $DP[i][c]$ come il massimo profitto che può essere ottenuto dai primi $i \leq n$ oggetti contenuti in uno zaino di capacità $c \leq C$.

Il massimo profitto del problema originale è rappresentato da $DP[n][C]$.

Definiamo la parte ricorsiva.
Consideriamo l'ultimo oggetto, se non lo prendiamo la capacità non cambia visto che non c'è profitto, quindi $DP[i][c] = DP[i-1][c]$
Se invece lo prendiamo dobbiamo sottrarre il peso dalla capacità e aggiungere il profitto relativo, quindi $DP[i][c] = DP[i-1][c-w[i]] + p[i]$.

Di conseguenza la soluzione migliore è quella con profitto massimo:

$$DP[i][c] = \max(\overbrace{DP[i-1][c-w[i]] + p[i]}^{\text{Preso}}, \overbrace{DP[i-1][c]}^{\text{Non preso}})$$

Vediamo ora i casi base:

- Il profitto massimo quando non abbiamo più oggetti o quando non abbiamo più capacità è 0
- Se la capacità è negativa dobbiamo fare in modo che non venga mai scelto quell'elemento, quindi impostiamo il profitto a -∞

E otteniamo:

$$DP[i][c] = \begin{cases} 0 & i = 0 \text{ or } c = 0 \\ -\infty & c < 0 \\ \max(DP[i-1][c-w[i]] + p[i], DP[i-1][c]) & \text{otherwise} \end{cases}$$

Un algoritmo iterativo che mette in pratica questa formula ricorsiva è il seguente:

```
int knapsack(int[] w, int[] p, int n, int C)
DP = new int[0...n][0...C]
for i = 0 to n do
    DP[i][0] = 0
for c = 0 to C do
    DP[0][c] = 0
for i = 1 to n do
    for c = 1 to C do
        if w[i] ≤ c then
            DP[i][c] = max(Preso: DP[i-1][c-w[i]] + p[i], Non preso: DP[i-1][c])
        else
            DP[i][c] = Non preso: DP[i-1][c]
return DP[n][C]
```

Esempio

```
w = [ 4, 2, 3, 4]
p = [10, 7, 8, 6]
C = 9
```

	c									
i	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	10	10	10	10	10	10
2	0	0	7	7	10	10	17	17	17	17
3	0	0	7	8	10	15	17	18	18	25
4	0	0	7	8	10	15	17	18	18	25

Risulta che la complessità computazionale della funzione $knapsack()$ è $O(n \cdot C)$.

Usciamo di nuovo dalla programmazione dinamica per far notare che in realtà questo algoritmo non è polinomiale.
C non è la dimensione dell'input, ma C è l'input, questo significa che C potrebbe essere assurdamente grande.
Questo perchè appunto l'algoritmo non è polinomiale, ma **pseudo-polinomiale**, perché sono necessari $k = \log\ C$ bit per rappresentare C e quindi la complessità è $O(n \cdot 2^k)$, quindi esponenziale rispetto alla dimensione dell'input.
Notiamo che tutti i problemi risolti usando la programmazione dinamica possono essere risolti in maniera ricorsiva con un metodo divide-et-impera, in quanto basta scrivere un algoritmo basato sulla formula ricorsiva.
Di conseguenza potrebbe essere che quando abbiamo C assurdamente grande ed n piccolo, potrebbe essere più vantaggioso usare un algoritmo di complessità $O(2^n)$ (come quello ricorsivo) piuttosto che questo con complessità $O(n \cdot C)$

```
int knapsack(int[] w, int[] p, int n, int C)
return knapsackRec(w, p, n, C)
```

```
int knapsackRec(int[] w, int[] p, int i, int c)
if c < 0 then
    return −∞
else if i == 0 or c == 0 then
    return 0
else
    int nottaken = knapsackRec(w, p, i − 1, c)
    int taken = knapsackRec(w, p, i − 1, c − w[i]) + p[i]
    return max(nottaken, taken)
```

Qual è la complessità della funzione knapsack() ricorsiva?

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n-1)+1 & n > 1 \end{cases}$$

$$T(n) = O(2^n)$$

Zaino con memoization

Riprendiamo il problema dello zaino e analizziamo la tabella che viene costruita con un esempio di esecuzione:

```
w = [ 4, 2, 3, 4]
p = [10, 7, 8, 6]        C = 9
```

	c									
i	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	0	**0**
1	**0**	0	**0**	**0**	**10**	**10**	**10**	**10**	10	**10**
2	0	0	**7**	7	10	**10**	**17**	17	17	**17**
3	0	0	7	8	10	**15**	17	18	18	**25**
4	0	0	7	8	10	15	17	18	18	**25**

Notiamo che **non tutti gli elementi della matrice sono necessari** alla risoluzione del nostro problema. Infatti vengono riempite (e calcolate) un sacco di caselle "inutili" per la soluzione finale.
Questo succede perché la programmazione dinamica essendo bottom-up risolve obbligatoriamente tutti i problemi, anche quelli “inutili”.

Vediamo quindi la tecnica di **memoization** (annotazione), la quale fonde l’approccio di memorizzazione della programmazione dinamica con l’approccio top-down di divide-et-impera.
Creiamo quindi una tabella DP, inizializzata con un **valore speciale** (ad esempio -1), il quale indica che il sottoproblema di quella casella non è ancora stato risolto.
Quando dobbiamo risolvere un sottoproblema controlliamo nella tabella se è già stato risolto:

- Se **si**, si usa il risultato della tabella
- Se **no**, si risolve il sottoproblema, si calcola il risultato e lo si memorizza

In questa maniera, ogni sottoproblema viene calcolato una sola volta e viene memorizzato come nella versione bottom-up.

```
int knapsack(int[] w, int[] p, int n, int C)
DP = new int[1...n][1...C]
for i = 1 to n do
    for c = 1 to C do
        DP[i][c] = -1
return knapsackRec(w, p, n, C, DP)
```

```
int knapsackRec(int[] w, int[] p, int i, int c, int[][] DP)
if c < 0 then
    return -∞
else if i == 0 or c == 0 then
    return 0
else
    if DP[i][c] < 0 then
        int nottaken = knapsackRec(w, p, i - 1, c, DP)
        int taken = knapsackRec(w, p, i - 1, c - w[i], DP) + p[i]
        DP[i][c] = max(nottaken, taken)
    return DP[i][c]
```

Ecco un esempio pratico della tabella che viene creata ora:

```
w = [ 4, 2, 3, 4]
p = [10, 7, 8, 6]
C = 9
```

	c									
i	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0										
1		-1	**0**	**0**	**10**	**10**	**10**	**10**	-1	**10**
2		-1	**7**	-1	-1	**10**	**17**	-1	-1	**17**
3		-1	-1	-1	-1	**15**	-1	-1	-1	**25**
4		-1	-1	-1	-1	-1	-1	-1	-1	**25**

Si potrebbe obiettare dicendo che il costo computazionale resta comunque $O(nC)$ visto che è il costo dell'inizializzazione della tabella, infatti applicata in questo modo non c'è alcun vantaggio nell'utilizzare la tecnica di memoization, ma permette tuttavia di tradurre in fretta le espressioni ricorsive.
Un vero vantaggio in termini di costi si ottiene usando un dizionario (hash table) al posto della tabella, così che non sia necessario eseguire l'inizializzazione.
Il costo di esecuzione è $O(min(2^n,\ nC))$, quindi "becchiamo" automaticamente la complessità migliore tra le due per ogni istanza del problema.

Anche in questo caso otteniamo il valore numerico della soluzione, ma non la soluzione vera e propria (ovvero quali oggetti prendere). Si può ricostruire la soluzione dalla tabella nella stessa maniera con cui è stato fatto in Hateville.

Per esercizio

```
w = [ 4, 2, 3, 4]
p = [10, 7, 8, 6]
C = 9
```

	c									
i	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0										
1		-1	**0**	**0**	**10**	**10**	**10**	**10**	-1	**10**
2		-1	**7**	-1	-1	**10**	**17**	-1	-1	**17**
3		-1	-1	-1	-1	**15**	-1	-1	-1	**25**
4		-1	-1	-1	-1	-1	-1	-1	-1	**25**

Variante dello zaino, senza limiti

Questo problema è equivalente a quello dello zaino solo che non ci sono i limiti di scelta, ovvero uno stesso oggetto può essere preso più volte.

Dato uno zaino di capacità C e n oggetti caratterizzati da peso w e profitto p, definiamo $DP[i][c]$ come il massimo profitto che può essere ottenuto dai primi $i \leq n$ oggetti contenuti in uno zaino di capacità $c \leq C$, **senza porre limiti al numero di volte che un oggetto può essere selezionato**.

Nella formula ricorsiva semplicemente eliminiamo il -1 che ci faceva escludere quell'oggetto quando lo sceglievamo:

$$DP[i][c] = \begin{cases} 0 & i = 0 \text{ or } c = 0 \\ -\infty & c < 0 \\ \max(DP[i \not\!/ 1][c - w[i]] + p[i], DP[i-1][c]) & \text{otherwise} \end{cases}$$

Essendo però che non ci interessano più gli indici (non dobbiamo più tener conto di che oggetti abbiamo preso visto che possiamo sempre prenderli tutti), possiamo trattare un vettore invece di una matrice (e vedere un approccio diverso), riducendo lo spazio occupato.

Ridefiniamo quindi la nostra tabella:

Dato uno zaino senza limiti di scelta di capacità C e n oggetti caratterizzati da peso w e profitto p, definiamo $DP[c]$ come il massimo profitto che può essere ottenuto da tali oggetti in uno zaino di capacità $c \leq C$. E otteniamo la seguente formula ricorsiva:

$$DP[c] = \begin{cases} 0 & c = 0 \\ \max_{w[i] \leq c}\{DP[c - w[i]] + p[i]\} & c > 0 \end{cases}$$

Vediamo l'implementazione tramite memoization:

```
int knapsack(int[] w, int[] p, int n, int C)
int[] DP = new int[0...C]
for i = 0 to C do
    DP[i] = -1
knapsackRec(w, p, n, C, DP)
return DP[C]
```

```
int knapsackRec(int[] w, int[] p, int n, int c, int[] DP)
if c == 0 then
    return 0
if DP[c] < 0 then
    int maxSoFar = 0
    for i = 1 to n do
        if w[i] ≤ c then
            int val = knapsackRec(w, p, n, c - w[i], DP) + p[i])
            maxSoFar = max(maxSoFar, val)
    DP[c] = maxSoFar
return DP[c]
```

La complessità della funzione $knapsack()$ è $O(nC)$ perchè dentro la ricorsione per ogni capacità $c \leq C$ dobbiamo scorrere al massimo 1 volta tutti gli oggetti.
Infatti nel caso pessimo dobbiamo riempire tutti i C elementi del vettore DP, e ogni elemento costa $O(n)$.

I vantaggi di questa soluzione sono che:

- La complessità in spazio è pari a $\Theta(C)$
- Non è detto che tutti gli elementi debbano essere riempiti

Gli svantaggi sono che questo approccio rende più difficile la ricostruzione della soluzione:

- Possiamo ispezionare tutti gli elementi per capire da dove deriva il massimo
- Conviene tuttavia memorizzare l'indice da cui deriva il massimo

Vediamo un'implementazione dove memorizziamo l'indice da cui deriva il massimo così da poter ricostruire la soluzione:

```
int knapsack(int[] w, int[] p, int n, int C)
int[] DP = new int[0...C]
int[] pos = new int[0...C]
for i = 0 to C do
    DP[i] = −1
    pos[i] = −1
knapsackRec(w, p, n, C, DP, pos)
return solution(w, C, pos)
```

```
int knapsackRec(int[] w, int[] p, int n, int c, int[] DP, int[] pos)
if c == 0 then
    return 0
if DP[c] < 0 then
    DP[c] = 0
    for i = 1 to n do
        if w[i] ≤ c then
            int val = knapsackRec(w, p, n, c − w[i], DP, pos) + p[i])
            if val ≥ DP[c] then
                DP[c] = val
                pos[c] = i
return DP[c]
```

La funzione che ritorna la soluzione è la seguente:

```
LIST solution(int[] w, int c, int[] pos)
if c == 0 or pos[c] < 0 then
    return List()
else
    LIST L = solution(w, c − w[pos[c]], pos)
    L.insert(L.head(), pos[c])
    return L
```

Questa funzione restituisce una lista di indici selezionati (multinsieme visto che gli indici possono comparire più volte).
Se $c=0$ lo zaino è stato riempito perfettamente invece se $pos[c]<0$ allora lo zaino non può essere riempito interamente (ad esempio ci sono solo pesi pari con una capacità dispari).

Sottosequenza comune massimale

Abbiamo una stringa di molecole chiamate basi (**A**denina, **C**itosina, **G**uanina, **T**imina).
Il problema è, date due sequenze di DNA, trovare quanto queste siano “simili”.

Una sequenza P è una **sottosequenza** di T se P è ottenuto da T rimuovendo uno o più dei suoi elementi. Alternativamente P è definito come l’insieme degli indici $[1,\dots,n]$ degli elementi di T che compaiono anche in P. I rimanenti elementi sono elencati nello **stesso ordine**, senza essere necessariamente contigui.

Esempi

- P = "AAATA"
- T = "AAAATTGA"

Note

La sequenza vuota $\emptyset$ è sottosequenza di ogni altra sequenza

Una sequenza X è una **sottosequenza comune** (common subsequence) di due sequenze T, U, se è sottosequenza sia di T che di U. In questo caso scriviamo $X \in CS(T,U)$

Una sequenza $X \in CS(T,U)$ è una **sottosequenza comune massimale** (longest common subsequence) di due sequenze T, U se non esiste nessun altra sottosequenza comune $Y \in CS(T,U)$ tale che Y sia più lunga di X, ovvero $|Y|>|X|$. In questo caso scriviamo $X \in LCS(T,U)$

Il problema quindi è, date due sequenze T e U, quello di trovare la LCS tra T e U.

Una soluzione ovviamente è quella di forza bruta, ovvero provare tutte le possibili sottosequenze:

```
int LCS(Item[] T, Item[] U)
Item[] maxsofar = nil
foreach subsequence X of T do
    if X is subsequence of U then
        if len(X) > len(maxsofar) then
            maxsofar = X
return maxsofar
```

Visto che, data una sequenza T lunga n, ci sono 2^n possibili sottosequenze e visto che controllare se una sequenza è sottosequenza di un altra costa $O(m+n)$, la complessità computazionale di questa funzione è $O(2^n(m+n))$.

Per trovare una soluzione migliore, definiamo il concetto di **prefisso**:
Data una sequenza T composta dai caratteri $t_1 t_2 \dots t_n$, $T(i)$ denota il prefisso di T dato dai primi i caratteri, quindi $T(i) = t_1 t_2 \dots t_i$

Esempi

- $T =$ "ABDCCAABD"
- $T(0) =$ ""
- $T(3) =$ "ABD"
- $T(6) =$ "ABDCCA"

Ora, date due sequenze T e U di lunghezza n e m, scriviamo una formula ricorsiva $LCS(T(i), U(j))$ che restituisca la LCS dei prefissi $T(i)$ e $U(j)$.

Il caso base ce l'abbiamo quando uno dei due prefissi è vuoto, quindi quando $i = 0$ oppure $j = 0$, in questo caso abbiamo che la LCS è l'insieme vuoto.
Se invece nessuno dei due prefissi è vuoto possono verificarsi due casi:

- Gli ultimi caratteri di $T(i)$ e $U(j)$ coincidono, ovvero $t_i = u_j$
  In questo caso possiamo dire con certezza che l'ultimo carattere di entrambe le sequenze farà parte della LCS di $T(i)$ e $U(j)$.
  Di conseguenza la LCS di $T(i)$ e $U(j)$ è uguale alla LCS di $T(i-1)$ e $U(j-1)$ più questo ultimo carattere.

$$LCS(T(i), U(j)) = LCS(T(i-1), U(j-1)) \oplus t_i$$

- Gli ultimi caratteri di $T(i)$ e $U(j)$ non coincidono, ovvero $t_i \neq u_j$
  In questo caso ci sono due sottocasi: o l'ultimo carattere di $T(i)$ non fa parte della LCS oppure l'ultimo carattere di $U(j)$ non fa parte della LCS:
  $LCS(T(i), U(j)) = longest(LCS(T(i-1), U(j)), LCS(T(i), U(j-1))$

La formula completa quindi è

$$LCS(T(i), U(j)) = \begin{cases} \emptyset & i = 0 \textbf{ or } j = 0 \\ LCS(T(i-1), U(j-1)) \oplus t_i & i > 0 \textbf{ and } j > 0 \textbf{ and } t_i = u_j \\ longest(LCS(T(i-1), U(j)), \\ \quad LCS(T(i), U(j-1)) & i > 0 \textbf{ and } j > 0 \textbf{ and } t_i \neq u_j \end{cases}$$

Vediamo la dimostrazione di questa formula:

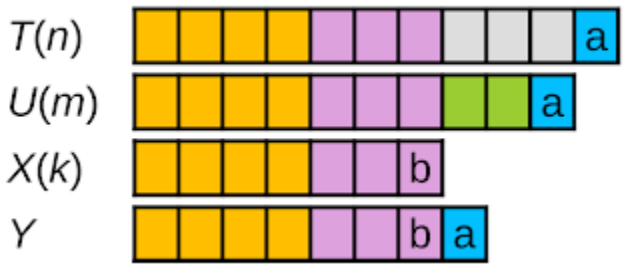
Teorema – Sottostruttura ottima

Date le due sequenze $T = (t_1, \ldots, t_n)$ e $U = (u_1, \ldots, u_m)$, sia $X = (x_1, \ldots, x_k)$ una LCS di T e U. Sono dati tre casi:

1. $t_n = u_m \Rightarrow x_k = t_n = u_m$ **and** $X(k-1) \in \mathcal{LCS}(T(n-1), U(m-1))$
2. $t_n \neq u_m \wedge x_k \neq t_n \Rightarrow X \in \mathcal{LCS}(T(n-1), U)$
3. $t_n \neq u_m \wedge x_k \neq u_m \Rightarrow X \in \mathcal{LCS}(T, U(m-1))$

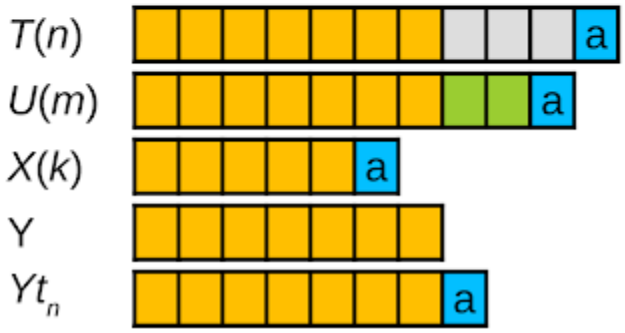
$t_n = u_m \Rightarrow x_k = t_n = u_m$

- Per assurdo: $x_k \neq t_n = u_m$.
- Si consideri $Y = X t_n$.
- Allora $Y \in \mathcal{CS}(T, U)$ e $|Y| > |X|$, contraddizione.



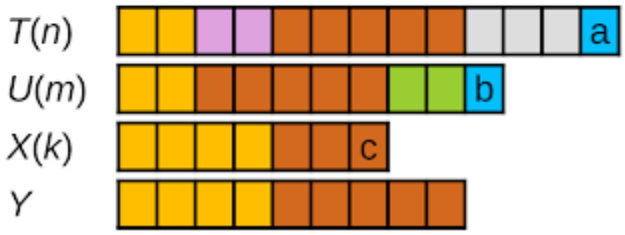
$t_n = u_m \Rightarrow X(k-1) \in LCS(T(n-1), U(m-1))$

- Per assurdo: $X(k-1) \notin \mathcal{LCS}(T(n-1), U(m-1))$.
- Allora $\exists Y \in \mathcal{LCS}(T(n-1), U(m-1))$ tale che $|Y| > |X(k-1)|$.
- Quindi $Yt_n \in \mathcal{CS}(T, U)$ e $|Yt_n| > |X(k-1)t_n| = X$, contraddizione.



$t_n \neq u_m \wedge x_k \neq t_n \Rightarrow X \in \mathcal{LCS}(T(n-1), U)$

- Per assurdo: $X \notin \mathcal{LCS}(T(n-1), U)$.
- Allora $\exists Y \in \mathcal{LCS}(T(n-1), U)$ tale che $|Y| > |X|$.
- Quindi è anche vero che $Y \in \mathcal{LCS}(T, U)$.
- Quindi X non è una LCS di T e U, assurdo.



Ora ricaviamo la formula di ricorrenza partendo da quella formula basandosi sulle lunghezza della LCS.
Date due sequenze T e U di lunghezza n e m, scrivere una formula ricorsiva $DP[i][j]$ che restituisca la **lunghezza** della LCS dei prefissi $T(i)$ e $U(j)$.

$$DP[i][j] = \begin{cases} 0 & i = 0 \textbf{ or } j = 0 \\ DP[i-1][j-1] + 1 & i > 0 \textbf{ and } j > 0 \textbf{ and } t_i = u_j \\ \max\{DP[i-1][j], DP[i][j-1]\} & i > 0 \textbf{ and } j > 0 \textbf{ and } t_i \neq u_j \end{cases}$$

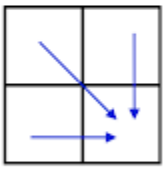
$DP[n][m]$ conterrà la lunghezza della LCS del problema originale.
Ecco l'implementazione per il calcolo della lunghezza della LCS:

```
int lcs(ITEM[] T, ITEM[] U, int n, int m)
int[][] DP = new int[0...n][0...m]
for i = 0 to n do
    DP[i][0] = 0
for j = 0 to m do
    DP[0][j] = 0
for i = 1 to n do
    for j = 1 to m do
        if T[i] == U[j] then
            DP[i][j] = DP[i-1][j-1] + 1
        else
            DP[i][j] = max(DP[i-1][j], DP[i][j-1])
return DP[n][m]
```

Ecco un esempio:

- **TACCBT**
- **ATBCBD**

↘ deriva da *i*-1,*j*-1
↓ deriva da *i*-1,*j*
→ deriva da *i*,*j*-1



i \ j		0	1	2	3	4	5	6
			A	T	B	C	B	D
0		0	0	0	0	0	0	0
1	T	0	↓0	↘ 1	→1	→1	→1	→1
2	A	0	↘ 1	↓ 1	↓ 1	↓ 1	↓ 1	↓ 1
3	C	0	↓ 1	↓ 1	↓ 1	↘ 2	→2	→2
4	C	0	↓ 1	↓ 1	↓ 1	↘ 2	↓ 2	→2
5	B	0	↓ 1	↓ 1	↘ 2	↓ 2	↘ 3	→3
6	T	0	↓ 1	↘ 2	↓ 2	↓ 2	↓ 3	↓ 3

$$DP[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ DP[i-1][j-1] + 1 & i > 0 \text{ and } j > 0 \text{ and } t_i = u_j \\ \max\{DP[i-1][j], DP[i][j-1]\} & i > 0 \text{ and } j > 0 \text{ and } t_i \neq u_j \end{cases}$$

Potenzialmente si potrebbe applicare memoization anche qui, però è meno efficace.
Per ricostruire la soluzione, parto dalla casella in basso a destra e guardo se le lettere della casella sono uguali, se si mi muovo in diagonale, altrimenti mi muovo verso il max tra la casella in verticale e quella in orizzontale.
Però come si vede dalla figura (le freccette), possiamo aiutare la ricostruzione della soluzione costruendo il percorso già durante il ciclo della programmazione dinamica. In questa situazione non ha molto senso perché dobbiamo fare il confronto tra sole due caselle, quindi il costo è comunque costante.
Ecco l'implementazione di come ricostruire la LCS:

```
int lcs(Item[] T, Item[] U, int n, int m)
...
return subsequence(DP, T, U, n, m)
```

```
List subsequence(int[][] DP, Item[] T, Item[] U, int i, int j)
if i == 0 or j == 0 then
    return List()
if T[i] == U[j] then
    S = subsequence(DP, T, U, i − 1, j − 1)
    S.insert(S.head(), T[i])
    return S
else
    if DP[i − 1][j] > DP[i][j − 1] then
        return subsequence(DP, T, U, i − 1, j)
    else
        return subsequence(DP, T, U, i, j − 1)
```

La complessità di *subsequence* è $O(m+n)$ (nel caso pessimo si continua a fare in maniera alternata su e sinistra)
La complessità di LCS è $O(mn)$

Nel caso in cui si voglia sapere solo la lunghezza della LCS (quindi non serva ricostruire la soluzione) possiamo migliorare la complessità in spazio, conservando solamente la riga precedente.

```
int lcs(Item[] T, Item[] U, int n, int m)
int[][] DP' = new int[0...m]
int[][] DP = new int[0...m]
for j = 0 to m do
    DP[j] = 0
for i = 1 to n do
    DP', DP = DP, DP'
    DP[0] = 0
    for j = 1 to m do
        if T[i] == U[j] then
            DP[j] = DP'[j − 1] + 1
        else
            DP[j] = max(DP'[j], DP[j − 1])
return DP[m]
```

Facciamo alcuni **commenti finali**:

- Non sempre è necessario memorizzare informazioni aggiuntive per ricostruire la soluzione
- Se non è necessario ricostruire la soluzione, è possibile risparmiare spazio conservando solo i dati ancora utili

Un'applicazione pratica di questo algoritmo è nel comando Unix *diff*, che esamina due file di testo e ne evidenzia le differenze a livello di riga, che significa che i confronti fra simboli sono in realtà confronti fra righe, dunque n e m sono il numero di righe del file.
Per distinguere le righe si utilizzano funzioni hash.

```
Questo è il testo originale
alcune linee non dovrebbero
cambiare mai
altre invece vengono
rimosse
altre vengono aggiunte
```

```
Questo è il testo nuovo
alcune linee non dovrebbero
cambiare mai
altre invece vengono
cancellate
altre vengono aggiunte
come questa
```

```
- Questo è il testo originale
+ Questo è il testo nuovo
  alcune linee non dovrebbero
  cambiare mai
  altre invece vengono
- rimosse
+ cancellate
  altre vengono aggiunte
+ come questa
```

Figura 13.4: Il file `original.txt` (a sinistra); il file `new.txt` (al centro); l'output di `diff original.txt new.txt` (a destra).

String matching approssimato

Nei motori di ricerca e in generale nelle applicazioni che si occupano di ricerca di testo, può capitare di dover risolvere il seguente problema.

Siano:

- $P = p_1 \dots p_m$ una stringa detta **pattern**
- $T = t_1 \dots t_n$ una stringa detta **testo**, con $n \geq m$

Un'**occorrenza k-approssimata** di P in T è una copia di P in T in cui sono ammessi k "errori" tra P e T. Gli errori sono del seguente tipo:

1. I corrispondenti caratteri in P e T sono diversi (**sostituzione**)
2. Un carattere in P non è incluso in T (**inserimento**)
3. Un carattere in T non è incluso in P (**cancellazione**)

Esempio

```
T = questoèunoscempio
P =        unesempio
```

Il problema consiste nel trovare un'occorrenza k-approssimata di P in T per cui k sia minimo ($0 \leq k \leq m$).
In sintesi dobbiamo:

- Trovare quale è il valore minimo di k per cui si trova un'occorrenza k-approssimata di P in T

- Dire dove si trova questa sequenza
- Dire con quali errori

La differenza con il problema della sottosequenza comune massimale è che lì le due stringhe avevano lo stesso ruolo, invece qui è ben definita una differenza.

Definiamo quindi la nostra tabella di programmazione dinamica.
Sia $DP[0\dots m][0\dots n]$ una tabella di programmazione dinamica tale che $DP[i][j]$ sia il minimo valore k per cui esiste un'occorrenza k-approssimata di $P(i)$ in $T(j)$ che termina nella posizione j di T.
Quando vogliamo definire il valore di $DP[i][j]$ abbiamo quattro possibilità:

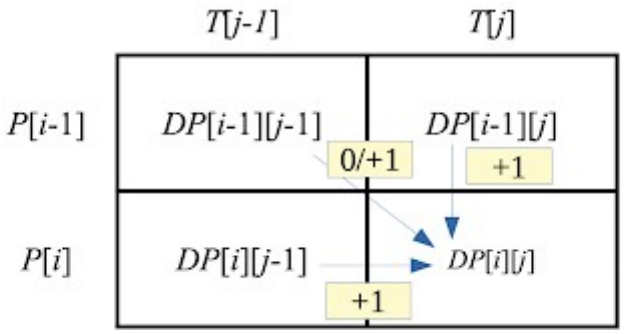
- Vale $DP[i-1][j-1]$ se $P[i]=T[j]$
  In questo caso avanziamo su entrambe le stringhe (**non c'è nessun errore**)
- Vale $DP[i-1][j-1]+1$ se $P[i]\neq T[j]$
  In questo caso avanziamo su entrambe le stringhe (**sostituzione**)
- Vale $DP[i-1][j]+1$
  In questo caso avanziamo solo sul pattern (**inserimento**)
- Vale $DP[i][j-1]+1$
  In questo caso avanziamo solo sul testo (**cancellazione**)

Dopo abbiamo i casi base:

- Se il pattern è vuoto (ovvero $i=0$ quindi $DP[0][j]$) il numero di errori è 0, perchè la stringa vuota occorre in ogni posizione di T
- Se il testo è vuoto (ovvero $j=0$ quindi $DP[i][0]$) il numero di errori è i (la lunghezza del pattern) perchè questi caratteri devono tutti essere cancellati, oppure inseriti nel testo

Mettendo tutto insieme otteniamo:

$$DP[i][j] = \begin{cases} 0 & i = 0 \\ i & j = 0 \\ \min\{DP[i-1][j-1] + \delta, & \delta = \text{iif}(P[i] = T[j], 0, 1) \\ \quad DP[i-1][j] + 1, & \\ \quad DP[i][j-1] + 1\} & \text{altrimenti} \end{cases}$$



Si noti che $DP[m][j]=k$ se e solo se esiste un'occorrenza k-approssimata di P in T che termina nella posizione j di T, di conseguenza la soluzione del problema è data dal più piccolo valore $DP[m][j]$ con $0 \le j \le n$.

Osservazione

Notiamo quindi che non è sempre detto che quando si utilizza programmazione dinamica la soluzione si trovi sempre nell'ultima casella della matrice (in basso a destra), ma questo dipende dal problema e soprattutto da come definiamo la matrice, infatti in questo caso la soluzione deve essere cercata all'interno della tabella DP.

Ecco un esempio della matrice dove vediamo in particolare due casi specifici di come ottenere il valore di una casella della matrice:

P \ T		A	B	A	B	A
	0	0	0	0	0	0
B	1	1	0	1	0	1
A	2	1	1	0	1	0
B	3	2	1	1	0	1

Il codice è il seguente:

```
int stringMatching(ITEM[] P, ITEM[] T, int m, int n)
int[][] DP = new int[0...m][0...n]
for j = 0 to n do DP[0][j] = 0                    % Caso base: i = 0
for i = 1 to m do DP[i][0] = i                    % Caso base: j = 0
for i = 1 to m do                                 % Caso generale
    for j = 1 to n do
        DP[i][j] = min(DP[i − 1][j − 1] + iif(P[i] == T[j], 0, 1),
                       DP[i − 1][j] + 1,
                       DP[i][j − 1] + 1)
int pos = 0                                       % Calcola minimo ultima riga
for j = 1 to n do
    if DP[m][j] < DP[m][pos] then
        pos = j
return pos
```

La complessità di questo algoritmo è $O(n \cdot m)$ poiché sono valutati esplicitamente tutti gli elementi di DP.

Per ricostruire la soluzione basta analizzare la tabella e vedere da quale delle tre caselle derivi la soluzione finale (come sempre in caso di “parità” significa che ci sono più soluzioni ottime, quindi diventa indifferente il percorso che si sceglie).

Prodotto di catena di matrici

Data una sequenza di *n* matrici $A_1,\ldots,A_n$ compatibili due a due al prodotto (ovvero il numero di colonne di A_i è uguale al numero di righe di A_{i+1}), vogliamo calcolare il loro prodotto.
Ricordiamo che il prodotto tra matrici non è commutativo (quindi non possiamo scambiare due matrici nell’ordine) però è associativo (quindi possiamo cambiare l’ordine di applicazione dell’operazione), quindi $\left(A_1 \cdot A_2\right) \cdot A_3 = A_1 \cdot \left(A_2 \cdot A_3\right)$.
Moltiplicare due matrici A_i di dimensione $c_{i-1} \times c_i$ e A_{i+1} di dimensione $c_i \times c_{i+1}$ produce una matrice di dimensione $c_{i-1} \times c_{i+1}$ e richiede $c_{i-1} \cdot c_i \cdot c_{i+1}$ moltiplicazioni scalari, in quanto ogni elemento della matrice risultante viene calcolato dalla formula seguente:

$$\sum_{k=1}^{c_i} A_i[h,k] \cdot A_{i-1}[k,j] \text{ con } 1 \le h \le c_{i-1} \text{ e } 1 \le j \le c_{i+1}$$

Quindi il prodotto tra matrici si basa sulla **moltiplicazione scalare** come operazione elementare, e il numero di moltiplicazioni scalari necessarie per moltiplicare *n* matrici dipende dall’ordine in cui vengono moltiplicate le matrici, ecco qualche esempio:

A	100×1
B	1×100
C	100×1

	# Moltiplicazioni	Memoria
$(A \cdot B)$	$100 \times 1 \times 100 = 10000$	10000
$((A \cdot B) \cdot C)$	$100 \times 100 \times 1 = 10000$	100
	20000	10100
$(B \cdot C)$	$1 \times 100 \times 1 = 100$	1
$(A \cdot (B \cdot C))$	$100 \times 1 \times 1 = 100$	100
	200	101

A	50×10
B	10×40
C	40×30
D	30×5

$(((A \cdot B) \cdot C) \cdot D)$: 87500 moltiplicazioni
$((A \cdot (B \cdot C)) \cdot D)$: 34500 moltiplicazioni
$((A \cdot B) \cdot (C \cdot D))$: 36000 moltiplicazioni
$(A \cdot ((B \cdot C) \cdot D))$: 16000 moltiplicazioni
$(A \cdot (B \cdot (C \cdot D)))$: 10500 moltiplicazioni

$(((A \cdot B) \cdot C) \cdot D)$: 87500

$(A \cdot B)$	$50 \times 10 \times 40 = 20000$
$((A \cdot B) \cdot C)$	$50 \times 40 \times 30 = 60000$
$((A \cdot B) \cdot C) \cdot D$	$50 \times 30 \times 5 = 7500$
	87500

Quindi vogliamo calcolare il prodotto delle n matrici impiegando il più basso numero possibile di moltiplicazioni scalari.
Caratterizziamo la soluzione nel modo seguente: una **parentesizzazione** $P_{i,j}$ del prodotto $A_i \cdot A_{i+1} \cdot \ldots \cdot A_j$ consiste:

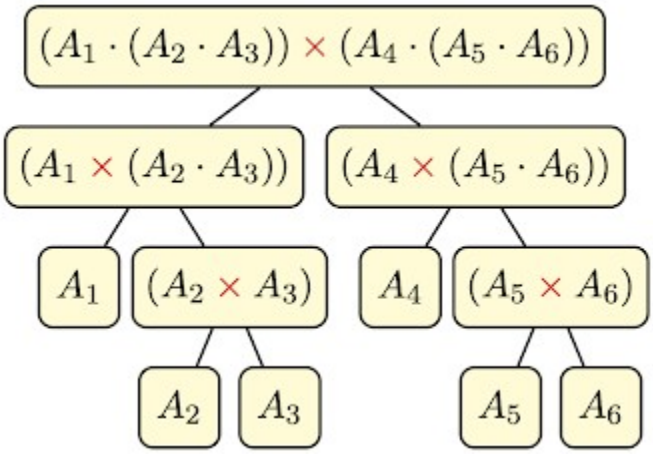
- Nella matrice A_i se $i = j$
- Nel prodotto di due parentesizzazioni $\left(P_{i,k} \cdot P_{k+1,j}\right)$ altrimenti

Ecco un esempio:

Esempio

$(A_1 \cdot (A_2 \cdot A_3)) \times (A_4 \cdot (A_5 \cdot A_6))$

In questo caso, $k = 3$ e il prodotto evidenziato è detto "ultimo prodotto"



Una parentesizzazione $P_{i,j}$ si dice ottima se non esiste altra parentesizzazione $P'_{i,j}$ con costo inferiore, quindi la parentesizzazione ottima è quella che richiede il minor numero di moltiplicazioni scalari per essere completata, fra tutte le parentesizzazioni possibili.
Si può dimostrare che data una parentesizzazione ottima $P_{i,j} = \left(P_{i,k} \cdot P_{k+1,j}\right)$, le parentesizzazioni $P_{i,k}$ e $P_{k+1,j}$ sono anch'esse ottime (lo vediamo dopo)

La motivazione di questo procedimento è che vale la pena in termini di tempo preprocessare i dati per cercare la parentesizzazione ottima, per risparmiare tempo dopo nel calcolo vero e proprio.

Abbiamo caratterizzato la nostra soluzione in maniera ricorsiva, e questo ci permetterebbe di scrivere una soluzione basata su divide-et-impera; ma in questo caso molti problemi verrebbero ripetuti inutilmente.
Definiamo con $P(n)$ il numero di parentesizzazioni possibili con n matrici $A_1 \cdot \ldots \cdot A_n$.
Notiamo che l'ultimo prodotto può occorrere in $n-1$ posizioni diverse, e una volta fissato l'indice k dell'ultimo prodotto, abbiamo:

- $P(k)$ parentesizzazioni per $A_1 \cdot \ldots \cdot A_k$
- $P(n-k)$ parentesizzazioni per $A_{k+1} \cdot \ldots \cdot A_n$

Nel caso di una singola matrice invece esiste un solo modo per esprimerla in parentesi
Il numero $P(n)$ di possibili parentesizzazioni è quindi definibile ricorsivamente nel modo seguente:

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n > 1 \end{cases}$$

n	1	2	3	4	5	6	7	8	9	10
$P(n)$	1	1	2	5	14	42	132	429	1430	4862

Numero di Catalan

$$P(n) = C(n) = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!} = \Theta\left(\frac{4^n}{n\sqrt{n}}\right)$$

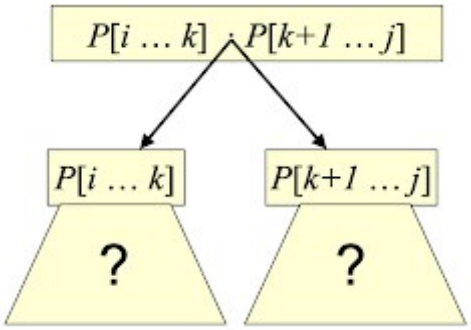
$P(n)$ è l'$(n-1)$-esimo numero di Catalan, quindi $P(n) = \Theta\left(\frac{4^n}{n\sqrt{\square}}\right)$, quindi un algoritmo ricorsivo basato su divide et impera è superpolinomiale, e quindi non va bene (è un algoritmo a forza bruta).

Prima di passare al prossimo approccio vediamo alcune definizione matematiche:

- $A_1 \cdot A_2 \cdot \ldots \cdot A_n$: il prodotto di n matrici da ottimizzare
- c_{i-1}: il numero di righe della matrice A_i (e numero di colonne della matrice A_{i-1})
- c_i: il numero di colonne della matrice A_i (e numero di righe della matrice A_{i+1})
- $A[i \ldots j]$: il sottoprodotto $A_i \cdot A_{i+1} \cdot \ldots \cdot A_j$
- $P[i \ldots j]$: una parentesizzazione per $A[i \ldots j]$ (non necessariamente ottima)

Quindi sia $A[i \ldots j]$ una sottosequenza del prodotto di matrici e si consideri una parentesizzazione ottima $P[i \ldots j]$ di $A[i \ldots j]$. Esiste un **ultimo prodotto**, ovvero esiste un indice k tale che

$$P[i \ldots j] = P[i \ldots k] \cdot P[k+1 \ldots j]$$



Teorema

Se $P[i \dots j] = P[i \dots k] \cdot P[k+1 \dots j]$ è una parentesizzazione ottima del prodotto $A[i \dots j]$, allora:

- $P[i \dots k]$ è una parentesizzazione ottima del prodotto $A[i \dots k]$
- $P[k+1 \dots j]$ è una parentesizzazione ottima del prodotto $A[k+1 \dots j]$

La dimostrazione viene fatta per assurdo:

Supponiamo esista una parentesizzazione ottima $P'[i \dots k]$ di $A[i \dots k]$ con costo inferiore a $P[i \dots k]$. Allora $P'[i \dots k] \cdot P[k+1 \dots j]$ sarebbe una parentesizzazione di $A[i \dots j]$ con costo inferiore a $P[i \dots j]$, ed è assurdo.

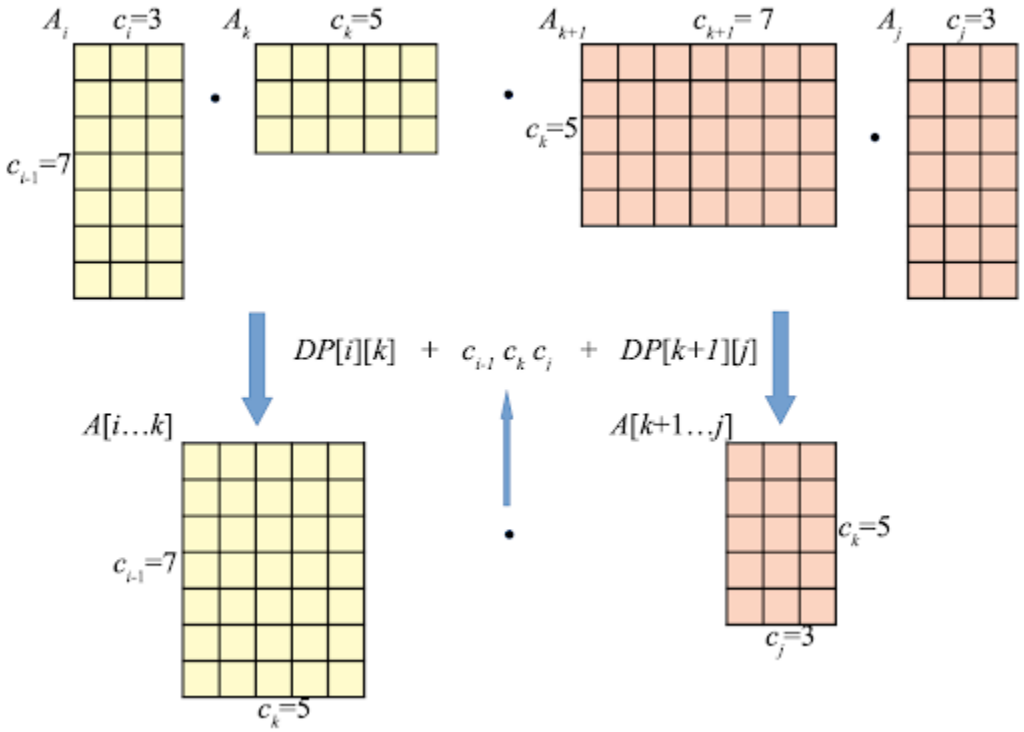
Quindi definiamo la nostra tabella di programmazione dinamica nel seguente modo:

Sia $DP[i][j]$ il numero di moltiplicazioni scalari necessarie per calcolare il prodotto $A[i \dots j]$.

Il caso base è quando abbiamo una sola matrice, ovvero $i = j$, e in questo caso non ci sono moltiplicazioni da fare e quindi $DP[i][j] = 0$.

Il passo ricorsivo invece è quando abbiamo $i < j$. Sappiamo che esiste una parentesizzazione ottima $P[i \dots j] = P[i \dots k] \cdot P[k+1 \dots j]$. Sfruttando la ricorsione possiamo dire che $DP[i][j] = DP[i][k] + DP[k+1][j] + c_{i-1} c_k c_j$.

Questo perché il costo della parentesizzazione ottima $P[i \dots j] = P[i \dots k] \cdot P[k+1 \dots j]$ può essere espresso come il costo delle parentesizzazioni ottime $P[i \dots k]$ e $P[k+1 \dots j]$ più $c_{i-1} c_k c_j$ moltiplicazioni scalari per moltiplicare tra loro le matrici risultanti di dimensione $c_{i-1} \times c_k$ e $c_k \times c_j$.



Il problema è che non sappiamo il valore di k. Visto che non lo conosciamo, l'unico modo per trovarlo è provarli tutti, facendo variare k da i a j - 1, quindi la formula finale diventa:

Formula finale

$$DP[i][j] = \begin{cases} 0 & i = j \\ \min_{i \le k < j}\{DP[i][k] + DP[k+1][j] + c_{i-1} \cdot c_k \cdot c_j\} & i < j \end{cases}$$

Vediamo alcuni esempi grafici per capire meglio il funzionamento di questa formula:

i \ j	1	2	3	4	5	6
1	0					
2	-	0				
3	-	-	0			
4	-	-	-	0		
5	-	-	-	-	0	
6	-	-	-	-	-	0

$$
\begin{aligned}
DP[1][2] &= \min_{1 \le k < 2} \{DP[1][k] + DP[k+1][2] + c_0 c_k c_2\} \\
&= DP[1][1] + DP[2][2] + c_0 c_1 c_2 \\
&= c_0 c_1 c_2
\end{aligned}
$$

i \ j	1	2	3	4	5	6
1	0					
2	-	0				
3	-	-	0			
4	-	-	-	0		
5	-	-	-	-	0	
6	-	-	-	-	-	0

$$
\begin{aligned}
DP[2][4] &= \min_{2 \le k < 4} \{DP[2][k] + DP[k+1][4] + c_1 c_k c_4\} \\
&= \min\{DP[2][2] + DP[3][4] + c_1 c_2 c_4, \\
&\qquad DP[2][3] + DP[4][4] + c_1 c_3 c_4\}
\end{aligned}
$$

i \ j	1	2	3	4	5	6
1	0					
2	-	0			◯	
3	-	-	0			
4	-	-	-	0		
5	-	-	-	-	0	
6	-	-	-	-	-	0

$$
\begin{aligned}
DP[2][5] &= \min_{2 \le k < 5} \{DP[2][k] + DP[k+1][5] + c_1 c_k c_5\} \\
&= \min\{DP[2][2] + DP[3][5] + c_1 c_2 c_5, \\
&\qquad DP[2][3] + DP[4][5] + c_1 c_3 c_5, \\
&\qquad DP[2][4] + DP[5][5] + c_1 c_4 c_5\}
\end{aligned}
$$

i \ j	1	2	3	4	5	6
1	0				◯	
2	-	0				
3	-	-	0			
4	-	-	-	0		
5	-	-	-	-	0	
6	-	-	-	-	-	0

$$
\begin{aligned}
DP[1][5] &= \min_{1 \le k < 5} \{DP[1][k] + DP[k+1][5] + c_0 c_k c_5\} \\
&= \min\{DP[1][1] + DP[2][5] + c_0 c_1 c_5, \\
&\qquad DP[1][2] + DP[3][5] + c_0 c_2 c_5, \\
&\qquad DP[1][3] + DP[4][5] + c_0 c_3 c_5, \\
&\qquad DP[1][4] + DP[5][5] + c_0 c_4 c_5\}
\end{aligned}
$$

i \ j	1	2	3	4	5	6
1	0					
2	-	0				
3	-	-	0			
4	-	-	-	0		
5	-	-	-	-	0	
6	-	-	-	-	-	0

$$
\begin{aligned}
DP[1][6] &= \min_{1 \le k < 6} \{ && DP[1][k] + DP[k+1][6] + c_0 c_k c_6 \} \\
&= \min\{ && DP[1][1] + DP[2][6] + c_0 c_1 c_6, \\
& && DP[1][2] + DP[3][6] + c_0 c_2 c_6, \\
& && DP[1][3] + DP[4][6] + c_0 c_3 c_6, \\
& && DP[1][4] + DP[5][6] + c_0 c_4 c_6, \\
& && DP[1][5] + DP[6][6] + c_0 c_5 c_6 \}
\end{aligned}
$$

Passiamo ora al codice.
Come input prendiamo un vettore $c[0 \ldots n]$ contenente le dimensioni delle matrici:

- $c[0]$ è il numero di righe della prima matrice (A_1)
- $c[i-1]$ è il numero di righe della matrice A_i
- $c[i]$ è il numero di colonne della matrice A_i

Come input prendiamo anche due indici i, j che rappresentano l'intervallo di matrici da moltiplicare.
In output verrà dato il numero di moltiplicazioni scalari per calcolare il prodotto delle matrice comprese tra i e j.
L'approccio ricorsivo è il seguente:

```
int recPar(int[] c, int i, int j)
if i == j then
    return 0
else
    minSoFar = +∞
    for int k = i to j − 1 do
        int val = recPar(c, i, k) + recPar(c, k + 1, j) + c[i − 1] · c[k] · c[j]
        if val < minSoFar then
            minSoFar = minSoFar
    return minSoFar
```

Questa soluzione ricorsiva top-down è $\Omega(2^n)$, quindi non è poi migliore dell'approccio basato su forza bruta, il problema è sempre che molti sottoproblemi vengono risolti più volte. Il numero di sottoproblemi è $\frac{n(n+1)}{2}$

Passiamo quindi alla programmazione dinamica con un approccio bottom-up.
Definiamo due matrici di dimensioni $n \times n$, DP e $last$:

- $DP[i][j]$ contiene il numero di moltiplicazioni scalari necessarie per moltiplicare le matrici $A[i \dots j]$
- $last[i][j]$ contiene il valore k dell'ultimo prodotto che minimizza il costo per il sottoproblema

Notiamo però che non possiamo riempire semplicemente una riga dopo l'altra, questo perché per riempire la casella $DP[i][j]$ abbiamo bisogno di tutte le caselle "sotto" e a "sinistra".
Infatti costruiamo la tabella partendo dalla diagonale principale e riempiendo diagonalmente.

```
computePar(int[] c, int n)
int[][] DP = new int[1...n][1...n]
int[][] last = new int[1...n][1...n]
for i = 1 to n do                                   % Fill main diagonal
    DP[i][i] = 0
for h = 2 to n do                                   % h: diagonal index
    for i = 1 to n − h + 1 do                       % i: row
        int j = i + h − 1                           % j: column
        DP[i][j] = +∞
        for k = i to j − 1 do                       % k: last product
            int temp = DP[i][k] + DP[k + 1][j] + c[i − 1] · c[k] · c[j]
            if temp < DP[i][j] then
                DP[i][j] = temp
                last[i][j] = k
return DP[1][n]
```

Ecco un esempio pratico:

$$
\begin{aligned}
DP[1][4] &= \min_{1 \le k < 4} \{\ DP[1][k] + DP[k+1][4] + c_0 \cdot c_k \cdot c_4\ \} \\
&= \min \{\ DP[1][1] + DP[2][4] + c_0 \cdot c_1 \cdot c_4, \\
&\qquad \{\ DP[1][2] + DP[3][4] + c_0 \cdot c_2 \cdot c_4, \\
&\qquad \{\ DP[1][3] + DP[4][4] + c_0 \cdot c_3 \cdot c_4\ \} \\
&= \min \{\ 0 + 112 + 7 \cdot 8 \cdot 3, \\
&\qquad \{\ 224 + 24 + 7 \cdot 4 \cdot 3, \\
&\qquad \{\ 176 + 0 + 7 \cdot 2 \cdot 3\ \} \\
&= \min \{\ 280 , 332 , 218\ \}
\end{aligned}
$$

DP	1	2	3	4	5	6
1	0	224	176	218	276	350
2		0	64	112	174	250
3			0	24	70	138
4				0	30	90
5					0	90
6						0

i	$c[i]$
0	7
1	8
2	4
3	2
4	3
5	5
6	6

last	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

Il costo computazionale di questo algoritmo è $O(n^3)$, in quanto ogni cella richiede tempo $O(n)$ per essere riempita, e la soluzione si trova nella cella $DP[1][n]$.

La tabella *last* è utile per ricostruire la soluzione trovata in $O(n)$ (altrimenti avremmo dovuto cercare nuovamente il valore k per ogni posizione, portando il costo della ricostruzione a $O(n^2)$).

Ecco la funzione che stampa la soluzione:

computePar(**int**[ ] c, **int** n)

```
[...]
printPar(last, 1, n)
```

printPar(**int**[ ][ ] $last$, **int** i, **int** j)

```
if i == j then
    print "A["; print i; print "]"
else
    print "("; stampaPar(last, i, last[i][j]); print "·";
     stampaPar(last, last[i][j] + 1, j); print ")"
```

Per effettuare il calcolo effettivo dobbiamo usare sempre la tabella *last*:

int[ ][ ] multiply(**matrix**[ ] A, **int**[ ][ ] $last$, **int** i, **int** j)

```
if i == j then
    return A[i]
else
    int[ ][ ] X = multiply(A, last, i, last[i][j])
    int[ ][ ] Y = multiply(A, last, last[i][j] + 1, j)
    return matrix-multiplication(X, Y)
```

Ecco un esempio:

$$A[1\dots6] = A[1\dots3] \cdot A[4\dots6]$$
$$A[1\dots3] = A_1 \cdot A[2\dots3]$$
$$A[4\dots6] = A[4\dots5] \cdot A_6$$
$$A[2\dots3] = A_2 \cdot A_3$$
$$A[4\dots5] = A_4 \cdot A_5$$

last	**1**	**2**	**3**	**4**	**5**	**6**
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

Risultato finale

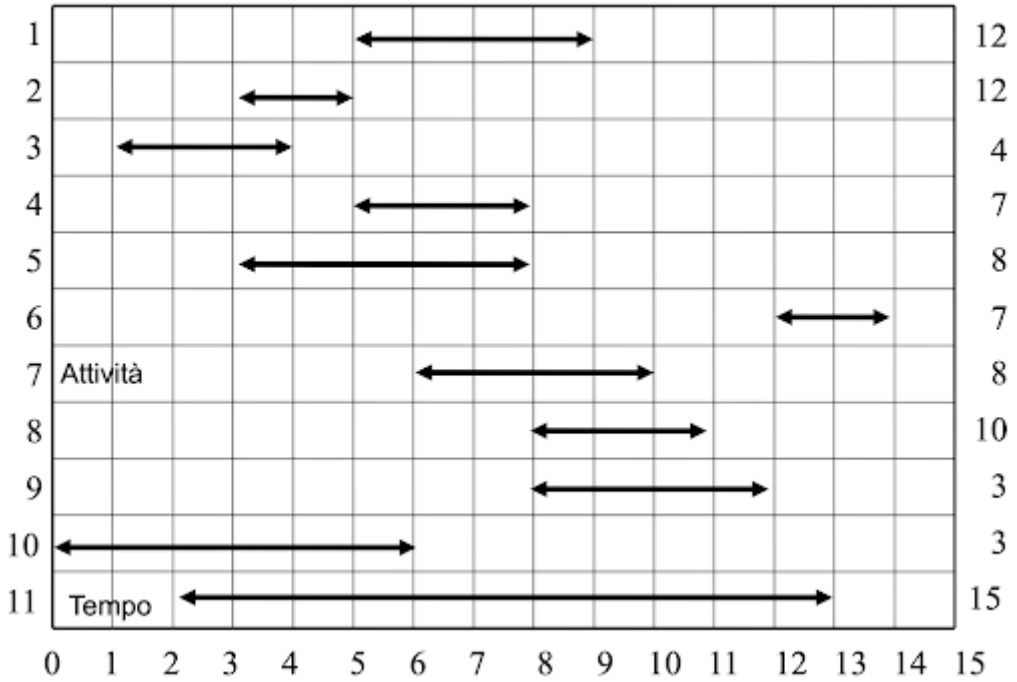
$$A = ((A_1 \cdot (A_2 \cdot A_3)) \cdot ((A_4 \cdot A_5) \cdot A_6))$$

Osservazione:
Questo esempio ci fa capire che bisogna fare attenzione a come si riempie la tabella, non è detto che riempire una riga dopo l'altra sia sempre possibile.

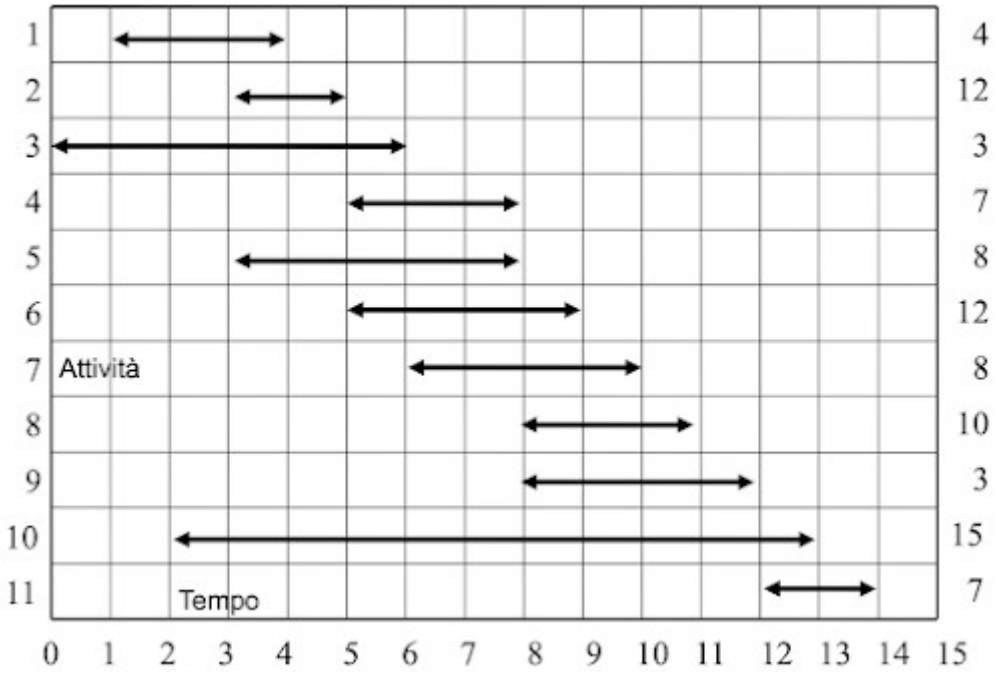
Insieme indipendente di intervalli pesati

Supponete di essere il manager di un grande albergo. La vostra sala riunioni è molto richiesta, e vi trovate a dover decidere come affittarla nel prossimo mese. Avete ricevuto un certo numero di richieste, ognuna caratterizzata da un orario di inizio e un orario di fine, e da un guadagno associato. Il vostro compito è massimizzare il guadagno dell'albergo. È facile vedere che il vostro problema è un caso particolare di questo problema classico:

Siano dati n intervalli distinti $[a_1, b_1[, \ldots, [a_n, b_n[$ della retta reale, aperti a destra, dove all'intervallo i è associato un peso/profitto w_i con $1 \leq i \leq n$.
Trovare un insieme indipendente di peso massimo, ovvero un sottoinsieme di intervalli tutti disgiunti tra di loro tale che la somma dei pesi degli intervalli nel sottoinsieme sia la più grande possibile.
Due intervalli i e j si dicono disgiunti se: $b_j \leq a_i$ oppure $b_i \leq a_j$



Gli intervalli sono aperti a destra in quanto un evento che finisce a mezzogiorno può essere seguito da un evento che inizia alla stessa ora.
Per risolvere questo problema con la programmazione dinamica, dobbiamo svolgere un po' di pre-elaborazione, ordinando gli intervalli per estremi finali non decrescenti $b_1 \leq b_2 \leq \ldots \leq b_n$.



Una prima versione della soluzione potrebbe essere questa:
Definiamo il vettore $DP[i]$ che contiene il profitto massimo ottenibile con i primi i intervalli.
Quando vogliamo calcolare $DP[i]$ possiamo:

- Non prendere in considerazione l'intervallo i, quindi prendere il valore $DP[i-1]$
- Prenderlo in considerazione, quindi provare a vedere se riusciamo ad aggiungerlo in maniera disgiunta ai massimi che abbiamo calcolato prima di lui

Con questo ragionamento otteniamo questo algoritmo:

Profitto massimo (Versione 1)

$DP[i]$ contiene il profitto massimo ottenibile con i primi i intervalli

$$DP[i] = \begin{cases} 0 & i = 0 \\ \max(DP[i-1], \max\{DP[j] + w_i : j < i \wedge b_j \leq a_i\}) & i > 0 \end{cases}$$

Il costo computazionale di questa soluzione è $O(n^2)$, perchè per ogni casella dobbiamo riscorrere tutte quelle prima di lei.

Una seconda possibile pre-elaborazione consiste nel pre-calcolare il predecessore $pred_i = j$ di i, dove $j < i$ è il massimo indice tale che $b_j \leq a_i$. Se non esiste tale indice mettiamo $pred_i = 0$.
In questo caso diventa molto più semplice definire $DP[i]$:

Profitto massimo (Versione 2)

$$DP[i] = \begin{cases} 0 & i = 0 \\ \max(DP[i-1], DP[pred_i] + w_i) & i > 0 \end{cases}$$

La funzione per calcolare i predecessori è la seguente:

```
int[] computePredecessor(int[] a, int[] a, int n)
int[] pred = new int[0...n]
pred[0] = 0
for i = 1 to n do
    j = i − 1
    while j > 0 and b[j] > a[i] do
        j = j − 1
    pred[i] = j
return pred
```

Con questo algoritmo i predecessori si calcolano in $O(n^2)$, e quindi non guadagniamo niente rispetto a prima, però è possibile usare una ricerca dicotomica facendo diventare questo algoritmo $O(n \log n)$, bisogna però prima ordinare gli intervalli.
Eventuali intervalli con lo stesso tempo di fine possono essere ordinati in qualunque modo, questo perché ogni valore $DP[i]$ rappresenta il massimo profitto ottenibile con i primi i intervalli, è quindi possibile escludere l'intervallo i-esimo se sceglierne uno precedente j (ma con lo stesso tempo di fine $b[i]=b[j]$) ha un valore $DP[j]$ più alto

Il costo della soluzione ottima del problema originario si trova nella casella $D[n]$.
La versione completa dell'algoritmo è questa:

```
SET maxinterval(int[] a, int[] b, int[]w, int n)
{ ordina gli intervalli per estremi di fine crescenti }
int[] pred = computePredecessor(a, b, n)
int[] DP = new int[0...n]
DP[0] = 0
for i = 1 to n do
    DP[i] = max(DP[i − 1], w[i] + DP[pred[i]])
i = n
SET S = Set()
while i > 0 do
    if DP[i − 1] > w[i] + DP[pred[i]] then
        i = i − 1
    else
        S.insert(i)
        i = pred[i]
return S
```

In cui abbiamo anche la ricostruzione della soluzione, che costa $O(n)$.

Costo computazionale

- Ordinamento intervalli: $O(n \log n)$
- Calcolo predecessori: $O(n \log n)$
- Riempimento tabella DP: $O(n)$
- Ricostruzione soluzione: $O(n)$
- Algoritmo completo: $O(n \log n)$

Osservazione

Da questo esempio impariamo che talvolta può essere necessario pre-processare l'input per poter applicare nella maniera più efficiente la programmazione dinamica

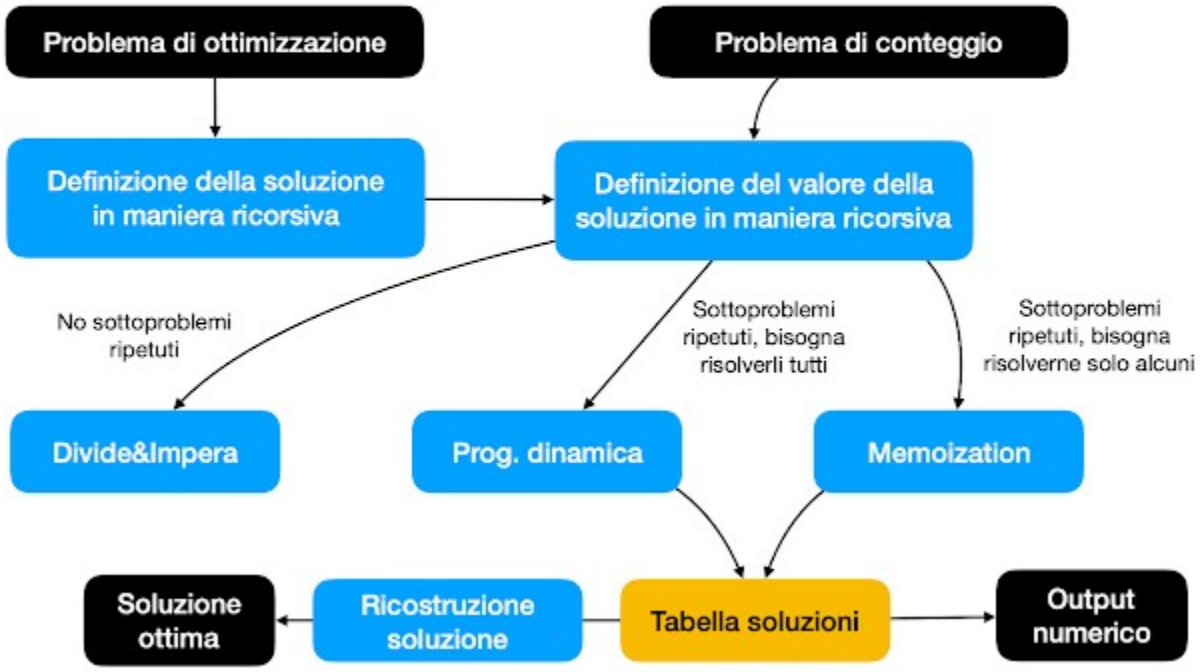
In conclusione:

La programmazione dinamica non è la soluzione di tutti i vostri problemi. Esistono altre tecniche che possono fare "meglio di così". Inoltre, è possibile che soluzioni ad-hoc possano essere migliori.

Esempi

- Longest increasing subsequence: esiste soluzione $O(n \log n)$
- Longest common subsequence: può essere risolto in tempo $O(mn/\log n)$ (con alfabeto limitato)
- Four Russians algorithm: tecnica generale che può essere applicata a vari problemi su matrice con alfabeto limitato $O(n^2/\log n)$
  - Edit distance
  - Transitive closure, ...

Approccio generale



Quindi il riassunto delle fasi delle tecniche di programmazione dinamica e memoization sono:

- **Caratterizzare** la **struttura** di una soluzione ottima
- **Dimostrare** che la soluzione gode di **sottostruttura ottima**
- **Definire** ricorsivamente il **valore** di una soluzione ottima
- **Calcolare** il **valore** di una soluzione ottima "bottom-up" (prog. dinamica) / "top-down" (memoization)
- **Ricostruzione** di una soluzione ottima

Strutture di dati e progettazione di algoritmi

Dato un problema non ci sono “ricette generali” per progettare algoritmi efficienti che lo risolvano, al contrario accade molto spesso di ripartire praticamente da zero ogniqualvolta si debba progettare un nuovo algoritmo.

Nel progettare algoritmi, comunque, possono essere evidenziate quattro “fasi” comuni attraverso le quali occorre usualmente passare:

1. **Classificazione del problema**
   Come prima cosa bisogna stabilire la classe più ampia di problemi di cui fa parte il nostro problema. Solitamente si usa distinguere tre classi di problemi:
   - **Problemi decisionali** (verificare se è soddisfatta una certa proprietà)
   - **Problemi di ricerca** (trovare una soluzione ammissibile)
   - **Problemi di ottimizzazione** (trovare la soluzione ottima)
2. **Caratterizzazione della soluzione**
   Si prova a caratterizzare matematicamente la soluzione del problema. Tale caratterizzazione di solito suggerisce un algoritmo di risoluzione semplice ma non molto efficiente
3. **Tecnica di progettazione**
   Si cerca di applicare certe tecniche di progettazione di algoritmi per rendere gli algoritmi più veloci, ad esempio divide-et-impera, programmazione dinamica, greedy, ricerca locale, backtrack, algoritmi probabilistici
4. **Strutture di dati**
   Si vede se c’è un modo astuto di strutturare i dati elaborati dall’algoritmo che permetta di eseguire velocemente le operazioni utilizzate nell’algoritmo

Di solito si usa questo ordine delle fasi, ma in alcuni casi si possono considerare solo alcune fasi in ordine qualsiasi e in altri si possono riconsiderare più volte certe fasi per raffinare e rendere più efficiente una versione successiva dell’algoritmo.

In questo capitolo vedremo le fasi di caratterizzazione matematica della soluzione e l’utilizzo di strutture dati nella progettazione di algoritmi efficienti.

Cammini minimi

L’input di questo problema consiste in:

- Un **grafo orientato** $G=(V,E)$
- Un **nodo sorgente** s
- Una **funzione di peso** $w : E \to R$

Definiamo il concetto di costo di un cammino: *Dato un cammino* $p=<v_1,v_2,\ldots,v_k>$ *con* $k>1$*, il* ***costo del cammino*** *è dato da* $w(p)=\sum_{i=2}^{k} w(v_{i-1},v_i)$

Il problema chiede di dare in output un cammino da s a u, per ogni nodo $u \in V$, il cui costo sia minimo, ovvero più piccolo o uguale del costo di qualunque altro cammino da s a u

Cammini minimi da singola sorgente

- Input: Grafo pesato, nodo radice s
- Output: i cammini minimi che vanno da s a tutti gli altri nodi

Cammino minimo tra una coppia di vertici

- Input: Grafo pesato, una coppia di vertici s, d
- Output: un cammino minimo fra s e d
- Si risolve il primo problema e si estrae il cammino richiesto
  Non si conoscono algoritmi più efficienti

Cammini mimimi tra tutte le coppie di vertici

- Input: Grafo pesato
- Output: i cammini minimi fra tutte le coppie di vertici.
- Soluzione basata su programmazione dinamica

I pesi dei cammini possono essere di varie tipologie, e i vari algoritmi possono funzionare o meno in base alle varie categorie di pesi. I pesi possono essere:

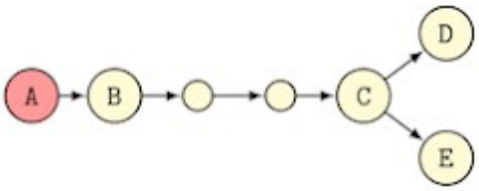
- **Solo positivi** oppure **positivi e negativi**
- **Numeri interi** oppure **numeri reali**

In generale non è detto che esista sempre una soluzione ottima da s ad un altro nodo u, questo perché:

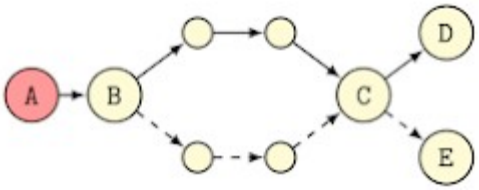
- u potrebbe non essere raggiungibile da s e di conseguenza non esiste alcuna soluzione
- Se sono ammessi pesi negativi, potrebbe esistere un ciclo per il quale la somma delle lunghezze degli archi nel ciclo dia un valore negativo, e quindi la lunghezza minima non sarebbe inferiormente limitata

Si osservi che due cammini minimi distinti possono avere un tratto iniziale comune, da s fino ad un certo nodo z, e poi divergere verso le rispettive destinazioni u e v, ma viceversa non possono convergere in un nodo comune d dopo aver percorso due tratti iniziali distinti, perché in tal caso ci sarebbero due cammini minimi distinti da s a z, mentre il problema ne richiede solo uno.

Si noti che due cammini minimi possono avere un tratto in comune $A \rightsquigarrow C \ldots$



$\ldots$ ma non possono convergere in un nodo comune C dopo aver percorso un tratto distinto

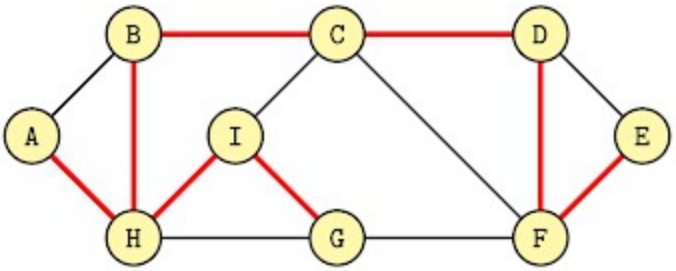


Dunque possiamo definire l'**albero dei cammini minimi**, che è un albero di copertura radicato in s avente un cammino da s a tutti i nodi raggiungibili da s.

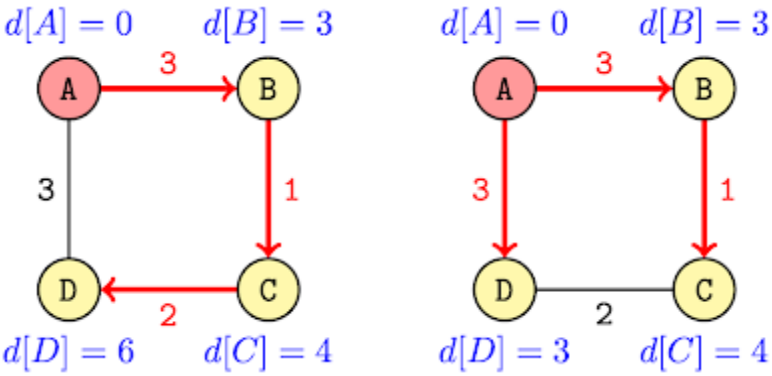
Albero di copertura (Spanning tree)

Dato un grafo $G = (V, E)$ non orientato e connesso, un albero di copertura di G è un sottografo $T = (V, E_T)$ tale che

- T è un albero
- $E_T \subseteq E$
- T contiene tutti i vertici di G



Una **soluzione ammissibile**, quindi, può essere descritta da un albero di copertura T radicato in s e da un vettore di distanze d, i cui valori $d[u]$ rappresentano il costo del cammino da s a u in T. Quindi una soluzione ammissibile è data da $n-1$ archi che formano $n-1$ cammini, ognuno dei quali parte dal medesimo nodo s, ma giunge ad uno ed uno solo dei rimanenti nodi.



Per rappresentare quest'albero di copertura utilizziamo la rappresentazione basata su vettore dei padri.

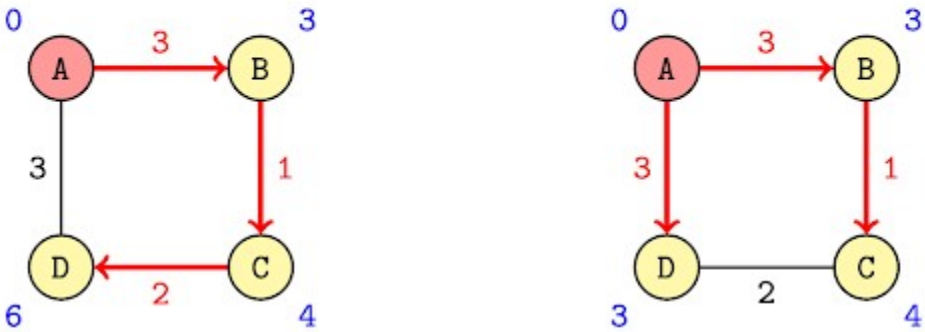
```
printPath(Node s, Node d, Node[] T)
if s == d then
    print s
else if T[d] == nil then
    print "error"
else
    printPath(s, T[d], T)
    print d
```

Vediamo ora il **Teorema di Bellman** che riguarda la soluzione ottima.
Una soluzione ammissibile T è ottima se e solo se valgono le seguenti condizioni:

- $d[v] = d[u] + w(u,v)$ per ogni arco $(u,v) \in T$
- $d[v] \leq d[u] + w(u,v)$ per ogni arco $(u,v) \in E$

$d[B] = d[A] + w(A,B)$ $\quad d[C] = d[B] + w(B,C)$ $\quad d[D] > d[A] + w(A,D)$
$d[D] = d[C] + w(C,D)$

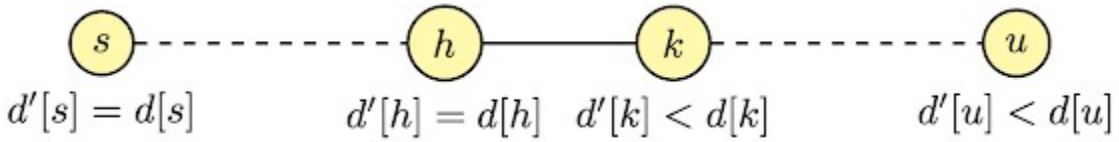
$d[B] = d[A] + w(A,B)$ $\quad d[C] = d[B] + w(B,C)$ $\quad d[D] \leq d[C] + w(C,D)$
$d[D] = d[A] + w(A,D)$



Vediamo la dimostrazione.
Sia T una soluzione ottima. Dobbiamo dimostrare che valgono le condizioni di Bellman.
Consideriamo un arco generico $(u,v) \in E$ e sia $w(u,v)$ la sua lunghezza.
Se $(u,v) \in T$ allora $d[v] = d[u] + w(u,v)$

Se $(u,v) \notin T$ allora $d[v] \leq d[u] + w(u,v)$, perché altrimenti esisterebbe nel grafo G un cammino da s a v più corto di quello in T, ed essendo T una soluzione ottima è assurdo.

Sia T una soluzione per cui valgono le condizioni di Bellman. Dobbiamo dimostrare che T è una soluzione ottima.
Supponiamo per assurdo che T non sia una soluzione ottima, allora esiste un cammino C da s ad un nodo u non ottimo. Allora esiste un albero ottimo T', in cui il cammino C' da s ad u ha distanza $d'[u] < d[u]$.
Poichè $d'[s] = d[s] = 0$, ma $d'[u] < d[u]$, allora esiste un arco (h,k) in C' tale che $d'[h] = d[h]$ e $d'[k] < d[k]$



Dunque $d'[k] = d'[h] + w(h,k)$ per costruzione, $d[k] \leq d[h] + w(h,k)$ per ipotesi e $d'[h] = d[h]$ per costruzione.
Combinando queste relazioni si ottiene $d'[k] = d'[h] + w(h,k) = d[h] + w(h,k) \geq d[k]$. Otteniamo quindi che $d'[k] \geq d[k]$, ma questo contraddice l'ipotesi.

Algoritmo prototipo

La caratterizzazione della soluzione tramite le condizioni di Bellman permette di formulare un algoritmo che si basa sulla verifica, arco per arco, di esse, con conseguente sostituzione degli archi che violano tali condizioni:

```
(int[], int[]) prototipoCamminiMinimi(GRAPH G, NODE s)
% Inizializza T ad una foresta di copertura composta da nodi isolati
% Inizializza d con sovrastima della distanza (d[s] = 0, d[x] = +∞)
while ∃(u, v) : d[u] + G.w(u, v) < d[v] do
    d[v] = d[u] + G.w(u, v)
    % Sostituisci il padre di v in T con u
return (T, d)
```

Dunque la foresta di copertura iniziale viene sostituita passo dopo passo dall'albero dei cammini minimi. Se al termine dell'esecuzione qualche nodo mantiene ancora la distanza $+\infty$, esso non è raggiungibile dalla radice r.
Per implementare la condizione ∃ si può mantenere una struttura dati S che permetta di aggiungere e togliere nodi. Inizialmente viene inserito solamente il nodo r. Selezionando un nodo u da S è possibile verificare le condizioni di Bellman su tutti gli archi (u,v) tali che v appartiene all'insieme di adiacenza $u.adj()$. Se viene modificata la distanza $d[v]$, il nodo v viene inserito nuovamente in S, visto che è possibile che v sia un nodo intermedio più conveniente per raggiungere altri nodi.

Vediamo ora un **algoritmo generico** che definisce meglio il prototipo di procedura appena abbozzato.

```
(int[], int[]) shortestPath(GRAPH G, NODE s)
int[] d = new int[1...G.n]          % d[u] è la distanza da s a u
int[] T = new int[1...G.n]          % T[u] è il padre di u nell'albero T
boolean[] b = new boolean[1...G.n]  % b[u] è true se u ∈ S
foreach u ∈ G.V() − {s} do
    T[u] = nil
    d[u] = +∞
    b[u] = false
T[s] = nil
d[s] = 0
b[s] = true
[...]
```

```
shortestPath(GRAPH G, NODE s) – Corpo principale
(1) DATASTRUCTURE S = DataStructure(); S.add(s)
    while not S.isEmpty() do
(2)     int u = S.extract()
        b[u] = false
        foreach v ∈ G.adj(u) do
            if d[u] + G.w(u, v) < d[v] then
                if not b[v] then
(3)                 S.add(v)
                    b[v] = true
                else
(4)                 % Azione da svolgere nel caso v sia già presente in S
                T[v] = u
                d[v] = d[u] + G.w(u, v)
    return (T, d)
```

Assumendo che la struttura prima o poi restituisca tutto quello che viene inserito, e in assenza di cicli negativi, tale procedura è corretta, ma non molto efficiente. La sua complessità dipende infatti dal numero massimo di inserzioni del medesimo nodo in S e dal numero massimo di miglioramenti delle distanze. Nel caso pessimo il tempo complessivo di esecuzione è superpolinomiale in n.

Per ottenere una procedura efficiente, non basta utilizzare una generica “struttura di dati” con un’altrettanto generica operazione $estrai()$, ma occorre specificarla ulteriormente.
Ad esempio delle scelte ragionevoli per la struttura S possono essere la coda con priorità, la coda e la pila. Nel seguito vedremo diverse implementazioni, e per ognuna di esse specificheremo come sostituire le righe (1)-(4), che devono essere specializzate a seconda della struttura dati utilizzata.

Algoritmo di Dijkstra

Se la struttura S è una coda con priorità, otteniamo **l’algoritmo di Dijkstra**.
Questo algoritmo è stato sviluppato nel 1956 da Edsger W. Dijkstra e fu pubblicato nel 1959.
Nella sua versione originale questo algoritmo veniva utilizzato per trovare la distanza minima tra due nodi e utilizzava il concetto di coda con priorità realizzata con lista o vettore non ordinato (gli heap sono stati proposti nel 1964).

Questo algoritmo funziona bene se tutti i pesi sono non negativi (altrimenti la complessità resta superpolinomiale).
Vediamo come modificare le 4 righe di codice utilizzando una coda con priorità.

La linea 1) consiste nell’inizializzazione di un vettore di lunghezza n, dove l’indice u rappresenta il nodo $u\text{-}esimo$. Tutte le priorità vengono inizializzate a $+\infty$ e la priorità di s è posta uguale a 0.
Questo costa $O(n)$

```
shortestPath(Graph G, Node s) – Corpo principale
(1) PriorityQueue Q = PriorityQueue(); Q.insert(s, 0)
    while not Q.isEmpty() do
(2)     int u = Q.deleteMin()
        b[u] = false
        foreach v ∈ G.adj(u) do
            if d[u] + G.w(u, v) < d[v] then
                [...]
```

La linea 2) consiste nell’estrazione del minimo. Dunque viene cercato il minimo all’interno del vettore e una volta trovato si “cancella” la sua priorità. Questo costa $O(n)$.

```
shortestPath(GRAPH G, NODE s) – Corpo principale
(1) PRIORITYQUEUE Q = PriorityQueue(); Q.insert(s, 0)
    while not Q.isEmpty() do
(2)     u = Q.deleteMin()
        b[u] = false
        foreach v ∈ G.adj(u) do
            if d[u] + G.w(u, v) < d[v] then
                [...]
    return (T, d)
```

La linea 3) consiste nell'inserimento in coda degli elementi che abbiamo modificato e che non sono ancora presenti nella coda. Dunque si inserisce la priorità nella posizione v - *esima*. Questo ha costo $O(1)$

```
shortestPath(GRAPH G, NODE s) – Corpo principale
[...]
if d[u] + G.w(u, v) < d[v] then
    if not b[v] then
(3)     Q.insert(v, d[u] + G.w(u, v))
        b[v] = true
    else
(4)     % Azione da svolgere nel caso v sia già presente in S
    T[v] = u
    d[v] = d[u] + G.w(u, v)
[...]
```

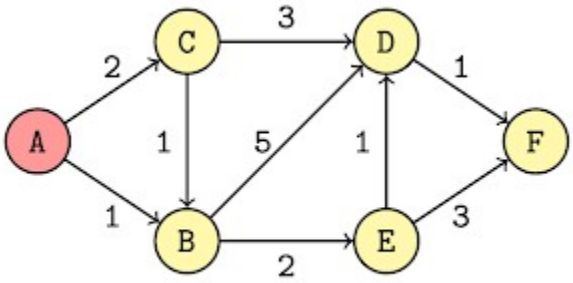
La linea 4) consiste nell'aggiornamento della priorità nel caso in cui l'elemento sia già nella coda.

Questo ha costo $O(1)$

shortestPath(GRAPH G, NODE s) – Corpo principale

```
[...]
if d[u] + G.w(u,v) < d[v] then
    if not b[v] then
(3)     Q.insert(v, d[u] + G.w(u,v))
        b[v] = true
    else
(4)     Q.decrease(v, d[u] + G.w(u,v))
    T[v] = u
    d[v] = d[u] + G.w(u,v)
[...]
```

Vediamo un esempio di Dijkstra con coda basata su vettore:



		A	B	C	E	D	F
A	0	0	~~0~~	~~0~~	~~0~~	~~0~~	~~0~~
B	∞	1	1	~~1~~	~~1~~	~~1~~	~~1~~
C	∞	2	2	2	~~2~~	~~2~~	~~2~~
D	∞	∞	6	5	4	4	~~4~~
E	∞	∞	3	3	3	~~3~~	~~3~~
F	∞	∞	∞	∞	6	5	5

In questa rappresentazione grafica, ogni colonna contiene lo stato del vettore d all'inizio di ogni ripetizione del ciclo $while\ \ not\ \ Q.isEmpty()$. Ogni riga v rappresenta l'evoluzione dello stato dell'elemento $d[v]$. L'evidenziatura in rosso rappresenta il nodo che viene estratto.

Se assumiamo pesi non negativi, ogni nodo viene estratto una e una sola volta, e al momento dell'estrazione la sua distanza è minima.
La dimostrazione viene fatta per induzione sul numero k di nodi estratti.
Il caso base è verificato visto che $d[s]=0$ e non ci sono pesi negativi.
Assumiamo quindi che l'ipotesi sia vera per i primi $k-1$ nodi (ovvero che sono stati estratti con la distanza minima). Quando viene estratto il $k-esimo$ nodo u:

- La sua distanza $d[u]$ dipende dai $k-1$ nodi già estratti (che hanno distanza minima per ipotesi)
- Non può dipendere dai nodi ancora da estrarre, in quanto hanno distanza $\geq d[u]$ e non essendoci pesi negativi non possono migliorare il cammino per arrivare a u

Dunque $d[u]$ è minimo e u non verrà più re-inserito, perchè non ci sono distanze negative.

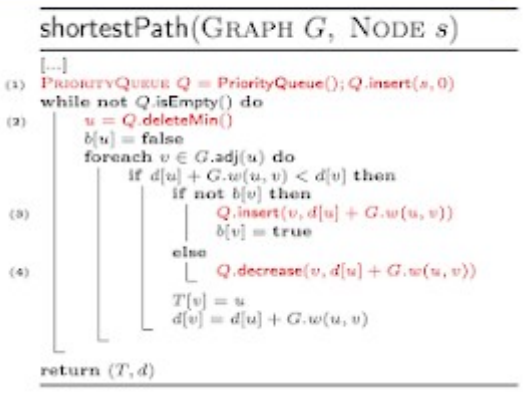
Dunque ogni nodo viene estratto da S una e una sola volta, e quindi il ciclo while viene eseguito n volte. All'interno del ciclo l'operazione più costosa è $deleteMin()$, che su una struttura non

ordinata richiede tempo $O(n)$. La $insert()$ costa $O(1)$ utilizzando una lista non ordinata e già sapendo (grazie al controllo sul vettore booleano) che l'elemento non è presente. Dunque il costo complessivo dell'algoritmo di Dijkstra è $O(n^2)$.

Costo computazionale

Riga	Costo	Ripet.
(1)	$O(n)$	1
(2)	$O(n)$	$O(n)$
(3)	$O(1)$	$O(n)$
(4)	$O(1)$	$O(m)$

Costo totale: $O(n^2)$



Algoritmo di Johnson

Qualora la struttura S sia una coda con priorità realizzata con uno **heap binario**, si ottiene un algoritmo proposto da D.B. Johnson nel 1977. Le linee da sostituire sono le stesse della versione precedente, però cambia la struttura dati.
Come prima se assumiamo lunghezze non negative, il ciclo while è eseguito esattamente n volte. All'interno del ciclo però le operazioni $deleteMin()$ e $decrease()$ richiedono tempo $O(\log\ n)$. Poiché, nel caso pessimo, $decrease()$ è richiamata ad ogni iterazione su tutti i nodi di $G.adj(u)$ la complessità totale è

$$O\left(n\ \log\ n+\left(\sum_{u\in G.V()}^{\square}\left|G.adj(u)\right|\log\ n\right)\right)$$

L'algoritmo di Johnson dunque ha complessità $O(m\ \log\ n)$. Per grafi sparsi, nei quali il numero di archi m è $O(n)$ la complessità diviene $O(n\ \log\ n)$ e questo algoritmo risulta più veloce di quello di Dijkstra. Altrimenti per grafi densi, ovvero $m=\Omega(n^2)$, la complessità risulta $O(n^2\ \log\ n)$
.

Riga	Costo	Ripet.
(1)	$O(n)$	1
(2)	$O(\log n)$	$O(n)$
(3)	$O(\log n)$	$O(n)$
(4)	$O(\log n)$	$O(m)$

Costo totale: $O(m \log n)$

Heap binario introdotto nel 1964

```
shortestPath(GRAPH G, NODE s)
[...]
(1) PRIORITYQUEUE Q = PriorityQueue(); Q.insert(s, 0)
while not Q.isEmpty() do
(2)     u = Q.deleteMin()
        b[u] = false
        foreach v ∈ G.adj(u) do
            if d[u] + G.w(u, v) < d[v] then
                if not b[v] then
(3)                 Q.insert(v, d[u] + G.w(u, v))
                    b[v] = true
                else
(4)                 Q.decrease(v, d[u] + G.w(u, v))
                T[v] = u
                d[v] = d[u] + G.w(u, v)
return (T, d)
```

Algoritmo di Fredman-Tarjan

La scarsa efficienza dell'algoritmo di Johnson nel caso di grafi densi deriva dal fatto che l'operazione *decrease*(), che viene eseguita fino a $O(m)$ volte, ha costo $O(\log\ n)$. Una possibile soluzione a questo problema prevede di adottare una struttura di dati esplicitamente studiata per migliorare le prestazioni del nostro algoritmo generico, gli heap di Fibonacci.
Questa struttura dati (non descritta in questo libro) conserva un costo di $O(\log\ n)$ per le operazioni *insert*() e *deleteMin*(), ma richiede un costo ammortizzato $O(1)$ per le operazioni *decrease*(). In questo modo, il costo del nostro algoritmo diventa $O(m+n\ \log\ n)$ e questo algoritmo del 1987 si chiama algoritmo di Fredman-Tarjan.

Riga	Costo	Ripet.
(1)	$O(n)$	1
(2)	$O(\log n)$	$O(n)$
(3)	$O(\log n)$	$O(n)$
(4)	$O(1)^{(*)}$	$O(m)$

Costo: $O(m + n \log n)$

(*) Costo ammortizzato

```
shortestPath(GRAPH G, NODE s)
[...]
(1) PRIORITYQUEUE Q = PriorityQueue(); Q.insert(s, 0)
while not Q.isEmpty() do
(2)     u = Q.deleteMin()
        b[u] = false
        foreach v ∈ G.adj(u) do
            if d[u] + G.w(u, v) < d[v] then
                if not b[v] then
(3)                 Q.insert(v, d[u] + G.w(u, v))
                    b[v] = true
                else
(4)                 Q.decrease(v, d[u] + G.w(u, v))
                T[v] = u
                d[v] = d[u] + G.w(u, v)
return (T, d)
```

Algoritmo di Bellman-Ford-Moore

Questo algoritmo è stato:

- proposto da Alfonso Shimbel nel 1955
- pubblicato da Lester Ford Jr. nel 1956

- pubblicato da Edward F. Moore nel 1958 (Shortest path faster algorithm, la versione che vediamo)
- pubblicato da Richard Bellman nel 1958

Questo algoritmo usa come struttura S una semplice coda, che è computazionalmente più pesante di Dijkstra. La forza di questo algoritmo è che ha complessità polinomiale anche se ci sono archi di lunghezza negativa!
Per ottenere questo algoritmo dobbiamo sostituire le 4 righe nel seguente modo:

La linea 1) consiste nell'inizializzazione di una coda di lunghezza n, e nell'inserimento di s nella coda. Questo costa $O(1)$

```
shortestPath(GRAPH G, NODE s) – Corpo principale
(1) QUEUE Q = Queue(); Q.enqueue(s)
    while not Q.isEmpty() do
(2)     u = Q.dequeue()
        b[u] = false
        foreach v ∈ G.adj(u) do
            if d[u] + G.w(u, v) < d[v] then
                [...]
    return (T, d)
```

La linea 2) consiste nell'estrazione del prossimo elemento della coda. Questo costa $O(1)$

```
shortestPath(GRAPH G, NODE s) – Corpo principale
(1) QUEUE Q = Queue(); Q.enqueue(s)
    while not Q.isEmpty() do
(2)     u = Q.dequeue()
        b[u] = false
        foreach v ∈ G.adj(u) do
            if d[u] + G.w(u, v) < d[v] then
                [...]
    return (T, d)
```

La linea 3) consiste nell'inserimento in coda dell'elemento v nel caso non sia già presente nella coda. Questo costa $O(1)$

shortestPath(GRAPH G, NODE s) – Corpo principale

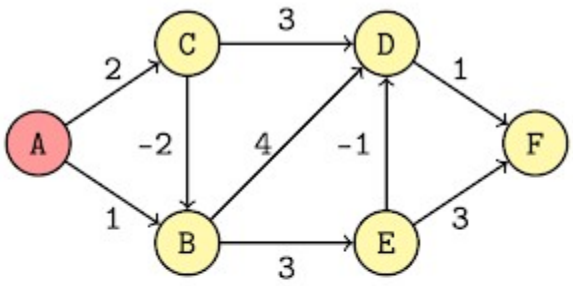
```
[...]
if d[u] + G.w(u, v) < d[v] then
    if not b[v] then
(3)     Q.enqueue(v)
        b[v] = true
    else
(4)     % Azione da svolgere nel caso v sia già presente in S
    T[v] = u
    d[v] = d[u] + G.w(u, v)
[...]
```

La linea 4) non serve, in quanto l'azione da compiere nel caso in cui v sia già presente nella coda è non fare nulla.

Questo algoritmo è computazionalmente più pesante di Dijkstra, però può essere usato in grafi con pesi negativi (benché non ci siano cicli la cui somma dei pesi sia negativa).
La struttura dell'algoritmo è in pratica quella di una visita BFS in cui la marcatura di un nodo consiste nel diminuirne la distanza ed il medesimo nodo può essere visitato, cioè estratto dalla coda S, più di una volta, ma non più di $n-1$ volte.
Per dimostrare questa proprietà, definiamo il concetto di **passata**. La passata è definita induttivamente come segue:

- Per $k=0$, la zeresima passata consiste nell'estrazione del nodo s dalla coda S
- Per $k>0$, la k-*esima* passata consiste nell'estrazione di tutti i nodi presenti in S al termine della passata $(k-1)$-*esima*

L'intuizione è che al termine della passata k, i vettori T e d descrivono i cammini minimi di lunghezza al più k. Al termine della passata $n-1$, i vettori T e d descrivono i cammini minimi (di lunghezza al più $n-1$)



- La prima riga contiene l'elemento estratto dalla coda
- L'ultima riga contiene lo stato della coda

		A	B	C	D	E	B	F	D	E	D	F
A	0	0	0	0	0	0	0	0	0	0	0	0
B	∞	1	1	0	0	0	0	0	0	0	0	0
C	∞	2	2	2	2	2	2	2	2	2	2	2
D	∞	∞	5	5	5	3	3	3	3	2	2	2
E	∞	∞	4	4	4	4	3	3	3	3	3	3
F	∞	∞	∞	∞	6	6	6	6	4	4	3	3
Q	A	BC	CDE	DEB	EBF	BFD	FDE	DE	E	D	F	

Siccome ad ogni passata la coda S contiene al più tutti i nodi e ad ogni passata si esaminano al più tutti gli archi, l'algoritmo di Bellman-Ford-Moore ha complessità $O(n(n+m)) = O(nm)$, sia con archi positivi sia con archi negativi.

Riga	Costo	Ripet.
(1)	$O(1)$	1
(2)	$O(1)$	$O(n^2)$
(3)	$O(1)$	$O(nm)$

Costo: $O(nm)$

Ogni nodo può essere inserito ed estratto al massimo $n-1$ volte

shortestPath(GRAPH G, NODE s) – Corpo principale

```
(1) QUEUE Q = Queue(); Q.enqueue(s)
    while not Q.isEmpty() do
(2)     u = Q.dequeue()
        b[u] = false
        foreach v ∈ G.adj(u) do
            if d[u] + G.w(u,v) < d[v] then
                if not b[v] then
(3)                 Q.enqueue(v)
                    b[v] = true
                T[v] = u
                d[v] = d[u] + G.w(u,v)
    return (T, d)
```

Cammini minimi su DAG

Se il problema viene ristretto a una tipologia più specifica di grafi, possiamo utilizzare le informazioni aggiuntive per formulare un algoritmo più specifico e efficiente.
Ad esempio se sappiamo che in input verrà dato un grafo DAG (Directed Acyclic Graph), possiamo dire con certezza che i cammini minimi sono sempre ben definiti, infatti anche in presenza di pesi negativi, non esistono cicli negativi.

Vogliamo provare e ridurre il più possibile il numero di volte in cui aggiorniamo il peso di una destinazione, per raggiungere questo obiettivo possiamo applicare un ordinamento topologico ai nodi del grafo e poi applicare l'algoritmo dei cammini minimi in quell'ordine. Le proprietà dell'ordinamento topologico ci consentono di dire che ogni nodo viene "analizzato" una volta sola, e una volta analizzato non verrà più guardato o modificato, perchè non ci sono cicli.

Prima di vedere l'algoritmo notiamo che se s è il primo nodo nell'ordinamento topologico tutto procede perfettamente, ma anche nel caso in cui s non sia il primo nodo dell'ordinamento topologico, quando verranno analizzati i nodi precedenti a s, avranno peso $+\infty$, e quindi i pesi rimarranno tutti invariati fino a che non verrà analizzato s che avrà sempre valore 0.

```
(int[], int[]) shortestPath(GRAPH G, NODE s)
int[] d = new int[1...G.n]          % d[u] è la distanza da s a u
int[] T = new int[1...G.n]  % T[u] è il padre di u nell'albero T
foreach u ∈ G.V() − {s} do
    T[u] = nil; d[u] = +∞
T[s] = nil; d[s] = 0
STACK S = topsort(G)
while not S.isEmpty() do
    u = S.pop()
    foreach v ∈ G.adj(u) do
        if d[u] + G.w(u, v) < d[v] then
            T[v] = u
            d[v] = d[u] + G.w(u, v)
return (T, d)
```

Questo algoritmo ha complessità $O(m+n)$, il costo dell'ordinamento topologico.

Vediamo il riassunto dei vari algoritmi che abbiamo visto per le varie tipologie di grafi con le loro complessità:

Complessità: quale preferire?

BFS	$O(m+n)$	Senza pesi
Dijkstra	$O(n^2)$	Pesi positivi, grafi densi
Johnson	$O(m \log n)$	Pesi positivi, grafi sparsi
Fredman-Tarjan	$O(m + n \log n)$	Pesi positivi, grafi densi, dimensioni molto grandi
Bellman-Ford	$O(mn)$	Pesi negativi
	$O(m+n)$	DAG
Bernstein,Nanongkai Wullf-Nilsen (2022)	$O(m \log^8 \nu \log W)$	Pesi negativi, interi

Cammini minimi da sorgente multipla

Questa parte viene svolta per completare il discorso sui cammini minimi e non c'entra nulla con la scelta della struttura dati.

Vediamo come risolvere il problema dei cammini minimi però non più da una singola sorgente s, bensì da più sorgenti.

La soluzione più semplice da applicare è quella di usare letteralmente gli algoritmi che abbiamo visto precedentemente per la singola sorgente, richiamandoli più volte sulle varie sorgenti:

Input	**Complessità**	**Approccio**
Pesi positivi, grafo denso	$O(n \cdot n^2)$	Applicazione ripetuta dell'algoritmo di Dijkstra
Pesi positivi, grafo sparso	$O(n \cdot (m \log n))$	Applicazione ripetuta dell'algoritmo di Johnson
Pesi negativi	$O(n \cdot nm)$	Applicazione ripetuta di Bellman-Ford, sconsigliata
Pesi negativi, grafo denso	$O(n^3)$	Algoritmo di Floyd e Warshall
Pesi negativi, grafo sparso	$O(nm \log n)$	Algoritmo di Johnson per sorgente multipla

Non vedremo l'algoritmo di Johnson per sorgente multipla ma ci concentreremo sull'algoritmo di Floyd e Warshall.

L'applicazione ripetuta di Bellman-Ford è $O(n^2 m)$ che equivale a $O(n^4)$.

Vediamo in questo paragrafo come tramite la programmazione dinamica sia possibile progettare un algoritmo di complessità inferiore a $O(n^4)$ anche per grafi con pesi negativi.

Come sempre assumiamo che esista un ordinamento fra i nodi del grafo e che i nodi vadano da 1 a n
Dato G, un **cammino k-vincolato** da u a v, con $0 \leq k \leq n$, è un cammino semplice da u a v che non attraversa i nodi $\{k, k+1, k+2, \ldots, n\} - \{u, v\}$, dunque il nodo di partenza e il nodo di arrivo sono esclusi dal vincolo.
Diciamo che p_{xy}^k è un **cammino minimo k-vincolato** fra x e y se esso ha il costo minimo tra tutti i cammini k-vincolati da x a y.

Concentriamoci sulla definizione del significato della tabella di programmazione dinamica.
Denotiamo con $d^k[x][y]$ il costo totale del cammino minimo k-vincolato fra x e y, se esiste.
Dunque

$$d^k[x][y] = \begin{cases} w(p_{xy}^k) & \text{se esiste } p_{xy}^k \\ +\infty & \text{altrimenti} \end{cases}$$

Notiamo che $d^n[x][y]$, che corrisponde al cammino minimo n-vincolato tra x e y, equivale al cammino minimo "generale" tra x e y.
Invece $d^0[x][y]$ corrisponde al peso degli archi che collegano x e y.

Per i cammini k-vincolati, vale la seguente proprietà di sottostruttura ottima: ogni sottocammino $d^k[i][j]$ di un cammino minimo k-vincolato $d^k[u][v]$ in G è esso stesso un cammino minimo k-vincolato in G (si dimostra per assurdo scomponendo il cammino in tre).

Sia $d^k[x][y]$ il costo di un cammino minimo k-vincolato fra u e v. Per la proprietà di sottostruttura ottima, è facile osservare che per un cammino minimo (k+1)-vincolato sono date due possibilità:

- O non passa nemmeno per il nodo k, e quindi il suo costo è anche quello di un cammino k-vincolato
- Oppure include anche il nodo k, e allora è uguale al costo del cammino minimo k-vincolato fra u e k più il costo del cammino minimo k-vincolato fra k e v

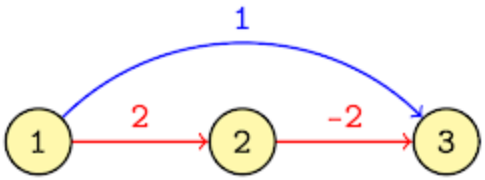
Otteniamo dunque la seguente formulazione ricorsiva:

Formulazione ricorsiva

$$d^k[x][y]) = \begin{cases} w(x,y) & k = 0 \\ \min(d^{k-1}[x][y], d^{k-1}[x][k] + d^{k-1}[k][y]) & k > 0 \end{cases}$$

Ecco un esempio pratico

Esempio

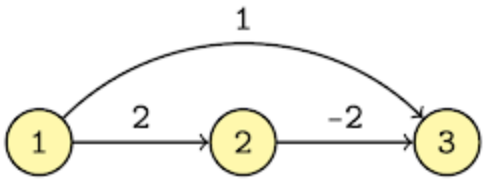


$$d^0[1][3] = 1$$
$$d^1[1][3] = 1$$
$$d^2[1][3] = \min(d^1[1][3], d^1[1][2] + d^1[2][3])$$
$$= \min(1, 0) = 0$$

La definizione che abbiamo appena fatto richiede di utilizzare k matrici, però si può facilmente notare che per calcolare un elemento, ci servono solamente gli elementi della matrice precedente. L'unico problema si ha quando si usa un elemento che è già stato calcolato per la matrice nuova, questo però non è un vero problema in quanto si ottiene comunque un risultato corretto quando si arriva alla fine, semplicemente verranno trovate soluzioni "sbagliate" per quel livello perchè ottimizzate con informazioni di livelli superiori.

Per poter ricavare oltre alla lunghezza del cammino minimo, anche il percorso del cammino minimo, facciamo utilizzo di una matrice dei padri T dove $T[x][y]$ rappresenta il predecessore di y nel cammino più breve da x a y.
Nel nostro esempio di prima:

Esempio



$$T[1][2] = 1$$
$$T[2][3] = 2$$
$$T[1][3] = 2$$

Ecco l'algoritmo completo facente uso di programmazione dinamica:

```
(int[][], int[][]) floydWarshall(GRAPH G)
int[][] d = new int[1...n][1...n]
int[][] T = new int[1...n][1...n]
foreach u, v ∈ G.V() do
    d[u][v] = +∞
    T[u][v] = nil
foreach u ∈ G.V() do
    foreach v ∈ G.adj(u) do
        d[u][v] = G.w(u, v)
        T[u][v] = u
for k = 1 to G.n do
    foreach u ∈ G.V() do
        foreach v ∈ G.V() do
            if d[u][k] + d[k][v] < d[u][v] then
                d[u][v] = d[u][k] + d[k][v]
                T[u][v] = T[k][v]
return (d, T)
```

Il costo computazionale di questo algoritmo è $\Theta(n^3)$, mentre utilizza soltanto $O(n^2)$ spazio di memoria.

Questo algoritmo è nato da un'intuizione di Warshall, il quale stava cercando di risolvere un altro problema, ovvero la chiusura transitiva.

La chiusura transitiva $G^* = (V, E^*)$ di un grafo $G = (V, E)$ è un grafo con gli stessi vertici del grafo di partenza ed è un grafo orientato tale che $(u, v) \in E^*$ se e solo se esiste un cammino da u a v in G.
Si nota subito che questo problema è molto simile a quello precedente, infatti supponendo di avere il grafo G rappresentato da una matrice di adiacenza M, la matrice M^n rappresenta la matrice di adiacenza di G^*. La matrice M^n è definita cosi:

Formulazione ricorsiva

$$M^k[x][y]) = \begin{cases} M[x][y] & k = 0 \\ M^{k-1}[x][y] \text{ or } (M^{k-1}[x][k] \text{ and } M^{k-1}[k][y]) & k > 0 \end{cases}$$

Viene utilizzato l'operatore OR visto che il problema si basa su una semplice condizione (true o false) e non più su un'ottimizzazione

Greedy

Nella vita di tutti i giorni si adottano strategie che danno immediatamente buoni risultati, ma che a lungo andare si rivelano scadenti. È questa la **strategia dell'ingordo** (o greedy):

Meglio un uovo oggi che una gallina domani

Il metodo greedy si può applicare a quei **problemi di ottimizzazione** in cui occorre selezionare un sottoinsieme S "ottimo" di oggetti, che verificano certe proprietà, da un insieme A di n oggetti.

La programmazione dinamica è molto efficiente per questi tipi di problema e valuta in maniera bottom-up tutte le decisioni possibili e sceglie la migliore, evitando però di ripetere sotto-problemi (decisioni) già percorse.
Gli algoritmi greedy invece selezionano solo una delle possibili decisioni, ovvero quella che sembra ottima (ovvero è localmente ottima). È però necessario dimostrare che si ottenga un ottimo globale.

Un algoritmo greedy **ordina** dapprima gli oggetti in base ad un **criterio di "appetibilità"**. La soluzione del problema è poi costruita in modo incrementale considerando gli oggetti uno alla volta e aggiungendo ogni volta l'oggetto più appetibile, se possibile.
In altri termini, l'algoritmo effettua una sequenza di scelte, preferendo ogni volta la scelta che fornisce immediatamente il miglior risultato. Fatta la scelta, è successivamente risolto un sottoproblema dello stesso tipo ma di dimensione più piccola. Il problema più piccolo dipende dalla sequenza di scelte passate, ma non dalle scelte future.

Ovviamente, non sempre esiste un algoritmo greedy che trovi la soluzione ottima di un qualsiasi problema di ottimizzazione (in alcuni casi soluzioni non ottimali possono essere comunque interessanti). Affinché un algoritmo greedy fornisca la soluzione ottima di un problema, occorre che siano verificate due proprietà (tra loro correlate):

- **Scelta greedy**
  Fra le molte scelte possibili, né può essere facilmente individuata una che porta sicuramente alla soluzione ottima
  Dopo aver fornito una caratterizzazione matematica della soluzione ottima, occorre dimostrare che tale soluzione può essere modificata in modo da utilizzare una prima scelta greedy, che riduce il problema ad un sottoproblema più piccolo dello stesso tipo, e

poi mostrare per induzione che il procedimento può essere esteso ad una sequenza di scelte greedy

- **Sottostruttura ottima**
  Fatta tale scelta, resta un sottoproblema con la stessa struttura del problema principale
  Per mostrare che una scelta greedy riduce il problema ad un sottoproblema più piccolo dello stesso tipo, occorre dimostrare che una soluzione ottima del problema contiene al suo interno le soluzioni ottime dei sottoproblemi

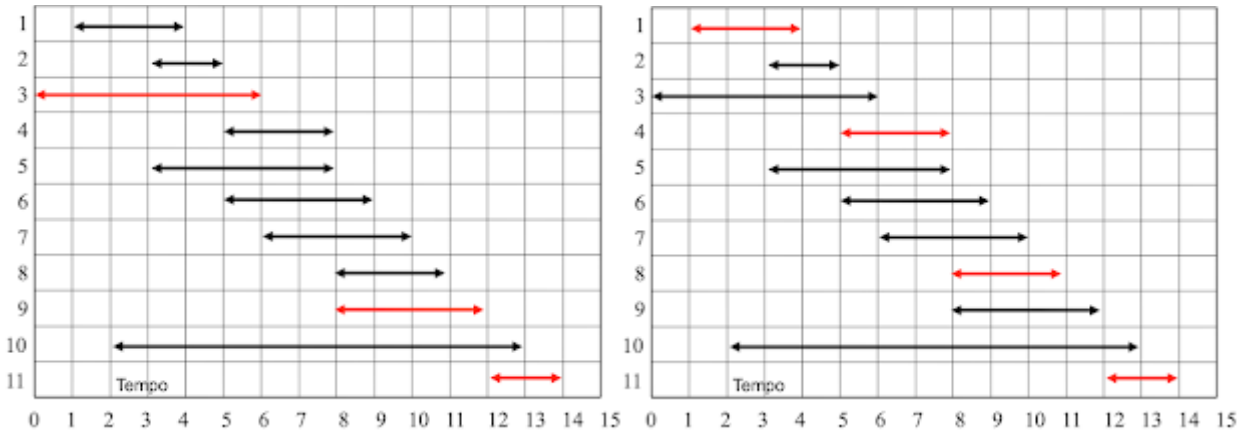
Insieme indipendente di intervalli

Nel capitolo della programmazione dinamica abbiamo visto un algoritmo per trovare l'insieme indipendente massimale per intervalli pesati. Consideriamo ora un caso particolare, in cui tutti gli intervalli hanno peso 1.

Sia $S = \{1,2,\ldots,n\}$ un insieme di intervalli della retta reale. Ogni intervallo $[a_i, b_i[$ con $i \in S$ è chiuso a sinistra e aperto a destra.
Un **insieme indipendente massimale** è un sottoinsieme di massima cardinalità formato da intervalli tutti disgiunti tra loro.

Vediamo due possibili soluzioni per un determinato insieme S:

i	a_i	b_i
1	1	4
2	3	5
3	0	6
4	5	7
5	3	8
6	5	9
7	6	10
8	8	11
9	8	12
10	2	13
11	12	14



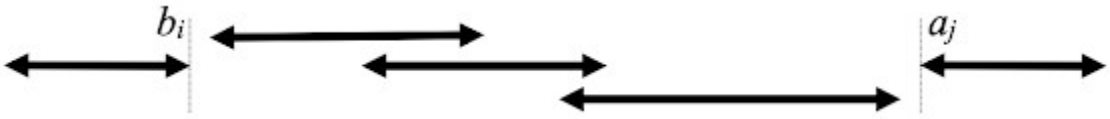
Per affrontare il problema iniziamo con la programmazione dinamica, dunque individuiamo una sottostruttura ottima, scriviamo una definizione ricorsiva per la dimensione della soluzione ottima e poi scriviamo una versione iterativa bottom-up dell'algoritmo.
Passiamo poi alla tecnica greedy, visto che abbiamo già una sottostruttura ottima, cerchiamo una possibile scelta ingorda e dimostriamo che la scelta ingorda porta alla soluzione ottima; infine scriviamo un algoritmo ricorsivo o iterativo che effettua sempre la scelta ingorda.

Vediamo una soluzione diversa da quella che abbiamo già visto con la programmazione dinamica perché possiamo sfruttare il fatto che gli intervalli non hanno peso (o hanno peso 1)
Iniziamo individuando la sottostruttura ottima, e possiamo utilizzare la stessa dimostrazione che abbiamo usato nel capitolo della programmazione dinamica:
Assumiamo che gli intervalli siano ordinati per tempo di fine (altrimenti costa $O(n \ \log \ n)$):

$$b_1 \leq b_2 \leq \ldots \leq b_n$$

Cambiamo approccio rispetto all'altra volta (in cui sceglievamo di prendere o meno l'ultimo intervallo).
Definiamo il sottoproblema $S[i \ldots j]$ come l'insieme di intervalli che iniziano dopo la fine di i e finiscono prima dell'inizio di j, dunque:

$$S[i \ldots j] = \{k \mid b_i \leq a_k < b_k \leq a_j\}$$



Per poter definire il problema iniziale, definiamo due intervalli fittizi:

- Intervallo 0: $b_0 = -\infty$
- Intervallo $n+1$: $a_{n+1} = +\infty$

Dunque il problema iniziale corrisponde al problema $S[0, n+1]$

Il teorema è il seguente:
Supponiamo che $A[i \ldots j]$ sia una soluzione ottimale di $S[i \ldots j]$ e sia k un intervallo che appartiene a $A[i \ldots j]$; allora:

- Il problema $S[i \ldots j]$ viene diviso in due sottoproblemi

    - $S[i \dots k]$: gli intervalli di $S[i \dots j]$ che finiscono prima di k
    - $S[k \dots j]$: gli intervalli di $S[i \dots j]$ che iniziano dopo di k
- La soluzione $A[i \dots j]$ contiene le soluzioni ottimali di $S[i \dots k]$ e $S[k \dots j]$
    - $A[i \dots j] \cap S[i \dots k]$ è la soluzione ottimale di $S[i \dots k]$
    - $A[i \dots j] \cap S[k \dots j]$ è la soluzione ottimale di $S[k \dots j]$

La dimostrazione di questo teorema si fa con il metodo cut-and-paste

Dunque possiamo definire la soluzione ricorsivamente nella seguente maniera:

$$A[i \dots j] = A[i \dots k] \cup \{k\} \cup A[k \dots j]$$

Come facciamo a scegliere un k che appartiene alla soluzione ottima? Dobbiamo analizzare tutte le possibilità.
Definiamo la tabella di programmazione dinamica: sia $DP[i][j]$ la dimensione del più grande sottoinsieme $A[i \dots j] \subseteq S[i \dots j]$ di intervalli indipendenti

$$DP[i][j] = \begin{cases} 0 & S[i \dots j] = \emptyset \\ \max_{k \in S[i \dots j]} \{DP[i][k] + DP[k][j] + 1\} & \text{altrimenti} \end{cases}$$

Dunque per ogni sottoproblema con $i < j$ dobbiamo scorrere tutti gli intervalli con costo $O(n)$ nel caso peggiore, e questo ci porta ad una soluzione con costo $O(n^3)$.

Sappiamo sicuramente che è possibile fare meglio di così visto che abbiamo una soluzione $O(n \log n)$ per gli intervalli pesati, dunque questa soluzione sembra inutile.

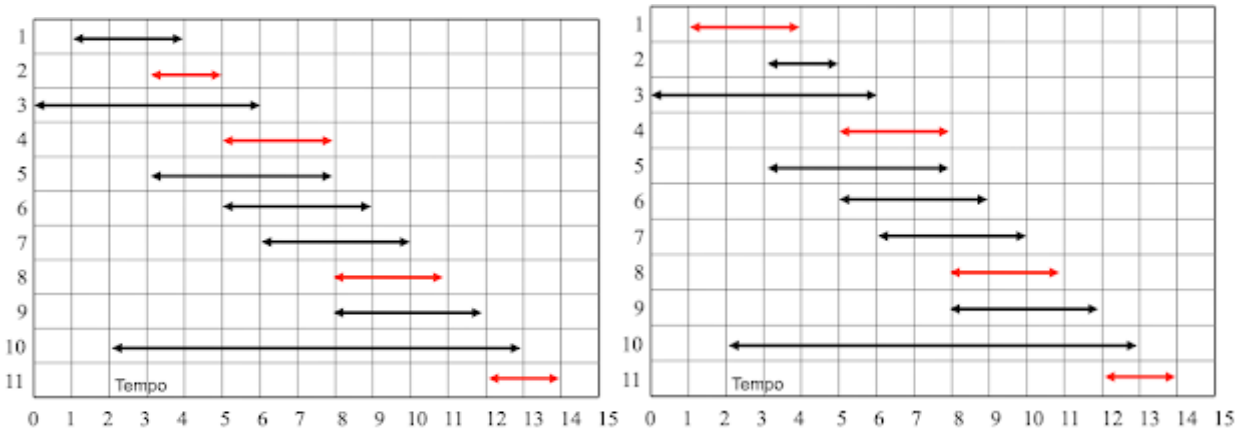
Però proviamo a pensare in maniera greedy… siamo sicuri che sia necessario analizzare tutti i valori di k oppure ne esiste uno facile da scegliere che sembri portare ad una soluzione ottima?
Ad esempio possiamo provare a prendere l'intervallo con il minor tempo di fine (nel caso in cui gli intervalli fossero ordinati per tempo di inizio, avremmo potuto prendere quello con il maggior tempo di inizio):
Questa idea infatti sembra buona logicamente in quanto il primo intervallo è quello che dà meno fastidio (si interseca meno) di tutti.
Sia $S[i \dots j]$ un sottoproblema non vuoto, e m l'intervallo di $S[i \dots j]$ con il minor tempo di fine, allora:

1. Il sottoproblema $S[i \dots m]$ è vuoto
2. m è sicuramente compreso in qualche soluzione ottima di $S[i \dots j]$

Dimostriamo 1, ovvero che $S[i \dots m]$ è vuoto.
Sappiamo che $a_m < b_m$ per definizione di intervallo.
Sappiamo che $\forall k \in S[i \dots j] : b_m \leq b_k$ visto che m ha minor tempo di fine, e dunque $\forall k \in S[i \dots j] : a_m \leq b_k$.
Visto che nessun intervallo in $S[i \dots j]$ termina prima di a_m, allora $S[i \dots m] = \varnothing$.

Dimostriamo 2.

Sia $A'[i \dots j]$ una soluzione ottima di $S[i \dots j]$ e sia m' l'intervallo con minor tempo di fine in $A'[i \dots j]$.
Sia $A[i \dots j] = A'[i \dots j] - \{m'\} \cup \{m\}$ una nuova soluzione. $A[i \dots j]$ è una soluzione ottima che contiene m in quanto ha la stessa dimensione di $A'[i \dots j]$ e gli intervalli sono indipendenti in quanto $b_m < b_{m'}$.
Dunque possiamo sempre trovare una soluzione ottima di cui m fa parte.
Vediamo un esempio pratico di questo concetto:



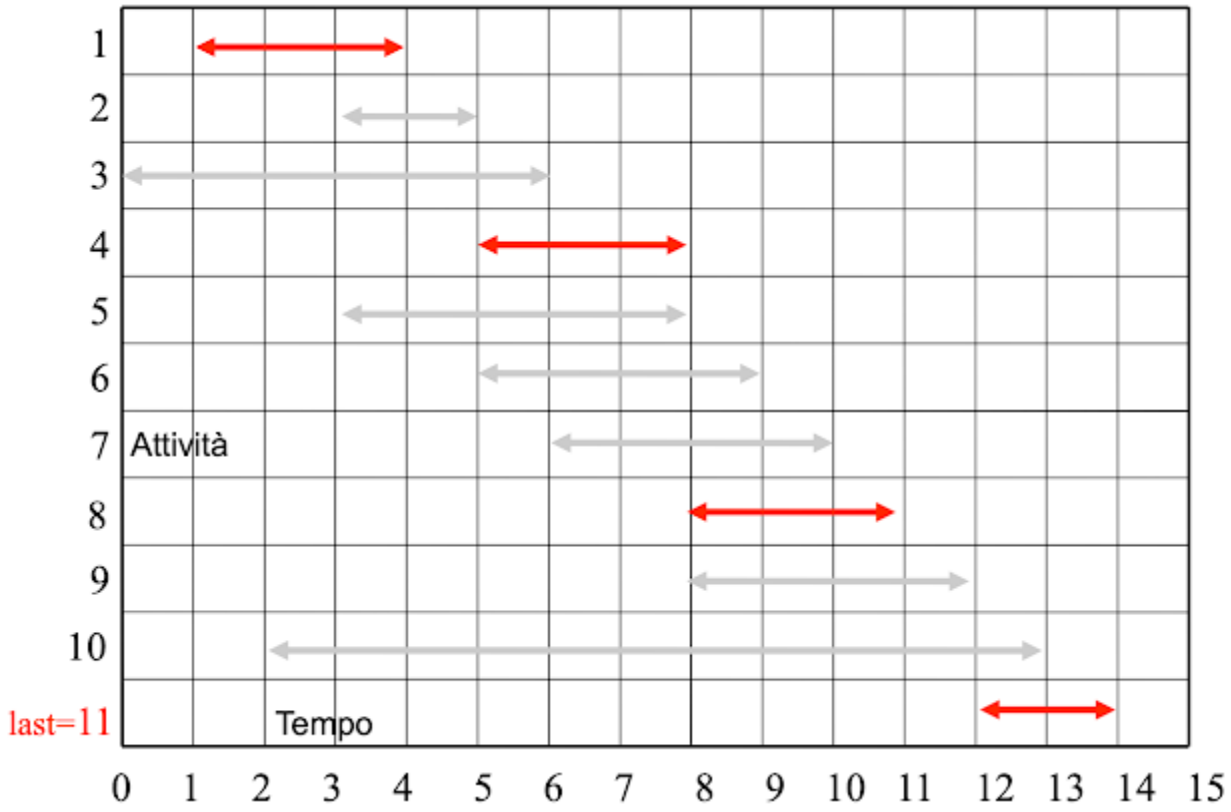
Le conseguenze di questa osservazione sono che:

- Non è più necessario analizzare tutti i possibili valori di k
  Infatti faccio una scelta "ingorda" ma sicura
- Non è più necessario analizzare due sottoproblemi
  Elimino tutte le attività che non sono compatibili con la scelta ingorda e mi resta un solo sottoproblema da risolvere: $S[m \dots j]$

L'algoritmo è il seguente:

```
SET independentSet(int[] a, int[] b)
{ ordina a e b in modo che b[1] ≤ b[2] ≤ ··· ≤ b[n] }
SET S = Set()
S.insert(1)
int last = 1                          % Ultimo intervallo inserito
for i = 2 to n do
    if a[i] ≥ b[last] then            % Controllo indipendenza
        S.insert(i)
        last = i
return S
```

Scorriamo tutti gli intervalli una e una sola volta, dunque l'algoritmo ha complessità $O(n)$ nel caso in cui gli intervalli siano già ordinati per tempo di fine. Se gli intervalli non sono ordinati l'algoritmo costa $O(n \log n)$



L'approccio che abbiamo appena visto consiste nel partire dalla programmazione dinamica (individuando la sottostruttura ottima e scrivendo una definizione ricorsiva per la dimensione della soluzione ottima). Successivamente abbiamo dimostrato la proprietà della scelta greedy, dunque abbiamo dimostrato che per ogni sottoproblema esiste una soluzione ottima che include la scelta greedy e poi abbiamo scritto un algoritmo iterativo che effettua sempre la stessa scelta ingorda.
Non sempre opereremo così, infatti si può saltare la parte della programmazione dinamica.

Resto

Il problema del resto consiste nel, dato un insieme di "tagli" di monete memorizzati in un vettore di interi positivi $t[1 \ldots n]$ e un intero R che indica il resto che dobbiamo restituire, trovare il più piccolo numero di pezzi necessari per dare un resto di R centesimi utilizzando i tagli disponibili, assumendo di avere un numero illimitato di monete per ogni taglio.
Formalmente bisogna trovare un vettore x di interi non negativi tale che:

$$R = \sum_{i=1}^{n} x[i] \cdot t[i] \qquad \text{e} \qquad m = \sum_{i=1}^{n} x[i] \text{ ha valore minimo}$$

Vediamo anche questa volta prima una soluzione basata su programmazione dinamica.
Definiamo la sottostruttura ottima:

Sia $S(i)$ il problema di dare un resto pari a i. Sia $A(i)$ una soluzione ottima del problema $S(i)$, rappresentata da un multi-insieme, e sia $j \in A(i)$.
Allora $S(i\text{-}t[j])$ è un sottoproblema di $S(i)$, la cui soluzione ottima è data da $A(i)\text{-}\{j\}$.

Possiamo quindi definire ricorsivamente la nostra tabella di programmazione dinamica: $DP[0 \ldots R]$.
$DP[i]$ rappresenta il numero minimo di monete per risolvere il problema $S(i)$ ed è definito ricorsivamente così:

$$DP[i] = \begin{cases} 0 & i = 0 \\ \min_{1 \le j \le n}\{DP[i - t[j]] \mid t[j] \le i\} + 1 & i > 0 \end{cases}$$

Dunque proviamo a usare tutti i tagli possibili e vediamo quale conviene, ecco l'algoritmo con programmazione dinamica:

```
int[] moneyChange(int[] t, int n, int R)
int[] DP = new int[0...R]                    % Value of the solution
int[] coin = new int[0...R]   % Coin to be used for a specific value
DP[0] = 0
for i = 1 to R do
    DP[i] = +∞
    for j = 1 to n do
        if i > t[j] and DP[i − t[j]] + 1 < DP[i] then
            DP[i] = DP[i − t[j]] + 1
            coin[i] = j
% Solution reconstruction
int[] x = new int[1...n] = {0} % Output vector, initialized to zero
while R > 0 do
    x[coin[R]] = x[coin[R]] + 1
    R = R − t[coin[R]]
return x
```

Come vediamo questa soluzione ha complessità $O(nR)$, data dal fatto che non sappiamo quale taglio convenga prendere e quindi le guardiamo tutte.

Proviamo a prendere come scelta greedy quella di selezionare la moneta j più grande tale per cui $t[j] \le R$, e poi risolvere il problema $S(R\text{-}t[j])$

Questa scelta però è corretta solo per determinati tagli di monete (si può usare per il sistema di monete dell'euro). Dunque la correttezza dell'algoritmo greedy dipende fortemente dal set di monete messe a disposizione t.

Ecco l'algoritmo:

```
int[] moneyChange(int[] t, int n, int R)
int[] x = new int[1...n]
{ Ordina le monete in modo decrescente }
for i = 1 to n do
    x[i] = ⌊R/t[i]⌋
    R = R − x[i] · t[i]
return x
```

il quale ha complessità $O(n)$ se l'input è già ordinato, oppure $O(n \log n)$ se bisogna ordinare l'input.

Dimostriamo che questa scelta funzione per il set di monete $t=[50,10,5,1]$.
Sia x una qualunque soluzione ottima, dunque tale che

$$R=\sum_{i=1}^{n} x[i]\cdot t[i] \qquad \text{e} \qquad m=\sum_{i=1}^{n} x[i] \text{ ha valore minimo}$$

Sappiamo che $t[k]\cdot x[k]<t[k-1]$, altrimenti x non sarebbe una soluzione ottima in quanto basterebbe sostituire un certo numero di monete di taglia $t[k]$ con una di taglia $t[k-1]$.

$$\begin{aligned} t[2]\cdot x[2] &= 10\cdot x[2] < t[1] = 50 \Rightarrow x[2]<5 \\ t[3]\cdot x[3] &= 5\cdot x[3] < t[2] = 10 \Rightarrow x[3]<2 \\ t[4]\cdot x[4] &= 1\cdot x[4] < t[3] = 5 \Rightarrow x[4]<5 \end{aligned}$$

Sia m_k la somma del valore delle monete di taglio inferiore a $t[k]$:

$$m_k=\sum_{i=k+1}^{4} x[i]\cdot t[i]$$

Se dimostriamo che $\forall k \ : \ m_k<t[k]$, allora la soluzione (ottima) è proprio quella calcolata dall'algoritmo greedy.

$$\begin{aligned} m_4 &= 0 && && < 1 && = t[4] \\ m_3 &= x[4]\cdot 1 + m_4 && \le 4\cdot 1 + m_4 && < 4+1 && = 5 = t[3] \\ m_2 &= x[3]\cdot 5 + m_3 && \le 1\cdot 5 + m_3 && < 5+5 && = 10 = t[2] \\ m_1 &= x[2]\cdot 10 + m_2 && \le 4\cdot 10 + m_2 && < 40+10 && = 50 = t[1] \end{aligned}$$

Dunque sul set di monete t possiamo applicare la tecnica greedy.

5 Domanda – Esempio di insieme di monete che non supporta la scelta greedy

Insieme: 12 8 1
Valore: 17

Scelta greedy: 12+1+1+1+1+1
Soluzione ottima: 8+8+1

Scheduling

Finora abbiamo sempre seguito un approccio che partiva dalla programmazione dinamica e poi arrivava alla scelta greedy.
Vediamo ora l'approccio senza passare dalla programmazione dinamica.

I passi da seguire sono:

- **Evidenziare i "passi di decisione"**
  Ovvero trasformare il problema di ottimizzazione in un problema di "scelte" successive
- **Evidenziare una possibile scelta ingorda**
  E dimostrare che tale scelta rispetti il "principio della scelta ingorda"
- **Evidenziare la sottostruttura ottima**
  Dimostrare che la soluzione ottima del problema "residuo" dopo la scelta ingorda può essere unito a tale scelta
- **Scrittura codice: top-down, anche in maniera iterativa**
  Nota: può essere necessario pre-processare l'input

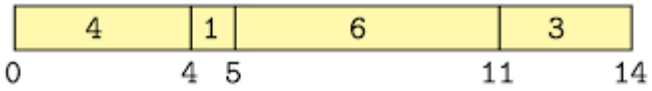
Il problema dello scheduling consiste in, dato un processore e n job da eseguire su di esso, ognuno caratterizzato da un tempo di esecuzione $t[i]$ noto a priori, trovare una sequenza di esecuzione (permutazione) che minimizzi il tempo di completamento medio.

Dato un vettore $A[1 \ldots n]$ contenente una permutazione di $(1, \ldots, n)$, il **tempo di completamento** dell'h-esimo job nella permutazione è:

$$T_A(h) = \sum_{i=1}^{h} t[A[i]]$$

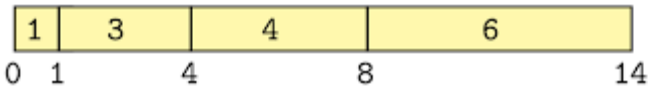
Ecco due esempi:

Esempio



Tempo di completamento medio:
$(4+5+11+14)/4 = 34/4 = 8.5$

Shortest job first

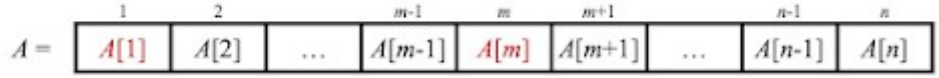


Tempo di completamento medio:
$(1+4+8+14)/4 = 27/4 = 6.75$

Vediamo che il problema dipende dalla scelta di quale job eseguire, e che possiamo fare la scelta ingorda del job con tempo di esecuzione minore (l'idea è che mettendo il job più corto per primo, "peserà" di meno nella sommatoria).

Dobbiamo dimostrare la correttezza della scelta greedy e la sottostruttura ottima.
Il teorema della sottostruttura ottima è il seguente:
Sia A una soluzione ottima di un problema con n job, in cui il job con minor tempo di fine m si trova in prima posizione. La permutazione dei seguenti $n-1$ job in A è una soluzione ottima al sottoproblema in cui il job m non viene considerato.

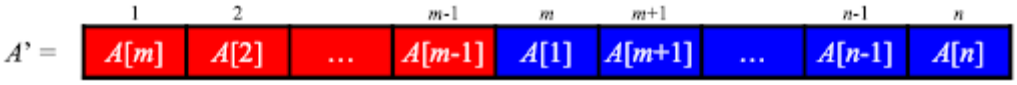
Il teorema della scelta greedy invece è il seguente:
Esiste una soluzione ottima A in cui il job con il minor tempo di fine m si trova in prima posizione.
Dimostriamo la scelta greedy.
Consideriamo una permutazione ottima A:



Sia m la posizione in A in cui si trova il job con minor tempo di fine.
Consideriamo la permutazione A' in cui i job in posizione 1 e m vengono scambiati:



Il tempo di completamento medio di A' è minore o uguale al tempo di completamento di A, infatti



Notiamo che il primo e il k-esimo job della permutazione A sono stati scambiati di posto; poiché la durata del k-esimo job è più breve di quella del primo, se ne deduce che quando viene

spostato al primo posto, ha un tempo di completamente inferiore; d'altra parte, il tempo di completamento del job in k-esima posizione in entrambe le sequenze è uguale, in quanto comunque devono essere completati tutti i primi k job.

$$T_{A'}(1) \leq T_A(1)$$
$$T_{A'}(k) = T_A(k)$$

I job in posizione $1,\ldots,m\text{-}1$ in A' hanno tempo di completamento ≤ dei job in posizione $1,\ldots,m\text{-}1$ in A, questo perché $A[m] \leq A[1]$.
I job in posizione $m,\ldots,n$ in A' hanno tempo di completamento = dei job in posizione $m,\ldots,n$ in A.
Però visto che A è ottima, A' non può avere tempo di completamento medio minore e quindi anche A' è ottima. Abbiamo quindi dimostrato che esiste una soluzione ottima che mette come primo elemento il job con minor tempo di esecuzione, quindi il nostro algoritmo greedy è corretto.

Zaino frazionario

Il problema è lo stesso dello zaino, solo che è possibile prendere frazioni degli oggetti.

Dati in input un intero positivo C (capacità dello zaino) e un insieme di n oggetti, tali che l'oggetto $i\text{-}esimo$ è caratterizzato da:

- Un profitto $p_i \in Z^+$
- Un peso $w_i \in Z^+$

Il problema consiste nello scegliere oggetti o porzioni di essi in modo da massimizzare il profitto degli oggetti scelti senza superare la capacità, ovvero determinare un vettore $x = [x_1,\ldots,x_n]$ di reali compresi tra 0 e 1 tali che il profitto totale $p(x) = \sum_{1 \leq i \leq n}^{\square} p_i x_i$ sia massimo e il peso $w(x) = \sum_{1 \leq i \leq n}^{\square} w_i x_i$ sia $\leq C$.

Abbiamo già visto come il problema dello zaino binario (0 o 1) goda di sottostruttura ottima, è facile adattare la dimostrazione nel caso di scelta frazionaria: una volta scelta una certa quantità di un oggetto, possiamo ridurci al sottoproblema in cui c'è un oggetto in meno e una capacità inferiore.

Abbiamo varie opzioni come scelta ingorda:

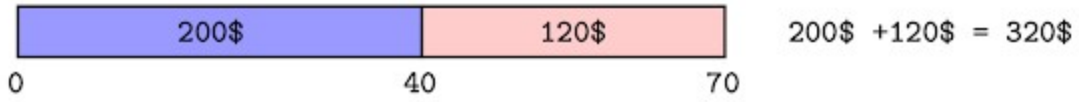
- Ordinare per profitto decrescente
- Ordinare per peso crescente

Vediamo dai seguenti esempi però che queste scelte ingorde non funzionano:

Consideriamo i tre oggetti a lato ed una capacità di 70

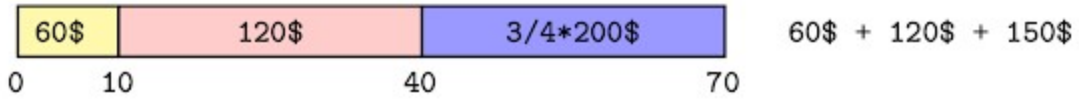
i	p_i	w_i
1	60$	10
2	200$	40
3	120$	30

Approccio 1: Ordinati per profitto decrescente



200$ +120$ = 320$

Approccio 2: Ordinati per peso crescente



60$ + 120$ + 150$ = 330$

La scelta ingorda giusta è quella di ordinare per profitto specifico in maniera decrescente, ovvero il rapporto tra profitto e peso (da notare che nel caso dello zaino 0/1 questa scelta non funziona, infatti per lo zaino 0/1 non si può applicare greedy):

Consideriamo i tre oggetti a lato ed una capacità di 70

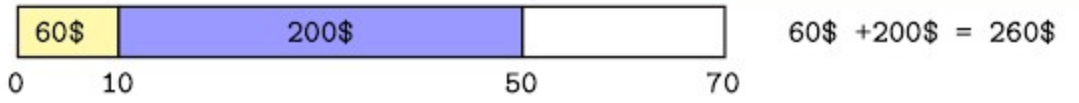
i	p_i	w_i	p_i/w_i
1	60$	10	6$
2	200$	40	5$
3	120$	30	4$

Approccio 3: Ordinati per profitto specifico p_i/w_i decrescente



60$ +200$ + 80$ = 340$

Approccio 3 non funziona per Zaino 0/1



60$ +200$ = 260$

Una volta scelto l'elemento con più alto profitto specifico, cerchiamo di prenderne la porzione più grande possibile senza superare la capacità C; dopodichè si risolve il sottoproblema dato dai rimanenti oggetti e dalla capacità residua.

Ecco l’algoritmo:

```
float[] zaino(float[] p, float[] w, float C, int n)
float[] x = new float[1...n]
{ ordina p e w in modo che p[1]/w[1] ≥ p[2]/w[2] ≥ ··· ≥ p[n]/w[n]}
for i = 1 to n do
    x[i] = min(C/w[i], 1)
    C = C − x[i] · w[i]
return x
```

Anche in questo caso la complessità è $O(n)$ se l’algoritmo è ordinato ed è $O(n \log n)$ in caso contrario.

$x[i] \in [0,1]$ rappresenta la porzione dell’oggetto i-esimo che deve essere prelevata

Per dimostrare la correttezza della scelta greedy assumiamo che gli oggetti siano ordinati per profitto specifico decrescente e che x sia una soluzione ottima.

Supponiamo che $x[1] < min\left(\frac{C}{w[i]}, 1\right)$, ovvero non contiene la porzione più grande possibile dell’oggetto 1, in altre parole per riempire lo zaino è stata scelta una quantità $min\left\{\frac{C}{w[i]} - x[1], w[1] - x[1]\right\}$ di altri oggetti il cui valore specifico e inferiore/uguale a quello dell’oggetto 1.

Possiamo allora costruire una nuova soluzione in cui $x'[1] = min\left(\frac{C}{w_i}, 1\right)$, ovvero massimizziamo la porzione dell’oggetto 1 inserita nella soluzione (e riduciamo di conseguenza la proporzione di altri 1 o più oggetti).
Otteniamo così una soluzione x' di profitto uguale o superiore a quello della soluzione x, visto che il profitto specifico dell’oggetto 1 è il più alto.

Alla fine saranno scelti per intero gli oggetti più appetibili, può essere scelta una frazione dell’oggetto immediatamente successivo, ed i rimanenti oggetti non saranno scelti.

Compressione di Huffman

Il problema della compressione consiste nel rappresentare i dati in maniera efficiente, ovvero impiegare il minor numero possibile di bit per la rappresentazione.
Una possibile tecnica di compressione è quella della codifica dei caratteri, quindi tramite una funzione di codifica $f : f(c) = x$ partendo da c che è un possibile carattere preso da un alfabeto Σ, otteniamo x che è una rappresentazione binaria e diciamo che c è rappresentato da x.

Una funzione di codifica che considera tutti i caratteri è quella ASCII, ma se abbiamo un file composto da un alfabeto di dimensioni ridotte, possiamo utilizzare una codifica personalizzata per utilizzare meno bit:

Esempio

- Supponiamo di avere un file di n caratteri
- Composto da caratteri nell'alfabeto `abcdef`
- Di cui conosciamo la frequenza relativa

Caratteri	a	b	c	d	e	f	Dim.
Frequenza	45%	13%	12%	16%	9%	5%	
ASCII	01100001	01100010	01100011	01100100	01100101	01100110	$8n$
Codifica 1	000	001	010	011	100	101	$3n$

Però dato il fatto che conosciamo la frequenza relativa, possiamo utilizzare una codifica "sbilanciata" rappresentando con meno bit i caratteri più frequenti e usando invece più bit per i caratteri poco frequenti:

Esempio

- Supponiamo di avere un file di n caratteri
- Composto da caratteri nell'alfabeto `abcdef`
- Di cui conosciamo la frequenza relativa

Caratteri	a	b	c	d	e	f	Dim.
Frequenza	45%	13%	12%	16%	9%	5%	
ASCII	01100001	01100010	01100011	01100100	01100101	01100110	$8n$
Codifica 1	000	001	010	011	100	101	$3n$
Codifica 2	0	101	100	111	1101	1100	$2.24n$

Costo totale: $(0.45 \cdot 1 + 0.13 \cdot 3 + 0.12 \cdot 3 + 0.16 \cdot 3 + 0.09 \cdot 4 + 0.05 \cdot 4) \cdot n = 2.24n$

La codifica due fa uso dei **codici a prefisso**.
In un codice a prefisso (meglio sarebbe "senza prefissi"), nessun codice è prefisso di un altro codice (questa condizione è strettamente necessaria per la decodifica).

Esempio 1

- Codice: "a" → 0, "b" → 10, "c" → 11
- "babaca": 10 · 0 · 10 · 0 · 11 · 0

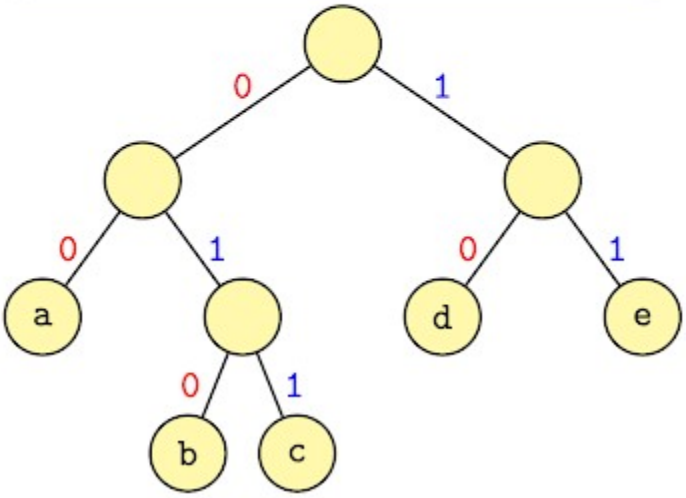
Esempio 2

- Codice: "a" → 0, "b" → 1, "c" → 11
- 111111?

Ovviamente bisogna usare bene questo tipo di codifica, perchè se il testo ha tanti caratteri che noi abbiamo codificato con tanti bit, questa codifica non porta vantaggi rispetto alla Codifica 1. Per la decodifica si usa una struttura ad albero dove i caratteri sono nelle foglie. Questo albero deve essere condiviso assieme alla compressione visto che la codifica, e di conseguenza la decodifica, è diversa per ogni file.

Alberi binari di decodifica

- Figlio sinistro/destro: 0 / 1
- Caratteri dell'alfabeto sulle foglie



a	b	c	d	e
00	010	011	10	11

Algoritmo di decodifica

```
parti dalla radice
while file non è finito do
    leggi un bit
    if bit è zero then
        vai a sinistra
    else
        vai a destra
    if nodo foglia then
        stampa il carattere
        torna alla radice
```

Definiamo formalmente il problema:
Dato un file F composto da caratteri nell'alfabeto Σ, vogliamo minimizzare il numero di bit richiesti per rappresentare il file.
Vediamo quanti bit sono richiesti per codificare il file dato un albero che rappresenta la codifica T.

Per ogni $c \in \Sigma$, sia $d_T(c)$ la profondità della foglia che rappresenta c. Il codice per c richiederà allora $d_T(c)$ bit.
Se $f[c]$ è il numero di occorrenze di c in F, allora la dimensione della codifica è

$$C(F,T) = \sum_{c \in \Sigma} f[c] \cdot d_T(c)$$

L'obiettivo come abbiamo già detto è minimizzare la lunghezza dei caratteri che compaiono più frequentemente e assegnare ai caratteri con la frequenza minore i codici corrispondenti ai percorsi più lunghi all'interno dell'albero.

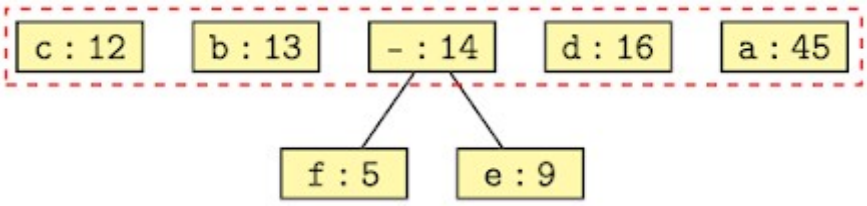
Quindi per ogni file dobbiamo:

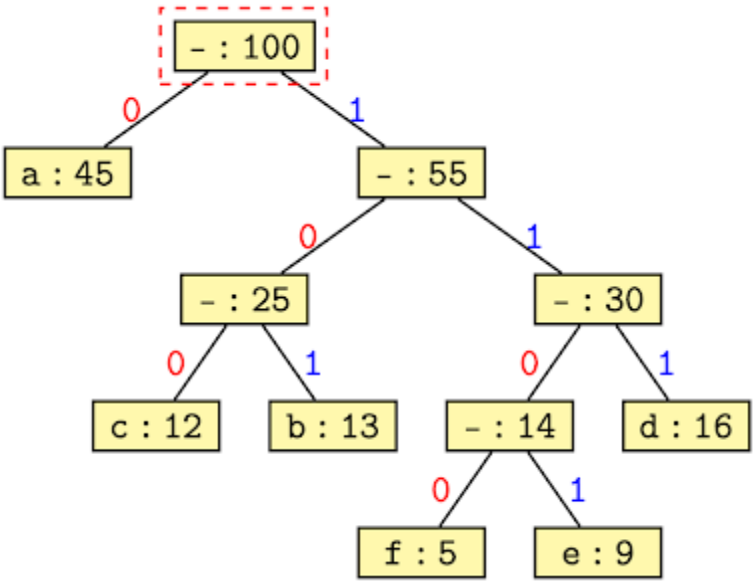
- Ottenere la frequenza di tutti i caratteri
- Costruire il codice
- Rappresentare il file tramite il codice
- Aggiungere al file una rappresentazione del codice, per la decodifica

L'algoritmo che ci consente di ottenere la codifica più conveniente in termini di spazio consiste in:

- Costruire un nodo foglia per ogni carattere, etichettato con la propria frequenza
- Rimuovere i due nodi con frequenze minori f_x e f_y
- Creare un nodo padre con etichetta - e frequenza $f_x + f_y$
- Collegare i due nodi rimossi con il nuovo nodo
- Aggiungere il nodo così creato all'insieme
- Si termina quando resta un solo nodo nell'insieme

Al termine, si etichettano gli archi dell'albero con bit 0 e 1





a	0
b	101
c	100
d	111
e	1101
f	1100

L'algoritmo per costruire l'albero è il seguente:

```
TREE huffman(int[] c, int[] f, int n)
PRIORITYQUEUE Q = MinPriorityQueue()
for i = 1 to n do
    Q.insert(Tree(c[i], f[i]), f[i])
for i = 1 to n − 1 do
    z1 = Q.deleteMin()
    z2 = Q.deleteMin()
    z = Tree(nil, z1.f + z2.f)
    z.left = z1
    z.right = z2
    Q.insert(z, z.f)
return Q.deleteMin()
```

Input

n : numero caratteri
$c[\,]$: caratteri alfabeto
$f[\,]$: frequenze

TREE	
c	% Carattere
f	% Frequenza
$left$	% Figlio sinistro
$right$	% Figlio destro

La complessità è $O(n \log n)$.

L'output dell'algoritmo di huffman per un dato file è un codice a prefisso ottimo.
La scelta greedy è che scegliere i due elementi con la frequenza più bassa conduce sempre ad una soluzione ottimale.
La sottostruttura ottima è che dato un problema sull'alfabeto Σ, è possibile costruire un sottoproblema con un alfabeto più piccolo.

Dimostriamo la correttezza della scelta greedy.
Siano

- Σ un alfabeto
- f un vettore di frequenze

E siano x e y i due caratteri con frequenza più bassa.

La tesi è che esiste un codice prefisso ottimo per Σ in cui x e y hanno la stessa profondità massima e i loro codici differiscono solo per l'ultimo bit (sono foglie sorelle).

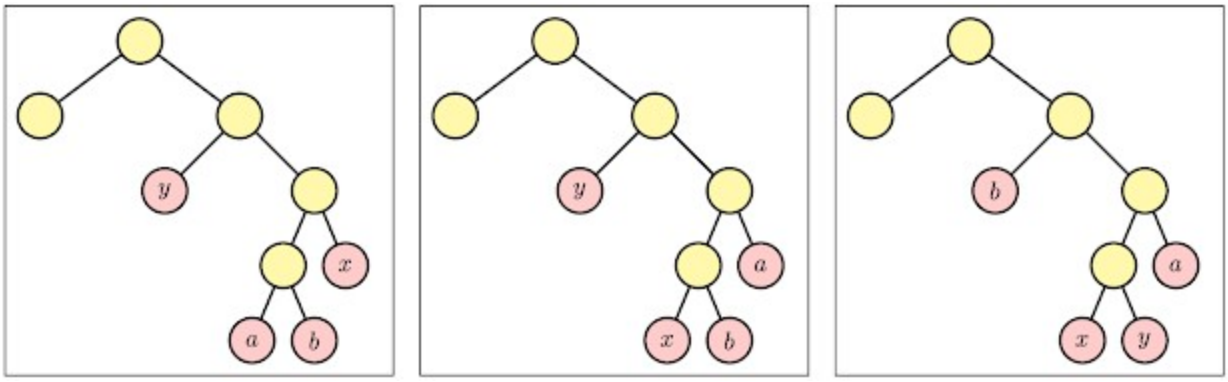
La dimostrazione come al solito è basata sulla trasformazione di una soluzione ottima.
Supponiamo che esista un codice ottimo T in cui i due caratteri a e b con profondità massima siano diversi da x e y.
Senza perdere di generalità assumiamo (a meno di scambi) che:

- $f[x] \leq f[y]$
- $f[a] \leq f[b]$

Poiché abbiamo detto che le frequenze di x e y sono minime, abbiamo che:

- $f[x] \leq f[a]$
- $f[y] \leq f[b]$

Scambiamo x con a ottenendo T' e y con b ottenendo T''



Dobbiamo dimostrare che $C(f,T'') \leq C(f,T') \leq C(f,T)$
Proviamo a calcolare $C(f,T) - C(f,T')$ e $C(f,T') - C(f,T'')$

$$
\begin{aligned}
C(f,T) - C(f,T') &= \sum_{c\in\Sigma} f[c]d_T(c) - \sum_{c\in\Sigma} f[c]d_{T'}(c) \\
&= (f[x]d_T(x) + f[a]d_T(a)) - (f[x]d_{T'}(x) + f[a]d_{T'}(a)) \\
&= (f[x]d_T(x) + f[a]d_T(a)) - (f[x]d_T(a) + f[a]d_T(x)) \\
&= (f[a] - f[x])(d_T(a) - d_T(x)) \\
&\geq 0 \\
C(f,T') - C(f,T'') &\geq 0 \qquad \text{Come sopra}
\end{aligned}
$$

Otteniamo che $C(f,T'') \leq C(f,T)$, ma poichè T è ottimo, allora anche T'' è ottimo.

Alberi di copertura di peso minimo

Si consideri il seguente problema di ottimizzazione su grafi pesati:

Minimo Albero di Copertura (*MINIMUM SPANNING TREE*). Dato un grafo non orientato e connesso $G=(V,E)$, con pesi $w(u,v)$ non negativi sugli archi, trovare un albero di copertura $G'=(V,T)$ per G, cioè un albero avente tutti i nodi di V, ma solo $n-1$ degli m archi in E, tale che la somma $\sum_{(u,v)\in T} w(u,v)$ sia la più piccola possibile.

In sintesi, dato un grafo pesato, determinare come interconnettere tutti i suoi nodi minimizzando il costo del peso associato ai suoi archi.

$w : V \times V \rightarrow R$ è una funzione di peso (ovvero il costo di connessione):

- Se $(u,v) \in E$, allora $w(u,v)$ è il peso dell'arco (u,v)
- Se $(u,v) \notin E$, allora $w(u,v)=+\infty$

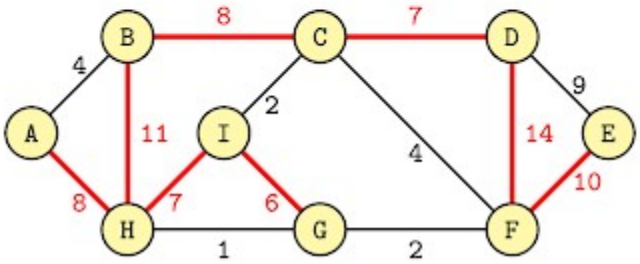
Poichè G non è orientato, $w(u,v)=w(v,u)$

Ricordiamo la definizione di **albero di copertura**:

Albero di copertura (Spanning tree)

Dato un grafo $G = (V, E)$ non orientato e connesso, un albero di copertura di G è un sottografo $T = (V, E_T)$ tale che

- T è un albero
- $E_T \subseteq E$
- T contiene tutti i vertici di G

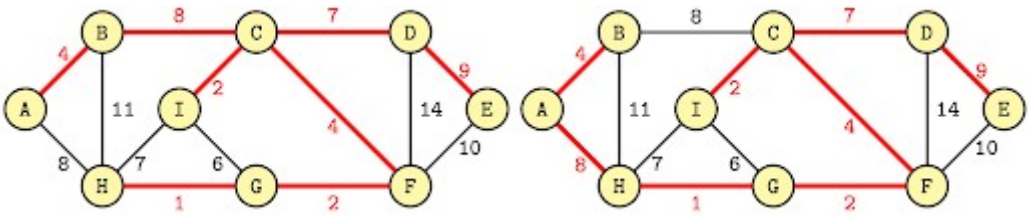


Dunque l'albero di copertura di un grafo G è un sottografo tale che sia un albero e contenga tutti i vertici del grafo originale G.

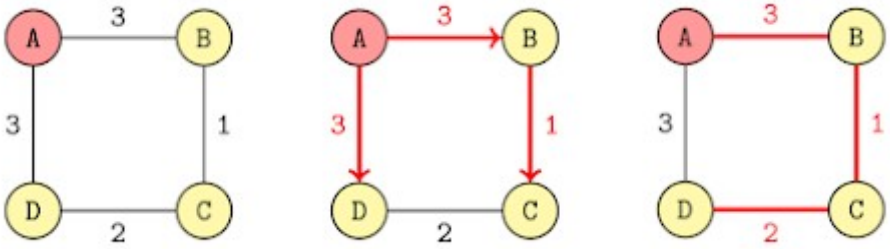
L'obiettivo è quello di trovare l'albero di copertura il cui peso totale sia minimo rispetto a ogni altro albero di copertura. Definiamo il peso dell'albero di copertura T come:

$$w(T)=\sum_{(u,v)\in E_T} w(u,v)$$

Non è detto però che l'albero di copertura minimo sia univoco:



Si può notare molto facilmente dal seguente esempio che l'albero dei cammini minimi da singola sorgente A (che è un albero di copertura) non coincide con l'albero di copertura di peso minimo:



Questo perché l'albero di copertura di peso minimo è definito sull'intero grafo e non a partire da un singolo nodo.

Questo problema può essere risolto con molti algoritmi, dei quali i più noti sono quelli di Kruskal (1956) e Prim (1957). Entrambi questi algoritmi sono basati sul principio **greedy** (prendono sempre l'arco di peso minimo); illustriamo prima un algoritmo ingordo generico, poi vediamo nel dettaglio gli algoritmi di Kruskal e Prim, e poi discutiamo i principi alla base della loro correttezza.

Algoritmo generico

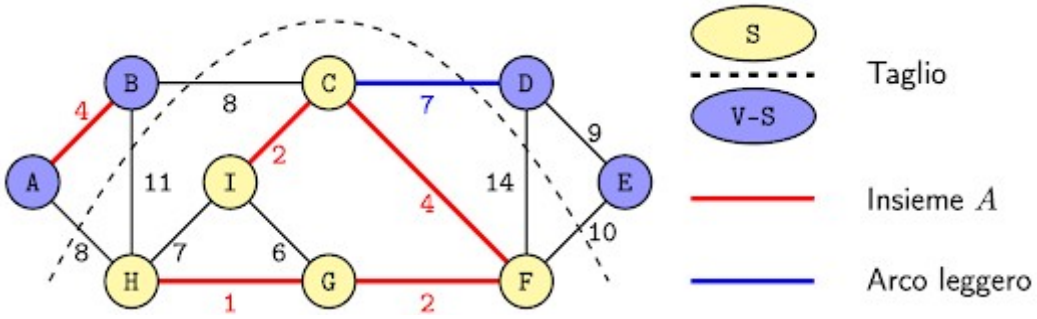
L'idea è di accrescere un sottoinsieme A di archi in modo tale che venga sempre rispettata la seguente invariante:

- A è un sottoinsieme di qualche albero di connessione minimo

Partiamo quindi dall'insieme vuoto e aggiungiamo piano piano gli archi.

Diamo alcune definizioni:

- Un **taglio** $(S, V - S)$ di un grafo non orientato $G = (V, E)$ è una partizione di V in due sottoinsiemi disgiunti
- Un arco (u, v) **attraversa** il taglio se $u \in S$ e $v \in V - S$
- Un taglio **rispetta** un insieme di archi A se nessun arco di A attraversa il taglio
- Un arco che attraversa un taglio è **leggero** nel taglio se il suo peso è minimo fra i pesi degli archi che attraversano il taglio



Definiamo **arco sicuro**, ovvero un arco che aggiunto ad A fa rimanere A un sottoinsieme di qualche albero di connessione minimo:

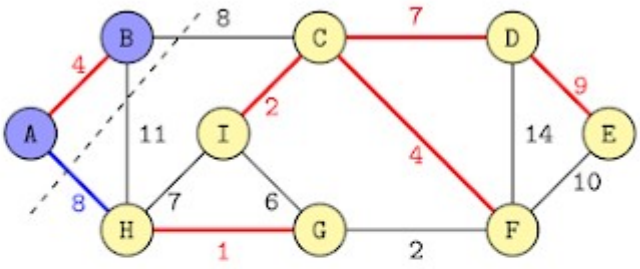
- Sia $G=(V,E)$ un arco non orientato e connesso
- Sia $w : V \times V \rightarrow R$ la funzione di peso per G
- Sia $A \subseteq E$ un sottoinsieme contenuto in un qualche albero di copertura minimo per G
- Sia $(S, V - S)$ un qualunque taglio che rispetti A
- Sia (u,v) un arco leggero che attraversi il taglio

Allora (u,v) è un **arco sicuro** per A

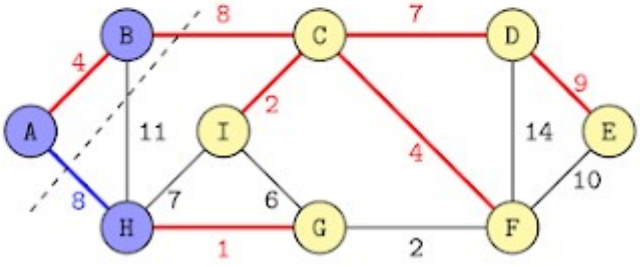
In altre parole il teorema dice: Sia A un sottoinsieme contenuto in qualche albero di copertura minimo per G e consideriamo un qualsiasi taglio che rispetti A. L'arco leggero per questo taglio allora è sicuro per A.

Esempio: arco non sicuro perché il taglio non rispetta A

Arco blu sicuro

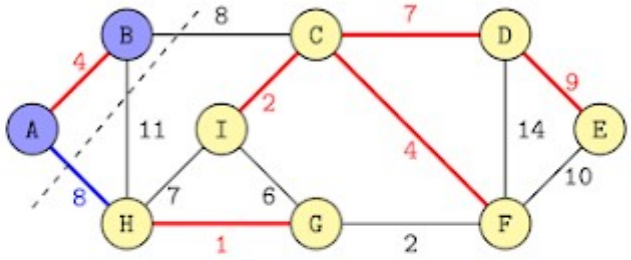


Arco blu non sicuro

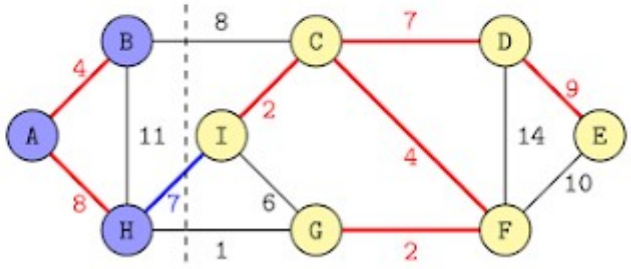


Esempio: arco non sicuro perché non leggero

Arco blu sicuro



Arco blu non sicuro

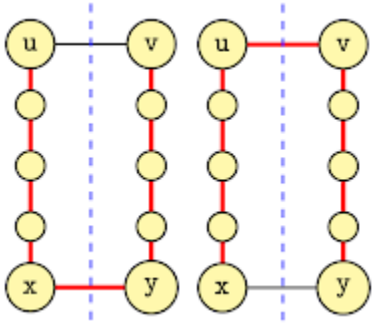


Dimostrazione:
Sia T un albero di copertura minimo che contiene A e sia (u,v) un arco che rispetta il teorema. Ci sono due casi:

- $(u,v) \in T$, allora (u,v) è sicuro per A
- $(u,v) \notin T$, allora trasformiamo T in un albero T' contenente (u,v) e dimostriamo che T' è un albero di copertura minimo

Per il secondo caso, quello che facciamo è togliere un arco specifico da T e aggiungere il nostro arco, mantenendo l'albero:

- Per definizione di albero di copertura, u e v sono connessi da un **cammino** $C \subseteq T$
- u e v, per ipotesi, si trovano in lati opposti del taglio (visto che (u,v) attraversa il taglio)
- Allora esiste sicuramente un arco $(x,y) \in C$ che attraversa il taglio (né possono esistere più di uno)
- Consideriamo $T' = T - \{(x,y)\} \cup \{(u,v)\}$
- T' è un albero di copertura, e vale $w(T') \leq w(T)$ perchè $w(u,v) \leq w(x,y)$, altrimenti (u,v) non sarebbe un arco leggero, però vale anche $w(T) \leq w(T')$ visto che T è minimo. Dunque il peso dei due archi è uguale e T' è anch'esso un albero di copertura minimo visto che è un albero di copertura e ha lo stesso peso dell'albero di copertura minimo



Dunque, operativamente, è necessario solamente aggiungere (u, v) ad A, in quanto (x, y) non fa sicuramente ancora parte di A, perché altrimenti il taglio eseguito non rispetterebbe A.

Un altro corollario per gli archi sicuri è:

- Sia $G=(V, E)$ un grafo non orientato e connesso
- Sia $w : V \times V \rightarrow R$ la funzione di peso per G
- Sia $A \subseteq E$ un sottoinsieme contenuto in un qualche albero di copertura minimo per G
- Sia C una componente connessa (un albero) nella foresta $G_A=(V, A)$
- Sia (u, v) un arco leggero che connette C a qualche altra componente in G_A (ovvero definisco il taglio con C da una parte e tutto il resto dall'altra e prendo l'arco leggero)

Allora (u, v) è un **arco sicuro** per A

Dimostrazione:
Definiamo un taglio $(C, V - C)$; per definizione, il taglio rispetta A. Poiché (u, v) attraversa il taglio, per il teorema precedente (u, v) è sicuro per A.

Algoritmo di Kruskal

L'idea dell'algoritmo di Kruskal è quella di ingrandire sottoinsiemi disgiunti di un albero di copertura minimo connettendoli fra di loro fino ad avere l'albero complessivo.
Dunque ad ogni passo si individua un arco sicuro scegliendo un arco (u, v) di peso minimo tra tutti gli archi che connettono due distinti alberi (componenti connesse) della foresta.
L'algoritmo è greedy perché ad ogni passo si aggiunge alla foresta un arco con il peso minore.

In particolare l'algoritmo analizza gli archi in ordine crescente di peso; un generico arco è aggiunto all'albero T se non forma circuiti con gli archi inseriti in T.

Il maggior sforzo computazionale dell'algoritmo di Kruskal è dato dall'ordinamento iniziale e dalla verifica che l'aggiunta di un nuovo arco non provochi la formazione di un circuito.
L'ordinamento iniziale degli archi costa $O(m \log m)$ tempo, che diventa $O(m \log n)$, dato che $\log m \leq \log n^2 = 2 \log n$.
Inoltre, poiché la costruzione di T avviene per unione di insiemi disgiunti, cioè di due componenti connesse che si "fondono" in una sola con l'aggiunta del nuovo arco, è possibile utilizzare una struttura merge-find $MFSET$ per mantenere le componenti connesse di T, inizializzato con n componenti distinte, ciascuna formata da un singolo nodo del grafo. In questo modo, la verifica che un arco appartenga a due componenti distinte (cioè non provochi un circuito in T) ed anche il suo eventuale inserimento in T richiedono tempo ammortizzato costante $O(1)$ usando le operazioni $find()$ e $merge()$. Poiché il ciclo **for** è ripetuto m volte, la complessità dell'algoritmo è data dall'ordinamento: $O(m \log n)$.

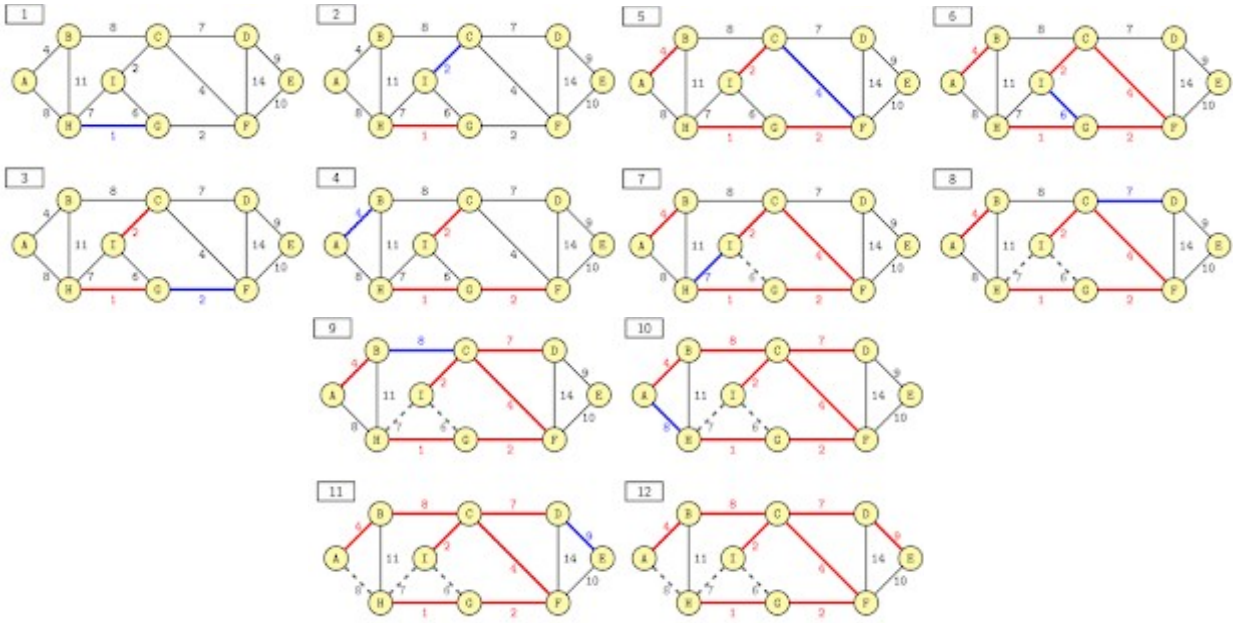
Per semplificare l'algoritmo di Kruskal, assumiamo che prenda in input un vettore di record $ARCO$, contenente i campi u, v e $peso$, corrispondenti agli estremi dell'arco e al suo peso.

Data l'usuale rappresentazione di un grafo con vettori di adiacenza, è facile vedere che il vettore degli archi può essere costruito in $O(n+m)$ tempo. L'albero T è restituito come un insieme di archi, che può essere realizzato con una lista non ordinata, in modo che l'inserimento di un arco in T richieda tempo $O(1)$.

Per risparmiare iterazioni inutili dopo che è stata raggiunta la soluzione ottima, la procedura *kruskal*() introduce una variabile intera c che conta il numero di inserimenti di archi nell'albero di copertura T, uscendo dal ciclo appena sono stati effettuati n - 1 inserimenti. L'ordine di grandezza della complessità resta comunque $O(m \log n)$.

```
SET kruskal(EDGE[] A, int n, int m)
SET T = Set()
MFSET M = Mfset(n)
{ ordina A[1,...,m] in modo che A[1].weight ≤ ··· ≤ A[m].weight }
int count = 0
int i = 1
% Termina quando l'albero ha n − 1 archi o non ci sono più archi
while count < n − 1 and i ≤ m do
    if M.find(A[i].u) ≠ M.find(A[i].v) then
        M.merge(A[i].u, A[i].v)
        T.insert(A[i])
        count = count + 1
    i = i + 1
return T
```

L'ordinamento serve a garantirmi che analizzerò sempre gli archi di peso minore, l'*MFSET* serve a garantirmi che l'arco attraversi il taglio.



Il tempo di esecuzione per l'algoritmo di Kruskal dipende dalla realizzazione della struttura dati per Merge-Find Set, utilizzando la versione con euristica sul rango + compressione, le cui operazioni hanno costo ammortizzato costante:

- Inizializzazione $MFSET$: 1 volta con costo $O(n)$
- Ordinamento archi: 1 volta con costo $O(m \log n)$
- Operazioni $find()$ e $merge()$: $O(m)$ volte con costo $O(1)$ (per ogni arco 2 operazioni di $find()$ e forse una di $merge()$)

Il costo totale quindi è $O(n+m \log n+m)=O(m \log n)$

Algoritmo di Prim

L'algoritmo di Prim, invece, procede mantenendo in A un singolo albero (l'albero viene allargato ad ogni passo, è più simile a Dijkstra, solo che il peso è quello effettivo degli archi, e non la somma dalla radice).
L'albero parte da un vertice arbitrario r (la radice) e cresce fino a quando non ricopre tutti i vertici.
Ad ogni passo viene aggiunto un arco leggero che collega un vertice in V_A con un vertice in V - V_A, dove V_A è l'insieme di nodi raggiunti da archi in A.

La correttezza è verificata in quanto $(V_A, V - V_A)$ è un taglio che rispetta A, e per il corollario gli archi leggeri che attraversano il taglio sono sicuri.

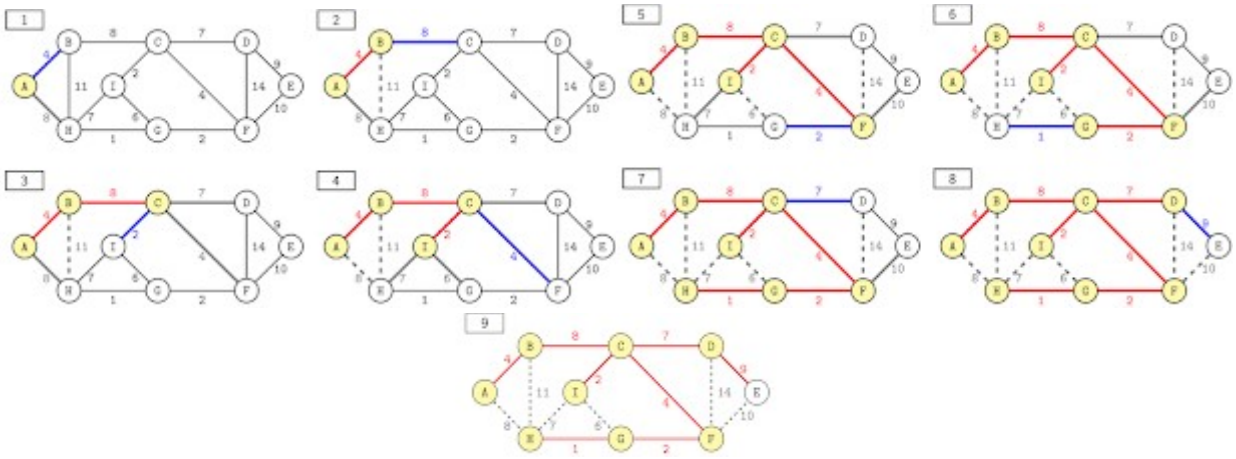
L'algoritmo di Prim parte da un albero T contenente uno specifico nodo radice r, e accresce T aggiungendo sempre l'arco con peso minore fra quelli che "escono" da T, ovvero fra quelli che uniscono un nodo di T con un nodo non incluso in T.

L'algoritmo prende in input un grafo G e un nodo radice r; come nel caso dei cammini minimi, la procedura ha accesso ad una funzione di costo w, lasciando alla realizzazione effettiva l'onere di definire dove debbano essere memorizzati tali costi. L'albero T viene rappresentato da un vettore di padri p, e A è mantenuto implicitamente proprio in p.

La struttura dati utilizzata in questo caso è una coda con priorità (min-heap). Tutti i nodi vengono inseriti nella coda con priorità $+\infty$, tranne il nodo r che ha priorità 0. La priorità del nodo u corrisponde al peso minimo di un arco che raggiunge u a partire da un nodo compreso in T, mentre $pos[u]$ è la posizione che u occupa nello heap. Quando un nodo u viene estratto dalla coda, $[p[u], u]$ rappresenta l'arco con peso minore fra quelli che escono dall'albero; sia l'arco $[p[u], u]$ che il nodo u entrano a far parte dell'albero T, indicato con $pos[u]=nil$. Tutti gli archi incidenti ad u vengono analizzati, e se permettono di raggiungere un nuovo nodo v non ancora nell'albero ($pos[v] \neq nil$) con costo inferiore, la priorità nella coda viene aggiornata, così come il vettore dei padri.

Utilizzando una coda con priorità realizzata tramite heap binario, il ciclo di inserimento dei nodi nella coda con priorità viene eseguito n volte, ognuna con costo $O(\log\ n)$; il ciclo principale viene eseguito n volte, ognuna con costo delete $O(\log\ n)$ per l'estrazione; il ciclo esterno viene eseguito m volte, ognuna con costo di increase priority $O(\log\ n)$. La complessità è quindi $O(m\ \log\ n)$, asintoticamente uguale all'algoritmo di Kruskal.

```
int[] prim(GRAPH G, NODE r)
PRIORITYQUEUE Q = MinPriorityQueue()
PRIORITYITEM[] pos = new PRIORITYITEM[1...G.n]
int[] p = new int[1...G.n]
foreach u ∈ G.V() − {r} do
    pos[u] = Q.insert(u, +∞)
pos[r] = Q.insert(r, 0)
p[r] = 0
while not Q.isEmpty() do
    NODE u = Q.deleteMin()
    pos[u] = nil
    foreach v ∈ G.adj(u) do
        if pos[v] ≠ nil and w(u, v) < pos[v].priority then
            Q.decrease(pos[v], w(u, v))
            p[v] = u
return p
```



Il tempo di esecuzione per l'algoritmo di Prim dipende dalla realizzazione della coda con priorità, utilizzando heap binario:

- Inizializzazione della coda con priorità: 1 volta con costo $O(n\ \log\ n)$
- Operazione $deleteMin()$: $O(n)$ volte con costo $O(\log\ n)$

- Operazione $decreasePriority()$: $O(m)$ volte con costo $O(\log\ n)$

Il costo totale quindi è $O(n+n\ \log\ n+m\ \log\ n)=O(m\ \log\ n)$, ovvero asintoticamente uguale a quello di Kruskal.

È possibile dare una versione dell'algoritmo di Prim che richiede tempo $O(n^2)$, sostituendo la struttura min-heap con una lista o un vettore non ordinati, analogamente a quanto visto per l'algoritmo di Dijkstra per i cammini minimi.
Utilizzando un vettore, l'operazione $deleteMin()$ ha costo $O(n)$ mentre l'operazione $decrease()$ ha costo $O(1)$. Il costo diventa $O(n^2)$, che è migliore di $O(m\ \log\ n)$ per grafi densi (dove $m=\Theta(n^2)$) mentre quella con min-heap è migliore per grafi sparsi (dove $m=O(n)$).

Utilizzando uno heap di Fibonacci al posto di un min-heap nella realizzazione della coda con priorità, l'operazione $deleteMin()$ ha costo $O(\log\ n)$ mentre l'operazione $decrease()$ ha costo $O(1)$. Il costo diventa $O(m+n\ \log\ n)$.

È possibile che vengano ottenuti alberi diversi utilizzando l'algoritmo di Kruskal e l'algoritmo di Prim, però sicuramente entrambi avranno lo stesso peso essendo minimi.

Facciamo ora delle osservazioni (soprattutto per quanto riguarda Kruskal):

- Il primo arco di peso minimo è sicuramente sicuro perchè fa sicuramente parte del minimo albero di copertura
- Il secondo arco di peso minimo è anch'esso sicuro, sia nel caso in cui sia staccato sia nel caso in cui sia attaccato (questo perchè il grafo è non orientato e quindi non può esistere un ciclo con due archi)
- Il terzo arco di peso minimo invece non è detto che sia sicuro

Un problema più o meno correlato a questo è quello dell'albero di copertura minima di un piano, in cui basta utilizzare uno di questi due algoritmi usando la distanza euclidea dei punti come peso (da non confondere con gli Steiner Tree in cui si ha la possibilità di aggiungere punti intermedi).

Applicazioni dirette per la progettazione

- Reti di telecomunicazione
- Reti idriche
- Reti di trasporto
- Reti elettriche

Alcuni utilizzi particolari

- Segmentazione di immagini
- Riconoscimento scrittura manuale
- Disegno di circuiti elettronici
- Progettazione tassonomie

Ecco la prospettiva storica per questo problema:

- $O(m \log n)$:
  - Primo algoritmo: Boruvka (1926)
  - Kruskal (1956)
  - Prim (1957), ma anche Jarnik (1930)
- $O(m + n \log n)$:
  - Fredman-Tarjan (1987)
  - Modifica di Prim che utilizza gli heap di Fibonacci
- $O(m + n)$:
  - Algoritmo probabilistico di Karger, Klein, Tarjan (1995)
  - Vari algoritmi in tempo lineare per casi particolari
  - Questione aperta se si possa risolvere il problema in tempo lineare deterministico

Dunque grazie all'heap di Fibonacci si sono scoperti algoritmi con complessità $O(m+n \log n)$, però l'heap di Fibonacci costa molto, quindi nella pratica non vengono utilizzati.

Si sono anche scoperti algoritmi probabilistici che costano $O(m+n)$, però non si conosce ancora nessun algoritmo deterministico con questo costo.

In conclusione sugli algoritmi Greedy possiamo dire:

Vantaggi

- Semplici da programmare
- Molto efficienti
- Quando è possibile dimostrare la proprietà di scelta ingorda, danno la soluzione ottima
- La soluzione sub-ottima può essere accettabile

Svantaggi

- Non sempre applicabili se si vuole la soluzione ottima

Ricerca locale

Talvolta, anche una strategia estremamente “miope” può dare buoni risultati:

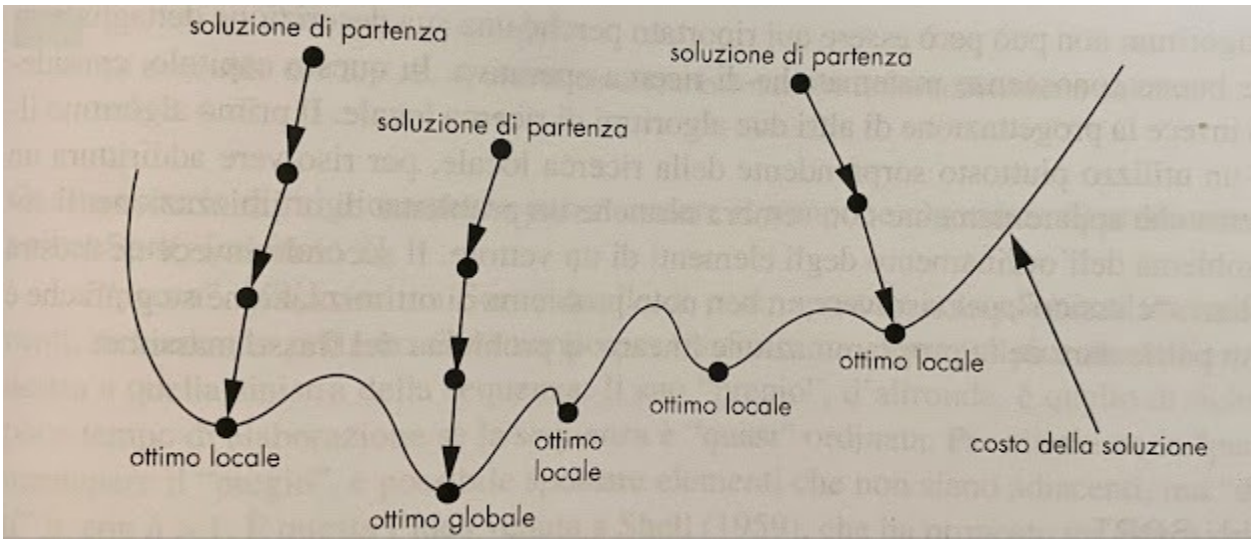
“Coltiviamo senza guardare più in là del proprio orticello”

In altri termini, se si conosce una soluzione ammissibile di un **problema di ottimizzazione**, si può cercare di trovarne una migliore che sia in qualche modo “vicina” a quella precedente.

Data una soluzione ammissibile Sol del problema (non necessariamente ottima), si definisce un **intorno** $I(Sol)$ di questa soluzione come un insieme di soluzioni ammissibili che sono ottenibili da essa in base a certi criteri di trasformazione. Si può successivamente sostituire la soluzione con una che fa parte del suo intorno ed è “migliore” di essa, e ripetere il procedimento finché non si riesce più a migliorare. Tale strategia è detta **ricerca locale** e gli algoritmi basati su di essa hanno la seguente struttura generale:

```
ricercaLocale()
Sol = una soluzione ammissibile del problema
while ∃S ∈ I(Sol) migliore di Sol do
    Sol = S
return Sol
```

In generale, il procedimento di ricerca locale si arresta su una soluzione che è un ottimo locale, cioè che non può essere migliorata attraverso le trasformazioni possibili, ma che può non essere l’ottimo globale del problema (esistono dei meccanismi per assicurarsi di ottenere l’ottimo globale). Inoltre, partendo da soluzioni iniziali diverse si possono raggiungere ottimi locali diversi, come si vede nell’immagine.



Ovviamente, se $I(Sol)$ comprende tutte le soluzioni ammissibili del problema, allora ogni ottimo locale è anche un ottimo globale, e la ricerca locale dà sempre la soluzione ottima del problema. Una tale ricerca, però, ha senso se la cardinalità di $I(Sol)$ è molto più piccola della cardinalità dell'insieme di tutte le soluzioni ammissibili, e tale restrizione sulla cardinalità di $I(Sol)$ può far arrestare la ricerca su un ottimo locale che non è quello globale.

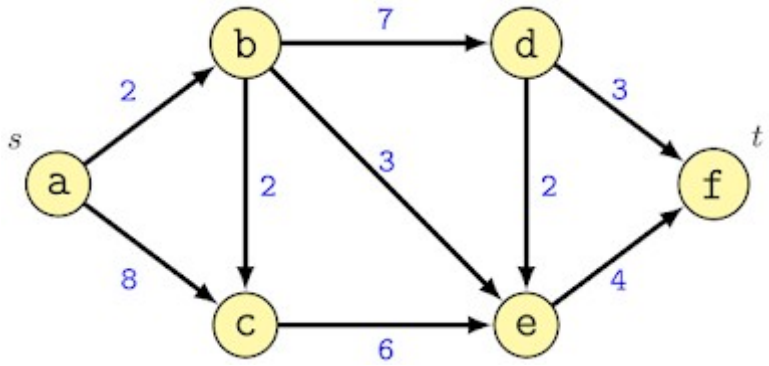
Per far si che l'intero procedimento abbia complessità polinomiale, occorre che la ricerca della soluzione S nell'intorno $I(Sol)$ abbia complessità polinomiale e che il numero complessivo di passi di "miglioramento", cioè di ripetizioni del ciclo **while**, sia anch'esso polinomiale. Per alcuni problemi, ma non per tutti, è possibile definire intorni di cardinalità "piccola" per i quali ogni ottimo locale è anche un ottimo globale che inoltre può essere trovato in tempo polinomiale con una ricerca locale.

Algoritmi di ricerca locale nel caso in cui non venga con certezza trovato un minimo globale ma solo locale, possono comunque essere utili nel caso in cui si stia cercando solamente una soluzione ragionevole.

Flusso massimo

La soluzione che vedremo per questo problema ci porterà sempre ad ottenere l'ottimo globale.

Una **rete di flusso** $G=(V,E,s,t,c)$ è data da un grafo orientato $G=(V,E)$, da una coppia di vertici V detti **sorgente** s e **pozzo** t, e da una funzione di **capacità** a valori interi positivi $c : V \times V \to Z^+ \cup \{0\}$, tale per cui $c(u,v)=0$ se $(u,v) \notin E$.
Vengono fatte le assunzioni che per ogni nodo $x \in V$, esiste un cammino $s \Rightarrow x \Rightarrow t$ da s a t che passa per x, altrimenti basterebbe ignorare il nodo x durante l'esecuzione dell'algoritmo.



Un **flusso** in G è una funzione a valori interi $F : V \times V \rightarrow Z$ che soddisfa le seguenti proprietà:

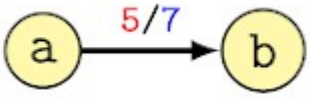
1) **Vincolo di capacità**

   $f(u,v) \leq c(u,v)$ per ogni coppia $u, v \in V$

Vincolo sulla capacità

Il flusso non deve eccedere la capacità sull'arco.

$$\forall x, y \in V : f(x,y) \leq c(x,y)$$



2) **Simmetria opposta**

   $f(u,v) = -f(v,u)$ per ogni coppia $u, v \in V$

Simmetria opposta

Il flusso che attraversa un arco in direzione (x, y) è l'opposto del flusso che attraversa l'arco in direzione (y, x).

$$\forall x, y \in V : f(x,y) = -f(y,x)$$



Il flusso viene definito in questo modo per semplificare la proprietà successiva e altre regole

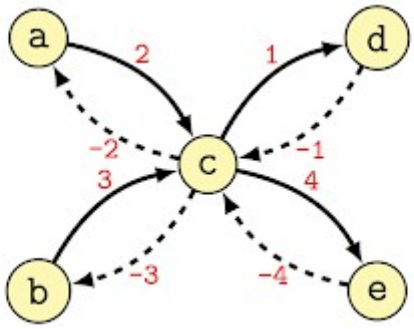
3) **Conservazione del flusso**

$\sum_{v \in V} f(u,v) = 0$ per ogni nodo $u \in V - \{s,t\}$

Conservazione del flusso

Per ogni nodo diverso da sorgente e pozzo, la somma dei flussi entranti deve essere uguale alla somma dei flussi uscenti.

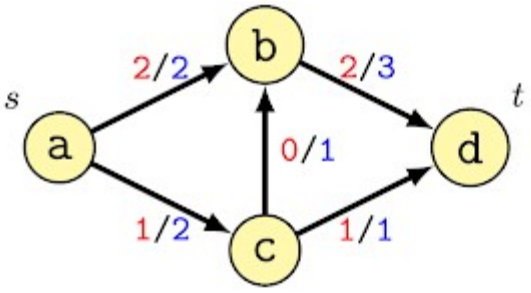
$$\forall x \in V - \{s,t\} : \sum_{y \in V} f(x,y) = 0$$



La quantità $f(u,v)$, che può essere positiva e negativa, è detta **flusso netto** dal nodo u al nodo v, mentre il **valore** $|f|$ **del flusso** f è il flusso netto totale che esce dalla sorgente (o che entra nel pozzo), ovvero

$$|f| = \sum_{v \in V} f(s,v) = \sum_{u \in V} f(u,t)$$

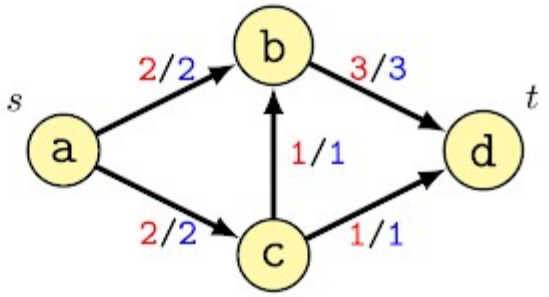
$|f| = 3$



Il problema del **flusso massimo** è definito nel modo seguente:
Data una rete di flusso $G = (V, E, s, t, c)$, trovare un flusso f^* che ha valore $|f^*|$ più grande possibile

$$|f^*| = max(|f|)$$

$|f^*| = 4$



Si osservi che la proprietà di simmetria opposta implica che $f(u,u)=0$ per ogni $u \in V$.
La stessa proprietà e i vincoli di capacità implicano che se sia $(u,v) \in E$ e $(v,u) \in E$, allora $f(u,v)=f(v,u)=0$. Pertanto, un flusso netto $f(u,v)\neq 0$ implica che $(u,v) \in E$ oppure $(v,u) \in E$.
Infine, la conservazione del flusso è la stessa legge della corrente elettrica di Kirchoff che afferma che il flusso netto totale che entra in un nodo (esclusi la sorgente ed il pozzo) eguaglia il flusso netto totale che esce dallo stesso nodo.

Vediamo il **metodo delle reti residue**.
L'algoritmo (informale) è il seguente:

- Si memorizza un flusso "corrente" f (inizialmente nullo)
- Si ripetono le operazioni seguenti
  - Si "sottrae" il flusso attuale dalla rete iniziale, ottenendo una rete residua
  - Si cerca un flusso g all'interno della rete residua
  - Si somma g ad f

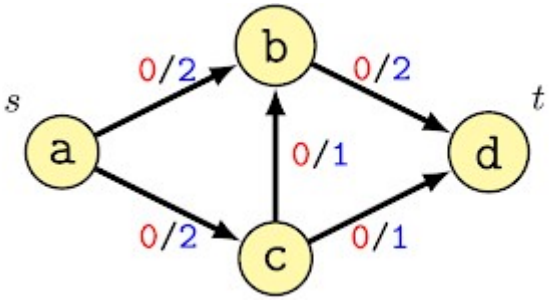
E ci si ferma quando non è più possibile trovare un flusso positivo g.

È possibile dimostrare (lo vedremo dopo) che questo approccio restituisce un flusso massimo.

Definiamo **flusso nullo** la funzione $f_0 : V \times V \rightarrow Z^+ \cup \{0\}$ tale che

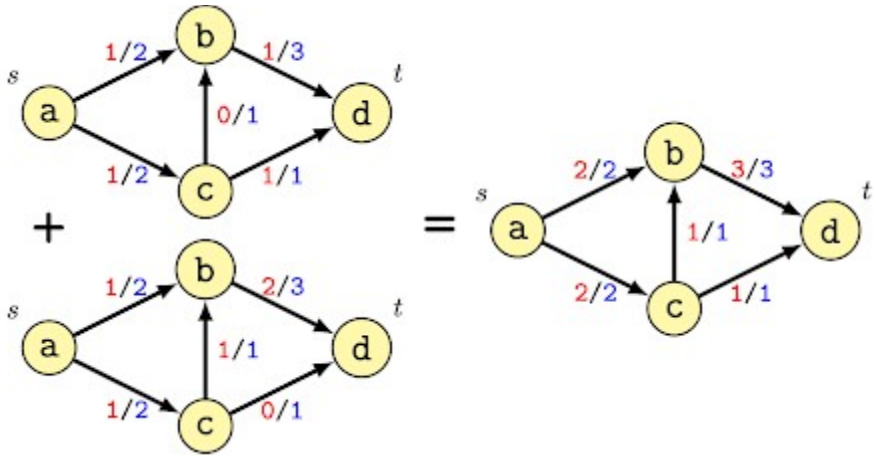
$$f(x,y)=0 \text{ per ogni } x,y \in V$$

$|f| = 0$



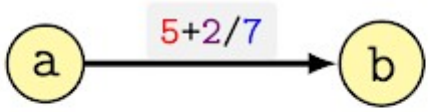
Per ogni coppia di flussi f_1 e f_2 in G, definiamo il **flusso somma** $g=f_1+f_2$ come un flusso tale che

$$\forall x,y \in V : g(x,y)=f_1(x,y)+f_2(x,y)$$



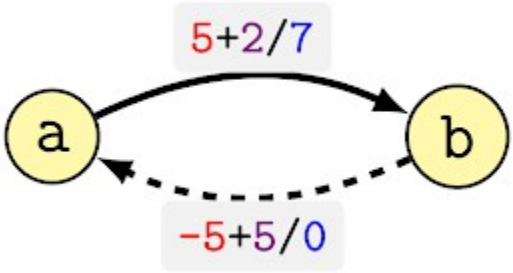
Definiamo **capacità residua** di un flusso f in una rete $G=(V,E,s,t,c)$ una funzione $c_f : V \times V \rightarrow Z^+ \cup \{0\}$ tale che

$$\forall x,y \in V : c_f(x,y)=c(x,y)-f(x,y)$$



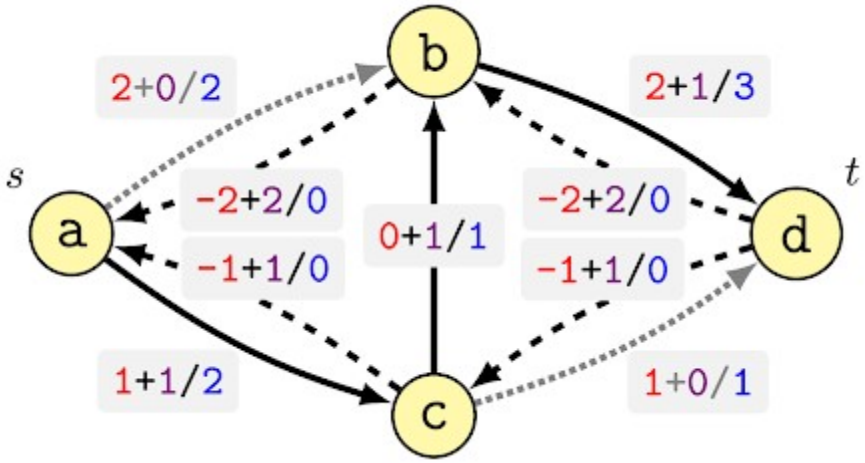
- Flusso in rosso
- Capacità residua in viola
- Capacità iniziale in blu

Si noti che in un flusso non nullo, la capacità residua in una coppia di nodi ha valore positivo o nullo sia sull'arco in avanti, che sull'arco di ritorno (per la definizione di flusso con valori negativi sull'arco di ritorno):



Data una rete di flusso $G=(V,E,s,t,c)$ e un flusso f su G, possiamo costruire una **rete residua** $G_f=(V,E_f,s,t,c_f)$, tale per cui:

$$\forall x,y \in V : (x,y) \in E_f \Leftrightarrow c_f(x,y)>0$$



Dunque le nuove capacità diventano le capacità residue.

La rete residua è una rete sugli stessi vertici del grafo originale. Per quanto riguarda gli archi, se un arco (u,v) ha un flusso associato $f(u,v)$ tale che $0<f(u,v)<c(u,v)$, allora questo arco si sdoppierà in G_f in:

- un arco (u,v) con capacità $c_f(u,v)=c(u,v)-f(u,v)$
- un arco (v,u) con capacità $c_f(v,u)=c(v,u)-f(v,u)=c(v,u)+f(u,v)=f(u,v)$

Il grafo G_f avrà al più il doppio degli archi di G. Si noti che se $f=f_0$, allora $G_f=G$.

Lo schema generale dell'algoritmo che andiamo a scrivere è il seguente:

```
int[][] maxFlow(GRAPH G, NODE s, NODE t, int[][] c)
f = f_0                                  % Inizializza un flusso nullo
c_f = c                                  % Capacità iniziale
repeat
    g = trova un flusso in c_f tale che |g| > 0, altrimenti f_0
    f = f + g
    c_f = Capacità residua del flusso f in G
until g = f_0
return f
```

Dobbiamo dimostrare il fatto che se f è un flusso in G e g è un flusso in G_f, allora $f+g$ è un flusso in G.
Per dimostrare che $f+g$ sia ancora un flusso in G, dobbiamo verificare che valgano le tre proprietà, ricordando che $G_f=(V,E_f,s,t,c_f)$, $\forall x,y\in V\ :\ (x,y)\in E_f \Leftrightarrow c_f(x,y)>0$ e $\forall x,y\in V\ :\ c_f(x,y)=c(x,y)-f(x,y)$:

- **Conservazione del flusso**

  Per ogni $x \in V - \{s,t\}$ deve valere $\sum_{y \in V} (f+g)(x,y) = 0$

  $$\sum_{y \in V} (f+g)(x,y) = \sum_{y \in V} \big(f(x,y)+g(x,y)\big) = \sum_{y \in V} f(x,y) + \sum_{y \in V} g(x,y) = 0+0=0$$
- **Simmetria opposta**

  Per l'antisimmetria di f e di g sappiamo che

  $f(x,y)+g(x,y) = -f(y,x)-g(y,x)$

  $f(x,y)+g(x,y) = -\big(f(y,x)+g(y,x)\big)$

  $(f+g)(x,y) = -\big((f+g)(y,x)\big)$
- **Vincolo di capacità**

  Sappiamo che g è un flusso in G_f dunque

  $g(x,y) \le c_f(x,y)$

  Sostituiamo $c_f(x,y)$ con $c(x,y)-f(x,y)$ che vale per definizione

  $g(x,y) \le c(x,y)-f(x,y)$

  Sommiamo da entrambi i lati il termine $f(x,y)$

  $f(x,y)+g(x,y) \le c(x,y)-f(x,y)+f(x,y)$

  E otteniamo

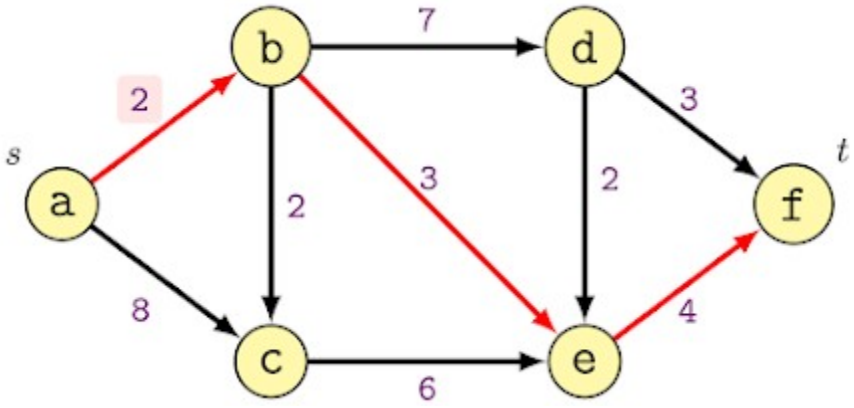
  $(f+g)(x,y) \le c(x,y)$

Dunque l'algoritmo ritornerà sicuramente un flusso.

Il problema principale dell'algoritmo che abbiamo proposto però è **come trovare un flusso aggiuntivo**?
L'algoritmo più semplice, proposto da Ford e Fulkerson (1956), richiede semplicemente di trovare un cammino aumentante qualsiasi, utilizzando un qualunque algoritmo di visita.

Si trova dunque un cammino $C = v_0, v_1, \ldots, v_n$ con $s = v_0$ e $t = v_n$ nella rete residua G_f, e si definisce **la capacità del cammino** come la minor capacità degli archi incontrati (collo di bottiglia):

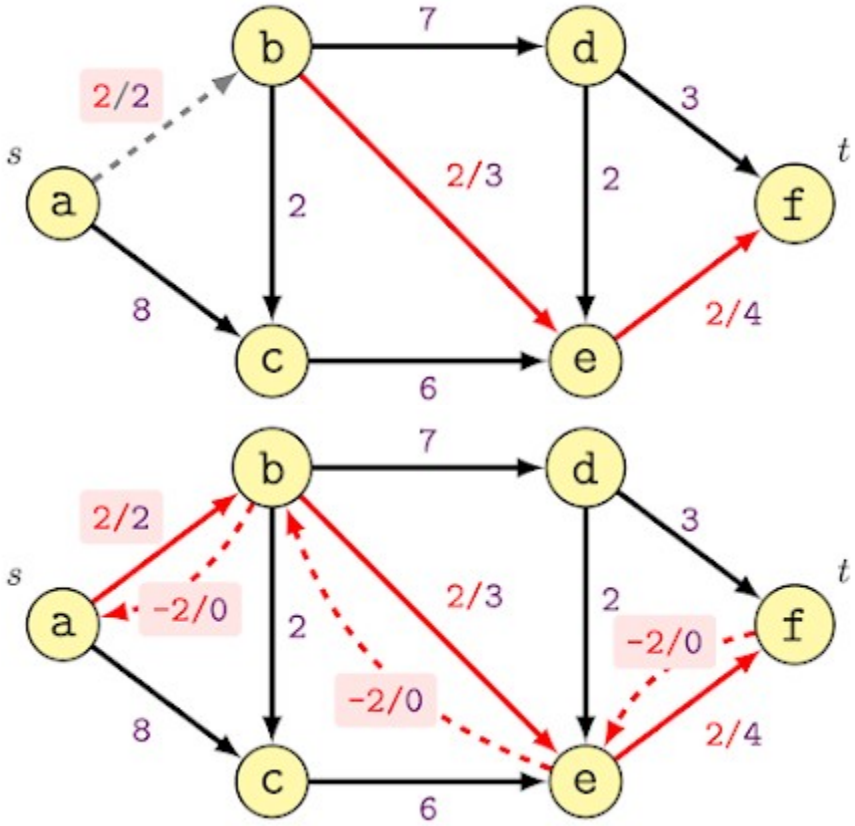
$$c_f(C) = min_{i=2\ldots n}\, c_f(v_{i-1}, v_i)$$



Si noti che se esiste un cammino, $c_f(C) > 0$ perché gli archi con capacità residua 0 sono già stati eliminati dal grafo.

Successivamente si crea un flusso addizionale g tale che

- $g(v_{i-1}, v_i) = c_f(C)$
- $g(v_i, v_{i-1}) = -c_f(C)$ (per antisimmetria)
- $g(x, y) = 0$ per tutte le altre coppie (x, y)



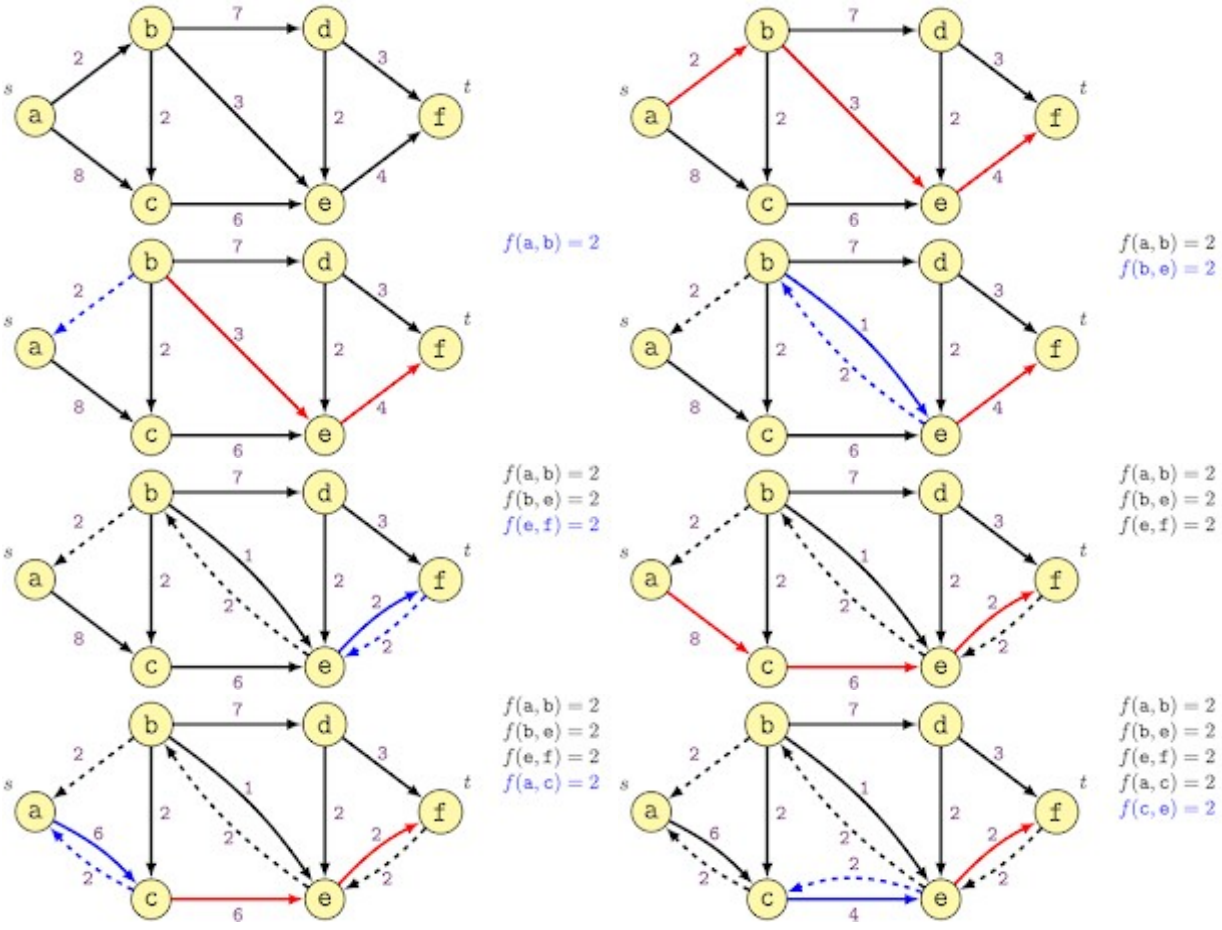
Alcune annotazioni:

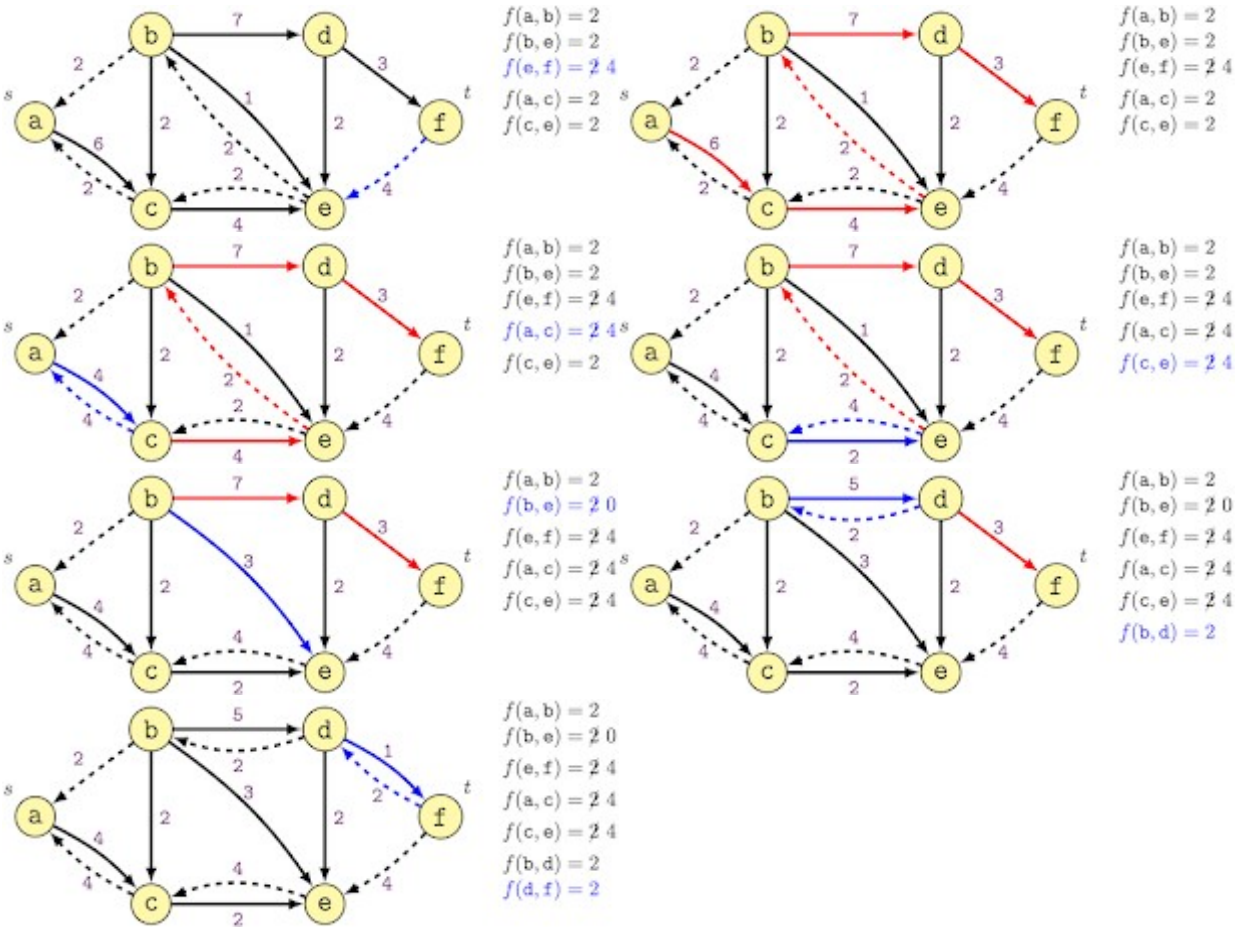
- La scelta del cammino è importante; scelte diverse portano ad algoritmi con prestazioni diverse.
- Il valore negativo assegnato al cammino inverso può potenzialmente “invertire” un flusso già assegnato; questo permette di migliorare una soluzione già trovata, e correggerla (ricerca locale).

L’algoritmo completo è il seguente:

```
int[][] maxFlow(GRAPH G, NODE s, NODE t, int[][] c)
int[][] f = new int[][]                          % Flusso parziale
int[][] g = new int[][]                          % Flusso da cammino aumentante
int[][] c_f = new int[][]                        % Rete residua
foreach x, y ∈ G.V() do
    f[x][y] = 0                                  % Inizializza un flusso nullo
    c_f[x][y] = c[x][y]                          % Copia c in r
repeat
    g = flusso associato ad un cammino aumentante in r, oppure f_0
    foreach x, y ∈ G.V() do
        f[x][y] = f[x][y] + g[x][y]              % f = f + g
        c_f[x][y] = c[x][y] − f[x][y]            % Calcola c_f
until g = f_0
return f
```

Gli archi all'indietro sono fondamentali per il corretto funzionamento dell'algoritmo come vediamo in questo esempio di esecuzione:





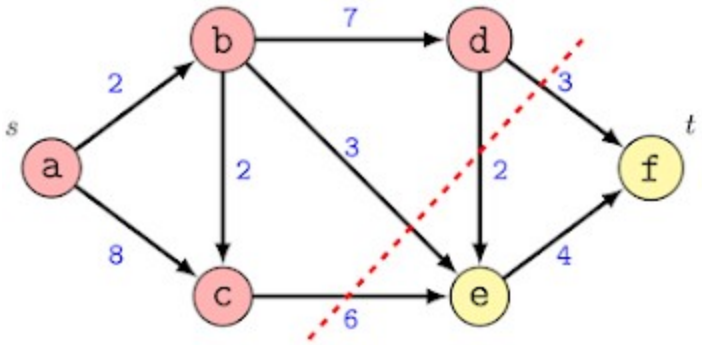
Ford e Fulkerson (1956) parlavano genericamente di una visita per trovare un cammino.
Edmonds e Karp (1972) suggerirono di utilizzare una visita in ampiezza.
La complessità della visita è dunque $O(V+E)$.

Abbiamo trattato solo il problema con sorgente unica e pozzo unico. Ma è semplice estendere questi algoritmi ai casi in cui ci siano più sorgenti e pozzi. E' sufficiente (i) aggiungere una super sorgente, collegata con un arco uscente a tutte le sorgenti; (ii) aggiungere un super pozzo, collegato con un arco entrante a tutti i pozzi. Tutti gli archi aggiunti hanno capacità infinite.

Dimostrazione correttezza

Per dimostrare che il nostro algoritmo termina correttamente, dobbiamo dimostrare che il flusso è massimo se e solo se non esiste un cammino da s a p nella rete residua. Per farlo, abbiamo bisogno ancora di alcune definizioni.
Un **taglio** (S,T) della rete di flusso $G=(V,E,s,t,c)$ è una partizione di V in due sottoinsiemi disgiunti S,T tali che:

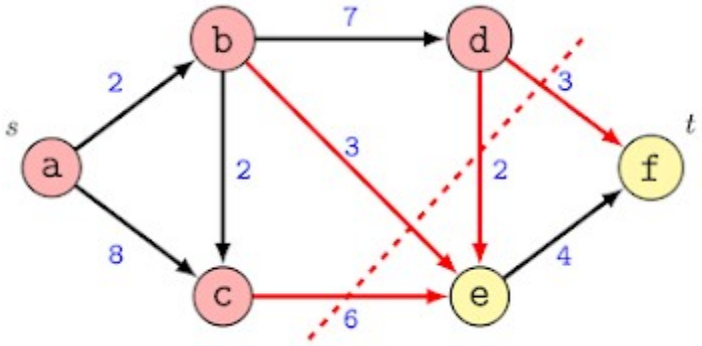
$$S = V - T$$
$$s \in S \wedge t \in T$$



$S = \{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{d}\}$
$T = \{\mathtt{e}, \mathtt{f}\}$

La **capacità** $C(S,T)$ **di un taglio** (S,T) è pari a:

$$C(S,T) = \sum_{x \in S, y \in T}^{\square} c(x,y)$$

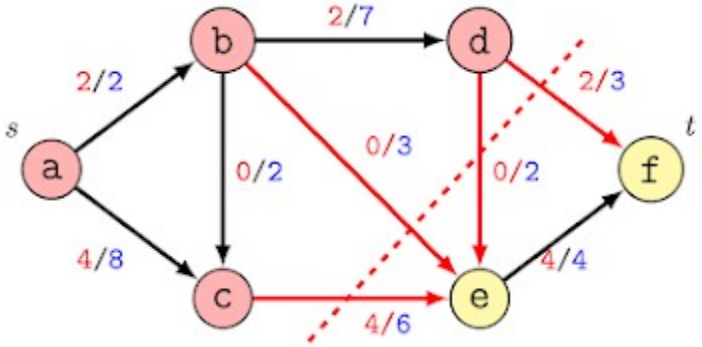


$S = \{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{d}\}$
$T = \{\mathtt{e}, \mathtt{f}\}$

$C(S,T) = 14$

Se f è un flusso in G, il **flusso netto** $F_f(S,T)$ attraverso il taglio (S,T) è:

$$F_f(S,T) = \sum_{x \in S, y \in T}^{\square} f(x,y)$$



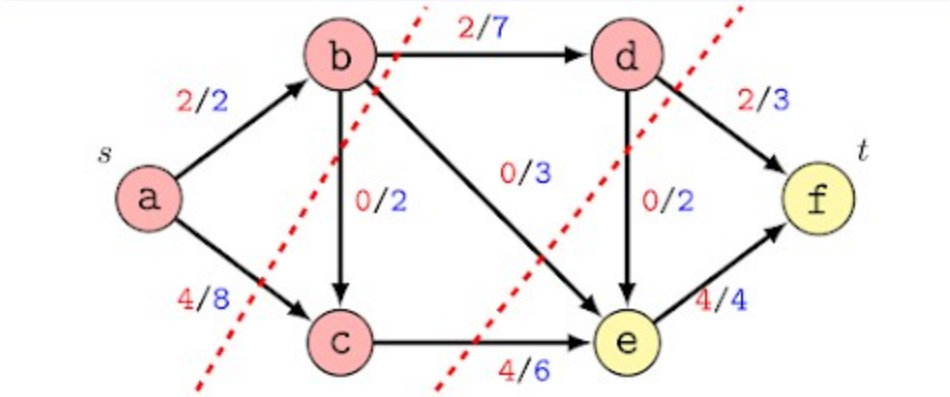
$S = \{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{d}\}$
$T = \{\mathtt{e}, \mathtt{f}\}$

$C(S,T) = 14$

$F_f(S,T) = 6$

Si noti che il flusso include valori positivi e negativi, mentre la capacità include solo valori positivi.

Vediamo il **lemma del valore del flusso di un taglio**.
Dato un flusso f e un taglio (S,T), la quantità di flusso $F_f(S,T)$ che attraversa il taglio è uguale a $|f|$.



Infatti abbiamo che

$$F_f(S,T) = \sum_{x \in S, y \in T} f(x,y)$$

Visto che $T = V - S$ vale

$$F_f(S,T) = \sum_{x \in S, y \in V} f(x,y) - \sum_{x \in S, y \in S} f(x,y)$$

Per l'antisimmetria vale $\sum_{x \in S, y \in S} f(x,y) = 0$ e quindi

$$F_f(S,T) = \sum_{x \in S, y \in V} f(x,y)$$

Visto che $s \in S$ abbiamo

$$F_f(S,T) = \sum_{x \in S} \left(\sum_{y \in V} f(x,y) \right)$$

$$F_f(S,T) = \sum_{x \in S-\{s\}} \left(\sum_{y \in V} f(x,y) + \sum_{y \in V} (s,y) \right)$$

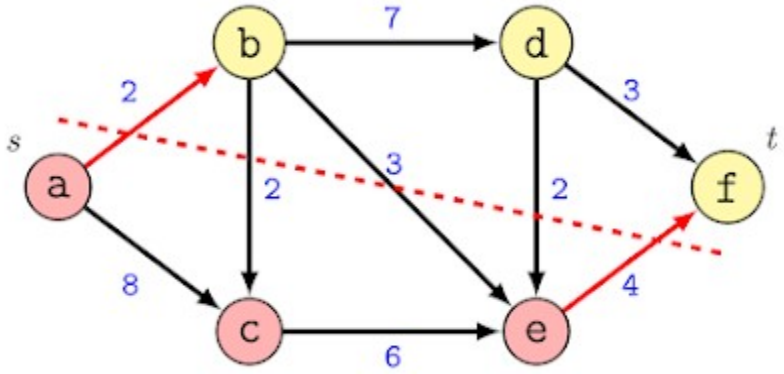
Per la conservazione del flusso abbiamo

$$F_f(S,T) = \sum_{x \in S-\{s\}} (0) + \sum_{y \in V} (s,y)$$

E quindi

$$F_f(S,T) = \sum_{y \in V} (s,y) = |f|$$

Vediamo il **lemma della capacità del taglio**.
Il **flusso massimo** è limitato superiormente dalla capacità del **taglio minimo**, ovvero il taglio la cui capacità è minore fra tutti i tagli.



Non consideriamo gli archi centrali perché nella definizione di capacità di un taglio dobbiamo considerare solo gli archi che vanno dal gruppo della sorgente al gruppo del pozzo.

Questo è verificato in quanto nessun flusso di un taglio può superare la capacità del taglio quindi

$$\forall f \ : \ F_f(S,T) \leq C(S,T) \text{ per ogni taglio } (S,T) \text{ in } V$$

Poi sappiamo che il flusso che attraversa il taglio è uguale al valore del flusso

$$\forall f \ : \ |f| = F_f(S,T) \text{ per ogni taglio } (S,T) \text{ in } V$$

Quindi, il valore del flusso è limitato superiormente dalla capacità di tutti i possibili tagli.

$$\forall f \ : \ |f| \leq C(S,T) \text{ per ogni taglio } (S,T) \text{ in } V$$

Il **teorema del taglio minimo / flusso massimo** è il seguente:
Le seguenti tre affermazioni sono equivalenti:

(1) f è un **flusso massimo**
(2) non esiste alcun cammino aumentante nella rete residua G_f
(3) esiste un **taglio minimo** (S,T) tale che $C(S,T) = |f|$

Procederemo con la dimostrazione circolarmente.
$(1) \Rightarrow (2)$
Dunque dobbiamo dimostrare che se f è un flusso massimo allora non esiste alcun cammino aumentante nella rete residua G_f.
Se esistesse un cammino aumentante, il flusso potrebbe essere aumentato e quindi non sarebbe massimo ed è assurdo.

$(2) \Rightarrow (3)$
Dunque dobbiamo dimostrare che se non esiste alcun cammino aumentante nella rete residua G_f allora esiste un taglio minimo (S,T) tale che $C(S,T) = |f|$
Poiché non esiste nessun cammino aumentante nella rete residua G_f, non esiste nessun cammino da s a t.
Sia S l'insieme dei vertici raggiungibili da s in G_f e definiamo $T = V - S$. Ovviamente $s \in S$ e $t \in T$, dunque (S,T) è un taglio.

Poichè t non è raggiungibile da s in G_f, tutti gli archi (x,y) con $x \in S$ e $y \in T$ sono saturati; ovvero $f(x,y)=c(x,y)$.

Per il lemma del valore del flusso di un taglio vale $|f|=\sum_{x \in S, y \in T}^{\square} f(x,y)$. Ne segue che

$$|f|=\sum_{x \in S, y \in T}^{\square} f(x,y)=\sum_{x \in S, y \in T}^{\square} c(x,y)=C(S,T)$$

(S,T) è minimo perchè vale $|f|=C(S,T)$ e per ogni taglio (S',T'), abbiamo che $|f| \leq C(S',T')$ per il lemma della capacità sul taglio.

$(3) \Rightarrow (1)$

Dunque dobbiamo dimostrare che se esiste un taglio minimo (S,T) tale che $C(S,T)=|f|$ allora f è un flusso massimo.

Poiché per un qualsiasi flusso f e un qualsiasi taglio (S,T) vale la relazione $|f| \leq C(S,T)$, il flusso che soddisfa $|f|=C(S,T)$ deve essere massimo.

Complessità

Se le capacità sono **intere**, l'algoritmo di **Ford-Fulkerson** (visita generale) ha complessità $O\left((V+E)|f^*|\right)$ (liste) o $O\left(V^2|f^*|\right)$ (matrice).

Questo perchè l'algoritmo parte dal flusso nullo e termina quando il valore totale del flusso raggiunge $|f^*|$ e ogni incremento del flusso aumenta il flusso di almeno un'unità, dunque ripetiamo al massimo $|f^*|$ volte.

Ogni ricerca di un cammino invece richiede una visita del grafo con costo $O(V+E)$ o $O(V^2)$ e la somma dei flussi e il calcolo della rete residua può essere effettuata sempre in tempo $O(V+E)$ o $O(V^2)$.

Si noti che esistono casi pessimi che effettivamente hanno questo tempo di esecuzione

Se le capacità sono **intere**, l'algoritmo di **Edmonds e Karp** (visita in ampiezza) ha complessità $O(V E^2)$ nel caso pessimo.

Dunque mettiamo assieme i due limiti superiori prendendo il più basso tra i due, ovvero $O\left(min\left((V+E)|f^*|, V E^2\right)\right)$.

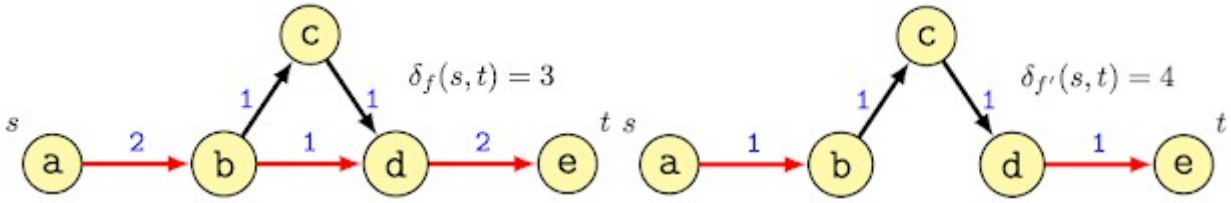
Verifichiamo che la complessità dell'algoritmo di Edmonds e Karp sia $O(V E^2)$.

Vengono eseguiti $O(VE)$ aumenti di flusso, ognuno dei quali richiede una visita in ampiezza $O(V+E)$. La complessità è quindi $O(VE(V+E))$.

Poichè $E=\Omega(V)$ (ogni nodo diverso da sorgente e pozzo ha almeno un arco entrante e un arco uscente), possiamo semplificare scrivendo che la complessità è $O(V E^2)$

Vediamo il **lemma di monotonia**.

Sia $\delta_f(s,x)$ la distanza minima in numero di archi da s a x in una rete residua G_f. Sia $f' = f + g$ un flusso nella rete iniziale, con g flusso non nullo derivante da un cammino aumentante. Allora $\delta_{f'}(s,x) \geq \delta_f(s,x)$.
Questo perché quando viene aumentato il flusso, alcuni archi si "spengono" (capacità residua 0). Questi archi erano utilizzati nei cammini minimi (BFS), dunque i cammini minimi non possono diventare più corti.



Vediamo il **lemma degli aumenti di flusso**.
Il numero totale di aumenti di flusso eseguiti dall'algoritmo di Edmonds e Karp è $O(VE)$.

- Sia G_f una rete residua
- Sia C un cammino aumentante di G_f
- (x,y) è un arco critico (collo di bottiglia) in C se $c_f(x,y) = min_{(u,v) \in C}\{c_f(u,v)\}$
- In ogni cammino esiste almeno un arco critico
- Una volta aggiunto il flusso associato a C, l'arco critico scompare dalla rete residua
- Poiché i cammini aumentanti sono cammini minimi, abbiamo che $\delta_f(s,y) = \delta_f(s,x) + 1$
- L'arco (x,y) potrà ricomparire se e solo se il flusso lungo l'arco diminuirà, ovvero se (y,x) appare in un cammino aumentante
- Sia g il flusso quando questo accade, come sopra abbiamo $\delta_g(s,x) = \delta_g(s,y) + 1$
- Per il lemma di monotonia, abbiamo anche che $\delta_g(s,y) \geq \delta_f(s,y)$, quindi $\delta_g(s,x) = \delta_g(s,y) + 1 \geq \delta_f(s,y) + 1 = \delta_f(s,x) + 2$
- Dal momento in cui un nodo è critico al momento in cui può tornare ad essere critico, il cammino minimo si è allungato almeno di due passi
- La lunghezza massima del cammino fino a x, tenuto conto che poi si deve ancora eseguire l'arco (x,y), è $V - 2$
- Dunque un arco può diventare critico al massimo $\frac{V-2}{2} = \frac{V}{2} - 1$ volte
- Poichè ci sono $O(E)$ archi che possono diventare critici $O(V)$ volte, abbiamo che il numero massimo di flussi aumentati è $O(VE)$
- Poiché ogni visita in ampiezza costa $O(E)$, il costo totale è $O(E^2 V) = O(m^2 n)$

Nome	Complessità	Note
Ford-Fulkerson	$O(E\|f^*\|)$	Converge con valori razionali
Edmonds-Karp	$O(VE^2)$	Specializzazione basata su BFS
Dinitz, blocking flow	$O(V^2E)$	In alcune reti particolari, $O(\min(V^{2/3}, E^{1/2})E)$
MPM	$O(V^3)$	Solo su DAG
Dinitz	$O(VE \log V)$	Struttura dati Dynamic trees
Goldberg e Rao	$O(\min(V^{2/3}, E^{1/2})E \cdot$ $\cdot \log(V^2/E+2) \log C)$	$C = \max_{(x,y)\in E} c(x,y)$
Orlin + King,Rao,Tarjan	$O(VE)$	2013
Chen, Kyng, Liu at al.	$O(V + E^{1+o(1)})$	"Quasi-linear", 112 pagine, 2022!

Abbinamento (matching) massimo nei grafi bipartiti

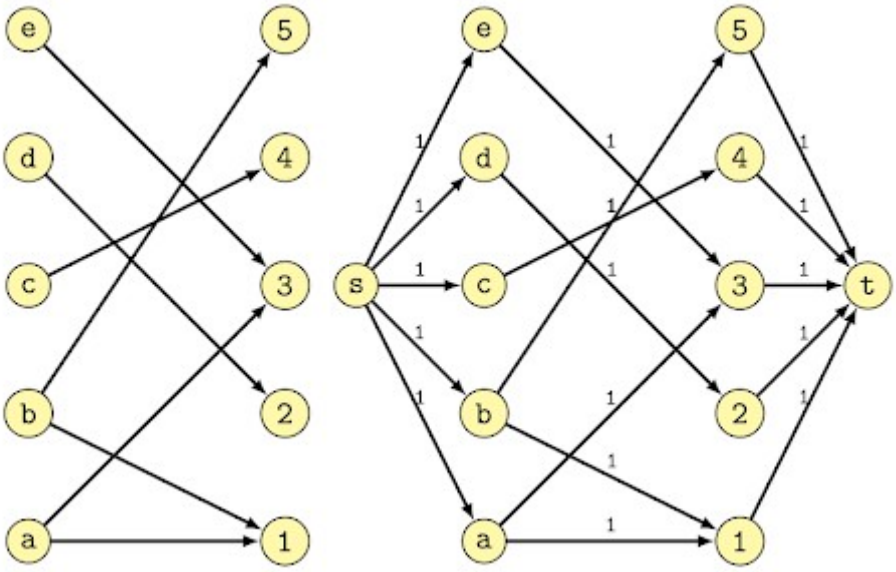
Alcuni problemi combinatori possono essere trattati come problemi di flusso, se opportunamente “adattati”. Dato un grafo non orientato $G=(V,E)$, un abbinamento (matching) è un sottoinsieme M di archi tale che, per ogni vertice $u \in V$, al massimo un arco di M sia incidente su u. Un abbinamento massimo è un abbinamento con cardinalità massima. Limitiamoci a considerare qui i grafi bipartiti, grafi in cui sia l’insieme V è diviso in due sottoinsiemi disgiunti L e R, e ogni arco in E collega un nodo di L a un nodo di R. Il problema di trovare un abbinamento massimo in un grafo bipartito ha molte applicazioni pratiche. Ad esempio, abbinare un insieme di L macchine a un insieme R di processi da eseguire contemporaneamente. La presenza di un arco significa che una particolare macchina $u \in L$ è in grado di eseguire un particolare processo. Un abbinamento massimo mette al lavoro il maggior numero di macchine.

Problema - Job Assignment - Input

- Un insieme J contenente n job
- Un insieme W contenente m worker
- Una relazione $R \subseteq J \times W$, tale che $(j,w) \in R$ se e solo se il job j può essere eseguito dal worker w

Problema - Job Assignment - Output

- Il più grande sottoinsieme $O \subseteq R$, tale che:
  - ogni job venga assegnato al più ad un worker
  - ad ogni worker venga assegnato al più un job



La soluzione consiste nel costruire una rete di flusso associata $G' = (V', E', s, t, c)$, dove:

- s e t sono nodi nuovi, che rappresentano la sorgente e il pozzo
- $V' = \{s\} \cup J \cup W \cup \{p\}$
- $E' = \{(s,u) \mid u \in J\} \cup \{(u,v) \mid u \in J, v \in W, (u,v) \in E\} \cup \{(v,t) \mid v \in W\}$; ovvero tutti gli archi in E sono ora direzionati da J a W, e sono stati aggiunti archi che collegano s a J e W a t
- $c(u,v) = 1$ se $(u,v) \in E$, $c(u,v) = 0$ altrimenti

Possiamo utilizzare per risolvere il problema in G'; e facile vedere che $|E'| = \Theta(E)$, e che il flusso massimo è $O(n)$; quindi, Ford-Fulkerson viene eseguito in tempo $O(mn)$.

Backtrack

Una tecnica non molto astuta, ma che talvolta può risultare utile, si basa sulla ovvia considerazione seguente:

"Prova a fare qualcosa e, se non funziona, disfala e riprova qualcos'altro"

Questa tecnica, detta **backtrack**, è alla base di molti algoritmi di visita di alberi e grafi, ed è usata per generare sistematicamente tutte le soluzioni ammissibili di un problema ("to backtrack" significa "tornare sui propri passi").

Dato un problema, una **soluzione ammissibile** è una soluzione che soddisfa un insieme di criteri. I criteri di ammissibilità di una soluzione dipendono dal problema.

- Zaino: sottoinsieme di oggetti di peso inferiore alla capacità
- Sottosequenza comune: una stringa che è sottosequenza di entrambe le stringhe in input

Nei problemi di ottimizzazione, viene definita una **funzione di costo/guadagno** definita sull'insieme delle soluzioni ammissibili.

- Zaino: somma dei guadagni degli oggetti selezionati
- Sottosequenza comune massimale: lunghezza della stringa

In alcuni problemi è richiesto o necessario **esplorare l'intero spazio delle soluzioni ammissibili** (perchè non esistono altri tipi di algoritmi che mi permettano di affrontare il problema). In questi problemi gli algoritmi backtrack, o detti anche **brute force** (di forza bruta) funzionano perfettamente:

- **Enumerazione**
  Questi problemi consistono nell'elencare tutte le soluzioni possibili (ad esempio elencare tutte le permutazioni di un insieme).
- **Ricerca**
  Questi problemi consistono nel trovare una soluzione ammissibile in uno spazio delle soluzioni molto grande. In questi problemi si applica un algoritmo di enumerazione, fermandosi alla prima soluzione trovata
- **Conteggio**
  Questi problemi consistono nel contare tutte le soluzioni ammissibili. Se non è possibile contare in maniera analitica (calcolo combinatorio), dobbiamo enumerare tutte le soluzioni possibili e contarle.
- **Ottimizzazione**
  Questi problemi consistono nel trovare la miglior soluzione ammissibile rispetto ad un certo criterio di valutazione. Enumeriamo tutte le soluzioni e prendiamo la migliore.

Il problema è che costruire tutte le soluzioni può essere molto costoso in quanto lo spazio delle possibili soluzioni può essere superpolinomiale, però a volte è l'unica strada possibile.
Dunque se esiste una soluzione polinomiale, bisogna evitare di provare questa tecnica.
A nostro vantaggio però, a volte lo spazio delle soluzioni non deve essere analizzato interamente.

Per enumerare tutte le soluzioni ammissibili bisogna esplorare lo spazio delle soluzioni in modo sistematico.
La tecnica del backtrack può essere applicata in due modi:

- Ricorsione
  Un metodo sistematico per esplorare uno spazio di ricerca, utilizzando la ricorsione per memorizzare le scelte fatte finora
- Iterazione
  Utilizza un approccio greedy, eventualmente tornando sui propri passi

Filosofia

"*Prova a fare qualcosa; se non va bene, disfalo e prova qualcos'altro*"
"*Ritenta, e sarai più fortunato*"

Enumerazione

Vediamo l'organizzazione generale di un problema di enumerazione.
Una soluzione viene rappresentata come un vettore $S[1 \dots n]$. Il contenuto degli elementi $S[i]$ è preso da un insieme di scelte C dipendente dal particolare problema.

Esempi

- C elementi di un insieme, possibili soluzioni sottoinsiemi di C
- C elementi di un insieme, possibili soluzioni permutazioni di C
- C mosse di gioco, possibili soluzioni sequenze di mosse
- C archi di un grafo, possibili soluzioni percorsi sul grafo

Ad ogni passo, si parte da una soluzione parziale $S[1 \dots i-1]$; se è possibile, si estende tale soluzione con una delle possibili scelte in una soluzione parziale $S[1 \dots i]$ e si valuta la nuova soluzione parziale ricorsivamente; altrimenti, si opera un passo di backtrack (ritornando indietro nella ricorsione) e si effettua una nuova scelta fra quelle possibili a partire da $S[1 \dots i-1]$.

La procedura $enumeration()$ realizza questo schema.
Ad ogni chiamata della procedura viene controllato se $S[1 \dots i-1]$ è una soluzione ammissibile, in caso positivo tale soluzione viene elaborata in qualche modo (stampata, contata, valutata in base ad una funzione di costo ecc.) dalla procedura $processSolution()$.
Altrimenti l'insieme C delle possibili opzioni per la scelta i-esima viene calcolato a partire dalle scelte precedenti $S[1 \dots i-1]$. Tutte le possibili strade vengono intraprese: uno dopo l'altro tutti i valori di C vengono scelti e memorizzati in $S[i]$ e la procedura $enumeration()$ chiama ricorsivamente se stessa, dando per assodate le prime i scelte e spostandosi sull' $(i+1)$-esima.
La ricorsione termina quando l'albero è stato visitato completamente.

Per supportare il caso in cui è sufficiente generare una e una sola soluzione ammissibile, le procedure $enumeration()$ e $processSolution()$ restituiscono un valore booleano, vero se una soluzione ammissibile è stata trovata, falso altrimenti.

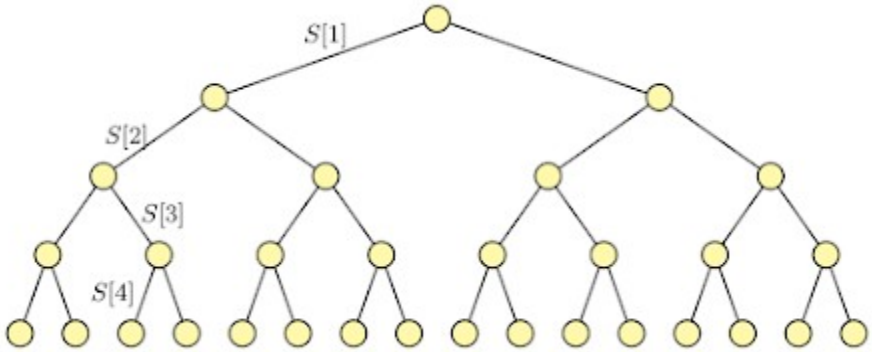
Ecco uno schema parziale della procedura $enumeration()$:

```
enumeration(⟨dati problema⟩, ITEM[] S, int i, ⟨dati parziali⟩)
% Verifica se S[1...i-1] contiene una soluzione ammissibile
if accept(⟨dati problema⟩, S, i, ⟨dati parziali⟩) then
    % "Processa" la soluzione (stampa, conta, etc.)
    processSolution(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
else
    % Calcola l'insieme delle scelte in funzione di S[1...i-1]
    SET C = choices(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
    % Itera sull'insieme delle scelte
    foreach c ∈ C do
        S[i] = c
        % Chiamata ricorsiva
        enumeration(⟨dati problema⟩, S, i+1, ⟨dati parziali⟩)
```

Il parametro i dunque rappresenta l'indice della prossima decisione da prendere.
La soluzione parziale $S[1 \ldots i-1]$ contiene le decisioni prese finora.
Il **caso base** della ricorsione si verifica quando $S[1 \ldots i-1]$ è una soluzione ammissibile, la quale viene processata e (assumendo che le soluzioni ammissibili non possano essere estese) la ricorsione termina.
Il **passo ricorsivo** consiste nel calcolare l'insieme C delle scelte possibili e poi, per ogni elemento $c \in C$, viene scritto c su $S[i]$ e viene chiamata ricorsivamente la funzione con indice $i+1$.

Viene costruito quindi un **albero delle decisioni**:

- Albero delle decisioni ≡ spazio di ricerca
- Radice ≡ soluzione parziale vuota
- Nodi interni dell'albero di decisione ≡ soluzioni parziali
- Foglie ≡ soluzioni ammissibili



Sottoinsiemi e permutazioni

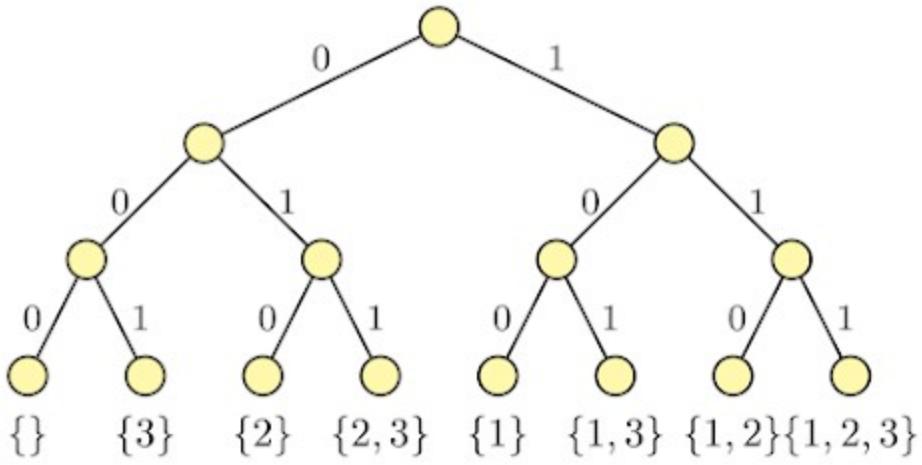
Vediamo come affrontare il problema dell'elencare tutti i possibili sottoinsiemi dell'insieme $\{1,\dots,n\}$.
Come dati del problema abbiamo solo il numero di elementi n.
Nel vettore delle scelte inseriamo 0 se non prendiamo l'elemento i e 1 se lo prendiamo.
Come caso base controlliamo se abbiamo compiuto n scelte, se si processiamo.
Altrimenti costruiamo l'insieme di scelte C che sono o "prendo l'elemento i" oppure "non prendo l'elemento i", e chiamiamo ricorsivamente la funzione.

```
subsetsRec(int n, int[] S, int i)
% S ammissibile dopo n scelte
if i > n then
    processSolution(S, n)
else
    % Non presente / presente
    SET C = {0, 1}
    foreach c ∈ C do
        S[i] = c
        subsetsRec(n, S, i + 1)
```

```
subsets(int n)
% Vettore delle scelte
int[] S = new int[1...n]
subsetsRec(n, S, 1)
```

```
processSolution(int[] S, int n)
print "{ "
for i = 1 to n do
    if S[i] then
        print i, " "
println "}"
```

Come richiesto dal problema, tutto lo spazio possibile viene esplorato con complessità $\Theta(n \cdot 2^n)$, perché abbiamo 2^n soluzioni e ognuna viene processata con costo n.
L'ordine è il seguente: si parte dall'insieme vuoto, poi vengono trovati tutti i sottoinsiemi che non comprendono il primo elemento e poi vengono trovati tutti i sottoinsiemi che lo comprendono (bisogna fare lo stesso ragionamento ricorsivamente).



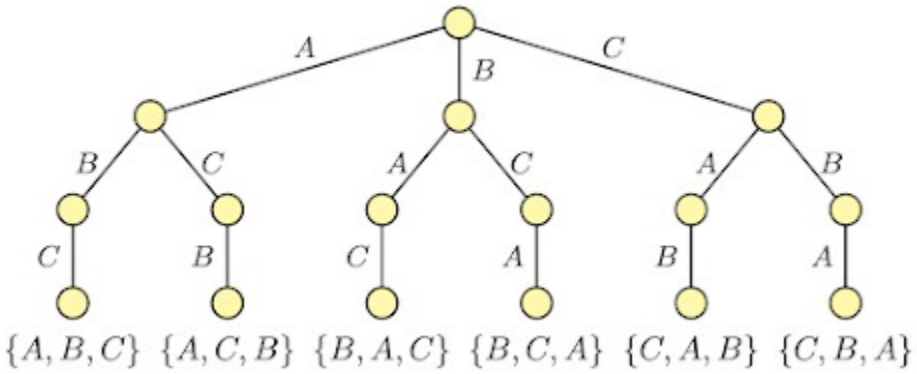
Si può costruire una versione iterativa semplicemente prendendo come vettori di scelte tutti i numeri binari da 0 a 2^n-1, ma la complessità resta la stessa:

```
subsets(int n)
for j = 0 to 2^n − 1 do
    print "{ "
    for i = 0 to n − 1 do
        if (j && 2^i) ≠ 0 then                    % Bitwise and
            print i, " "
    println "}"
```

Per il problema dell'enumerazione delle permutazioni invece come scelta scegliamo uno degli elementi rimasti nell'insieme $\{0,\ldots,n\}$.
Anche qui ci fermiamo quando abbiamo fatto n scelte.

```
permRec(SET A, ITEM[] S, int i)
% Se A è vuoto, S è ammissibile
if A.isEmpty() then
    print S
else
    % Copia A per il ciclo foreach
    SET C = copy(A)
    foreach c ∈ C do
        S[i] = c
        A.remove(c)
        permRec(A, S, i + 1)
        A.insert(c)
```

```
permutations(SET A)
int n = size(A)
int[] S = new int[1...n]
permRec(A, S, 1)
```



La complessità della stampa dell'array S è sempre $\Theta(n)$.

Il costo delle copie del vettore lungo un cammino radice-foglia è $\sum_{i=1}^{n} O(i) = O(n^2)$

Il numero di foglie è $n!$

Dunque la complessità è $O(n^2 n!)$, dunque non è molto efficiente come algoritmo per generare le permutazioni (esistono algoritmi molto più efficienti come l'Algoritmo di Heap)

Una versione del codice più pulita ed efficiente, che però non segue il template del backtrack che abbiamo visto, è la seguente che si basa sugli swap degli elementi.

```
permRec(ITEM[] S, int i)
% Caso base, un elemento
if i == 1 then
    println S
else
    for j = 1 to i do
        swap(S, i, j)
        permRec(S, i − 1)
        swap(S, i, j)
```

```
permutations(ITEM[] S, int n)
permRec(S, n)
```

Complessità

- $n!$ permutazioni
- $\Theta(n)$ per stamparle tutte
- $2n$ swap per ogni permutazione
- Costo totale $\Theta(n \cdot n!)$

Lo schema di backtrack che abbiamo visto può essere ulteriormente raffinato per risolvere altri tipi di problemi.
Ad esempio ci sono alcuni problemi in cui ad un certo punto si può dire "da qui in avanti non è più possibile trovare una soluzione con le scelte che abbiamo preso", dunque non ha senso continuare e conviene piuttosto tornare indietro, interrompendo lo sviluppo attraverso quelle scelte.

```
enumeration(⟨dati problema⟩, ITEM[] S, int i, ⟨dati parziali⟩)
if accept(⟨dati problema⟩, S, i, ⟨dati parziali⟩) then
    % Processo la soluzione S in quanto ammissibile
    processSolution(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
else if reject(⟨dati problema⟩, S, i, ⟨dati parziali⟩) then
    return
else
    % Ricorsione
    SET C = choices(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
    foreach c ∈ C do
        S[i] = c
        enumeration(⟨dati problema⟩, S, i + 1, ⟨dati parziali⟩)
```

Prendiamo il problema dei sottoinsiemi ma imponiamo il criterio che vogliamo solo sottoinsiemi composti da k elementi. In questa situazione quando abbiamo già scelto k elementi non ha senso continuare con lo sviluppo della soluzione, visto che sicuramente non porterà ad altre soluzioni accettabili.
Vediamo un primo tentativo di soluzione per il problema dei k-sottoinsiemi

```
kssRec(int n, int k, int[] S, int i)
int size = count(S, n)
if i > n and size == k then
    processSolution(S, n)
else if i > n and size ≠ k then
    return
else
    foreach c ∈ {0, 1} do
        S[i] = c
        kssRec(n, k, S, i + 1)
```

```
kSubsets(int n, int k)
int[] S = new int[1 ... n]
kssRec(n, k, S, 1)
```

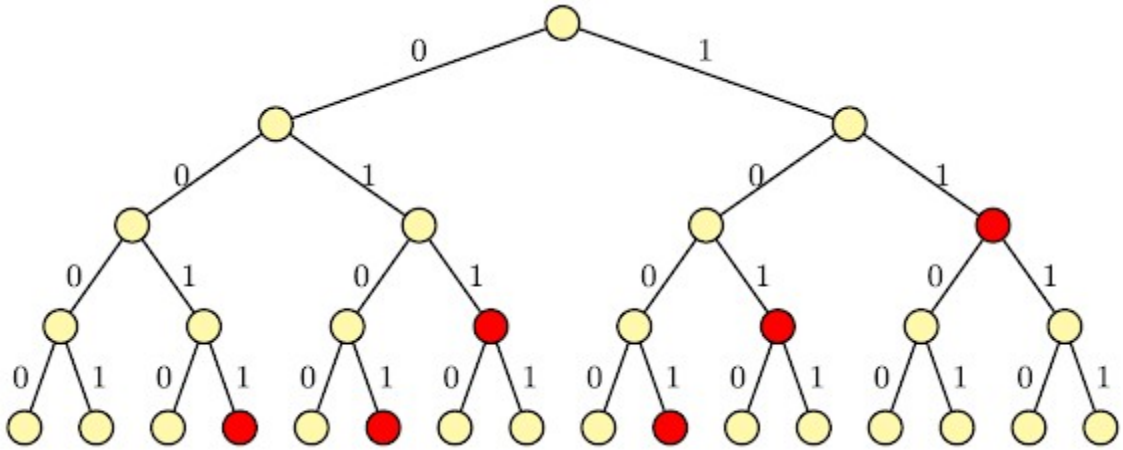
```
int count(int[] S, int n)
int count = 0
for j = 1 to n do
    count = count + S[j]
return count
```

Con questa soluzione non solo non guadagnamo nulla in termini di complessità, ma la peggioriamo anche visto che contiamo ogni volta quanti elementi abbiamo preso, perchè facciamo il controllo solo dopo aver preso tutte le scelte, semplicemente scartando le soluzioni non ammissibili.

Per evitare di ricontare tutte le volte quanti elementi abbiamo preso fino a quel momento, possiamo usare una variabile *missing*, inizializzata ad n e che viene decrementata ogni volta che prendiamo un elemento

```
kssRec(int n, int missing, int[] S, int i)
if i > n and missing == 0 then
    processSolution(S, n)
else if i > n and missing ≠ 0 then
    return
else
    foreach c ∈ {0, 1} do
        S[i] = c
        kssRec(n, missing − c, S, i + 1)
```

Notiamo però che una volta che abbiamo raggiunto $missing = 0$ non ha senso continuare, in quanto da quel punto in poi l'unica soluzione ammissibile è quella di non prendere più elementi.



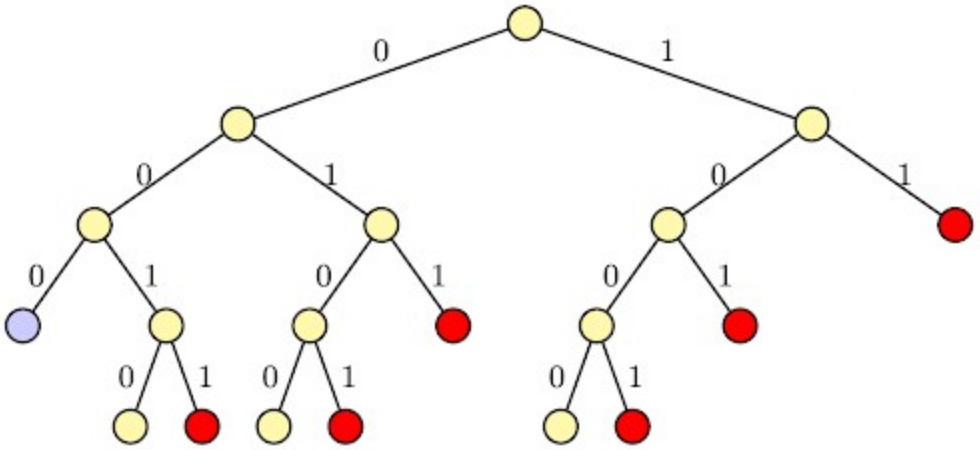
Quindi non appena abbiamo $missing = 0$ fermiamo la ricorsione e processiamo quella soluzione.

```
kssRec(int n, int missing, int[] S, int i)
if i > n and missing == 0 then
    processSolution(S, i − 1)
else if i > n or missing < 0 then
    return
else
    foreach c ∈ {0, 1} do
        S[i] = c
        kssRec(n, missing − c, S, i + 1)
```

In pratica "potiamo" (in inglese potare = **pruned**) i rami dell'albero che sicuramente non portano a soluzioni ammissibili.

Notiamo che questo ragionamento possiamo farlo anche all'incontrario, ovvero che se *missing* è maggiore del numero di elementi che mi mancano da considerare, allora sicuramente non potrò arrivare ad un sottoinsieme grande k.

```
kssRec(int n, int missing, int[] S, int i)
if missing == 0 then
    processSolution(S, i − 1)
else if i > n or missing < 0 or n − (i − 1) < missing then
    return
else
    foreach c ∈ {0, 1} do
        S[i] = c
        kssRec(n, missing − c, S, i + 1)
```



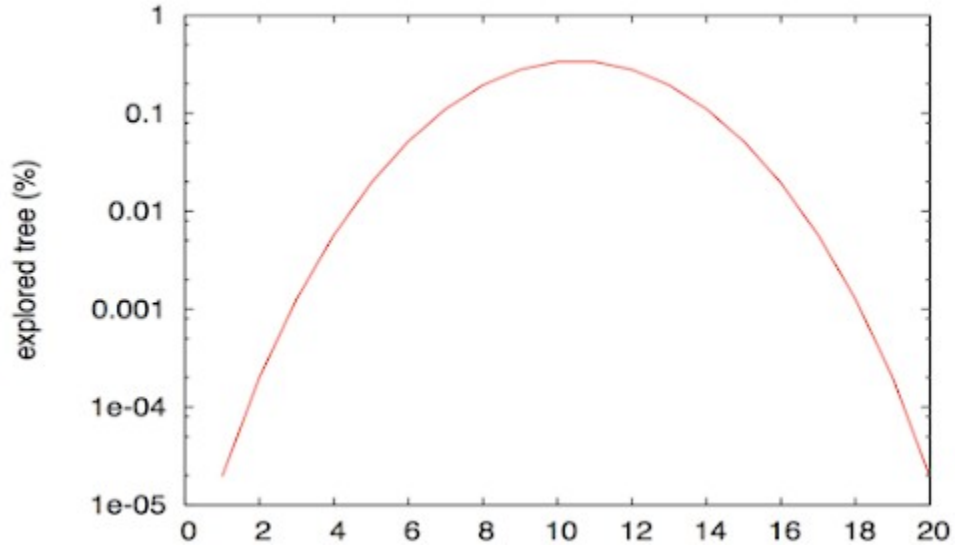
Pulendo un po' il codice che abbiamo scritto otteniamo la seguente soluzione

```
kssRec(int n, int missing, int[] S, int i)
if missing == 0 then
    processSolution(S, i − 1)
else if i ≤ n and 0 < missing ≤ n − (i − 1) then
    foreach c ∈ {0, 1} do
        S[i] = c
        kssRec(n, missing − c, S, i + 1)
```

Per capire i vantaggi di questi accorgimenti, ricordiamo che noi sappiamo esattamente quanti sono i sottoinsiemi di k elementi in un insieme n, ovvero $\frac{k}{n}$.

Con $n = 20$ e $k = 10$ la percentuale di albero scoperto è $50\,\%$. Con altri valori di k (molto piccoli e molto grandi) invece la percentuale è sempre più bassa.

Dunque specializzando l'algoritmo generico, possiamo ottenere una versione più efficiente (il miglioramento è parziale solo verso $\frac{n}{2}$).
Con l'algoritmo iterativo invece non posso ottenere la stessa efficienza perchè sono obbligato ad analizzare tutti i numeri.



Subset sum

Vediamo ora il problema della subset sum (somma di sottoinsieme).
Dati un vettore A contenente n interi positivi ed un intero positivo k, esiste un sottoinsieme $S \subseteq \{1 \dots n\}$ tale che $\sum_{i \in S} a[i] = k$?

Ecco un esempio con due soluzioni (una in rosso e una in blu):

i	1	2	3	4	5	6	7	8
A	1	4	3	12	7	2	21	55

$$k = 23$$
$$S_1 = \{2, 4, 5\}$$
$$S_2 = \{6, 7\}$$

Questo problema può essere risolto tramite programmazione dinamica (come abbiamo visto per lo zaino) in tempo $O(kn)$ (quindi complessità pseudopolinomiale), ma noi vedremo la soluzione backtrack in tempo $O(2^n)$, che potrebbe essere migliore di quella della programmazione dinamica in base al valore di k.
Dunque non siamo interessati a tutte le soluzioni, ma ce ne basta una. Quindi interromperemo l'esecuzione alla prima soluzione trovata.

L'idea è che non appena trovo una soluzione accettabile, la processo e ritorno *true* in modo che il *true* risalga tutta la catena di ricorsione fermando l'algoritmo.
Ecco l'algoritmo generico per risolvere questo tipo di problemi:

```
enumeration(⟨dati problema⟩, ITEM[] S, int i, ⟨dati parziali⟩)
if accept(⟨dati problema⟩, S, i, ⟨dati parziali⟩) then
    processSolution(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
    return true                % Trovata soluzione, restituisco true
else if reject(⟨dati problema⟩, S, i, ⟨dati parziali⟩) then
    return false  % Impossibile trovare soluzioni, restituisco false
else
    SET C = choices(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
    foreach c ∈ C do
        S[i] = c
        if enumeration(⟨dati problema⟩, S, i + 1, ⟨dati parziali⟩) then
            return true        % Trovata soluzione, restituisco true
    return false               % Nessuna soluzione, restituisco false
```

Dunque per il nostro algoritmo specifico, oltre all'array A e la sua dimensione n, ci portiamo avanti il parametro *missing* che all'inizio corrisponde a k e che viene decrementato del peso che prendiamo ogni volta che compiamo la scelta di prendere l'elemento. Non appena abbiamo che *missing* $= 0$, possiamo stampare l'array delle scelte che abbiamo preso fino a quel momento.

```
boolean ssRec(int[] A, int n, int missing, int[] S, int i)
if missing == 0 then
    processSolution(S, i − 1)        % Stampa gli indici della soluzione
    return true
else if i > n or missing < 0 then
    return false               % Terminati i valori o somma eccessiva
else
    foreach c ∈ {0, 1} do
        S[i] = c
        if ssRec(A, n, missing − A[i] · c, S, i + 1) then
            return true
    return false
```

```
subsetSum(int[] A, int n, int k)
int[] S = new int[1...n]
ssRec(A, n, k, S, 1)
```

Anche in questo caso “poto” l’albero delle soluzioni quando arrivo ad un punto in cui non posso più ottenere una soluzione (quando $missing < 0$).

Notiamo che se avessimo informazioni sulle somme cumulative, potremmo potare ulteriormente l’albero.

Giochi

Vediamo ora come risolvere alcuni giochi usando la tecnica del backtrack.

Il problema delle otto regine

Il problema consiste nel posizionare n regine in una scacchiera $n \times n$, in modo tale che nessuna regina ne “minacci” un’altra.

La prima soluzione tiene conto dell’idea è che ci sono n^2 caselle dove piazzare una regina.
Dunque per ogni casella scegliamo se piazzare una regina oppure no.
Dunque possiamo usare un vettore delle scelte $S[1 \dots n^2]$ binario in cui $S[i] = true$ significa che c’è una regina in $S[i]$.
Controlliamo la soluzione quando abbiamo scorso tutta la scacchiera $i = n^2$ e ad ogni passo le scelte possibili sono $Choices(S, n, i) = [true, false]$.
Il pruning lo facciamo ogni volta che inseriamo una nuova regina e vediamo che minaccia quelle già esistenti, e in quel caso restituiamo 0.
Con questo algoritmo per $n = 8$ abbiamo che le soluzioni sono $2^{n^2} = 2^{64} \simeq 1.84 \cdot 10^{19}$, dunque la complessità è alta e il problema potrebbe essere di rappresentazione (ovvero che la matrice binaria è molto sparsa).

Proviamo a cambiare approccio, sull’idea che dobbiamo piazzare n regine e ci sono n^2 caselle.
Inseriamo le regine una alla volta.
Usiamo un vettore delle scelte $S[1 \dots n]$ dove ogni scelta $S[i]$ corrisponde alle coordinate su cui abbiamo messo la regina i.
La soluzione la controlliamo quando inserisco tutte le regine, quindi quando $i = n$ e ad ogni passo le scelte possibili sono $Choices(S, n, i) = [1, \dots, n^2]$ dove ogni numero rappresenta una casella della scacchiera.
Il pruning lo applichiamo per restringere il sottoinsieme delle possibili mosse a quelle legali.
Con questo algoritmo per $n = 8$ abbiamo che le soluzioni sono $(n^2)^n = 64^8 = 2^{48} \simeq 2.81 \cdot 10^{14}$.
Abbiamo quindi ottenuto un miglioramento, ma lo spazio di esplorazione è ancora molto grande, e in più abbiamo un altro problema, ovvero come facciamo a distinguere due soluzioni che sono una la permutazione dell’altra.

Dunque per migliorare ancora l'algoritmo precedente, non mettiamo regine in caselle precedenti a quelle già scelte, ovvero consideriamo solo le permutazioni ordinate delle posizioni delle regine, semplicemente mettendo come condizione di pruning $S[i] > S[i-1]$.
Con questo algoritmo per $n=8$ abbiamo che le soluzioni sono $\frac{(n^2)^n}{n!} = \frac{2^{48}}{40320} = 6.98 \cdot 10^9$.

Un altro accorgimento è che ogni riga deve contenere esattamente una regina. Scegliamo quindi per ogni riga, in che colonna mettere la regina.
Usiamo un vettore delle scelte $S[1 \dots n]$ dove ogni scelta $S[i]$ corrisponde alla colonna su cui abbiamo messo la regina che sta nella riga i.
La soluzione la controlliamo quando inserisco tutte le regine, ovvero quando ho finito le righe, quindi quando $i=n$ e ad ogni passo le scelte possibili sono $Choices(S,n,i) = [1,\dots,n]$, le colonne.
Il pruning lo applichiamo per restringere il sottoinsieme delle possibili colonne su cui possiamo piazzare una regina.
Con questo algoritmo per $n=8$ abbiamo che le soluzioni sono $n^n = 8^8 = 2^{48} \simeq 1.67 \cdot 10^7$.
Abbiamo quindi ottenuto un gran miglioramento.

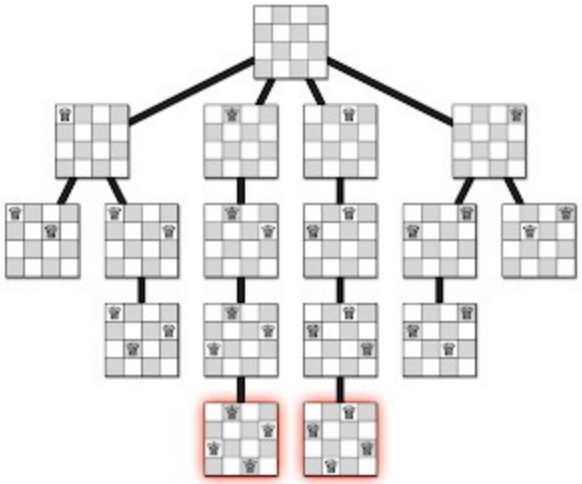
L'ultimo miglioramento è quello di applicare lo stesso ragionamento anche alle colonne e mettere come pruning l'eliminazione delle diagonali.
Con questo algoritmo per $n=8$ abbiamo che le soluzioni sono $n! = 8! = 40320$. E le soluzioni che vengono effettivamente visitate sono 15720

```
queens(int n, int[] S, int i)
if i > n then
    print S
else
    for j = 1 to n do  % Prova a piazzare la regina nella colonna j
        boolean legal = true
        for k = 1 to i − 1 do          % Verifica le regine precedenti
            if S[k] == j or S[k] == j + i − k or S[k] == j − i + k then
                legal = false
        if legal then
            S[i] = j
            queens(n, S, i + 1)
```

Vediamo come si crea un albero e come certe scelte non mi permettono di proseguire

n	1	2	3	4	5	6	7	8	9	10	11	12
#Sol	1	0	0	2	10	4	40	92	352	724	2680	14200

- In alto, il numero di soluzioni al variare di n
- A destra, l'albero della ricorsione per $n = 4$



Per questo problema esistono altre soluzioni euristiche più efficienti.

Il **Sudoku**
Il problema consiste, data una matrice con alcune caselle già riempite, ottenere una matrice finale completamente riempita di numeri da 1 a 9 in modo che ogni colonna, ogni riga e ogni box non abbia ripetizioni.
Esistono varie tecniche per risolvere il sudoku, una di queste è applicare la forza bruta (provarle tutte).
Quindi per ogni casella scelgo tra i numeri che non creano conflitti e provo ad andare avanti con quella scelta fino a che o trovo la soluzione o mi accorgo che quella scelta non andava bene.

```
boolean sudoku(int[][] S, int i)
if i == 81 then
    processSolution(S, n)
    return true
int x = i mod 9
int y = ⌊i/9⌋
Set C = moves(S, x, y)
int old = S[x][y]
foreach c ∈ C do
    S[x][y] = c
    if sudoku(S, i + 1) then
        return true
S[x][y] = old
return false
```

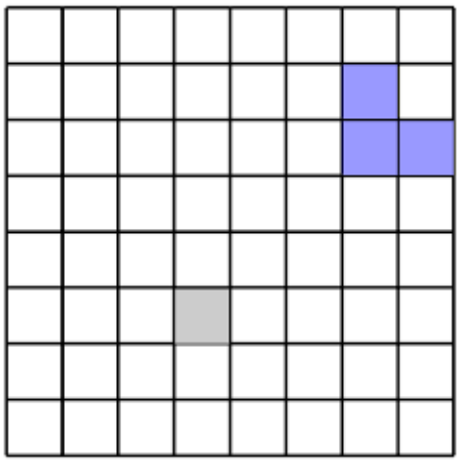
```
Set moves(int[][] S, int x, int y)
Set C = Set()
if S[x][y] ≠ 0 then
    % Numero pre-inserito
    C.insert(S[x][y])
else
    % Verifica conflitti
    for c = 1 to 9 do
        if check(S, x, y, c) then
            C.insert(c)
return C
```

Vediamo che l'insieme di mosse cambia a seconda che in quella casella ci sia un numero preinserito o meno. Ovviamente se c'è un numero preinserito non posso fare una scelta quindi prendo quello per forza.

La funzione check() controlla le colonne, le righe e le box in base alle scelte prese fino a quel momento

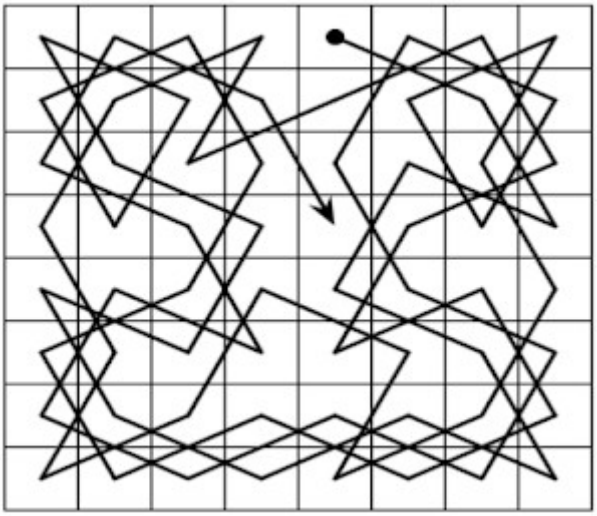
```
boolean check(int[][] S, int x, int y, int c)
for j = 0 to 8 do
    if S[x][j] == c then
        return false                    % Controllo sulla colonna
    if S[j][y] == c then
        return false                    % Controllo sulla riga
int b_x = ⌊x/3⌋
int b_y = ⌊y/3⌋
for i_x = 0 to 2 do
    for i_y = 0 to 2 do                 % Controllo sulla sottotabella
        if S[b_x · 3 + i_x][b_y · 3 + i_y] = c then
            return false
return true
```

Un altro **problema puzzle**
Consideriamo una scacchiera $n \times n$ con n potenza di 2, quindi $n = 2^k$.
Qualsiasi scacchiera di questo tipo con una cella rimossa, può essere ricoperta da triomini a forma di L



Anche qui possiamo applicare backtracking.

Knight tour
Consideriamo ancora una scacchiera $n \times n$. Lo scopo è quello di trovare un “giro di cavallo”, ovvero un percorso di mosse valide del cavallo in modo che ogni casella venga visitata al più una volta



La soluzione consiste in una matrice $n \times n$ di scelte S, in cui se una cella contiene 0 significa che non è ancora stata visitata, se contiene i significa che è stata visitata al passo i-esimo.
Ad ogni passo le mosse vengono scelte in modo da non andare su una casella già visitata.

```
boolean knightTour(int[][] S, int i, int x, int y)
% Se i = 64, ho fatto 63 mosse e ho completato un tour (aperto)
if i == 64 then
    processSolution(S)
    return true
SET C = moves(S, x, y)
foreach c ∈ C do
    S[x][y] = i
    if knightTour(S, i + 1, x + m_x[c], y + m_y[c]) then
        return true;
    S[x][y] = 0
return false
```

```
SET moves(int[][] S, int x, int y)
SET C = Set()
for i = 1 to 8 do
    n_x = x + m_x[i]
    n_y = y + m_y[i]
    if 1 ≤ n_x ≤ 8 and 1 ≤ n_y ≤ 8 and S[n_x][n_y] == 0 then
        C.insert(i)
return C
```

$$m_x = \{-1, +1, +2, +2, +1, -1, -2, -2\}$$
$$m_y = \{-2, -2, -1, +1, +2, +2, +1, -1\}$$

Con questo algoritmo, ad ogni passo ho al massimo 8 caselle disponibili, dunque ne visito al più $8^{63} \simeq 7.84 \cdot 10^{55}$. Grazie al pruning ne visito molte meno, ma resta a prescindere un problema non affrontabile per valori grandi di *n*.
Questo è un esempio del più generale "problema del cammino hamiltoniano", per il quale non esistono soluzioni polinomiali. Per questo particolare problema, tuttavia, esistono degli algoritmi di costo lineare nel numero di caselle, basati su divide-impera.

Per **uscire da un labirinto** una tecnica è quella di appoggiare la mano destra sulla siepe di destra e procedere senza mai levare la mano. In questa maniera eseguiamo un backtrack e prima o poi troveremo sicuramente l'uscita (anche se potrebbe essere subito a sinistra).

Backtracking iterativo

//FINO A SLIDE 72

Algoritmi probabilistici

Capita a volte di vedersi la strada bloccata da una persona, spostarsi a destra per passare, proprio mentre quella persona si sposta anch'essa nella stessa direzione; allora ci si sposta a sinistra, e di nuovo ci si trova bloccati. Se l'algoritmo seguito è "**if** destra **then** sinistra **else** destra", questo balletto non avrà fine. Ma se invece di fare una scelta deterministica ci si affida alla sorte, e si sceglie casualmente una direzione, la probabilità che la strada sia ancora bloccata dopo k mosse è $\left(\frac{1}{2}\right)^k$, un valore che diventa arbitrariamente piccolo con la crescita di k; prima o poi, sarà possibile passare.

Questo aneddoto suggerisce un approccio "audace" alla progettazione di algoritmi, che potrebbe essere riassunto da Virgilio in questo modo.

"La sorte aiuta gli audaci"

Quando non si sa quale opzione prendere di fronte ad una scelta, oppure l'algoritmo per decidere in maniera corretta ha un costo computazionale troppo alto, è possibile affidarsi alla casualità: si sceglie un'opzione a caso.

Abbiamo già incontrato il concetto di probabilità parlando di analisi del caso medio, ove il tempo di calcolo è stato valutato mediando su tutti i possibili dati di ingresso, dopo aver individuato (o per lo più assunto arbitrariamente) una distribuzione di probabilità in accordo alla quale i dati di ingresso possono presentarsi. Per esempio, il caso medio del QuickSort parte dall'assunzione che tutte le permutazioni di un vettore di n elementi siano equiprobabili.

Nel caso di algoritmi **probabilistici** (detti anche randomizzati) il calcolo delle probabilità è applicato non tanto ai dati di ingresso, quanto piuttosto ai dati di uscita. In un algoritmo probabilistico, la computazione dipende dai valori prodotti da un generatore di numeri casuali. Si assuma di avere a disposizione una funzione $random(h,k)$, che quando è richiamata restituisce con la stessa probabilità un qualsiasi intero generato casualmente tra h e k, estremi inclusi.

Gli algoritmi probabilistici si dividono in due macro categorie:

- Algoritmi la cui correttezza è probabilistica (**Montecarlo**)
  Ovvero progettare algoritmi che nel risolvere un problema possono fornire un risultato scorretto (o addirittura non fornirne alcuno), ma che risultano tuttavia utilissimi qualora la probabilità che tali eventi si verifichino possa essere resa arbitrariamente piccola
- Algoritmi corretti, il cui tempo di funzionamento è probabilistico (**Las Vegas**)
  Ovvero permutare casualmente i dati di ingresso di un problema. In questo modo una distribuzione casuale sui dati d'ingresso viene impostata preventivamente, anziché assunta successivamente nell'analisi di complessità.

Per concludere, si noti che questi non sono i primi esempi di algoritmi probabilistici del presente libro: le tabelle hash, infatti, si avvalgono di funzioni hash che distribuiscono le chiavi in maniera uniforme.

Algoritmi Montecarlo

Un problema classico e interessante che può essere risolto in maniera elegante tramite un algoritmo probabilistico è quello della *PRIMALITÀ*:

Test di primalità

Algoritmo per determinare se un numero in input n è primo.

Il problema può essere risolto con la seguente procedura deterministica:

```
boolean isPrimeNaif(int n)
for i = 2 to ⌊√n⌋ do
    if n/i == ⌊n/i⌋ then
        return false
return true
```

Una soluzione inefficiente testa tutti gli interi fra 2 e $\lfloor\sqrt{n}\rfloor$

La funzione *isPrimeNaif*$()$ prova a dividere n per ciascun numero i compreso tra 2 e $\sqrt{\square}$, e restituisce come risultato *true* se non è stato trovato alcun j tale che $n \mod j = 0$, cioè tale che il resto della divisione di n per j sia zero. La sua complessità è $O\left(n^{\frac{1}{2}}\right)$, che non è polinomiale nella dimensione del problema. Infatti, poiché un numero intero n codificato in base 2 necessita di $d = \log n + 1$ bit, la dimensione del problema è d. La complessità della funzione *isPrimeNaif*$()$ è pertanto $O\left(2^d\right)$, cioè esponenziale nella dimensione del problema (secondo il criterio di costo logaritmico).

Un altro algoritmo per risolvere *PRIMALITÀ* fa uso del teorema di Fermat.

Piccolo teorema di Fermat

Se n è primo, allora:

$$\forall b, 2 \leq b < n: \qquad b^{n-1} \bmod n = 1$$

Si può scegliere b randomicamente e vedere se è verificata questa proprietà:

- Se la proprietà non è verificata, allora sicuramente n non è un numero primo, ed è sicuramente composto
- Altrimenti n potrebbe essere un numero primo, ma non se ne ha la sicurezza

Test di primalità di Fermat

```
boolean isPrime1(int n)
b = random(2, n − 1)
if b^{n−1} mod n ≠ 1 then
    return false
return true
```

Questo algoritmo è corretto?

Esistono numeri composti (pseudo-primi in base b) tali che:

$\exists b, 2 \leq b < n-1 : b^{n-1} \bmod n = 1$

Esempio :

$n = 341 = 11 \times 13$

$2^{340} \bmod 341 = 1$

Quello che possiamo fare è ripetere questa procedura per k volte, diminuendo la possibilità che escano "falsi positivi"

Test di primalità di Fermat

```
boolean isPrime2(int n)
for i = 1 to k do
    b = random(2, n − 1)
    if b^{n−1} mod n ≠ 1 then
        return false
return true
```

Questo algoritmo è corretto?

Esistono numeri composti tali che:

$\forall b, 2 \leq b < n-1 : b^{n-1} \bmod n = 1$

(Numeri di Carmichael)

Dunque più grande sarà k, più aumenterà la probabilità di ottenere un risultato corretto.
Con questo algoritmo però non siamo in grado di determinare la probabilità con cui escano risultati corretti.

Vediamo quindi un altro algoritmo probabilistico. L'algoritmo più famoso per risolvere *PRIMALITÀ* è quello probabilistico di Rabin (1980). Tale algoritmo si basa sul concetto di testimone di compostezza.
Sia $n>2$ il numero intero (dispari, altrimenti si sa a prescindere che non è primo) in ingresso del quale si vuole verificare la primalità, e sia b un intero compreso tra 1 ed $n-1$. Se n è primo, devono valere le tre proprietà seguenti. In primo luogo, $mcd(n,b)=1$ per definizione di numero primo, dove $mcd()$ è il massimo comune divisore. Poi deve essere $b^{n-1} \mod n = 1$ per il Piccolo Teorema di Fermat. Infine, si ha che $b^2 \mod n = 1$ se e solo se $b=1$ oppure $b=n-1$.
Combinando queste tre proprietà dopo aver riscritto $n-1=2^v m$, dove v è il massimo esponente assegnabile a 2 ed m è dispari, si può dimostrare che se n è primo devono valere le condizioni:

(1) $mcd(n,b)=1$

(2) $b^{n-1} \mod n = 1$ oppure $\exists i,\ 0 \leq i \leq v-1\ :\ b^{2^i m} \mod n = -1$

L'intero b è un testimone che n è composto (cioè non primo) se almeno una delle condizioni (1) e (2) è falsa. Se b è un testimone che n è composto, allora n è effettivamente un numero

composto. Se invece b non testimonia che n è composto, allora n può essere primo oppure composto.

Teorema

Se n è primo , allora per ogni intero b, con $2 \leq b < n$ valgono entrambe le seguenti condizioni:

1. $\text{mcd}(n, b) = 1$
2. $b^m \bmod n = 1 \vee \exists i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n = n - 1$

con $n - 1 = m \cdot 2^v$, m dispari

Contrapposizione

Se esiste un intero b, con $2 \leq b < n$ tale che almeno una delle seguenti condizioni è vera:

1. $\text{mcd}(n, b) \neq 1$
2. $b^m \bmod n \neq 1 \wedge \forall i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n \neq n - 1$

allora n è composto

La verifica che b sia un testimone di compostezza per n può essere effettuata usando $O(\log\ n)$ operazioni aritmetiche su numeri interi di $O(\log\ n)$ bit. Infatti, v ed m si possono calcolare dimezzando n-1 per $O(\log\ n)$ volte. La condizione (1) è verificabile in tempo $O(\log\ n)$ con il ben noto algoritmo di Euclide. Il calcolo di $b^m\ mod\ n$ si può svolgere ancora in tempo $O(\log\ n)$ per esponenziazioni successive di b, mentre le potenze $b^{2^i m}\ mod\ n$ si calcolano sempre con lo stesso tempo per quadrature successive di $b^m\ mod\ n$.

Verifica primalità n – Esempio: $n = 229$ numero primo

Sia n un numero dispari	$n = 229$
Sia $n - 1 = m \cdot 2^v$, con m dispari	$228 = 57 \cdot 2^2$
$n - 1$ in binario è pari a: m in binario seguito da v zeri	$\overbrace{111001}^{m=57}\underbrace{00}_{v=2}$
se $\exists b, 2 \leq b < n$ per cui una delle seguenti affermazioni è vera, n è composto:	Proviamo $b = 137$
(1) $\text{mcd}(n, b) \neq 1$	$\text{mcd}(229, 137) = 1$
(2) $b^m \bmod n \neq 1$ **and**	$137^{57} \bmod 229 = 188 \neq 1$
$\forall i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n \neq n - 1$	$137^{57 \cdot 2^0} \bmod 229 = 122 \neq 228$
	$137^{57 \cdot 2^1} \bmod 229 = 228$

Verifica primalità n – Esempio: $n = 221 = 13 \times 17$

Sia n un numero dispari	$n = 221$
Sia $n - 1 = m \cdot 2^v$, con m dispari	$220 = 55 \cdot 2^2$
$n-1$ in binario è pari a: m in binario seguito da v zeri	$\overbrace{110111}^{m=55}\underbrace{00}_{v=2}$
se $\exists b, 2 \leq b < n$ per cui una delle seguenti affermazioni è vera, n è composto:	$b = 137$ (Witness)
(1) $\text{mcd}(n, b) \neq 1$	$\text{mcd}(221, 137) = 1$
(2) $b^m \bmod n \neq 1$ **and**	$137^{55} \bmod 221 = 188 \neq 1$
$\forall i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n \neq n - 1$	$137^{55 \cdot 2^0} \bmod 221 = 188 \neq 220$
	$137^{55 \cdot 2^1} \bmod 221 = 205 \neq 220$

Verifica primalità n – Esempio: $n = 221 = 13 \times 17$

Sia n un numero dispari	$n = 221$
Sia $n - 1 = m \cdot 2^v$, con m dispari	$220 = 55 \cdot 2^2$
$n-1$ in binario è pari a: m in binario seguito da v zeri	$\overbrace{110111}^{m=55}\underbrace{00}_{v=2}$
se $\exists b, 2 \leq b < n$ per cui una delle seguenti affermazioni è vera, n è composto:	$b = 174$ (Strong liar)
(1) $\text{mcd}(n, b) \neq 1$	$\text{mcd}(221, 174) = 1$
(2) $b^m \bmod n \neq 1$ **and**	$174^{55} \bmod 221 = 47 \neq 1$
$\forall i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n \neq n - 1$	$174^{55 \cdot 2^0} \bmod 221 = 47 \neq 220$
	$174^{55 \cdot 2^1} \bmod 221 = 220$

Rabin ha dimostrato che se n è un numero composto, allora ci sono almeno $\frac{3}{4}(n-1)$ numeri compresi tra 1 ed $n-1$ che ne testimoniano la compostezza. Ciò implica che scegliendo a caso un numero b compreso tra 1 ed $n-1$ e verificando se b testimonia la compostezza di n oppure no, c'è una probabilità inferiore ad $\frac{1}{4}$ di rispondere erroneamente che n è primo quando invece è composto.
Questo porta alla progettazione del seguente algoritmo probabilistico:

```
boolean isPrime(int n)
for i = 1 to k do
    b = random(2, n − 1)
    if isComposite(n, b) then
        return false
return true
```

Il precedente algoritmo ripete la verifica di compostezza su k valori indipendenti e uniformemente distribuiti di b e assume che n sia primo se e solo se nessuno di tali k valori è un testimone della compostezza. Mentra la risposta *false* (cioè che n è composto) è sempre corretta, la risposta *true* (cioè che n è primo) è corretta con probabilità maggiore o uguale a $1-\left(\frac{1}{4}\right)^k$. In altri termini, la probabilità di errore, cioè di assumere che n sia primo quando invece è composto, non supera $\left(\frac{1}{4}\right)^k$. Inoltre, poiché il valore di k è una costante, il numero di operazioni aritmetiche della funzione $primo()$ resta $O(\log\ n)$.

L'algoritmo di rabin mostra come la probabilità di errore possa essere resa arbitrariamente piccola iterando il procedimento probabilistico per un opportuno numero di volte, permettendo così la risoluzione probabilistica in tempo polinomiale di molti problemi decisionali.
Per concludere, è interessante notare che fino a qualche anno fa l'approccio probabilistico di Rabin era l'unico conosciuto e non erano noti algoritmi deterministici di costo polinomiale. La situazione è cambiata nel 2002, quando i tre matematici indiani Agrawal, Kayal e Saxena introdussero un algoritmo di complessità polinomiale $O(\log^{12+\epsilon}\ n)$. Da allora, enormi sforzi di ricerca hanno portato alla scoperta di innumerevoli varianti, alcune delle quali basate su congetture matematiche non ancora dimostrate. Al momento la versione migliore sembra essere quella di Pomerance e Lenstra, che ha complessità $O(\log^{6+\epsilon}\ n)$.
Nonostante la presenza di algoritmi deterministici polinomiali, tuttavia, l'algoritmo di Rabin è ancora il più veloce e il più usato.

Vediamo ora *ESPRESSIONE _ POLINOMIALE _ NULLA*

Problema

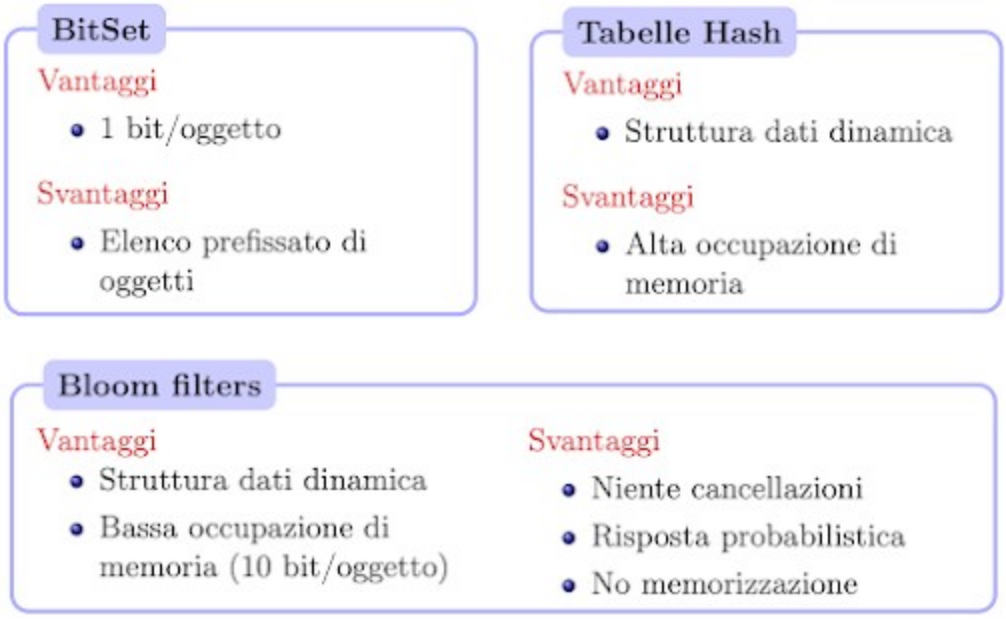
Data un'espressione algebrica polinomiale $p(x_1, \ldots, x_n)$ in n variabili, determinare se p è identicamente nulla oppure no.

Assumiamo che l'espressione non sia in forma di monomi (altrimenti si risolverebbe banalmente verificando se esiste oppure no un coefficiente diverso da 0).
Il miglior algoritmo deterministico noto, basato sul metodo di semplificazione, richiede un tempo di calcolo molto elevato.

È possibile definire un algoritmo probabilistico che dapprima genera casualmente una n-upla di valori $v_1,\ldots,v_n$ e poi valuta $p(v_1,\ldots,v_n)$. Se $p(v_1,\ldots,v_n)\neq 0$, allora l'espressione polinomiale è

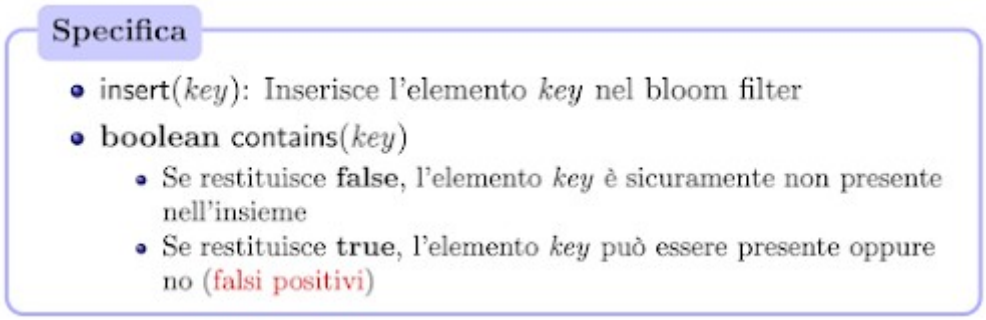
sicuramente non nulla. Se $p(v_1,\ldots,v_n)=0$, c'è la possibilità che l'espressione sia nulla. Indicato con d il grado massimo delle variabili in p, è stato dimostrato che generando ciascun valore v_i tramite $random(1,2d)$ la probabilità di errore non supera $\frac{1}{2}$. Iterando la valutazione di p più volte e assumendo l'espressione nulla se $p(v_1,\ldots,v_n)=0$ per tutte le n-ple generate, la probabilità può essere resa piccola a piacere.

Mettendo assieme i BitSet (come possibilità per rappresentare un insieme) e le tabelle hash, otteniamo i **Bloom Filters**.



Con i Bloom Filters siamo in grado di utilizzare "pochi" bit qualunque sia la dimensione dell'input, però:

- Non permette cancellazioni
- Da una risposta probabilistica
- Non può memorizzare i valori



Nei bloom filter la probabilità di errore dipende da quanta memoria mettiamo a disposizione:

Trade-off fra occupazione di memoria e probabilità di falso positivo

- Sia ϵ la probabilità di falso positivo
- I bloom filter richiedono $1.44 \log_2(1/\epsilon)$ bit per elemento inserito

ϵ	Bit
10^{-1}	4.78
10^{-2}	9.57
10^{-3}	14.35
10^{-4}	19.13

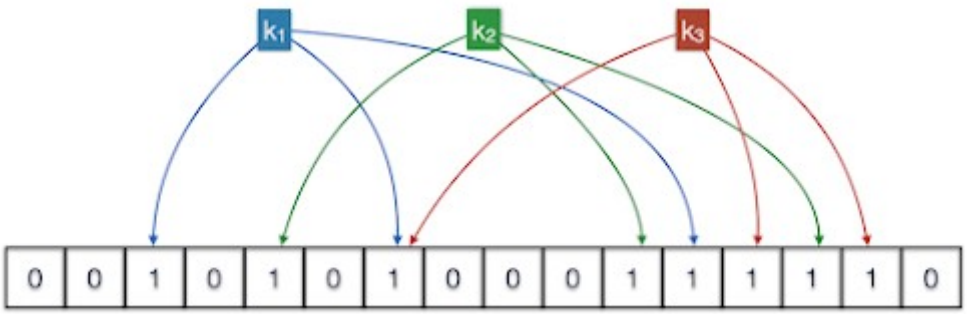
Un'applicazione pratica dei Bloom filter è il Chrome Safe Browsing, infatti Chrome contiene un database delle URL associate a siti con malware, costantemente aggiornato che fino al 2012, memorizzato con un Bloom Filter.

- Chrome verifica l'appartenenza di ogni URL al database
  - Se la risposta è **false**, non appartiene
  - Se la risposta è **true**, potrebbe appartenere e viene fatta una verifica tramite un servizio centralizzato di Google

L'implementazione viene fatta tramite un vettore booleano A di m bit, inizializzato a *false*, e k funzioni hash $h_1, h_2, \ldots, h_k : U \to [0, m-1]$.
Quando dobbiamo inserire un elemento, applichiamo tutte e k le funzioni hash all'elemento e "accendiamo" tutti i bit risultanti. Ovviamente possono verificarsi collisioni e più funzioni hash ci sono più collisioni ci saranno.

insert(key)

for $i = 1$ **to** k **do**
  $A[h_i(key)] =$ **true**



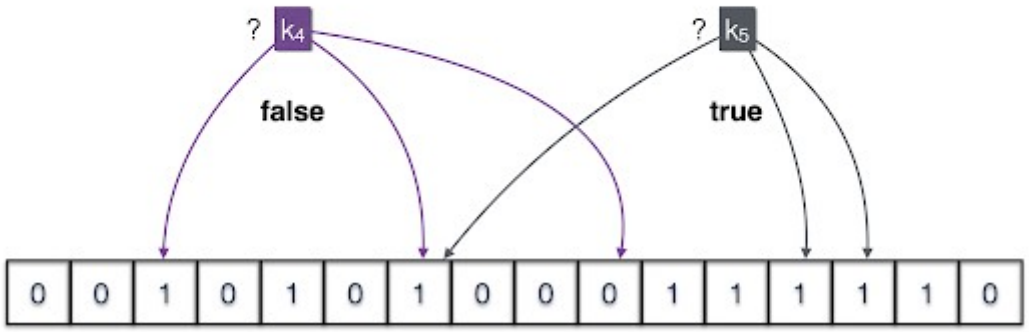
Per verificare se un elemento è presente nel Bloom Filter vado ad analizzare tutti i bit associati a quell'elemento:

- Se non sono tutti a 1, allora sicuramente l'elemento non è presente
- Se sono tutti a 1, allora l'elemento potrebbe essere presente.

boolean contains(key)

for $i = 1$ **to** k **do**
  if $A[h_i(key)] ==$ **false then return false**

return true



Qualche formula (senza dimostrazione)

- Dati n oggetti, m bit, k funzioni hash, la probabilità di un falso positivo è pari a:

$$\epsilon = \left(1 - e^{-kn/m}\right)^k$$

- Dati n oggetti e m bit, il valore ottimale per k è pari a

$$k = \frac{m}{n} \ln 2$$

- Dati n oggetti e una probabilità di falsi positivi ϵ, il numero di bit m richiesti è pari a:

$$m = -\frac{n \ln \epsilon}{(\ln 2)^2}$$

Algoritmi Las Vegas

Il più semplice utilizzo della funzione *random*() consiste nel permutare casualmente i dati di ingresso di un problema. In questo modo una distribuzione casuale sui dati d'ingresso viene imposta preventivamente, anziché assunta successivamente nell'analisi di complessità.

Vediamo questa soluzione risolvendo il problema della *SELEZIONE*, in cui si chiede di identificare il k-esimo elemento più piccolo all'interno di un insieme, senza dover ordinare tale insieme.

Dato un vettore A di n interi, la **mediana** è il numero che si troverebbe nella posizione centrale se questi fossero ordinati. A parte una difficoltà nel fornire la posizione centrale nel caso di n pari (si sceglie arbitrariamente l'intero superiore di $\frac{n}{2}$ sia nel caso pari che nel caso dispari), è facile vedere che la mediana può essere calcolata in tempo $O(n \log n)$ tempo ordinando il vettore. Ma è possibile domandarsi:

- Se questo ordinamento sia necessario
- Se (ancora più importante) il tempo $\Omega(n \log n)$ sia necessario

Ragionando su tali questioni, può essere interessante generalizzare e cercare di risolvere il problema della *SELEZIONE*.

Selezione

Dato un array A contenente n valori e un valore $1 \leq k \leq n$, trovare l'elemento che occuperebbe la posizione k se il vettore fosse ordinato

Si noti che per $k = 1$ ($k = n$) il problema si riduce a quello di trovare il minimo (massimo), mentre $k = \frac{n}{2}$ corrisponde al mediano.

Per piccoli valori di k si può utilizzare uno heap, estraendo $k - 1$ valori prima di trovare il nostro valore.

```
int heapSelect(ITEM [] A, int n, int k)
buildHeap(A)
for i = 1 to k − 1 do
    deleteMin(A, n)
return deleteMin(A, n)
```

La complessità è $O(n + k \log n)$, se k è piccolo, ovvero $k = O\left(\frac{n}{\log n}\right)$, il costo è $O(n)$.

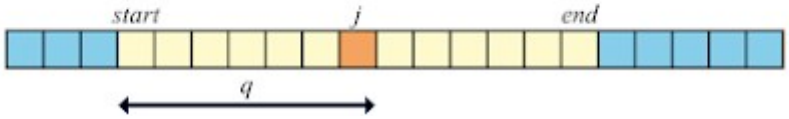
Se k è grande invece, ad esempio $k = \frac{n}{2}$, non va bene.

L'idea quindi è quella di utilizzare un approccio divide-et-impera simile a QuickSort.
Essendo un problema di ricerca non è necessario cercare in entrambe le partizioni come QuickSort, ma basta cercare in una sola di esse. Ovviamente bisogna fare attenzione agli indici.

Per trovare il k-esimo elemento più piccolo senza ordinare il vettore, si può utilizzare la procedura $pivot()$ di QuickSort. Ricordiamo che la procedura $pivot(A, first, last)$ scambia tra loro elementi di $A[first \ldots last]$ e restituisce l'indice j del "perno" tale che $A[i] \leq A[j]$, per $first \leq i \leq j$ e $A[i] \geq A[j]$, per $j \leq i \leq last$. La seguente procedura $selection()$, proposta da Hoare (1961), richiama $pivot()$ ed individua così il q-esimo elemento più piccolo di $A[first \ldots last]$,

dove $q = j - first + 1$ è il numero di elemento di $A[first \dots j]$. Se $k \leq q$, allora $selection()$ è riapplicata ad $A[first \dots j]$; se invece $k > q$, allora $selection()$ è riapplicata ad $A[j+1 \dots last]$, ma per ricercare il $(k-q)$-esimo elemento più piccolo. La chiamata iniziale è $selection(A, 1, n, k)$.

```
ITEM selection(ITEM[] A, int first, int last, int k)
if first == last then
    return A[first]
else
    int j = pivot(A, first, last)
    int q = j − first + 1
    if k == q then
        return A[j]
    else if k < q then
        return selection(A, first, j − 1, k)
    else
        return selection(A, j + 1, last, k − q)
```



L'algoritmo nel caso pessimo si comporta "male" come Quicksort, nel caso ottimo invece si comporta molto bene.

Caso pessimo

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n-1) + n & n > 1 \end{cases}$$
$$T(n) = O(n^2)$$

Caso ottimo

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n/2) + n & n > 1 \end{cases}$$
$$T(n) = O(n)$$

Nel caso medio, assumendo che $pivot()$ restituisca con la stessa probabilità una qualsiasi posizione j del vettore A, il numero medio di confronti $T(n)$ tra elementi di A effettuati da $selection()$ è dato dalla relazione di ricorrenza:

Caso medio

Assumiamo che pivot() restituisca con la stessa probabilità una qualsiasi posizione j del vettore A

$$T(n) = n + \frac{1}{n}\sum_{q=1}^{n} T\left(\max\{q-1, n-q\}\right) \qquad \text{Media su } n \text{ casi}$$

$$\leq n + \frac{1}{n}\sum_{q=\lfloor n/2 \rfloor}^{n-1} 2T(q), \qquad \text{per } n > 1$$

Esempi

- $n = 4$: $\max\{0,3\} + \max\{1,2\} + \max\{2,1\} + \max\{3,0\}$
- $n = 5$: $\max\{0,4\} + \max\{1,3\} + \max\{2,2\} + \max\{3,1\} + \max\{4,0\}$

perchè $max(q-1, n-q)$ è uguale a $q-1$ se $q \geq \frac{n}{2}$, mentre è uguale ad $n-q$ se $q < \frac{n}{2}$.

n è sempre il costo del calcolo del pivot e faccio la media di tutti i pivot possibili, però tengo sempre in conto di essere sfortunato e assumo di andare sempre nella parte più grande (con *max*).

Effettuiamo un ragionamento informale per cercare di ricavare una buona soluzione sulla quale applicare la tecnica di dimostrazione per tentativi.
In media, $selection()$ dovrebbe essere richiamata ogni volta su una porzione dimezzata del vettore A. Poichè la $pivot()$ richiede tempo lineare nella lunghezza della porzione di A, il numero complessivo di confronti dovrebbe crescere come $n + \frac{n}{2} + \frac{n}{4} + \ldots + 1 \leq 2n$. Pertanto, una soluzione promettente di $T(n)$ con la quale tentare sembra essere $O(n)$.

Tentiamo quindi con $T(n) \leq cn$ per qualche costante c. La base dell'induzione è banalmente verificata perché $0 = T(0) = T(1) \leq c$. Si assuma che valga l'ipotesi induttiva $T(q) \leq cq$ per ogni $q \leq n-1$. Sostituendo, si ricava:

$$
\begin{aligned}
T(n) &\leq n + \frac{1}{n}\sum_{q=\lfloor n/2 \rfloor}^{n-1} 2 \cdot cq \leq n + \frac{2c}{n}\sum_{q=\lfloor n/2 \rfloor}^{n-1} q && \text{Sostituzione, raccolgo } 2c \\
&= n + \frac{2c}{n}\left(\sum_{q=1}^{n-1} q - \sum_{q=1}^{\lfloor n/2 \rfloor - 1} q\right) && \text{Sottrazione prima parte} \\
&= n + \frac{2c}{n} \cdot \left(\frac{n(n-1)}{2} - \frac{(\lfloor n/2 \rfloor)(\lfloor n/2 \rfloor - 1)}{2}\right) \\
&\leq n + \frac{2c}{n} \cdot \left(\frac{n(n-1)}{2} - \frac{(n/2 - 1)(n/2 - 2)}{2}\right) && \text{Rimozione limite inferiore} \\
&= n + \frac{\cancel{2}c}{n} \cdot \frac{\left(n^2 - n - \left(1/4\, n^2 - 3/2\, n + 2\right)\right)}{\cancel{2}} \\
&= n + c/n \cdot \left(3/4\, n^2 + 1/2\, n - 2\right) \\
&\leq n + c/n \cdot \left(3/4\, n^2 + 1/2\, n\right) = n + 3/4\, cn + 1/2 \overset{?}{\leq} cn && \text{Vera per } c \geq 6 \text{ e } n \geq 1
\end{aligned}
$$

Dunque la relazione $T(n) \leq cn$ vale per ogni $n \geq 1$ e $c \geq 6$. Pertanto, la funzione $selection()$ trova il k-esimo elemento più piccolo di A in tempo lineare nel caso medio.

Tutta questa analisi è partita dall'assunzione che $pivot()$ restituisca con la stessa probabilità una qualsiasi posizione j del vettore A; questo non è vero per particolari casi dell'input, quali ad esempio un vettore già ordinato. Per ovviare a questo problema, è possibile forzare una scelta casuale del valore su cui impostare il pivot, scambiando subito prima della chiamata $pivot()$ un elemento casuale fra *first* e *last* con l'elemento $A[first]$ (questo accorgimento vale anche per QuickSort).

$$A[random(first, last)] \leftrightarrow A[first]$$

Gli effetti sfavorevoli dovuti al caso pessimo sono pertanto ridotti e l'analisi della complessità nel caso medio risulta così più affidabile.

Tanto per curiosità, esistono altri algoritmi più sofisticati per la $selection()$ che richiedono tempo $O(n)$ nel caso pessimo, il primo dei quali è stato quello di Blum-Floyd-Pratt-Rivest-Tarjan (1973), deterministico.

Supponiamo ora di avere un algoritmo "black box" che mi ritorni il valore mediano di n valori in tempo $O(n)$. Per risolvere $selection()$ mi basta mettere il mediano in prima posizione, chiamare $pivot()$ e in questa maniera sono sicuro che verrà sempre fuori un caso ottimo $O(n)$.

Purtroppo una black box di questo tipo è impossibile averla, però, con meno pretese, è possibile avere una black box che mi ritorni un valore che dista al più $\frac{3}{10}n$ dal mediano (nell'ordinamento). In questa maniera posso utilizzare questo valore come pivot, e sicuramente evito il caso pessimo, avvicinandomi al caso ottimo.

Potrei utilizzare l'algoritmo della mediana approssimata per scegliere il pivot in perno(), e quindi ottenere partizioni che sono al più $\frac{3}{4}n$. A questo punto la divisione darebbe origine alla seguente equazione di ricorrenza:

$$T(n) = T(3n/4) + O(n)$$

Per il Master Theorem, otteniamo che la complessità è comunque $O(n)$.

Idea

- Suddividi i valori in gruppi di 5. Chiameremo l'i-esimo gruppo S_i, con $i \in [1, \lceil n/5 \rceil]$
- Trova il mediano M_i di ogni gruppo S_i
- Tramite una chiamata ricorsiva, trova il mediano m delle mediane $[M_1, M_2, \ldots, M_{\lceil n/5 \rceil}]$
- Usa m come pivot e richiama l'algoritmo ricorsivamente sull'array opportuno, come nella selection() randomizzata
- Quando la dimensione scende sotto una certa dimensione, possiamo utilizzare un algoritmo di ordinamento per trovare il mediano

```
ITEM select(ITEM[] A, int first, int last, int k)
% Se la dimensione è inferiore ad una soglia (10), ordina il vettore e
% restituisci il k-esimo elemento di A[first...last]
if last − first + 1 ≤ 10 then
    InsertionSort(A, first, last)        % Versione con indici inizio/fine
    return A[first + k − 1]

% Divide A in ⌈n/5⌉ sottovettori di dim. 5 e ne calcola la mediana
M = new int[1...⌈n/5⌉]
for i = 1 to ⌈n/5⌉ do
    M[i] = median5(A, first + (i − 1) · 5, last)

% Individua la mediana delle mediane e usala come perno
ITEM m = select(M, 1, ⌈n/5⌉, ⌈⌈n/5⌉/2⌉)
int j = pivot(A, first, last, m)         % Versione con m in input
[...]
```

```
ITEM select(ITEM[] A, int first, int last, int k)
[...]
% Calcola l'indice q di m in [first...last]
% Confronta q con l'indice cercato e ritorna il valore conseguente
int q = j − first + 1
if q == k then
    return m
else if q < k then
    return select(A, first, q − 1, k)
else
    return select(A, q + 1, last, k − q)
```

- Il calcolo dei mediani $M[]$ richiede al più $6\lceil n/5 \rceil$ confronti.
- La prima chiamata ricorsiva dell'algoritmo select() viene effettuata su $\lceil n/5 \rceil$ elementi
- La seconda chiamata ricorsiva dell'algoritmo select() viene effettuata al massimo su $7n/10$ elementi (esattamente $n - 3\lceil \lceil n/5 \rceil /2 \rceil$)
- L'algoritmo select() esegue nel caso pessimo $O(n)$ confronti

$$T(n) = T(n/5) + T(7n/10) + 11/5n$$

1	4	**7**	10	13	2	5	**8**	11	14	3	6	**9**	12	15

Quale preferire?

- Algoritmo probabilistico Las Vegas in tempo atteso $O(n)$
- Algoritmo deterministico in tempo $O(n)$, con fattori moltiplicativi più alti

Note storiche

- Nel 1883 Lewis Carroll (!) notò che il secondo premio nei tornei di tennis non veniva assegnato in maniera equa.
- Nel 1932, Schreier dimostrò che $n + \log n - 2$ incontri sono sempre sufficienti per trovare il secondo posto
- Nel 1973, a opera di Blum, Floyd, Pratt, Rivest e Tarjan, appare il primo algoritmo deterministico

La soluzione deterministica consiste nel suddividere tutti i valori in blocchetti da 5 elementi e trovare il mediano per ogni blocchetto. Tramite chiamata ricorsiva poi trovare il mediano dei mediani e usare quel mediano come pivot. Non appena la dimensione scende sotto una certa dimensione, utilizziamo un algoritmo di ordinamento classico per trovare il mediano.

Teoria dell' NP-completezza

Per quasi tutti i problemi trattati nei capitoli precedenti (ad esempio *CAMMINI MINIMI*, *ORDINAMENTO*, *STRING MATCHING*), oltre ad algoritmi banali del tipo "prova tutte le possibilità" (backtrack), erano stati introdotti algoritmi molto efficienti se non addirittura ottimi. Finora infatti abbiamo considerato solo problemi con soluzioni in tempo polinomiale, ovvero $O(n^k)$ per qualche k.

Esistono però dei problemi per cui, al contrario, non solo non è conosciuto un algoritmo migliore di uno esponenziale, ma si suppone addirittura che un tale algoritmo non esista!
In più esistono problemi per i quali non esiste proprio alcuna soluzione (esistono comunque degli algoritmi non completamente corretti ma che possono comunque risultare utili).

In questo capitolo discuteremo una classe di problemi per cui non è chiaro se esiste un algoritmo polinomiale che li risolva oppure no. I problemi appartenenti a questa classe sono "tutti sulla stessa barca", infatti sono tra loro computazionalmente correlati e destinati ad "affogare" o "salvarsi" insieme (possono essere tutti risolti in tempo polinomiale, oppure nessuno).
Ognuno può essere risolto in tempo superpolinomiale, ma per nessuno è noto un algoritmo polinomiale. Inoltre, vedremo in questo capitolo che se uno solo fosse risolubile in tempo polinomiale, allora lo sarebbero tutti, ma si suppone che non esista alcun algoritmo polinomiale per nessuno di essi.

Un **problema astratto** è una relazione binaria $R \subseteq I \times S$ tra un insieme I di istanze del problema e un insieme S di soluzioni.

Esempio – SHORTEST-PATH

- Un'istanza è una quadrupla (V, E, u, v)
- Una soluzione è una sequenza ordinata di vertici $v_1 \dots v_k$
- Nota: possono esistere più soluzioni associate alla stessa istanza

In base al tipo di problema potrebbero esistere più soluzioni associate alla stessa istanza.

Ricordiamo le diverse tipologie di problemi che esistono:

- Problemi di **ottimizzazione**
  Data un'istanza, trovare la "migliore" soluzione (secondo criteri specifici).
  Ad esempio $SHORTEST\text{-}PATH\ \ (OPT)$: Dati un grafo G e due nodi u e v, trovare il cammino più breve fra di essi
- Problemi di **ricerca**
  Data un'istanza, trovare una possibile soluzione fra quelle esistenti
  Ad esempio $PATH$: Dati un grafo G e due nodi u e v, trovare un cammino fra di essi
- Problemi di **decisione**
  Data un'istanza, verificare se soddisfa o meno una data proprietà. La relazione R è una funzione $R : I \rightarrow \{0,1\}$ (visto che non può assumere sia il valore *true* che il valore *false*)
  Ad esempio $SHORTEST\text{-}PATH\ \ (DEC)$: Dati un grafo G, due nodi u e v e un valore k, esiste un cammino tra u e v di lunghezza minore o uguale a k?

In questa lezione tratteremo solo problemi decisionali, visto che le versioni decisionali sono matematicamente più semplici da trattare. I problemi decisionali hanno una relazione molto stretta con i corrispondenti problemi di ottimizzazione, infatti

se posso risolvere efficientemente la versione di ottimizzazione ⇒ posso risolvere efficientemente la versione di decisione

dunque per le proprietà logiche

se **non** posso risolvere efficientemente la versione di decisione ⇒ **non** posso risolvere efficientemente la versione di ottimizzazione

Esempio – Shortest path

- Se conosco il cammino più breve, posso rispondere alla domanda decisionale per qualunque valore di k

In più per tali problemi non dobbiamo preoccuparci del tempo necessario per restituire la risposta, ma solo del tempo di calcolo. Stesso ragionamento per lo spazio di memoria.

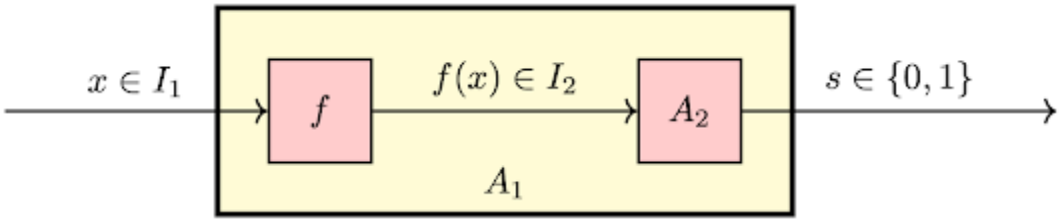
Purtroppo, i problemi "interessanti" sono spesso di ottimizzazione; questo non si accorda con la nostra scelta di utilizzare problemi di decisione. Tuttavia, è possibile "trasformare" un problema di ottimizzazione in un problema equivalente di decisione, e viceversa.

Riducibilità polinomiale

L'output dei problemi decisionali è sempre o *true* o *false*.

Vediamo la definizione di **riduzione polinomiale**.
Dati due problemi decisionali $R_1 \subseteq I_1 \times \{0,1\}$ e $R_2 \subseteq I_2 \times \{0,1\}$, R_1 è **riducibile polinomialmente** a R_2, e si scrive $R_1 \leq_p R_2$, se esiste una funzione $f : I_1 \to I_2$ con le seguenti proprietà:

- f è calcolabile in tempo polinomiale
- per ogni istanza x del problema R_1 e ogni soluzione $s \in \{0,1\}$, $(x,s) \in R_1 \Leftrightarrow (f(x),s) \in R_2$



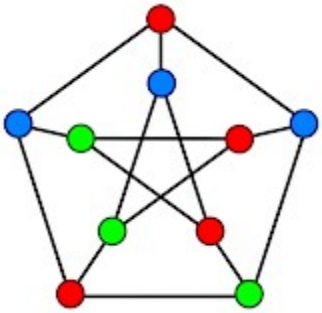
In sintesi, R_1 è riducibile polinomialmente a R_2 se, con le dovute trasformazioni, i due problemi danno la stessa risposta per ogni istanza.
In pratica, se si dimostra che $R_1 \leq_p R_2$, si trasforma un qualsiasi dato d'ingresso per R_1 in un particolare dato d'ingresso per R_2 e quindi R_1 risulta essere, in un certo senso, un sottoproblema di R_2!

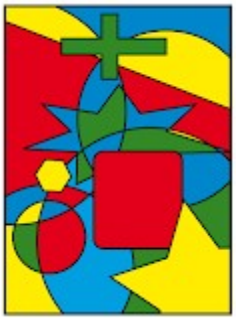
Vediamo ora una serie di problemi.

Il **problema della colorazione dei grafi** (*GRAPH - COLORING*):

Dati un grafo non orientato $G=(V,E)$ e un insieme di colori C, una **colorazione** dei vertici è un assegnamento $f : V \to C$:

- che "colora" ogni nodo con uno dei valori in C
- tale per cui nessuna coppia di nodi adiacenti ha lo stesso colore





Il *GRAPH - COLORING* di **ottimizzazione** è:
Dato un grafo non orientato $G=(V,E)$, restituire la colorazione che richiede il minimo numero di colori
Il *GRAPH - COLORING* **decisionale** è:
Dato un grafo non orientato $G=(V,E)$ e un valore k, determinare se esiste una colorazione di G con k colori

Il **problema del Sudoku** (*SUDOKU - PRECOLORED*):
Il problema generale del Sudoku richiede di inserire dei numeri fra 1 e n^2 in una matrice di $n^2 \times n^2$ elementi suddivisa in $n \times n$ sottomatrici di dimensione $n \times n$, in modo tale che nessun numero compaia più di una volta in ogni riga, colonna e sottomatrice.
Il *SUDOKU - PRECOLORED* **decisionale** è:
Data una matrice $n^2 \times n^2$ elementi con alcuni numeri già presenti nella matrice, determinare se esiste un modo per assegnare i numeri in modo da rispettare le regole del Sudoku

Vediamo un esempio di **riduzione da problema particolare a problema generale**.
Se noi costruiamo un grafo, in cui ogni vertice corrisponde ad una casella e costruiamo i lati su tutte le caselle che non possono avere lo stesso numero

Riduzione in tempo polinomiale

$V = \{(x,y) : 1 \le x \le n^2, 1 \le y \le n^2\}$
$[(x,y),(x',y')] \in E \Leftrightarrow$

- $x = x'$; oppure,
- $y = y'$; oppure,
- $(\lceil x/n \rceil = \lceil x'/n \rceil) \wedge (\lceil y/n \rceil = \lceil y'/n \rceil)$

$C = \{1,\ldots,n\}$.

Possiamo risolvere $SUDOKU\text{-}PRECOLORED$ utilizzando $GRAPH\text{-}COLORING$.
Infatti il sudoku è risolvibile solo se il grafo che ricaviamo in questa maniera è colorabile.

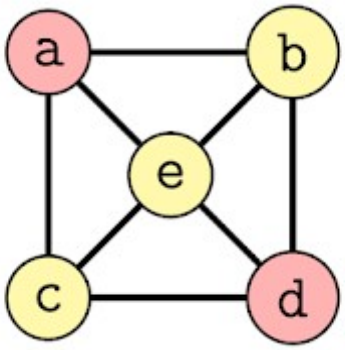
Dunque vale

$$SUDOKU\text{-}PRECOLORED \leq_p GRAPH\text{-}COLORING$$

Se abbiamo una soluzione algoritmica per la colorazione, allora abbiamo una soluzione algoritmica per il Sudoku.

Il **problema dell'insieme indipendente** ($INDEPENDENT\text{-}SET$):
Sia dato un grafo non orientato $G=(V,E)$; un insieme $S \subseteq V$ è un insieme indipendente $\Leftrightarrow$ nessun arco unisce due nodi in S.

$$\forall (x,y) \in E \ : \ x \notin S \vee y \notin S$$



L'$INDEPENDENT\text{-}SET$ di **ottimizzazione** è:
Dato un grafo non orientato $G=(V,E)$, restituire il più grande insieme indipendente presente nel grafo.
L'$INDEPENDENT\text{-}SET$ **decisionale** è:
Dato un grafo non orientato $G=(V,E)$ e un valore k, determinare se esiste un insieme indipendente di dimensione almeno k.

Questo è un esempio di **packing problem**, un problema in cui si cerca di selezionare il maggior numero di oggetti, la cui scelta è però soggetta a vincoli di esclusione (lo zaino è un packing problem).

Il **problema della copertura di vertici** ($VERTEX\text{-}COVER$):
Dato un grafo non orientato $G=(V,E)$, un insieme $S \subseteq V$ è una copertura di vertici $\Leftrightarrow$ ogni arco in E ha almeno un estremo in S

$$\forall (x,y) \in E \ : \ x \in S \vee y \in S$$

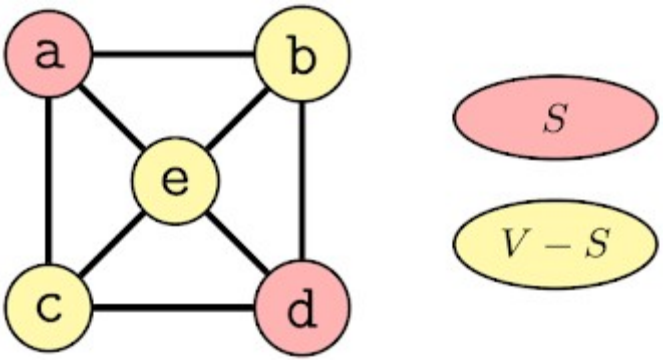
Il $VERTEX\text{-}COVER$ di **ottimizzazione** è:
Dato un grafo non orientato $G=(V,E)$, restituire la copertura dei vertici di dimensione minima
Il $VERTEX\text{-}COVER$ **decisionale** è:
Dato un grafo non orientato $G=(V,E)$ e un valore k, determinare se esiste una copertura di vertici di dimensione al massimo k

Questo è un esempio di **covering problem**, un problema in cui si cerca di ottenere il più piccolo insieme in grado di coprire un insieme arbitrario di oggetti con il più piccolo sottoinsieme di questi oggetti.

Vediamo una **riduzione tra problemi duali**.
Dimostriamo che i due problemi sono riducibili polinomialmente l'uno con l'altro.

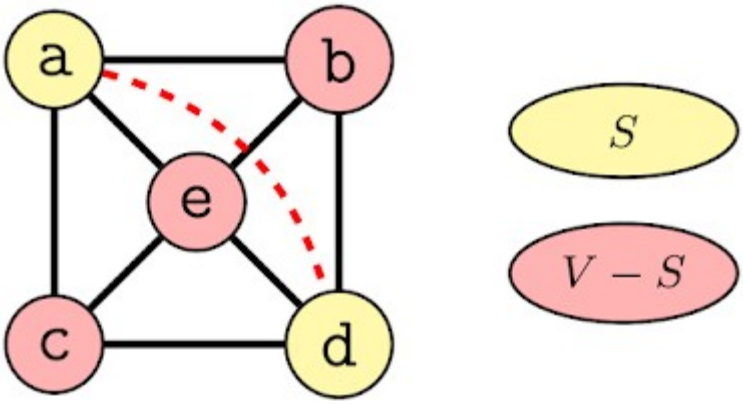
Se $S \subseteq V$ è un independent set, allora $V - S$ è un vertex cover.
Se S è un insieme indipendente:

- Ogni arco (x, y) non può avere entrambi gli estremi in S
- Dunque almeno uno dei due vertici deve appartenere a $V - S$ (come volevasi dimostrare)



Se $V - S$ è un vertex cover, allora $S \subseteq V$ è un independent set.
Supponiamo per assurdo che S non sia un independent set:

- Allora esiste un arco (x, y) che unisce due nodi in S
- Dunque nessuno dei due estremi sta in $V - S$
- Questo implica che $V - S$ non è vertex cover, ed è assurdo



Abbiamo dimostrato che i due problemi sono equivalenti, ovvero:

$$VERTEX\text{-}COVER \leq_p INDEPENDENT\text{-}SET$$
$$INDEPENDENT\text{-}SET \leq_p VERTEX\text{-}COVER$$

Il **problema di soddisfacibilità di formule booleane** (*SATisfiability*):
Dato un insieme V contenente *n* variabili:

- un **letterale** è una variabile v oppure la sue negazione $\underline{v}$
- una **clausola** è una disgiunzione di letterali (letterali separati da or)
- una **formula in forma normale congiuntiva** è una congiunzione di clausole (clausole separate da and)

Esempio

$$(x \vee \overline{y} \vee z) \wedge (\overline{x} \vee w) \wedge y$$

Data un'espressione in forma normale congiuntiva, il problema della soddisfacibilità consiste nel decidere se esiste una assegnazione di valori di verità alle variabili che rende l'espressione vera.

Esempio

$$(x \vee \overline{y} \vee z) \wedge (\overline{x} \vee w) \wedge y$$

$$x = \mathbf{true}, y = \mathbf{true}, w = \mathbf{true}, z = \mathbf{true}\}$$

$$(\mathbf{true} \vee \mathbf{false} \vee \mathbf{true}) \wedge (\mathbf{false} \vee \mathbf{true}) \wedge \mathbf{true}$$

Il $3\text{-}SAT$ è una forma ridotta di SAT:
Data un'espressione in forma normale congiuntiva in cui le clausole hanno esattamente 3 letterali, il problema della soddisfacibilità consiste nel decidere se esiste una assegnazione di valori di verità alle variabili che rende l'espressione vera.

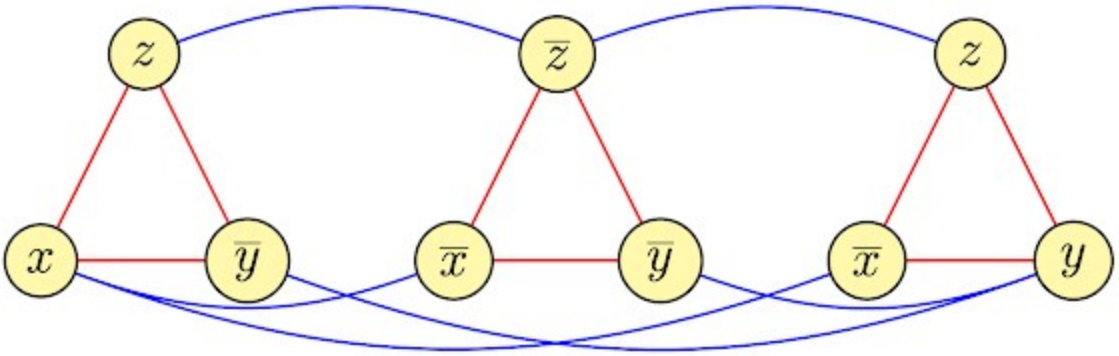
Essendo una forma ridotta di SAT, sappiamo che

$$3\text{-}SAT \leq_p SAT$$

Vogliamo dimostrare che $3\text{-}SAT \leq_p INDEPENDENT\text{-}SET$

Data una formula $3\text{-}SAT$ costruiamo un grafo nella seguente maniera:

- Per ogni clausola, aggiungiamo un terzetto di nodi, collegati fra di loro da archi
- Per ogni letterale che compare in modo normale e in modo negato, aggiungere un arco fra di essi (arco di conflitto).



La formula $3\text{-}SAT$ è soddisfacibile se e solo se è possibile trovare un independent set di dimensione esattamente k, dunque

$$3\text{-}SAT \leq_p INDEPENDENT\text{-}SET$$

Inoltre è possibile dimostrare che

$$SAT \leq_p 3\text{-}SAT$$

Infatti è possibile trasformare una formula SAT in una formula $3\text{-}SAT$ usando due semplici trucchi:

- Se la clausola è più lunga di tre elementi, si introduce una nuova variabile e si divide la clausola in due:
  $(a \vee b \vee c \vee d) \equiv (a \vee b \vee z) \wedge (\underline{z} \vee c \vee d)$
- Se la clausola è più corta di tre elementi si fa "padding":
  $(a \vee b) \equiv (a \vee a \vee b)$

Visto che la nozione di riducibilità polinomiale gode della proprietà transitiva abbiamo:

$$SAT \leq_p 3\text{-}SAT \leq_p INDEPENDENT\text{-}SET \leq_p VERTEX\text{-}COVER$$

Classi P, $PSPACE$

Definizione di **algoritmo**:
Dati un problema di decisione R e un algoritmo A (scritto in un modello di calcolo Turing-equivalente) che lavora in tempo $f_t(n)$ e spazio $f_s(n)$, diciamo che A risolve R se A restituisce s su un'istanza x se e solo se $(x, s) \in R$.

Definizione di **classi di complessità**:
Data una qualunque funzione $f(n)$, chiamiamo:

- $TIME(f(n))$ l'insieme dei problemi decisionali risolvibili da un algoritmo che lavora in tempo $O(f(n))$
- $SPACE(f(n))$ l'insieme dei problemi decisionali risolvibili da un algoritmo che lavora in spazio $O(f(n))$

Tuttavia, queste definizioni sono troppo dettagliate, quindi definiamo alcune classi più ampie.

La **classe** P è la classe dei problemi decisionali risolvibili in tempo polinomiale nella dimensione n dell'istanza di ingresso:

$$P = ¿\, c = 0 \,¿ \infty \, TIME\left(n^c\right)$$

La **classe** $PSPACE$ è la classe dei problemi decisionali risolvibili in spazio polinomiale nella dimensione n dell'istanza di ingresso:

$$PSPACE = ¿\, c = 0 \,¿ \infty \, SPACE\left(n^c\right)$$

Notiamo che $P \subseteq PSPACE$ perchè sicuramente lo spazio che userò per trovare una soluzione sarà al massimo uguale al tempo che ci impiego, oppure più piccolo.

Notiamo che secondo questa definizione, lo zaino non appartiene a P.

Certificati polinomiali

Oltre ad essere tutti di tipo decisionale, i problemi elencati precedentemente hanno in comune un'altra semplice proprietà. Benché sia estremamente difficile rispondere Sì o No, è abbastanza facile convincersi se un'ipotetica soluzione dia risposta Sì. Formalmente, tutti questi problemi possiedono un certificato polinomiale, cioè un algoritmo che, data una presunta soluzione del problema, verifica in tempo polinomiale che tale soluzione sia effettivamente una soluzione che dà risposta Sì.

Dato un problema decisionale R e un'istanza di input x tale che $\left(x, true\right) \in R$, un **certificato** è un insieme di informazioni che permette di provare che $\left(x, true\right) \in R$.

Esempi

- SAT: un assegnamento di verità alle variabili della formula
- GRAPH-COLORING: un'associazione nodo-colore $f : V \to \{1, \ldots, k\}$
- INDEPENDENT-SET: un sottoinsieme $S \subseteq V$ di k elementi

Tutti questi "certificati" hanno dimensione polinomiale nella dimensione dell'input.

I certificati suddetti possono essere verificati in tempo polinomiale:

- SAT: si calcola il valore di verità della formula a partire dall'assegnamento di verità delle variabili in tempo $O(n)$
- GRAPH-COLORING: si verifica che nodi adiacenti non abbiano lo stesso colore in tempo $O(m+n)$
- INDEPENDENT-SET: si verifica che nodi in S non abbiano nodi adiacenti in S in tempo $O(m+n)$

Classe NP

Fino ad ora, abbiamo visto solo algoritmi deterministici, ovvero algoritmi in cui ogni passo è determinato dallo stato attuale del sistema. Un modo alternativo per vedere la classe NP (da cui la N) è considerare algoritmi non deterministici.
In un algoritmo non deterministico, tutte le volte che ci si pone di fronte ad una scelta, c'è la possibilità di "indovinare" la strada giusta. Ad esempio, nel problema SAT, l'algoritmo potrebbe indovinare ad ogni passo il valore di una variabile, e risolvere quindi il problema in tempo lineare. La computazione non è più quindi espressa come una sequenza di passi, ma con un albero di possibili scelte. Nel caso di SAT, il problema corrisponde nel cercare un cammino nell'albero che porti ad una foglia, tale per cui gli assegnamenti portano al successo.

La **classe** NP è la classe di tutti i problemi decisionali risolvibili in tempo polinomiale con algoritmi non deterministici (ovvero non è detto che seguano sempre gli stessi passi per arrivare alla soluzione).

Un altra definizione è che la classe NP è l'insieme di tutti i problemi che ammettono un **certificato verificabile in tempo polinomiale**.
Esistono infatti dei problemi che non possono essere associati ad un certificato di dimensione polinomiale.

Quantified Boolean Formula (QBF)

Il problema QBF è una generalizzazione del problema SAT nel quale ad ogni variabile possono essere applicati quantificatori universali e esistenziali

Esempio

$$\forall x\ \exists y\ \exists z\ ((x \vee z) \wedge y)$$

Esercizio...

Progettate un certificato per QCB che possa essere verificato in tempo polinomiale

Si ritiene che un certificato del genere non esista

- $\mathbb{P} \subseteq \mathbb{NP}$: infatti, tutti i problemi decidibili in tempo polinomiale possono semplicemente ignorare il certificato e prendere la loro decisione;
- $\mathbb{NP} \subseteq \mathbb{PSPACE}$: la fase deterministica di verifica può essere condotta solo se il certificato ha dimensioni polinomiali;
- $\mathbb{PSPACE} \subseteq \mathbb{EXPTIME}$: intuitivamente, n^c caselle di memoria possono trovarsi al più in 2^{n^c} stati diversi, che può essere necessario esplorare tutti;

Si ritiene che tutte queste inclusioni siano in realtà strette, ma sono tutti problemi aperti.
Perché studiamo queste classi? Perché $\mathbb{P}$ è l'insieme dei problemi "gestibili"; si noti che $\mathbb{P}$ include anche complessità come n^{100}, ma queste raramente si presentano in pratica, visto che la maggior parte dei problemi si trova entro la complessità n^3.

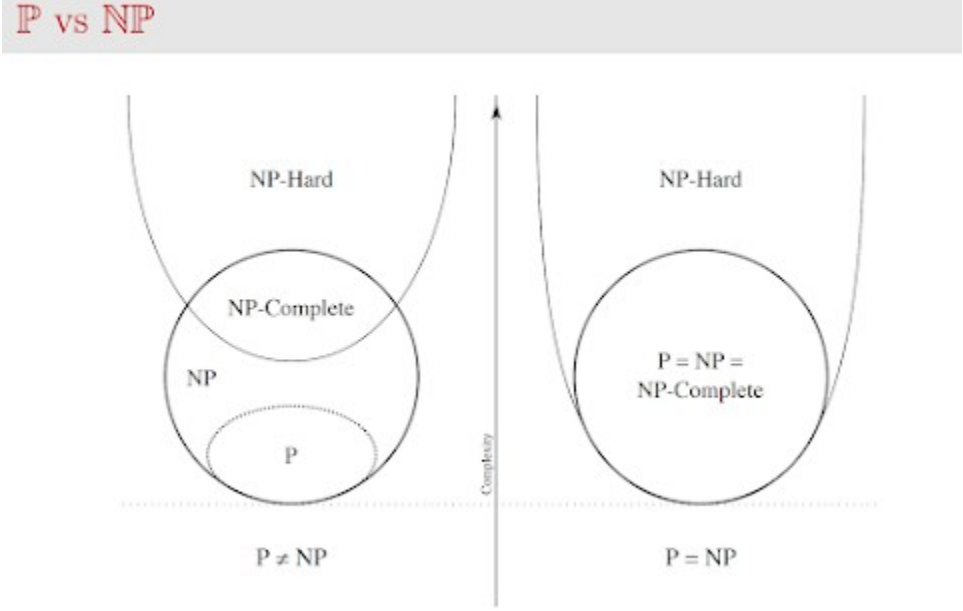
Problemi NP-Completi

Vediamo come prima cosa un lemma.
Se $R_1 \leq_p R_2$ e $R_2 \in P$, allora anche R_1 è contenuto in P.
Dimostrazione:
Sia $T_f(n) = O\left(n^{k_f}\right)$ il tempo necessario per trasformare un input di R_1 in un input di R_2 tramite una funzione f.
Sia $T_2(n) = O\left(n^{k_2}\right)$ il tempo necessario per risolvere R_2.
La complessità per risolvere R_1 è $T_1(n) = O\left(n^{k_f \cdot k_2}\right)$ che è polinomiale

Vediamo altre definizioni.
Un problema decisionale R si dice NP**-arduo** se ogni problema $Q \in NP$ è riducibile polinomialmente a R ($Q \leq_p R$).
Un problema decisionale R si dice NP**-completo** se appartiene alla classe NP ed è NP-hard (infatti possono esistere problemi NP-hard, ma non appartenenti a NP)

Ci chiediamo se $P = NP$? La risposta sarebbe sì se un qualunque problema decisionale NP-completo appartenesse a P. In questa situazione la classe NP non esisterebbe più e tutti i problemi apparterrebbero a P. In questo momento non sappiamo se vale o meno

Ecco la visualizzazione grafica delle due situazioni possibili.



Se troviamo una soluzione per uno qualunque di questi problemi, la troviamo per tutti.

Esistono problemi NP-completi?
Dimostrare che un problema è contenuto in NP è semplice.
Dimostrare che un problema è NP-completo richiede una dimostrazione difficile, apparentemente impossibile: Tutti i problemi in NP sono riducibili polinomialmente a tale problema, anche i problemi che non conosciamo!

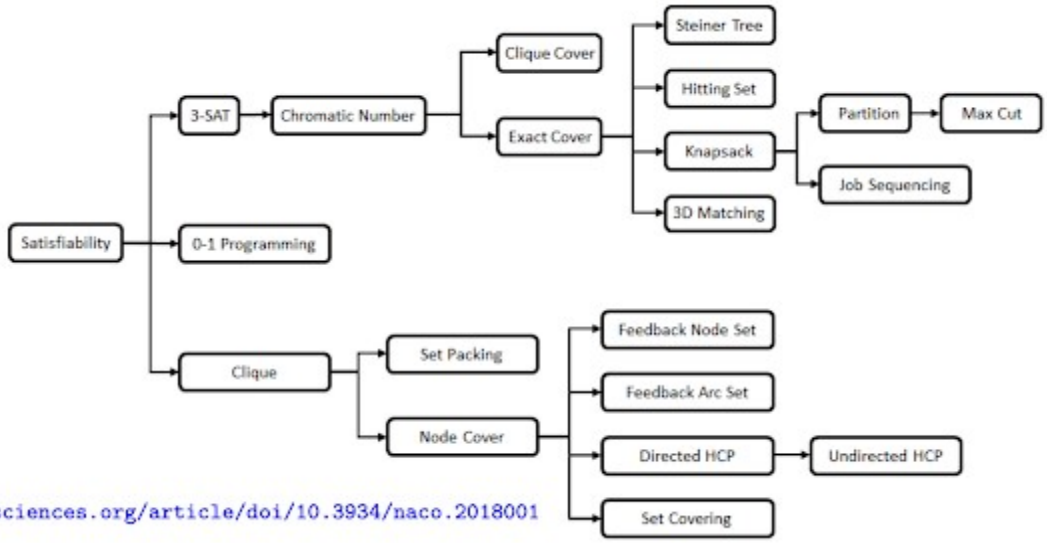
Il **teorema di Cook-Levin** dimostrò che SAT è NP-completo (tramite una dimostrazione complessa basata sugli stati della macchina di Turing).
La dimostrazione è enormemente complessa, e quindi non presentabile qui; sostanzialmente, Cook e in grado di esprimere, in modo algoritmico, una formula booleana equivalente a qualunque problema in NP.

Dunque, partendo dalle riduzioni viste oggi e utilizzando il teorema di Cook-Levin, otteniamo:

$$SAT \leq_p 3\text{-}SAT \leq_p INDEPENDENT\text{-}SET \leq_p VERTEX\text{-}COVER \leq_p SAT$$

In altre parole, tutti i problemi sono equivalenti e sono tutti NP-completi

Ad oggi sono stati dimostrati 21 problemi NP-completi:

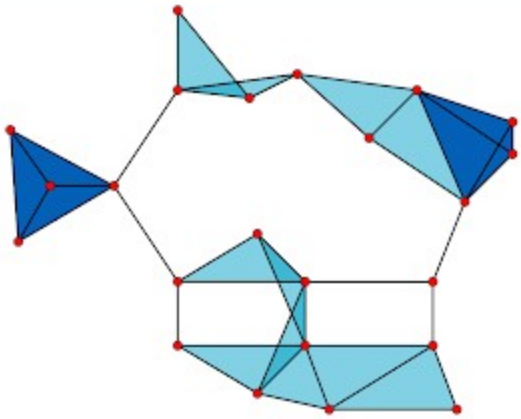


://aimsciences.org/article/doi/10.3934/naco.2018001

Vediamo altri problemi NP-completi “classici”:

Cricca (CLIQUE)

Dati un grafo non orientato ed un intero k, esiste un sottoinsieme di almeno k nodi tutti mutuamente adiacenti?



Applicazioni

- Bioinformatica
- Ingegneria elettronica
- Chimica

Commesso viaggiatore (Traveling salesperson - TSP)

Date n città, le distanze tra esse, ed un intero k, è possibile partire da una città, attraversare ogni città esattamente una volta tornando alla città di partenza, percorrendo una distanza non superiore a k?



Programmazione lineare $0/1$

Data una matrice A di elementi interi e di dimensione $m \times n$, ed un vettore b di m elementi interi, esiste un vettore x di n elementi $0/1$ tale che $Ax \leq b$?

Esempio

$$\begin{aligned} x_1 + x_2 + x_3 + x_4 &\leq 2 \\ x_1 - x_2 - x_3 + x_4 &\leq 0 \\ x_1 \qquad + x_3 + x_4 &\leq 1 \end{aligned}$$

Il sistema è verificato per $x_1 = x_2 = 1$ ed $x_3 = x_4 = 0$.

Copertura esatta di insiemi /EXACT COVER

Dato un insieme X e una collezione $\mathcal{Y} = \{Y_1, \ldots, Y_n\}$ di sottoinsiemi di X, esiste una sottocollezione $\mathcal{Z} \subseteq \mathcal{Y}$ che partizioni X?

$X = \{1, 2, 3, 4, 5, 6, 7\}$

$\mathcal{Y} = \{A, B, C, D, E, F\}$

$A = \{1, 4, 7\}$

$B = \{1, 4\}$

$C = \{4, 5, 7\}$

$D = \{3, 5, 6\}$

$E = \{2, 3, 6, 7\}$

$F = \{2, 7\}$

$\mathcal{Z} = \{B, D, F\}$

Partizione (PARTITION)

Dato un vettore A contenente n interi positivi, esiste un sottoinsieme $S \subseteq \{1 \ldots n\}$ tale che $\sum_{i \in S} A[i] = \sum_{i \notin S} A[i]$?

Somma di sottoinsieme (Subset sum)

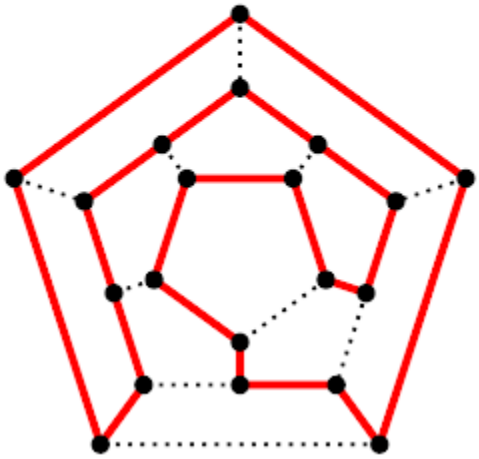
Dati un vettore A contenente n interi positivi ed un intero positivo k, esiste un sottoinsieme $S \subseteq \ldots n\}$ tale che $\sum_{i \in S} a[i] = k$?

Zaino (KNAPSACK)

Dati un intero positivo C (la capacità dello zaino) e un insieme di n oggetti, tali che l'oggetto i è caratterizzato da un "profitto" $p[i] \in \mathbf{Z}^+$ e da un peso $w[i] \in \mathbf{Z}^+$, esiste un sottoinsieme $S \subseteq \{1, \ldots, n\}$ tale che il peso totale $w(S) = \sum_{i \in S} w[i] \leq C$ e il profitto totale $p(S) = \sum_{i \in S} p[i]$ è maggiore o uguale a k?

Circuito hamiltoniano (HAMILTONIAN-CIRCUIT)

Dato un grafo non orientato G, esiste un circuito che attraversi ogni nodo una e una sola volta?



Notiamo che la complessità si nasconde dove non ce l'aspettiamo, infatti cambiando una sola parola di un problema, cambia completamente la complessità:

Circuito hamiltoniano

Dato un grafo non orientato G, esiste un circuito che attraversi ogni nodo una e una sola volta?

$\mathbb{NP}$-completo

Circuito euleriano

Dato un grafo non orientato G, esiste un circuito che attraversi ogni arco una e una sola volta?

In $\mathbb{P}$

Cammini massimi

Dato un grafo $G = (V, E)$ e una funzione di peso w sugli archi, trovare il cammino con peso massimo

$\mathbb{NP}$-completo

Cammini minimi

Dato un grafo $G = (V, E)$ e una funzione di peso w sugli archi, trovare il cammino con peso minimo

In $\mathbb{P}$

Ci sono tutt'ora problemi che non sappiamo se appartengano a P oppure a NP (ad esempio il problema dell'isomorfismo fra grafi, che richiede di determinare se due grafi finiti sono isomorfi).

Il problema della primalità si è scoperto da poco essere in P.

Il problema della fattorizzazione è sicuramente in NP, e si presume non sia in P e non sia NP-completo

Isomorfismo fra grafi

Il problema dell'isomorfismo fra grafi richiede di determinare se due grafi finiti sono isomorfi.

- Non sappiamo ancora.... $\mathbb{NP}$-completo? $\mathbb{P}$?
- Dibattito ancora in corso (Ottobre 2017)

Primalità

Dato un numero n, determinare se n è primo.

- Incluso in $\mathbb{P}$
- AKS primality test (2002-2005): $\tilde{O}(\log^6 n)$

Fattorizzazione

Dato un numero n, individuare i fattori primi che lo compongono.

- Sicuramente in $\mathbb{NP}$, si presume nè in $\mathbb{P}$, nè $\mathbb{NP}$-completo

Tecniche risolutive per problemi intrattabili

Se si deve risolvere un particolare problema, e si riesce a dimostrare la sua NP-completezza (ovvero non è risolvibile in tempo polinomiale), che cosa si può fare?
In generale, bisogna adottare un approccio diverso, che può essere riassunto dalla massima seguente:

"*Chi si accontenta gode*"

In sintesi non si può avere tutto dalla vita; bisogna rinunciare a qualcosa. Le 4 caratteristiche a cui possiamo rinunciare sono:

- **Generalità**
  Il caso generale del problema non è risolvibile in tempo polinomiale, ma si può vedere se esiste qualche valore particolare dell'input che renda il problema risolubile in tempo polinomiale (utilizzando un **algoritmo pseudo-polinomiale**).
- **Ottimalità** (il più seguito)
  Si può cercare di individuare un **algoritmo di approssimazione** che non restituisca necessariamente la soluzione ottima, ma che garantisca di ottenerne una non troppo "distante" da essa (è possibile dimostrare la "bontà" di un algoritmo di approssimazione matematicamente).

- **Formalità**
  Si possono ricercare **algoritmi euristici**, di solito basati su tecniche greedy o di ricerca locale, che: siano particolarmente facili da programmare; abbiano se possibile complessità polinomiale; e forniscano sperimentalmente risultati buoni anche se non vi è alcuna dimostrazione formale (matematica) che questo avvenga sempre.
- **Efficienza**
  Si possono progettare **algoritmi esponenziali** di tipo **branch-&-bound**, che cercano di evitare di enumerare tutte le soluzioni con un'accurata potatura. Dunque vengono comunque utilizzate soluzioni esponenziali ma si cerca di limitare il più possibile lo spazio di ricerca con un'accurata potatura.

Algoritmi pseudo polinomiali

Consideriamo il problema ¿_∑¿ (somma di sottoinsieme), che abbiamo già visto e risolto tramite backtracking $O(2^n)$ ed è risolvibile anche con programmazione dinamica tramite lo zaino $O(kn)$ (in questo caso trovando anche gli indici).

Il problema è il seguente:
Dati un vettore A contenente n interi positivi ed un intero positivo k, esiste un sottoinsieme $S \subseteq (1 \dots n)$ tale che $\sum_{i \in S} a[i] = k$?

Concentriamoci quindi sulla versione decisionale del problema (ovvero capire solamente se esistono degli indici).

Possiamo risolvere il problema tramite **programmazione dinamica** definendo una tabella booleana $DP[0 \dots n][0 \dots k]$.
$DP[i][r]$ è uguale a *true* se esiste un sottoinsieme dei primi i valori memorizzati in A la cui somma è pari a r, *false* altrimenti.

$$DP[i][r] = \begin{cases} \textbf{true} & r = 0 \\ \textbf{false} & r > 0 \wedge i = 0 \\ DP[i-1][r] & r > 0 \wedge i > 0 \wedge A[i] > r \\ DP[i-1][r] \textbf{ or } DP[i-1][r - A[i]] & r > 0 \wedge i > 0 \wedge A[i] \leq r \end{cases}$$

Il codice dell'algoritmo è il seguente:

```
boolean subsetSum(int[] A, int n, int k)
boolean[][] DP = new boolean[0...n][0...k] = {false}
for i = 0 to n do
    DP[i][0] = true                                  % Obiettivo raggiunto
for r = 1 to k do
    DP[0][r] = false                                 % Valori terminati
for i = 1 to n do
    for r = 1 to A[i] − 1 do
        DP[i][r] = DP[i − 1][r]                      % Valore troppo grande
    for r = A[i] to k do
        DP[i][r] = DP[i − 1][r] or DP[i − 1][r − A[i]]   % Valore ok
return DP[n][k]
```

Il costo dell'algoritmo basato su programmazione dinamica è $\Theta(kn)$, visto che viene riempita completamente una tabella grande $n \times k$.
Dunque la soluzione non è polinomiale nella dimensione del problema. Infatti assumendo $a[i] \leq k$ per $1 \leq i \leq n$, poiché altrimenti $a[i]$ può essere eliminato, la dimensione d dei dati in ingresso è $O(n \log k)$, ma non $O(nk)$.
Se k è limitata da un polinomio in n, cioè k è $O(n^c)$ con c costante, allora la funzione $subsetSum()$ ha complessità polinomiale, poiché nk risulta essere $O(n^{c+1})$. Se invece k è superpolinomiale in n, allora la complessità della funzione risulta essere anch'essa superpolinomiale. Infatti, se k è $O(2^n)$, allora nk risulta essere $O(n\ 2^n)$.
In altri termini, la complessità di $subsetSum()$ non dipende solamente dalla cardinalità n del sottoinsieme in ingresso, ma anche dai valori degli elementi del sottoinsieme. Tale complessità risulta polinomiale oppure no a seconda che i valori degli elementi in ingresso siano "piccoli" oppure "grandi" rispetto al loro numero (situazione analoga per lo zaino).

Ecco un esempio dell'algoritmo all'opera:

Esempio

$$A = [5, 9, 10]$$
$$n = 3$$
$$k = 24$$

i \ r	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	A
0	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	
1	1	◦	◦	◦	◦	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	5
2	1	◦	◦	◦	◦	1	◦	◦	◦	1	◦	◦	◦	◦	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	9
3	1	◦	◦	◦	◦	1	◦	◦	◦	1	1	◦	◦	◦	1	1	◦	◦	◦	1	◦	◦	◦	◦	1	10

Notiamo che con questo particolare esempio, la tabella viene “calcolata” tutta, ma poi i valori effettivamente utilizzati per la decisione finale sono pochi.

Dunque viene spontaneo provare ad utilizzare una tecnica ricorsiva di **backtracking**, così da calcolare solamente i valori che sono effettivamente utili al calcolo del risultato finale.

```
boolean ssRec(int[] A, int i, int r)
if r == 0 then
    return true                                  % Obiettivo raggiunto
else if i == 0 then
    return false                                 % Valori terminati
else if A[i] > r then
    return ssRec(A, i − 1, r)                    % Valore troppo grande
else
    return ssRec(A, i − 1, r) or ssRec(A, i − 1, r − A[i])   % Valore ok
```

	r	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	A
i	0	1	◦	◦	◦	◦	0	◦	◦	◦	0	0	◦	◦	◦	0	0	◦	◦	◦	◦	◦	◦	◦	◦	0	
	1	◦	◦	◦	◦	◦	1	◦	◦	◦	◦	◦	◦	◦	◦	0	0	◦	◦	◦	◦	◦	◦	◦	◦	0	5
	2	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	1	◦	◦	◦	◦	◦	◦	◦	◦	◦	0	9
	3	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	◦	1	10

Questo algoritmo costa $O(2^n)$.

Dunque da un lato abbiamo l’algoritmo di programmazione dinamica di complessità $O(nk)$ e dall’altro l’algoritmo di backtrack di complessità $O(2^n)$.
Utilizzando **memoization** possiamo combinare i due algoritmi combinando le due complessità a rendendo validi entrambi i limiti. Per la tabella di programmazione dinamica non utilizziamo una matrice ma un dizionario.

```
boolean ssRec(int[] A, int i, int r, DICTIONARY DP)
if r == 0 then
    return true                                   % Obiettivo raggiunto
else if i == 0 then
    return false                                  % Valori terminati
else
    boolean res = DP.lookup(⟨i, r⟩)
    if res == nil then
        res = ssRec(A, i − 1, r, DP)              % Valore non preso
        if A[i] < r then
            res = res or ssRec(A, i − 1, r − A[i], DP)   % Valore preso
        DP.insert((i, r), res)
    return res
```

i \ r	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	A
0	1	∘	∘	∘	∘	0	∘	∘	∘	0	0	∘	∘	∘	0	0	∘	∘	∘	∘	∘	∘	∘	∘	0	
1	∘	∘	∘	∘	∘	1	∘	∘	∘	∘	∘	∘	∘	∘	0	0	∘	∘	∘	∘	∘	∘	∘	∘	0	5
2	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	1	∘	∘	∘	∘	∘	∘	∘	∘	∘	0	9
3	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	∘	1	10

Esempio

$A = [1, 1, 1, 1, 1]$

$n = 5$

$k = 5$

i \ r	0	1	2	3	4	5	A
0	1	0	0	0	0	0	
1		1	0	0	0	0	1
2			1	0	0	0	1
3				1	0	0	1
4					1	0	1
5						1	1

Questa soluzione dunque si “adatta” in base all’input.

Programmazione dinamica	$\Theta(nk)$
Backtracking	$O(2^n)$
Memoization con dizionario	$O(nk), O(2^n)$

Abbiamo visto che la complessità dell’algoritmo di programmazione dinamica è esponenziale perché k è l’input e non la dimensione dell’input, infatti k viene rappresentato da $t = \lceil \log k \rceil$ cifre binarie, quindi la complessità è $O(nk) = O(n\ 2^t)$, esponenziale, dunque quasi sempre il limite superiore è dato da 2^n.

NP-completezza “forte”

Ci si potrebbe chiedere sotto quali condizioni un problema NP-completo può essere risolto con un algoritmo polinomiale per valori “piccoli” dei dati in ingresso.
Dato un problema decisionale R, con insieme di dati di ingresso I, distinguiamo tra la “dimensione” d di I, cioè la lunghezza della stringa che codifica I, ed il “più grande numero” intero # che appare in I.

In ¿_∑¿, d è $O(n \ \log \ \#)$ in quanto abbiamo $n+1+1$ elementi e $\log \ \#$ è il numero di bit che servono per rappresentare il più grande numero presente nell’input.

In cricca (*CLIQUE*), ovvero:
Dati un grafo non orientato ed un intero k, esiste un sottoinsieme di almeno k nodi tutti mutuamente adiacenti?
Come # prendiamo il massimo tra m, n e k perché il grafo, non essendoci pesi sugli archi, è una matrice di adiacenza.
Se rappresentiamo l’input come matrice di adiacenza, d è $O(n+m+\log \ \#)$

In commesso viaggiatore (*TSP*), ovvero:
Date n città e una matrice simmetrica d di distanze positive, dove $d[i][j]$ è la distanza fra i e j, trovare un percorso che, partendo da una qualsiasi città, attraversi ogni città esattamente una volta e ritorni alla città di partenza, in modo che la distanza totale percorsa sia minima.
Come # prendiamo il massimo tra n, k e $max(d)$ perché il grafo questa volta ha i pesi sugli archi e quindi è una matrice di pesi.

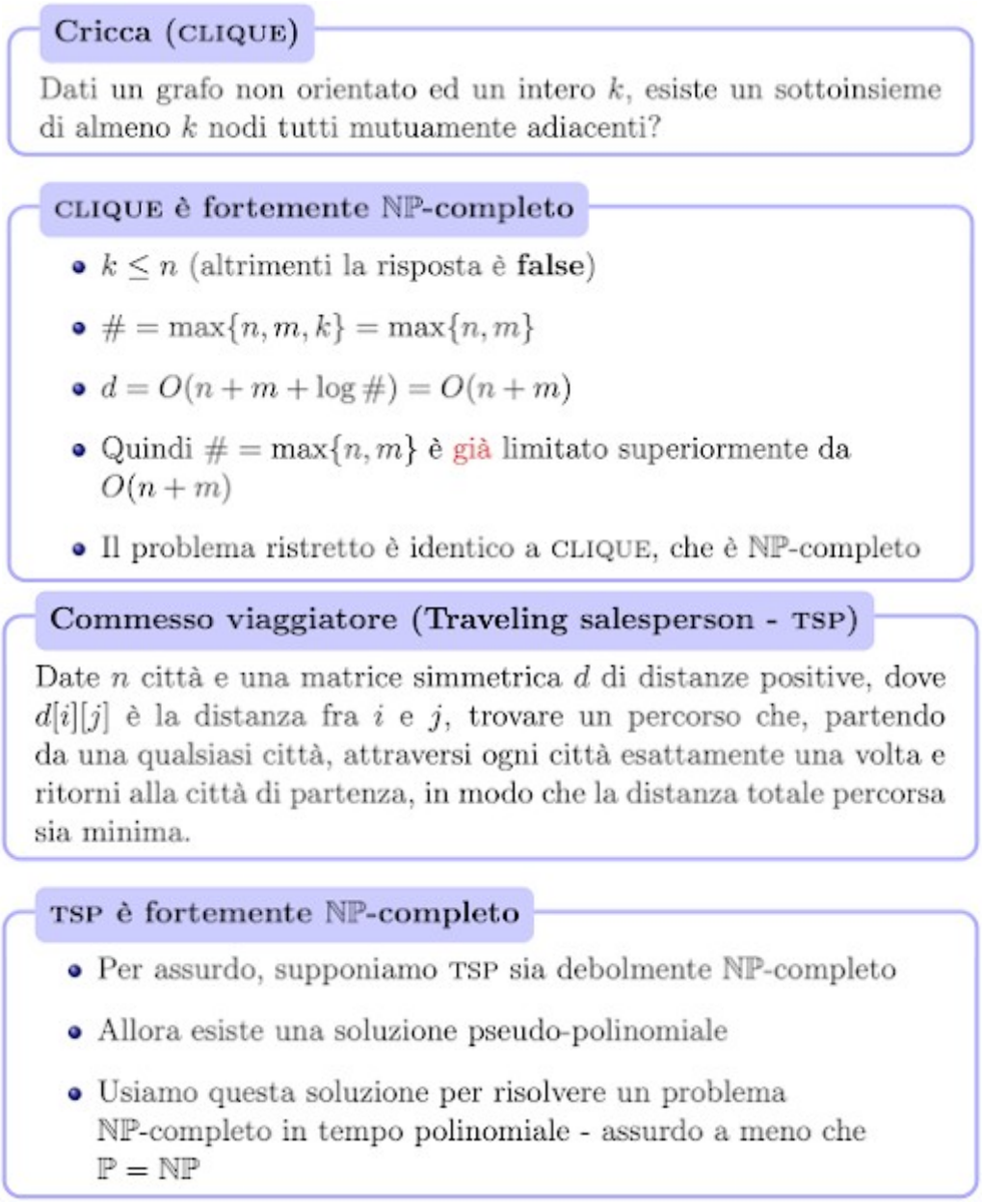
Esempi

Nome	I	#	d
SUBSET-SUM	$\{n, k, A\}$	$\max\{n, k, \max(A)\}$	$O(n \log \#)$
CLIQUE	$\{n, m, k, G\}$	$\max\{n, m, k\}$	$O(n + m + \log \#)$
TSP	$\{n, k, d\}$	$\max\{n, k, \max(d)\}$	$O(n^2 \log \#)$

Sia R_{pol} il problema R ristretto a quei dati d’ingresso per i quali # è limitato superiormente da $p(d)$, con p funzione polinomiale in d. R è **fortemente NP-completo** se R_{pol} è NP-completo.

Ad esempio, in ¿_∑¿ possiamo assumere (come già detto) che $a[i] \leq k$ per $1 \leq i \leq n$.
Se k è limitata da un polinomio in n, cioè k è $O(n^c)$ con c costante, allora
$\# = max(n, k, a_1, \ldots, a_n)$ che è $O(n^c)$ per le assunzioni che abbiamo fatto.
La soluzione basata su programmazione dinamica ha complessità $O(nk) = O(n^{c+1})$, quindi in P.
Quindi ¿_∑¿ non è fortemente NP-completo.

CLIQUE e *TSP* sono fortemente NP-completi.



Se un problema non è fortemente NP-completo, allora è **debolmente NP-completo**.

In pratica, sono **fortemente** NP-completi tutti quei problemi che possono essere dimostrati NP-completi con riduzioni che non usano numeri interi esponenzialmente grandi, mentre sono “debolmente” NP-completi quei problemi la cui dimostrazione di intrattabilità dipende pesantemente da riduzioni che usano numeri interi esponenzialmente grandi.

Un algoritmo ha complessità pseudo-polinomiale se risolve un certo problema R, per qualsiasi dato I d'ingresso, in tempo $p(\#, d)$, con p funzione polinomiale (non costante) in # e d (gli algoritmi per ¿ _ $\sum$ ¿ e *KNAPSACK* sono pseudo polinomiali).

Vale il seguente risultato:
Nessun problema fortemente NP-completo può essere risolto da un algoritmo pseudo-polinomiale, a meno che non sia $P = NP$.
Questo perché se R è fortemente NP-completo, allora il problema ristretto R_{pol} è NP-completo per qualche polinomio pol. Se esistesse un algoritmo pseudo-polinomiale A per R, allora A risolverebbe R_{pol} in tempo polinomiale, che è assurdo, a meno che non sia $P = NP$.

Se *TSP* fosse debolmente NP-completo, allora potremmo risolvere *HAMILTONIAN _ CIRCUIT* con un algoritmo polinomiale (visto che basta impostare i valori delle distanze ad 1 e 2)

Dimostriamo che TSP è fortemente $\mathbb{NP}$-completo

- Sia $G = (V, E)$ un grafo non orientato
- Definiamo una matrice di distanze a partire da G

$$d[i][j] = \begin{cases} 1 & (i,j) \in E \\ 2 & (i,j) \notin E \end{cases}$$

- Il grafo G ha un circuito hamiltoniano se e solo se è possibile trovare un percorso da commesso viaggiatore di costo n
- Se esistesse un algoritmo pseudopolinomiale A per TSP, HAMILTONIAN CIRCUIT potrebbe essere risolto da A in tempo polinomiale

Il più famoso problema fortemente NP-completo, in cui i numeri interi hanno un ruolo rilevante, è la cosiddetta 3 _ *PARTITION*, che generalizza appunto il problema della *PARTITION*.

PARTITION:
Dato un vettore A contenente *n* interi positivi, esiste un sottoinsieme $S \subseteq \{1 \ldots n\}$ tale che $\sum_{i \in S} A[i] = \sum_{i \notin S} A[i]$?

Questo problema è riducibile a ¿ _ $\sum$ ¿ scegliendo come valore k la metà di tutti i valori presenti:

Esempio

14	6	12	3	7	2

$$k = \frac{\sum_{i=1}^{n} A[i]}{2} = \frac{44}{2} = 22$$

Poiché è possibile risolverlo tramite un algoritmo pseudopolinomiale, $PARTITION$ è debolmente NP-completo.

$3_PARTITION$:
Dati $3n$ interi $\{a_1, \ldots, a_{3n}\}$, esiste una partizione in n triple $T_1, \ldots, T_n$, tale che la somma dei tre elementi di ogni T_j è la stessa, per $1 \leq j \leq n$?

Nonostante $3_PARTITION$ sia molto simile a $PARTITION$, esso non è debolmente NP-completo, bensì fortemente NP-completo.

Per dimostrare l'NP-completezza di un sottoproblema R, non è sempre necessario dimostrare l'NP-completezza di un sottoproblema R_{pol}. Infatti, è sufficiente provare l'esistenza di un altro problema Q, di cui si conosce già l'NP-completezza forte, che si può ridurre a R con una trasformazione che richiede tempo pseudo-polinomiale. Tale trasformazione deve inoltre garantire che $\#_R$, il più grande intero introdotto nei dati di R, sia limitato superiormente da una funzione $p(d_Q, \#_Q)$ polinomiale nella dimensione d_Q dell'input di Q e nel più grande intero $\#_Q$ che appare in tale input.

Algoritmi di approssimazione

Molti problemi di ottimizzazione di rilevante interesse pratico sono NP-ardui, poiché gli stessi problemi in forma decisionale sono NP-completi. Per tali problemi, c'è poca speranza di progettare un algoritmo che trovi la soluzione ottima in tempo polinomiale.
In molti casi, però, il fatto di non poter ottenere la soluzione ottima non è poi così drammatico, e ci si può accontentare di algoritmi che, in tempo polinomiale, trovino una soluzione ammissibile del problema che sia abbastanza "vicina" a quella ottima. Se è possibile dimostrare che la soluzione ammissibile fornita dall'algoritmo si discosta, nel caso pessimo o in quello medio, dalla soluzione ottima per uno scarto (o errore) relativo limitato per tutti i possibili dati di ingresso del problema, allora si ha un **algoritmo di "approssimazione"**.
Vedremo un po' di algoritmi di approssimazione e poi vedremo come non tutti i problemi NP-ardui ammettano algoritmi di approssimazione.

Dato un problema di ottimizzazione P, con funzione costo non negativa c, un **algoritmo** si dice **di $\alpha(n)$-approssimazione** se fornisce una soluzione ammissibile x il cui costo $c(x)$ non si

discosta dal costo $c(x^*)$ della soluzione ottima x^* per più di un fattore $\alpha(n)$, per qualunque input di dimensione n:

$$c(x^*) \leq c(x) \leq \alpha(n) \cdot c(x^*) \text{ con } \alpha(n) > 1 \text{ (**minimizzazione**)}$$
$$\alpha(n) \cdot c(x^*) \leq c(x) \leq c(x^*) \text{ con } \alpha(n) < 1 \text{ (**massimizzazione**)}$$

In altri termini, l'algoritmo garantisce che la percentuale di scostamento dalla soluzione ottima non superi una certa costante. Tale scostamento vale per la medesima costante e per tutti gli input.
Identificare un valore $\alpha(n)$ e dimostrare che l'algoritmo lo rispetta è ciò che rende un buon algoritmo un algoritmo di approssimazione.

Per iniziare, consideriamo un problema NP-arduo molto noto, il $BIN_PACKING$:

Bin packing

Dati:

- un vettore A contenente n interi positivi (i volumi degli oggetti)
- un intero positivo k (la capacità di una scatola, tale che $\forall i : A[i] \leq k$),

si vuole trovare una partizione di $\{1, \ldots, n\}$ nel minimo numero di sottoinsiemi disgiunti ("scatole") tali che $\sum_{i \in S} A[i] \leq k$ per ogni insieme S della partizione



Si consideri l'algoritmo FF (First-Fit, greedy) in cui gli oggetti sono considerati in un ordine qualsiasi (quello di "arrivo") e ciascun oggetto è assegnato alla prima scatoletta che lo può contenere, e dimostriamo che FF è un algoritmo di approssimazione.

Esempio

3	7	2	5	4	3	5

$k = 8$

3, 2, 3	7	5	4	5

Proviamo a "valutare" FF. Assumiamo che sia $A[i] \leq k$ per ciascun i. Se il numero N di scatolette usate da FF è maggiore di uno, allora tale numero deve essere minore di $\frac{2}{k} \sum_{i=1}^{n} A[i]$.
Questa proprietà deriva dal fatto che non ci possono essere due scatolette riempite al più per

metà. Infatti se ci fossero allora il primo oggetto inserito nella scatoletta di indice maggiore avrebbe potuto essere inserito nella scatoletta di indice minore e l'algoritmo FF non l'avrebbe inserito in un'altra scatoletta.

D'altro canto, il minimo numero di scatolette N^* non può essere inferiore a $\frac{1}{k}\sum_{i=1}^{n} A[i]$ (la soluzione ottima). Pertanto, $N < 2N^*$, quindi $\alpha(n) = 2$ ed FF non utilizza più del doppio del numero ottimo di scatolette.
È interessante rimarcare che è possibile dimostrare per un'altra via, molto più complicata, un risultato migliore per FF, e cioè che $N < \frac{17}{10}N^* + 2$. Un'approssimazione ancora migliore si ottiene con una variante di FF, detta FFD (First-Fit-Decreasing) dove gli oggetti sono considerati secondo l'ordine non crescente $A[1] \geq \ldots \geq A[n]$. Per FFD è stato dimostrato che $N < \frac{11}{9}N^* + 4$.
Queste sono dimostrazioni di limiti superiori per il fattore $\alpha(n)$, per casi particolari l'approssimazione può essere migliore.

Vediamo ora un esempio più complesso, ovvero un algoritmo di approssimazione per una variante semplificata del problema del *COMMESSO_VIAGGIATORE* (che abbiamo già visto).

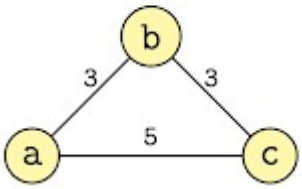
Commesso viaggiatore con dis. triangolari (Δ-TSP)

Siano date n città e le distanze (positive) $d[i][j]$ tra esse, tali per cui vale la regola delle diseguaglianze triangolari:

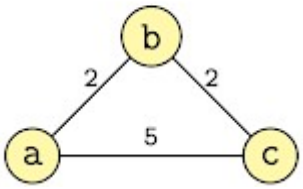
$$d[i][j] \leq d[i][k] + d[k][j] \quad \forall i, j, k : \quad 1 \leq i, j, k \leq n$$

Trovare un percorso che, partendo da una qualsiasi città, attraversi ogni città esattamente una volta e ritorni alla città di partenza, in modo che la distanza complessiva percorsa sia minima.

Con diseguaglianza triangolare



Senza diseguaglianza triangolare



La diseguaglianza triangolare in sintesi significa che risulta sempre più efficiente andare da un nodo ad un altro con un cammino diretto piuttosto che con un cammino indiretto.

La versione decisionale di ΔTSP è NP-completa. Infatti, nella prova di NP-completezza forte di TSP sono state usate distanze $d_{ij} \in \{1,2\}$, per $1 \leq i, j \leq n$, che ovviamente verificano la proprietà di triangolarità. Dunque si può applicare la stessa dimostrazione per ΔTSP, e visto che $HAMILTONIAN_CIRCUIT$ è NP-completo, allora lo è anche ΔTSP.

Lemma

HAMILTONIAN-CIRCUIT $\leq_p$ Δ-TSP

Dimostrazione

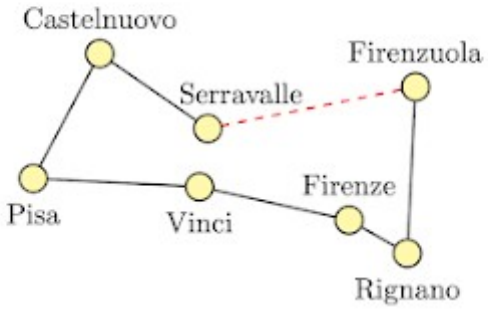
- Sia $G = (V, E)$ un grafo non orientato
- Definiamo una matrice di distanze a partire da G:

$$d[i][j] = \begin{cases} 1 & (i,j) \in E \\ 2 & (i,j) \notin E \end{cases}$$

- Diseguaglianze triangolari: $d[i][j] \leq 2 \leq d[i][k] + d[k][j]$
- G ha un circuito hamiltoniano $\Leftrightarrow$
  esiste un percorso Δ-TSP in d di lunghezza n

Per semplicità, ΔTSP può essere reinterpretato come il problema di trovare un circuito Hamiltoniano di costo minimo in un grafo K_n di n nodi, non orientato, pesato e completo, cioè formato da una cricca di n nodi.

- Si consideri un circuito hamiltoniano e si cancelli un suo arco
- Si ottiene un albero di copertura



Un qualsiasi circuito Hamiltoniano di K_n in cui si cancelli un arco è un particolare albero di copertura per esso, dunque si può facilmente osservare che il costo dell'albero di copertura è inferiore a quello del circuito hamiltoniano da cui è stato ricavato.

Teorema:
Qualunque circuito hamiltoniano π ha costo $c(\pi)$ superiore al costo mst di un albero di copertura di peso minimo, ovvero $mst < c(\pi)$.

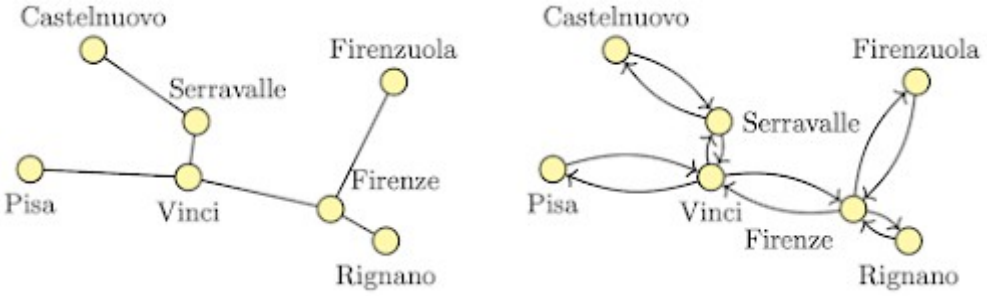
Dimostrazione

Per assurdo

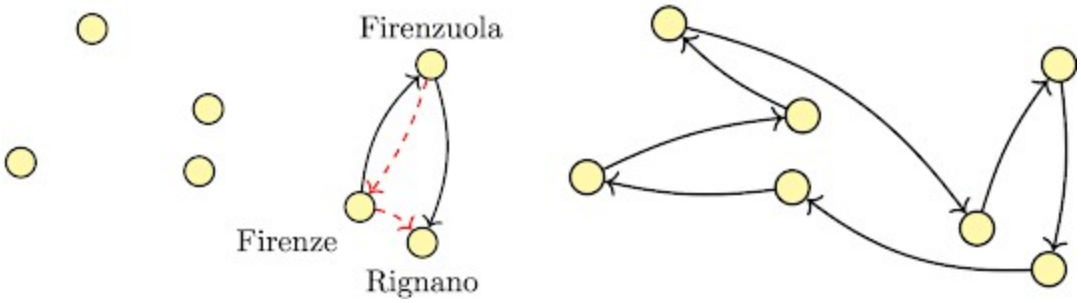
- Supponiamo che esista un circuito hamiltoniano π di costo $c(\pi) \leq mst$
- Togliamo un arco, otteniamo un albero di copertura con peso inferiore $mst' < c(\pi) \leq mst$
- Contraddizione, visto che mst è il costo minimo fra tutti gli alberi di copertura

Di conseguenza possiamo anche dire che l'albero di copertura di peso minimo ha sicuramente costo inferiore del ciclo hamiltoniano di peso minimo.
Quindi un minimo albero di copertura per K_n non può avere un costo superiore alla soluzione ottima di ΔTSP (che trova un circuito hamiltoniano di peso minimo).

Se si individua un minimo albero di copertura di peso mst e se ne percorrono gli archi due volte, la prima volta in un senso e l'altra nel senso opposto, si può visitare ciascuna città almeno una volta e la distanza complessiva percorsa, ovvero $2 \cdot mst$, non supera il doppio della distanza ottima di ΔTSP. Non abbiamo però un circuito hamiltoniano.



Visitando le città in modo da "saltare" una città già visitata, si ottiene, per la triangolarità delle distanze, un circuito Hamiltoniano π il cui costo $c(\pi)$ è sicuramente minore o uguale di $2 \cdot mst$.
Per il teorema possiamo anche dire che $c(\pi) < 2 \cdot c(\pi^*)$, con $c(\pi^*)$ costo del circuito Hamiltoniano minimo, visto che vale $2 \cdot mst < 2 \cdot c(\pi^*)$.
Dunque $\alpha(n) = 2$.



La complessità di questo algoritmo di approssimazione è $O(n^2 \log n)$:

- $O(n^2 \log n)$ per l'algoritmo di Kruskal
- $O(n)$ per la visita in profondità del MTS raddoppiato con $2n$ archi

È possibile individuare dati di ingresso particolarmente "perversi" per i quali il fattore di approssimazione tende a 2.

L'algoritmo di Christofides (1976) ha un fattore di approssimazione di $\frac{3}{2}$, ed è il migliore risultato al momento (fino al 2021).

Non approssimabilità

Purtroppo, per molti problemi di ottimizzazione, trovare una soluzione approssimata è tanto difficile quanto trovarne una ottima! Questo è vero, per esempio, per le versioni di ottimizzazione di $CRICCA$, $COLORAZIONE$ e $COMMESSO_VIAGGIATORE$ (senza diseguaglianze triangolari).
Questo è vero a meno che P non sia uguale a NP, ma in quel caso molti altri teoremi non varrebbero più.

Algoritmi euristici

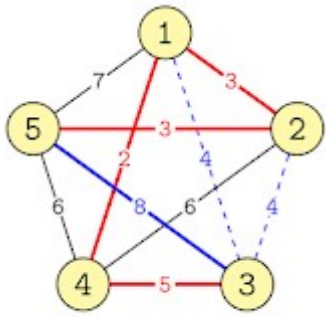
Quando si è presi dalla disperazione a causa dell'enorme difficoltà di un problema di ottimizzazione NP-arduo, si può ricorrere ad algoritmi "euristici" che forniscono una soluzione ammissibile, non necessariamente ottima né approssimata. Tali euristiche sono di solito basate su tecniche **greedy** oppure di **ricerca locale**.
Vengono utilizzati dunque algoritmi che funzionano molto bene nella pratica, ma di cui non si riesce a dimostrare la correttezza.

Greedy

Iniziamo progettando un algoritmo euristico di tipo greedy per il problema $COMMESSO_VIAGGIATORE$.

Shortest edge first
Consideriamo l'input come grafo non orientato completo e pesato. Ordiniamone gli archi per pesi non decrescenti e proviamo ad aggiungere archi alla soluzione seguendo questo ordine finché non sono stati aggiunti $n-1$ archi, dove n è il numero di nodi. Per ogni nodo, contiamo quanti archi incidenti in quel nodo sono stati aggiunti alla soluzione. Per poter aggiungere una arco, occorre verificare che per ciascuno dei suoi nodi non siano stati già scelti due archi e che non si formino circuiti. Per verificare quest'ultima proprietà, si utilizza un $MFSET$, terminando di aggiungere archi quando l'$MFSET$ è formato da una sola componente.
A questo punto si è trovata una catena Hamiltoniana e si chiude il circuito aggiungendo l'arco tra i due nodi estremi della catena, ciascuno dei quali ha un solo arco incidente.

	1	2	3	4	5
1		3	4	2	7
2	3		4	6	3
3	4	4		5	8
4	2	6	5		6
5	7	3	8	6	



Costo: 21

Questa euristica richiede tempo $O(n^2 \log n)$, ovvero il costo dell'ordinamento degli archi, e si può riassumere nella procedura *greedyTsp*().

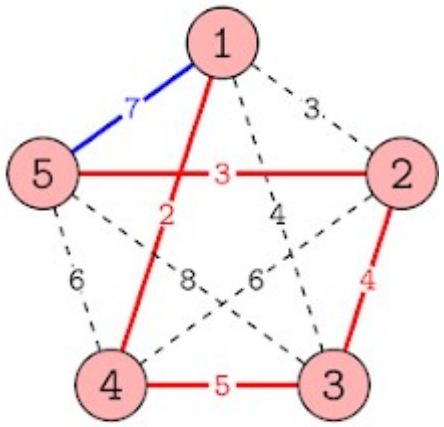
```
SET greedyTsp(GRAPH G)
SET result = Set()
MFSET M = Mfset(G.n)
int[] edges = new int[1...n] = { 0 }          % N. archi nella catena
A = {ordina gli archi per peso non decrescente}
foreach (u, v) ∈ A do
    if edges[u] < 2 and edges[v] < 2 and M.find(u) ≠ M.find(v) then
        result.insert( (u, v) )
        edges[u] = edges[u] + 1
        edges[v] = edges[v] + 1
        M.merge(u, v)
int u = 1; while edges[u] ≠ 1 do u = u + 1
int v = u + 1; while edges[v] ≠ 1 do v = v + 1
result.insert( (u, v) )
return result
```

Nearest neighbor

In questo approccio si parte da una città, si seleziona come prossima città quella più vicina e si va avanti così evitando città già visitate. Quando si sono visitate tutte le città, si torna alla città di partenza.

Si può fare un algoritmo che lavori direttamente sulla matrice di adiacenza.

	1	2	3	4	5
1		3	4	2	7
2	3		4	6	3
3	4	4		5	8
4	2	6	5		6
5	7	3	8	6	



Costo: 21

Questa euristica, invece, richiede tempo $O(n^2)$.

La soluzione che otteniamo con questi due approcci può:

- Essere utilizzata come base di partenza per un algoritmo branch-&-bound
- Essere migliorata ulteriormente tramite ricerca locale

Ricerca locale

Per introdurre le tecniche euristiche basate su ricerca locale, si consideri ancora il problema del *COMMESSO_VIAGGIATORE* e sia π un circuito Hamiltoniano del grafo completo non orientato K_n. Un'euristica ragionevole di ricerca locale può essere progettata sulla base del seguente intorno:

$$I_2(\pi) = \{\pi' : \pi' \text{ è ottenuto da } \pi \text{ cancellando due archi non consecutivi del circuito e sostituendoli con due archi non compresi nel circuito}\}$$

Si noti che $|I_2(\pi)| = \frac{n(n-1)}{2} - n$. Infatti, cancellati due archi non consecutivi, c'è un unico modo per riconnettere i due spezzoni in un circuito con archi che non fanno parte di π, mentre il numero di tutte le possibili coppie di archi non consecutivi di π è dato da quello di tutte le $\frac{n(n-1)}{2}$ coppie meno le n coppie di archi consecutivi. Pertanto, in un'euristica di ricerca locale, il passaggio da una soluzione π ad una $\pi' \in I_2(\pi)$ richiede $O(n^2)$ tempo.

Ovviamente, una soluzione iniziale può essere generata casualmente oppure costruita tramite l'euristica $greedyTsp()$.
Ovviamente non vi è alcuna garanzia che la soluzione trovata sia ottima globalmente, ma solamente ottima localmente. Questo perché può esistere una soluzione ottenibile scambiando 3 o 4 archi anziché solamente 2.

"A heuristic is an algorithm that doesn't work"

Algoritmi branch-&-bound

Per risolvere un problema di ottimizzazione NP-arduo si può modificare la procedura $enumeration()$ vista nella sezione su backtrack, in modo da evitare di effettuare certe sequenze di scelte che si rivelino incapaci di generare la soluzione ottima.

Si assuma che il problema da risolvere sia di minimo (tutto va ribaltato nel caso di massimo), che ogni sequenza di scelte abbia costo non negativo, e che ogni scelta, aggiunta alle scelte già effettuate, non faccia diminuire il costo della soluzione parziale così costruita.

Ricordiamo la procedura $enumeration()$:

```
enumeration(⟨dati problema⟩, ITEM[] S, int i, ⟨dati parziali⟩)
if isAdmissible(⟨dati problema⟩, S, i, ⟨dati parziali⟩) then
    processSolution(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
    return true                % Trovata soluzione, restituisco true
else if reject(⟨dati problema⟩, S, i, ⟨dati parziali⟩) then
    return false  % Impossibile trovare soluzioni, restituisco false
else
    SET C = choices(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
    foreach c ∈ C do
        S[i] = c
        if enumeration(⟨dati problema⟩, S, i + 1, ⟨dati parziali⟩) then
            return true        % Trovata soluzione, restituisco true
    return false               % Nessuna soluzione, restituisco false
```

Si supponga di avere a disposizione una opportuna funzione "**lower bound**" $lb(S,i)$, che dipenda dalla sequenza di scelte fatte $S[1,\ldots,i]$ e garantisca che tutte le soluzioni ammissibili generabili aggiungendo nuove scelte a tale sequenza abbiano costo maggiore o uguale a $lb(S,i)$. Allora, è possibile mantenersi la migliore soluzione ammissibile $minSol$ tra quelle finora esaminate, insieme al suo costo $minCost$ (**upper bound**), e valutare $lb(S,i)$ ogniqualvolta è effettuata una nuova scelta $S[i]$ in un nodo dell'albero delle scelte. Se $lb(S,i)$ è maggiore o uguale a $minCost$, allora si può evitare di generare ed esplorare il sottoalbero delle scelte radicato in tal nodo.

In sintesi, nel momento in cui sono sicuro che in quel "percorso" non posso più trovare una soluzione, non ha senso continuare.

Questo metodo, detto **branch-&-bound**, non migliora la complessità della procedura $enumeration()$, che resta superpolinomiale nel caso pessimo, ma ne abbassa drasticamente il tempo di esecuzione nella pratica. Ciò vale in special modo se si riesce ad individuare una "buona" funzione lb, che sia cioè il più vicino possibile al costo della soluzione ottima.

```
branch&bound(⟨dati problema⟩, ITEM[] S, int i, ⟨dati parziali⟩)
SET C = choices(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
foreach c ∈ C do
    S[i] = c
    int lb = lb(⟨dati problema⟩, S, i, ⟨dati parziali⟩)
    if lb < minCost then
        if i < n then
            branch&bound(⟨dati problema⟩, S, i + 1, ⟨dati parziali⟩)
        else
            if cost(S, i) < minCost then
                minSol = S                    % Variabile globale
                minCost = cost(S, i)          % Variabile globale
```

In pratica, il tempo di computazione di una procedura branch-&-bound è influenzata da svariati fattori:

- **Soluzione di partenza**
  Anziché partire con $minCost = +\infty$, è in genere più conveniente determinare il costo di una soluzione ammissibile del problema ed inizializzare $minCost$ al valore di tale costo
- **Branch**
  Prestazioni migliori si possono ottenere variando la cosiddetta regola di branch, ovvero il criterio per determinare le scelte da effettuare e generare i figli di un nodo nell'albero delle scelte
- **Bound**
  La "migliore" regola per calcolare $lb()$ è quella che dà il lower bound più stretto, cioè più vicino possibile al costo della soluzione ottima, ma che sia nel contempo calcolabile velocemente
- **Ordine di visita**
  L'ordine più usato per generare e visitare l'albero delle scelte è senza dubbio quello anticipato (LIFO) precedentemente esposto, perché richiede meno spazio; infatti, utilizzando una pila, il massimo numero di "record di attivazione" è proporzionale alla lunghezza di un percorso radice-foglia, che è $O(n)$; talvolta, sono usati criteri diversi, in cui l'albero è visitato per livelli (FIFO) o utilizzando una priorità (per esempio considerando prima quel figlio avente il più promettente lower bound).
- **Scarto dall'ottimo**
  Anzichè raggiungere la soluzione ottima a tutti i costi, l'esecuzione può essere interrotta quando lo scarto dall'ottimo scende sotto una certa soglia prestabilita, cioè quando $\frac{minCost - lb^*}{lb^*} \le e$, con e costante prefissata ed lb^* uguale al più piccolo lower bound calcolato per i nodi generati ma non ancora visitati dell'albero delle scelte

Commesso viaggiatore

Come esempio di tecnica branch-&-bound, risolviamo il problema del $COMMESSO_VIAGGIATORE$ (senza diseguaglianze triangolari), per il quale sappiamo che non può esistere né un algoritmo ottimo pseudo-polinomiale né un algoritmo di approssimazione polinomiale (a meno che $P = NP$). Cerchiamo dunque di specializzare la procedura $branch\&bound()$ vista precedentemente.

Sia n il numero di città e $d[h][k]$ la distanza, intera non negativa, tra le città h e k, $1 \le h,k \le n$. Si assuma che al passo i-esimo siano state fatte le scelte $S[1,\ldots,i]$ prese dall'insieme $[1,\ldots,n]$. Ovviamente, un qualsiasi percorso ammissibile del commesso viaggiatore generabile "espandendo" $S[1,\ldots,i]$ deve attraversare le città $S[1]$, $S[2]$, …, $S[i]$ in quest'ordine, passare da $S[i]$ ad una qualsiasi delle rimanenti $n-i$ città, attraversare queste ultime città in un ordine qualsiasi, e da una di queste ritornare ad $S[1]$.

Degli esempi di lower bound sono:

- Somma del costo attuale più il numero di scelte rimanenti (perchè per ogni città avrò almeno 1 di costo)
- Somma del costo attuale più il numero di scelte rimanenti moltiplicato per il peso minimo degli archi

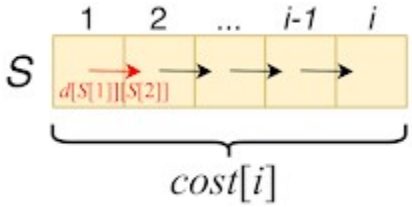
Versione 1

Il lower bound è il costo effettivo della soluzione finora trovata, ovvero la somma totale degli archi percorsi nel cammino percorso finora.

$$cost[i] = \sum_{h=2}^{i} d[S[h-1]][S[h]]$$

e dunque

$$lb(d,S,i) = cost[i]$$



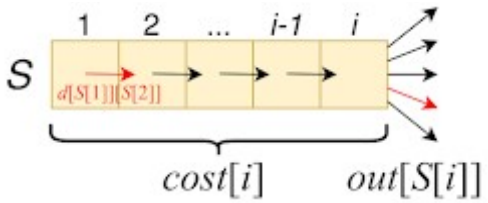
Versione 2

Il lower bound è il costo effettivo della soluzione finora trovata sommato al lower bound per "uscire" dall'ultimo nodo, ovvero il valore minimo fra tutti gli archi che escono da $S[i]$ e vanno in uno dei nodi non ancora scelti.

$$out[S[i]] = min_{k \notin S[1\ldots i]} d[S[i]][k]$$

e dunque

$$lb(d,S,i) = cost[i] + out[S[i]]$$



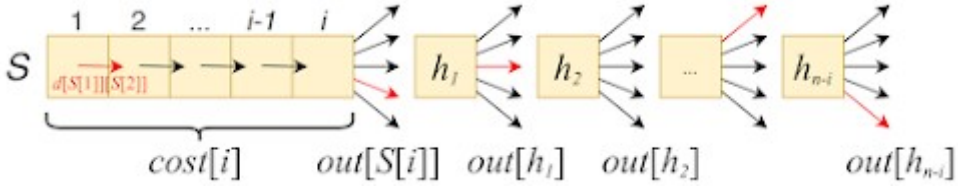
Versione 3

Il lower bound è il costo effettivo della soluzione finora trovata sommato al lower bound per "uscire" dai nodi non ancora scelti, ovvero per ogni nodo h non scelto, il valore minimo fra gli archi che escono da esso e vanno in un nodo non ancora scelto oppure in $S[1]$ (chiusura del circuito)

$$\forall h \notin S[1 \ldots h] \; : \; out[h] = min_{k \notin S[2 \ldots i]} d[h][k]$$

e dunque

$$lb(d, S, i) = cost[i] + out[S[i]] + \sum_{h \notin S[1 \ldots i]} out[h]$$



Versione 4

Il lower bound è il costo effettivo della soluzione finora trovata sommato al lower bound per "attraversare" i nodi non ancora scelti, ovvero per ogni nodo h non scelto, il costo minimo dato dalla somma di due archi (p, h) e (h, q) distinti che entrano e escono da h, proveniendo da $S[i]$ o da un nodo non ancora scelto; andando in un nodo non ancora scelto oppure in $S[1]$

$$transfer[h] = min_{p \notin S[1 \ldots i-1], q \notin S[2 \ldots i], \; p \neq q \neq h} \{ d[p][h] + d[h][q] \}$$

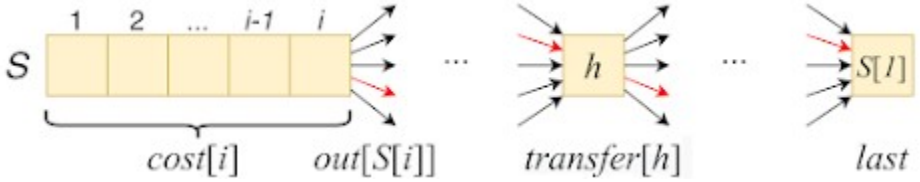
bisogna sommare anche il lower bound per tornare al primo nodo, ovvero il valore minimo fra tutti gli archi che vanno da uno dei nodi non ancora scelti a $S[1]$.

$$last = min_{h \notin S[1 \ldots i]} d[h][S[1]]$$

e dunque

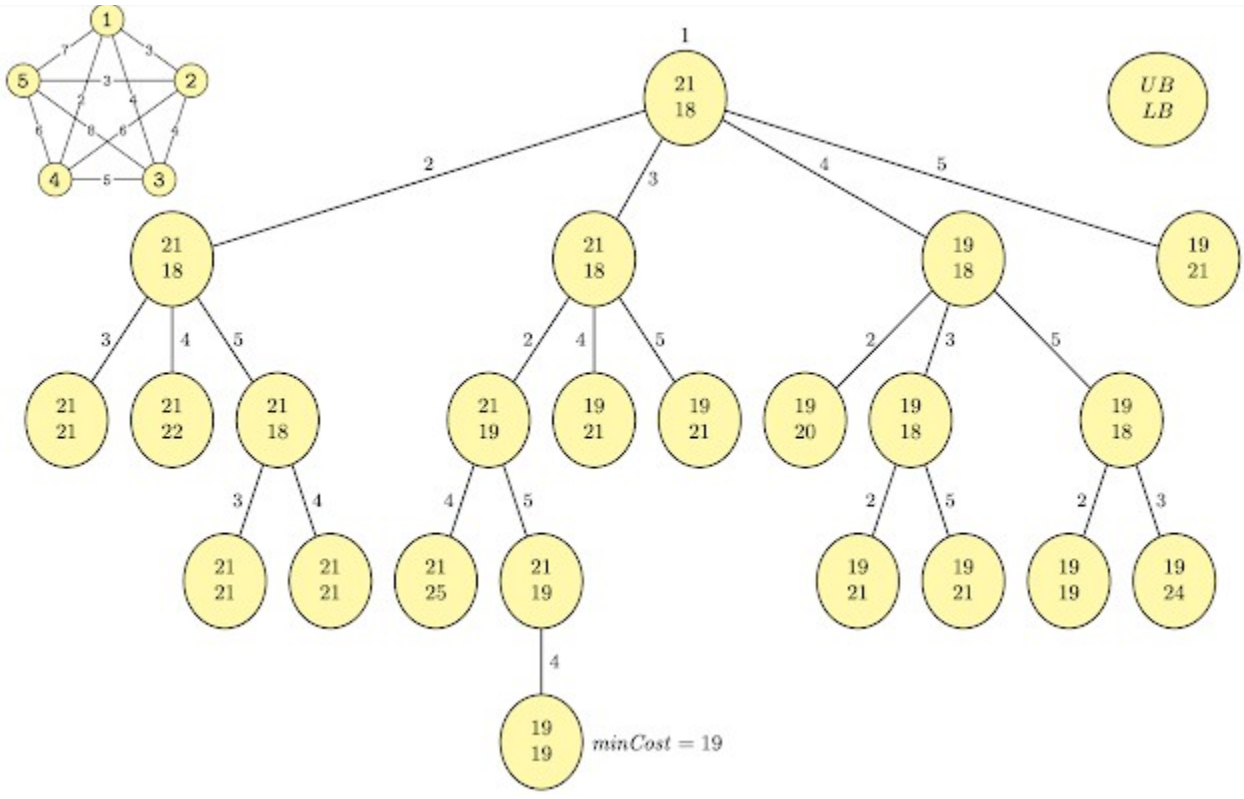
$$\text{lb}(d, S, i) = cost[i] + \left\lceil \frac{out + \sum_{h \notin S} transfer[h] + last}{2} \right\rceil$$

Si divide per un fattore 2 perché tutti gli archi vengono contati due volte, in uscita per un nodo e in entrata per un altro.



Ecco il codice:

```
bbTsp(ITEM[] S, int cost, SET R, int n, int i)
SET choices = copy(R)
foreach c ∈ choices do
    S[i] = c
    R.remove(c)
    if i < n then
        {calcola out, last, e transfer[h] per ogni h ∈ R}
        int lb = cost[i] + ⌈(out + Σ_{h∉S} transfer[h] + last) /2⌉
        if lb < minCost then
            bbTsp(S, cost + d[S[i − 1]][S[i]], R, n, i + 1)
    else
        cost = cost + d[S[i]][S[1]]
        if cost < minCost then
            minSol = S                          % Variabile globale
            minCost = costo                     % Variabile globale
    R.insert(c)
```



In questo semplice esempio, è stato possibile "potare" 42 su 65 nodi

Possibili miglioramenti

- È possibile variare l'ordine di visita dell'albero delle scelte
  - DFS vs Best-first
- È possibile variare il meccanismo di branching
  - Sui nodi, sugli archi, etc.
- È possibile cercare dei lower bound più stretti
  - Held, M., and Karp, R. M. (1971), "The Traveling Salesman Problem and Minimum Spanning Trees: part II", Mathematical Programming 1:6-25

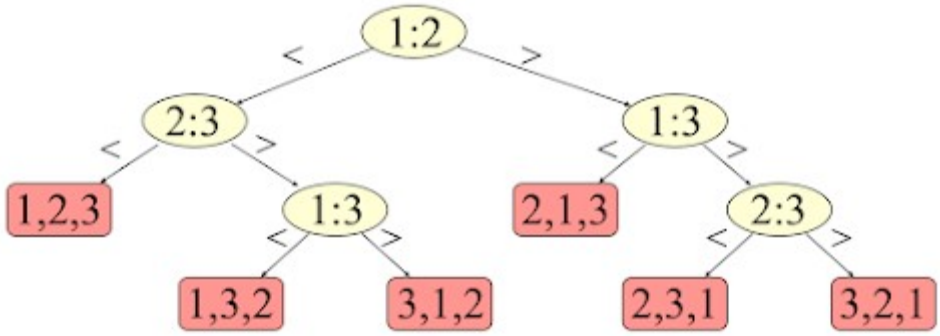
Algoritmi di ordinamento

Sommario - Algoritmi di ordinamento

- SelectionSort - $\Theta(n^2)$
- InsertionSort - $\Omega(n)$, $O(n^2)$
- ShellSort - $\Omega(n)$, $O(n^{3/2})$
- MergeSort - $\Theta(n \log n)$
- HeapSort - $\Theta(n \log n)$
- QuickSort - $\Omega(n \log n)$, $O(n^2)$

Tutti questi algoritmi sono basati su confronti tra due elementi, e gli algoritmi migliori sono $O(n \log n)$ (tralasciando InsertionSort e ShellSort, che sono più veloci solo in alcuni casi particolari)

È possibile dimostrare che qualunque algoritmo di ordinamento **basato su confronti** ha complessità $\Omega(n \log n)$.
Dimostrazione:
Consideriamo un qualunque algoritmo A basato su confronti. Assumiamo che tutti i valori siano distinti (no perdita di generalità).
L'algoritmo A può essere rappresentato tramite un **albero di decisione**, un albero binario che rappresenta i confronti tra gli elementi.



Un **cammino radice-foglia** in un albero di decisione è sequenza di confronti eseguiti dall'algoritmo corrispondente, dunque è un'esecuzione dell'algoritmo.
L'**altezza** dell'albero di decisione è il numero di confronti eseguiti dall'algoritmo corrispondente nel caso pessimo.

Sappiamo che un albero di decisione per l'ordinamento di n elementi contiene almeno $n!$ foglie (potrebbero essere di più nel caso in cui l'algoritmo possa arrivare in modi diversi alla stessa permutazione).

Sappiamo che se T è un albero binario in cui ogni nodo interno ha esattamente 2 figli e k è il numero delle sue foglie, l'altezza dell'albero è almeno $\log\ k$, ovvero $\Omega(\log\ k)$.

Dunque l'altezza minima dell'albero di decisione per l'ordinamento è
$\Omega(\log\ n!) = \Omega(\log\ n^n) = \Omega(n\ \log\ n)$

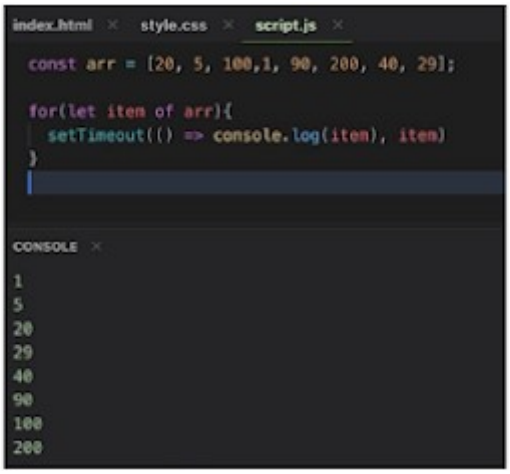
Esistono algoritmi di ordinamento che non sono basati su confronti, ad esempio SpaghettiSort:

Algoritmo Spaghetti Sort – $O(n)$

1. Prendi n spaghetti
2. Taglia lo spaghetto i-esimo in modo proporzionale all'i-esimo valore da ordinare
3. Con la mano, afferra gli n spaghetti e appoggiali verticalmente sul tavolo
4. Prendi il più lungo, misuralo e metti il valore corrispondente in fondo al vettore da ordinare
5. Ripeti (4) fino a quando non hai terminato gli spaghetti

Solo che SpaghettiSort assume che sia possibile confrontare contemporaneamente tutti i valori.

Un altro algoritmo simile a SpaghettiSort è TimerSort:



Un'ulteriore tecnica di ordinamento che non si basa su confronti è quella utilizzata da CountingSort.
I numeri da ordinare sono compresi in un intervallo $[1 \dots k]$. Si costruisce un array $B[1 \dots k]$ che conta il numero di volte in cui un valore compreso in $[1 \dots k]$ compare in A. Ricolloca i valori così ottenuti nel vettore da ordinare A
L'intervallo non deve necessariamente iniziare in 1 e finire in k; qualunque intervallo di cui conosciamo gli estremi può essere utilizzato nel CountingSort.

countingSort(**int**[] A, **int** n, **int** k)

```
int[] B = new int[1...k]
for i = 1 to k do
    B[i] = 0
for j = 1 to n do
    B[A[j]] = B[A[j]] + 1
j = 1
for i = 1 to k do
    while B[i] > 0 do
        A[j] = i
        j = j + 1
        B[i] = B[i] - 1
```

La complessità è $O(n+k)$, se $k = O(n)$ allora la complessità è $O(n)$.

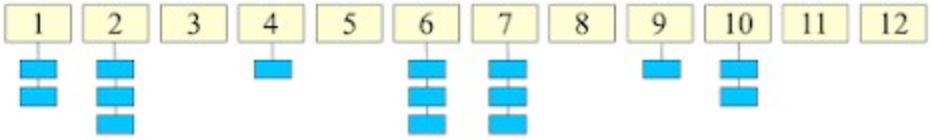
Counting Sort e limiti inferiore per l'ordinamento

- Counting Sort non è basato su confronti
- Abbiamo cambiato le condizioni di base
- Pseudopolinomiale: se k è $O(n^3)$, questo algoritmo è peggiore di tutti quelli visti finora

Pigeonhole Sort

Casellario

- Cosa succede se i valori non sono numeri interi, ma record associati ad una chiave da ordinare?
- Non possiamo usare counting
- Ma possiamo usare liste concatenate!



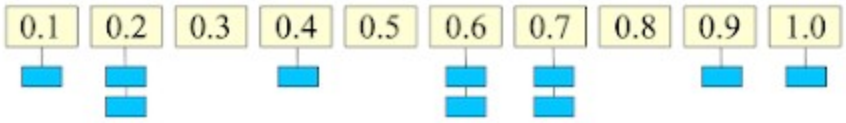
Bucket Sort

Ipotesi sull'input

- Valori reali uniformemente distribuiti nell'intervallo $[0, 1)$
- Qualunque insieme di valori distribuiti uniformemente può essere normalizzato nell'intervallo $[0, 1)$ in tempo lineare

Idea

- Dividere l'intervallo in n sottointervalli di dimensione $1/n$, detti bucket, e poi distribuire gli n numeri nei bucket
- Per l'ipotesi di uniformità, il numero atteso di valori nei bucket è 1
- Possono essere ordinati con Insertion Sort



Proprietà degli algoritmi di ordinamento

Stabilità

Un algoritmo di ordinamento è stabile se preserva l'ordine iniziale tra due elementi con la stessa chiave

Domande

- Quali dei seguenti algoritmi sono stabili? Insertion Sort, Merge Sort, Heap Sort, Quick Sort, Pigeonhole Sort
- Come si può rendere un qualunque algoritmo stabile?

Risposte

- Stabili: Insertion Sort, Merge Sort, Pigeonhole Sort
- Basta usare come chiave di ordinamento la coppia (chiave, posizione iniziale)

Riassunto ordinamento

Insertion Sort

$\Omega(n)$, $O(n^2)$, stabile, sul posto, iterativo. Adatto per piccoli valori, sequenze quasi ordinate.

Merge Sort

$\Theta(n \log n)$, stabile, richiede $O(n)$ spazio aggiuntivo, ricorsivo (richiede $O(\log n)$ spazio nello stack). Buona performance in cache, buona parallelizzazione.

Heap Sort

$\Theta(n \log n)$, non stabile, sul posto, iterativo. Cattiva performance in cache, cattiva parallelizzazione. Preferito in sistemi embedded.

Quick Sort

$O(n \log n)$ in media, $O(n^2)$ nel caso peggiore, non stabile, ricorsivo (richiede $O(\log n)$ spazio nello stack). Buona performance in cache, buona parallelizzazione, buoni fattori moltiplicativi.

Counting Sort

$\Theta(n+k)$, richiede $O(k)$ memoria aggiuntiva, iterativo. Molto veloce quando $k = O(n)$

Pigeonhole Sort

$\Theta(n+k)$, stabile, richiede $O(n+k)$ memoria aggiuntiva, iterativo. Molto veloce quando $k = O(n)$

Bucket Sort

$O(n)$ nel caso i valori siano distribuiti uniformemente, stabile, richiede $O(n)$ spazio aggiuntivo

Shell Sort

$O(n\sqrt{n})$, stabile, adatto per piccoli valori, sequenze quasi ordinate.